

STRATOSPHERIC

**From Zero to
Production
with Spring
Boot and
AWS**



**BJÖRN WILMSMANN, PHILIP RIECKS,
AND TOM HOMBERGS**

Stratospheric

From Zero to Production with Spring Boot and AWS

Tom Hombergs, Björn Wilmsmann, and Philip Riecks

This book is available at <http://leanpub.com/stratospheric>

This version was published on 2024-07-23



This is a [Leanpub](#) book. Leanpub empowers authors and publishers with the Lean Publishing process. [Lean Publishing](#) is the act of publishing an in-progress ebook using lightweight tools and many iterations to get reader feedback, pivot until you have the right book and build traction once you do.

© 2020 – 2024 Tom Hombergs, Björn Wilmsmann, and Philip Riecks

Also By These Authors

Books by [Tom Hombergs](#)

[Stratospheric \(Edición Española\)](#)

[Get Your Hands Dirty on Clean Architecture \(2nd edition\)](#)

Books by [Björn Wilmsmann](#)

[Stratospheric \(Edición Española\)](#)

Books by [Philip Riecks](#)

[Stratospheric \(Edición Española\)](#)

[Testing Spring Boot Applications Demystified](#)

[Testing Spring Boot Applications Masterclass Workbook](#)

[Java Testing Toolbox](#)

Contents

Introduction	1
Why Spring Boot & AWS?	3
Who Should Read This Book?	4
Prerequisites for the Hands-On Examples	5
What to Expect from This Book?	6
Getting in Touch	6
Resources	7
About the Authors	7

Part I: Deploying with AWS

1. Getting Warm with AWS	10
Getting Ready	10
Inspecting the “Hello World” Todo App	13
Publishing the “Hello World” App to Docker Hub	13
Getting Started with AWS Resources	16
Inspecting the CloudFormation Templates	18
Inspecting the Deployment Scripts	25
Inspecting the AWS Console	28
2. An Overview of AWS Services	32
AWS CloudFormation	33
AWS Cloud Development Kit (CDK)	33

CONTENTS

Amazon CloudWatch	34
Amazon Cognito	34
Amazon DynamoDB	34
Amazon Elastic Compute Cloud (EC2)	35
Amazon Elastic Container Registry (ECR)	35
Amazon Elastic Container Service (ECS)	36
Amazon MQ	36
Amazon Relational Database Service (RDS)	36
Amazon Route 53	37
Amazon Simple Email Service (SES)	37
Amazon Simple Queue Service (SQS)	37
Amazon Simple Storage Service (S3)	38
Amazon Virtual Private Cloud (VPC)	38
AWS Certificate Manager	39
AWS Identity and Access Management (IAM)	39
AWS Lambda	39
AWS Secrets Manager	40
AWS Systems Manager (SSM)	40
Elastic Load Balancing (ELB)	40
3. Managing Permissions with IAM	42
Users, Groups, and Roles	43
Root Users vs. Individual Users	45
Defining Policies	46
Creating AWS Access Keys for Each User	50
Managing IAM Resources Programmatically	51
Best Practices for Managing Permissions with IAM	52
4. The Evolution of Automated Deployments	53
An Anecdote about Manual Deployments	53

CONTENTS

Self-Service Deployments with the AWS Console	59
Scriptable Deployments with the AWS CLI	60
Declarative Deployments with CloudFormation	61
Programmable Deployments with CDK	62
5. First Steps with CDK	66
Creating Our First CDK App	66
Deploying a Spring Boot App with a CDK Construct	74
Why Not Stop Here?	80
6. Designing a Deployment Project with CDK	82
The Big Picture	82
Working with CDK	85
The Docker Repository CDK App	86
The Network CDK App	96
The Service CDK App	104
Playing with the CDK Apps	112
7. Building a Continuous Deployment Pipeline	113
GitHub Actions Concepts	114
Bootstrapping a New Environment	115
Deploying a Shared Network	117
Deploying an Application Environment	119
Building a Continuous Deployment Workflow	120
Supporting High-Frequency Deployments with Amazon SQS and AWS Lambda	126
Addendum: Configuring HTTPS and a Custom Domain with Route 53 and ELB	138
Domain Name System (DNS)	139
HTTPS and Transport Layer Security (TLS)	141

CONTENTS

Registering or Transferring a Domain	143
Creating an SSL Certificate with CDK	144
Creating an HTTPS Listener Using the Network App	149
Associating a Custom Domain For the ELB	152

Part II: Spring Boot & AWS156

8. The Sample Todo Application	158
Features	158
Application Architecture	160
Domain Model	161
Skeleton Application Setup	162
9. Local Development	174
The Challenges of Local Cloud Development	174
LocalStack – Our Local AWS Cloud	176
Local Amazon RDS & Amazon Cognito	179
Bringing It All Together	179
10. Building User Registration and Login with Amazon Cognito	181
What’s OAuth2?	181
OAuth 2.0 Terminology	183
OpenID Connect 1.0 (OIDC)	186
Alternatives to OAuth2 & OpenID Connect	187
Using Amazon Cognito for User Management	188
Using Amazon Cognito as an Identity Provider With Spring Security . .	201
Enabling Local Development	223
11. Connecting to a Database with Amazon RDS	227
Introduction to AWS Relational Database Service (RDS)	228
Setting up IAM Permissions	229

CONTENTS

Creating a Database CDK App	229
Strategies for Initializing the Database Structure	241
Configuring the Database in the Todo App	243
Using the Database for Storing and Retrieving Todos	245
Enabling Local Development	252
12. Sharing Todos with Amazon SQS and Amazon SES	254
Using Amazon SQS for Asynchronous Workloads	255
Sending Emails with Amazon SES	272
Enabling Local Development	283
13. Push Notifications with Amazon MQ	287
What Are Push Notifications Anyway?	287
Push Notifications for Live Updates	290
AWS Services for Implementing Push Notifications	291
Setting up a Message Broker with CDK	295
Implementing Push Notifications in the Todo App	304
Enabling Local Development	320
14. Tracing User Actions with Amazon DynamoDB	322
Use Case: Tracing User Actions	323
Amazon RDS vs. Amazon DynamoDB	324
Implementing User Tracing in the Todo App	333
Enabling Local Development	345
Part III: Production Readiness with AWS	347
15. Structured Logging with Amazon CloudWatch	348
Logging with AWS	349
CloudWatch Logging Terminology	350
Status Quo: Logging Unstructured Text	350

CONTENTS

Logging and Querying Structured Data	357
16. Metrics with Amazon CloudWatch	369
Introduction to Monitoring Metrics with Amazon CloudWatch	370
Sending Metrics from AWS Services	372
Sending Metrics from Our Spring Boot Application	379
Monitoring Metrics with Amazon CloudWatch	388
17. Alerting with Amazon CloudWatch	400
Introduction to Alerting with Amazon CloudWatch	401
Creating Alarms with the AWS CDK	404
Working and Living with Alarms and Incidents	416
18. Synthetic Monitoring with Amazon CloudWatch	421
Introduction to CloudWatch Synthetics	422
Recording a Canary Script for the Todo App	423
Keeping it Simple	428
Automating Canary Deployment with CDK	429
Alerting on Canary Failure	432
Closing Thoughts	434
Conquer the Cloud	435
Additional Resources	437
Appendix	440
Technical GitHub Actions IAM User	440
Deployment Guide	441
Changelog	448

Introduction

Have you browsed job descriptions for developer jobs lately? Don't worry, we're all doing it. Chances are the job descriptions require you to have vast experience in "cloud development" in addition to being an expert in at least one tech stack.

But what is "cloud development", anyway?

The term "cloud" has been hazy since its inception a decade or so ago. Some people upload their photos into the cloud. Some organizations use "cloud services" to manage their employees, customers, and vendors. Does that mean that when I'm ordering pizza online that I'm using "the cloud", too?

We believe the main idea of "the cloud" is the concept of self-service. Where you had to fill out forms and sign contracts before, you can now simply sign up for a service online and use it right away (provided that your credit card is funded). From the vast range of services "in the cloud", you can choose the ones you need and add them to your toolbox.

So, if you ask us: yes, ordering pizza online is ordering pizza "in the cloud".

The term "cloud development" takes the concept of self-service into the domain of software development. Instead of requesting new storage or computing resources by filing tickets for the IT department, we create them ourselves with our cloud provider. Instead of writing release notes to tell the operations team how to deploy the software we've written, we deploy it ourselves - using the services of a cloud provider. Instead of asking the operations team for logs of our misbehaving software in production, we browse them ourselves on demand.

The concept of self-service has disrupted conventional software development

processes considerably. Software teams can now use a range of cloud services not only to build software but also to bring it into production and operate it. Done right, this improves time-to-market and learning from user feedback through iterative releases – one of the pillars of agile software development.

Software teams now have great power over their software development process – and the equally great responsibility of operating software in production. The collection of practices and philosophies that emerged from this new paradigm is commonly referred to as “DevOps”.

The major players in tech understand that cloud development enables DevOps and self-organized teams and they have used it to great effect. Self-organized teams make a business far easier to scale because such teams have all the (cloud) resources they need at their fingertips. This creates a culture of accountability because each team owns their services in production. With this culture comes a feeling of “being in control” for the software team, which can significantly impact motivation.

This book takes a particular tech stack (Spring Boot) and a specific cloud provider (AWS) and follows a complete software development process to present tools and methods that support a DevOps culture.

In Part I, we’ll learn everything we need to know about deploying a Spring Boot application to AWS. In the first chapter, we’ll already deploy a Docker container and access it via the browser! Then, we’ll learn how to use AWS CloudFormation and the AWS Cloud Development Kit (CDK) to automate deployments and finally build a full-fledged continuous deployment pipeline with GitHub Actions.

In Part II, we’ll learn about several AWS services that we can use for common tasks. We’ll build a user registration and login without implementing it ourselves by leveraging the Amazon Cognito service. Then, we’ll connect our Spring Boot application with a relational database and a NoSQL database.

Furthermore, we'll send out emails and subscribe to messaging systems – all “self-serve” and fully managed by AWS.

Finally, in Part III, we'll dive into aspects important for running an application in production. We'll explore how to use Amazon CloudWatch to view logs and metrics. By actively monitoring our application and creating alerts, we'll increase the likelihood of detecting failures early. The book closes with a chapter on setting up HTTPS and a custom domain for our application.

With this, we hope that you will have as much fun reading this book as we had writing it!

Why Spring Boot & AWS?

We could have chosen any combination of tech stack and cloud provider for this book. We chose the combination that we were most intrigued with because we think it has a lot of potential (and, of course, because we have a bit of experience with it).

Spring Boot is the leading framework for building applications in the JVM ecosystem. It makes it easy to build production-ready software. Between the three of us, we have built countless applications with Spring Boot and have become deeply familiar with it.

AWS is the leading cloud platform. We can make use of a vast array of AWS cloud services to help us architect, build, and deploy a software application. Big players like Netflix and Atlassian have gone all-in with AWS, running all of their SaaS applications on AWS infrastructure. The days of deploying your software on a server in the basement of your company are over (or will soon be over, if your company is not quite there, yet).

As you can imagine, the combination of Spring Boot and AWS is quite powerful. At the time of writing, there is no comprehensive resource about integrating Spring Boot applications with AWS, so we wrote this book to fill that void.

After reading this book, you will know how to deploy a Spring Boot application to AWS and how to use many AWS services to make your life easier.

Who Should Read This Book?

This book is aimed at developers building software on the JVM with Spring Boot. You should probably have a bit of experience with Java and Spring Boot. We'll explain the Spring Boot features we're using, but not in very deep detail.

As for AWS, on the other hand, *this book requires no previous knowledge*. We'll start from scratch. If you have never heard of the AWS services we're using in this book, don't worry. We'll go into enough detail to get you up to speed.

When we're developing an application “in the cloud”, as we'll do in this book, we're automatically moving from plain “development” toward DevOps. We're still developing software (the “dev” in DevOps), but we're also concerned with operating it in the cloud (the “ops” in DevOps).

Many developers dislike operational concerns. That's often because they have been trained by years of conventional software development practices that there is an ops team to take the burden of running their application in production.

We didn't like to deal with operational concerns either. The feeling of “being in control” that comes with cloud development, however, awakened our curiosity for operational matters.

So, if you are a developer who is not comfortable with – or even dislikes –

operations, this book is for you, too. We're hoping that this book will spark some joy for cloud operations in you.

That said, this book is definitely for software engineers and not for system administrators or Site Reliability Engineers (SREs). They would probably be disappointed that we're not going into the nitty-gritty details of AWS infrastructure.

Prerequisites for the Hands-On Examples

This is a hands-on book with a lot of code examples and things to try out yourself. You can read the book from top to bottom without running the code examples yourself, but you'll have the best learning effect if you play around with the code and deploy it to AWS yourself. For this to work, you need to install some software.

To build and run the sample application on your machine, you need JDK 17 (or above). If you don't have one installed yet you can get it from the [Adoptium website](#) (formerly AdoptOpenJDK).

Also, you need to have Docker and Docker Compose running. We'll need it to spin up some Docker containers when testing and running the app, and when playing around with deploying to AWS. You can get Docker from the [Docker website](#).

In this book, when we're talking about the command line we assume a Unix command line. If you're working on a Windows machine make sure you have a Bash emulator like [GitBash](#) installed. Better yet, install the [Windows Subsystem for Linux \(WSL\)](#) (which should be called "Linux Subsystem for Windows" in our opinion).

What to Expect from This Book?

This book is not a guide to any of the AWS certifications. If you want to prepare for an AWS certification, you're probably better off finding a resource specific to that certification.

You will, however, learn a lot about different AWS services in this book. If you're coming from Spring Boot and want to expand into AWS, this book will be a better start into the topic than a book preparing you for a certification.

What this book offers is hands-on knowledge about getting a Spring Boot application into the cloud and operating it there. We'll build a continuous deployment pipeline, access the most common AWS services from a Spring Boot app, and learn how to monitor and nurture the app once it's live.

Expect hands-on code examples. Expect playing around with the code yourself – all code examples are part of a GitHub repository with a working Spring Boot application. Also, expect discussions of why we're doing things the way we do them in this book and when we should rather choose a different way.

Getting in Touch

We welcome any feedback on this book. Since it's self-published, we can update it at any time. You can help us make it better for everyone by letting us know when you find an error or anything that's unclear.

You can post any issues on the public issue tracker on [GitHub](#). You can reach all of us by sending an email to info@stratospheric.dev. Alternatively, you can reach out to each of us individually by following the links in the section about the authors below.

Resources

We provide some additional resources that complement the content in this book.

Feel free to [join our Slack community](#) to discuss the book, ask questions, and work through the book together.

[Browse our GitHub repos](#), which contain most of the code used in this book. Clone those repos and get everything up and running locally while reading the book.

[Subscribe to the newsletter](#) to be notified whenever we update the book or have news to share about the Spring Boot and AWS technology combo.

Enroll for the [complementary Stratospheric online course](#) for a more interactive and dynamic learning experience. This ebook establishes the baseline for the online course. In the online course we cover various topics in greater detail.

About the Authors

Tom Hombergs

Tom is a seasoned software engineer with a passion for building systems in the simplest way possible. He is regularly blogging about Java, Spring, and AWS and is the author of [Get Your Hands Dirty on Clean Architecture](#), giving hands-on advice on implementing a hexagonal architecture.

Find out more about Tom on [reflectoring.io](#) and his [Twitter profile](#).

Björn Wilmsmann

Björn Wilmsmann is an independent IT consultant who helps companies transform their business into a digital business. He designs and develops business solutions and enterprise applications for his clients. Björn provides hands-on training in technologies such as Angular and Spring Boot.

Find out more about Björn on bjoernkw.com and his [Twitter profile](#).

Philip Riecks

Under the slogan “Testing Java Applications Made Simple” Philip provides recipes and tips & tricks to accelerate your testing success and make testing joyful (or at least less painful). Apart from blogging, he’s a course instructor for various Java-related online courses and active on YouTube.

Find out more about Philip on rieckpil.de and his [Twitter profile](#).

Part I: Deploying with AWS

In the first part of this book, we'll look at how to get an application into the cloud.

We'll start by diving right in by deploying a “Hello World” app to the cloud with AWS CloudFormation. Follow the steps to deploy the app yourself for a first feeling of success!

After this hands-on section, we'll cover some AWS basics. We'll give an introduction to the AWS services used throughout the book and explain how AWS does IAM (Identity and Access Management).

We strongly believe that automation is a key to successful software development. So, the rest of this part is dedicated to diving into what the AWS CDK (Cloud Development Kit) has to offer to automate the hell out of the deployment process.

After this part, we'll have built a fully automated deployment pipeline with CDK and GitHub Actions.

For now, we don't care much about which application we're deploying. Any web application in a Docker image would do. For demo purposes, we'll use a “Hello World” version of the Todo application we're going to build in Part II.

1. Getting Warm with AWS

Before we start building any features for our Todo app, we want to get comfortable with AWS. We'll start building out the features in Part II of the book.

The first thing we'll do is to deploy a "Hello World" version of the Todo app to get a quick boost of dopamine that'll keep us going!

Don't expect this chapter to dive very deep into each topic. It's superficial by design so that you get a feel for AWS even if you don't know anything about it. We'll dive deep into topics in the later chapters of this book.

If you already know how to deploy a Docker image to AWS Fargate with CloudFormation, you may want to skip this chapter.

Getting Ready

If you've never deployed an app to the cloud before, you're in for a treat. We're going to deploy a first version of our Todo app to AWS with only a couple of CLI commands (it requires some preparation to get these CLI commands working, though).

We're going to use Docker to make our app runnable in a container, AWS CloudFormation to describe the infrastructure components we need, and the AWS CLI to deploy that infrastructure and our app.

The goal of this chapter is not to become an expert in all things AWS, but instead to learn a bit about the AWS CLI and CloudFormation because the coming

chapters will build on them. And there's no better way to learn than to get your hands dirty.

We'll start at zero and set up our AWS account first.

Setting up an AWS Account

To do anything with AWS, you need an account with them. If you don't have an account yet, go ahead and [create one now](#).

If you already have an account running serious applications, you might want to create an extra account just to make sure you're not messing around with your serious business while playing around with this book.

Installing the AWS CLI

To do magic with AWS from our command line, we need to install the AWS CLI.

The AWS CLI is a beast of a command-line interface that provides commands for many and many different AWS services (224 at the time of this writing). In this chapter, we're going to use it to deploy the application and then to get some information about the deployed application.

Installing the AWS CLI differs across operating systems, so please follow the [official instructions](#) for your operating system to install version 2 of the AWS CLI on your machine.

Once it's installed, run `aws configure`. You will be asked to provide 4 parameters:

```
~ aws configure
AWS Access Key ID [*****Kweu]:
AWS Secret Access Key [*****CmqH]:
Default region name [ap-southeast-2]:
Default output format [yaml]:
```

You can get the “AWS Access Key ID” and “AWS Secret Access Key” after you have [logged into your AWS account](#) when you click on your account name and then “My Security Credentials”. There, you open the tab “Access keys” and click on “Create New Access Key”. Copy the values into the prompt of the AWS CLI.

The AWS CLI is authorized to make calls to the AWS APIs in your name, now.

Next, the `aws configure` command will ask you for a “Default region name”.

The AWS services are distributed across “regions” and “availability zones”. Each geographical region is fairly isolated from the other regions for reasons of data residency and low latency. Each region has 2 or more availability zones to make the services resilient against outages.

Each time we interact with an AWS service, it will be with the service’s instance in a specific region. So, choose the region nearest to your location from [the list of service endpoints provided by AWS](#) and enter the region code into the `aws configure` prompt (for example “us-east-1”).

Finally, the `aws configure` command will prompt you for the “Default output format”. This setting defines the way the AWS CLI will format any output it presents to you.

You can choose between two evils: JSON or YAML. We’re not going to judge you on your choice.

We’re done configuring the AWS CLI now. Run the following command to test it:

```
aws ec2 describe-regions
```

This command lists all the AWS regions in which we can make use of EC2 instances. EC2 stands for “Elastic Cloud Compute” which is the AWS service providing virtual machines that we can deploy our applications into. If the command prints out a list of regions, you’re good to go.

Inspecting the “Hello World” Todo App

Let’s take a quick peek at the Todo app we’re going to deploy to AWS.

You’ll find the source code for the app in the [Stratospheric GitHub repository](#). Feel free to clone it or to inspect it on GitHub.

At this point, the app is no more than a stateless “Hello World” Spring Boot app. We’ll build actual features into this app later in Part II of the book.

The app has a single controller named `IndexController` that shows nothing more than the message “Welcome to the Todo Application!”. Feel free to start the application via this command:

```
./gradlew bootrun
```

Then, navigate to `http://localhost:8080` to see the message.

To deploy the app to AWS, we need to publish it as a Docker image next.

Publishing the “Hello World” App to Docker Hub

If you know how to package a Spring Boot app in a Docker image, you can safely skip this section. We have published the app on Docker Hub already, so you can use that Docker image in the upcoming steps.

If you're interested in the steps to create and publish a basic Docker image, stay tuned.

First, we need a `Dockerfile`. The repository already contains a `Dockerfile` with this content:

```
FROM eclipse-temurin:17-jre

ARG JAR_FILE=build/libs/*.jar
COPY ${JAR_FILE} app.jar

ENTRYPOINT ["java", "-jar", "/app.jar"]
```

This file instructs Docker to create an image based on a basic `eclipse-temurin` image, which bundles OpenJDK 17 with a Linux distribution.



The Eclipse Adoptium project is the successor of AdoptOpenJDK and the `temurin` runtimes are highly-quality Java runtimes delivered by the Eclipse foundation. The image itself has nothing to do with the Eclipse IDE. Read more about the AdoptOpenJDK transition to the Eclipse Foundation in their [official announcement](#).

Starting with version 2.3.0, Spring Boot supports more sophisticated ways of creating Docker images, including cloud-native Buildpacks. We're not going to dive into that, but if you're interested, [this blog post](#) gives an introduction on what you can do in that respect.

We create the argument `JAR_FILE` and tell Docker to copy the file specified by that argument into the file `app.jar` within the container.

Then, Docker will start the app by calling `java -jar /app.jar`.

Before we can build a Docker image, we need to build the app with

```
./gradlew build
```

This will create the file `/build/libs/todo-application-0.0.1-SNAPSHOT.jar`, which will be caught by the `JAR_FILE` argument in the Docker file.

To create a Docker image we can now call this command:

```
docker build -t stratospheric/todo-app-v1:latest .
```

Docker will now build an image in the namespace `stratospheric` and the name `todo-app-v1` and tag it with the tag `latest`. If you do this yourself, make sure to use your Docker Hub username as the namespace because you won't be able to publish a Docker image into the `stratospheric` namespace.

A call to `docker image ls` should list the Docker image now:

```
~ docker image ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
stratospheric/todo-app-v1	latest	5d3ef7cda994	3 days ago	647MB

To deploy this Docker image to AWS, we need to make it available to AWS somehow. One way to do that is to publish it to Docker Hub, which is the official registry for Docker images (later in this book, we'll use Amazon's ECR service to deploy Docker images). To do this, we call `docker login` and `docker push`:

```
docker login
docker push stratospheric/todo-app-v1:latest
```

The login command will ask for your credentials, so you need to have an account at hub.docker.com. The push command will upload the image to the Docker Hub so that anyone can pull it from there with this command:


```
docker pull stratospheric/todo-app-v1:latest
```

Great! The app is packaged in a Docker image and the image is published. Time to talk about deploying it to AWS.

Getting Started with AWS Resources

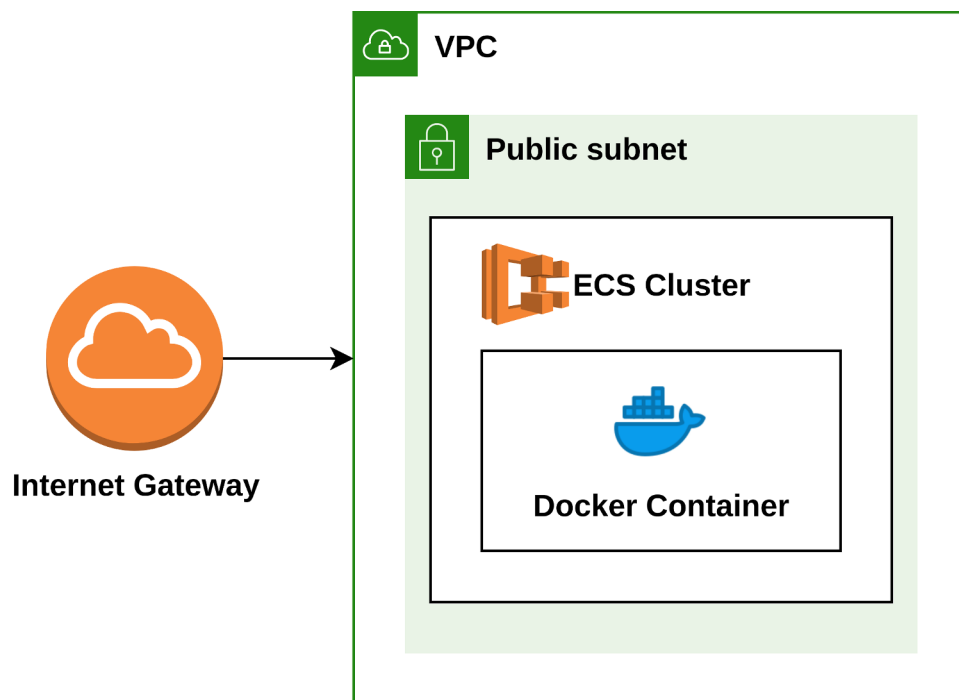
As mentioned above, we'll be using AWS CloudFormation to deploy some infrastructure and finally our Docker image to the cloud.

In a nutshell, CloudFormation takes a YAML or JSON file as input and provisions all the resources listed in that file to the cloud. This way, we can spin up a whole network with load balancers, application clusters, queues, databases, and whatever else we might need.

Pretty much every AWS service provides some resources that we can provision with CloudFormation. Almost everything that you can do via the AWS web interface (called the AWS Console), you can also do with CloudFormation. The docs provide a [list of the available CloudFormation resources](#).

The advantage of this is clear: With CloudFormation, we can automate what we would otherwise have to do manually.

Let's have a look at what we're going to deploy in this chapter:



We're deploying an ECS cluster within a public subnet in a virtual private cloud.

For deploying our Todo app, we're starting with just a few resources so we don't get overwhelmed. We're deploying the following resources:

A **Virtual Private Cloud (VPC)** is the basis for many other resources we deploy. It spins up a virtual network that is accessible only to us and our resources.

A VPC contains **public and private subnets**. A public subnet is reachable from the internet, a private subnet is not. In our case, we deploy a single public subnet only. For production deployments, we'd usually deploy at least two subnets, each in a different availability zone (AZ) for higher availability.

To make a subnet public, we need an **internet gateway**. An internet gateway allows outbound traffic from the resources in a public subnet to the internet. It also performs network address translation (NAT) to route inbound traffic from the internet to the resources in a public subnet.

Not being attached to an internet gateway makes a subnet a private subnet.

Into our public subnet, we deploy an **ECS cluster**. ECS (Elastic Container Service) is an AWS service that automates much of the work to deploy Docker images.

Within an ECS cluster, we can define one or more different services that we want to run. For each service, we can define a so-called **task**. A task is backed by a Docker image. We can decide how many instances of each task we want to run and ECS takes care of keeping that many instances alive at all times.

If the health check of one of our application instances (i.e. task instances) fails, ECS will automatically kill that instance and restart a new one. If we want to deploy a new version of the Docker image, we give ECS the URL to the new Docker image and it will automatically do a rolling deployment, keeping at least one instance alive at all times until all old instances have been replaced with new ones.

Let's get our hands dirty and have a look at the files that describe this infrastructure!

Inspecting the CloudFormation Templates

You can find the CloudFormation templates in the [cloudformation folder](#) on GitHub.

In that folder, we have two YAML files - `network.yml` and `service.yml` - as well as two shell scripts - `create.sh` and `delete.sh`.

The YAML files are the CloudFormation templates that describe the resources we want to deploy. The shell scripts wrap some calls to the AWS CLI to create (i.e. deploy) and delete (i.e. destroy) the resources described in those files. `network.yml` describes the basic network infrastructure we need, and `service.yml` describes the application we want to run in that network.

Before we look at the CloudFormation files, we need to discuss the concept of “stacks”.

A **stack** is CloudFormation’s unit of work. We cannot create single resources with CloudFormation unless they are wrapped in a stack.

A YAML file (or JSON file, if you enjoy chasing closing brackets more than chasing spaces) always describes the resources of a stack. Using the AWS CLI, we can interact with this stack by creating it, deleting it, or modifying it.

CloudFormation will automatically resolve dependencies between the resources defined in a stack. If we define a subnet and a VPC, for example, CloudFormation will create the VPC before the subnet, because a subnet always refers to a specific VPC. When deleting a stack, it will automatically delete the subnet before deleting the VPC.

The Network Stack

With the CloudFormation basics in mind, let’s have a look at the first couple of lines of the network stack defined in `network.yml`:

AWSTemplateFormatVersion: '2010-09-09'

Description: A basic network stack that creates a VPC with a single public subnet and some ECS resources that we need to start a Docker container within this subnet.

Resources:

...

A stack file always refers to a version of the CloudFormation template syntax. The last version is from 2010. It’s hard to believe that it hasn’t changed for more than 10 years, but the syntax is rather simple, as we’ll see shortly, so it makes sense that it’s stable.

Next is a description of the stack and then a big section with the key Resources that describes the resources we want to deploy in this stack.

In the network stack, we want to deploy the basic resources we need to deploy our Todo application onto. That means we want to deploy a VPC with a public subnet, an internet gateway to make that subnet accessible from the internet, and an ECS cluster that we can later put our Docker image into.

The first resource we define within the Resources block is the VPC:

```
VPC:
  Type: AWS::EC2::VPC
  Properties:
    CidrBlock: '10.0.0.0/16'
```

The key VPC we can choose as we see fit. We can reference the resource by this name later in the template.

A resource always has a Type. There are a [host of different resource types](#) available since almost every AWS service allows us to create resources via CloudFormation. In our case, we want to deploy a VPC – a virtual private cloud in which we put all the other resources.

Next, a resource may require some Properties to work. Most resources do require properties. To find out which properties are available, have a look at the reference documentation of the resource you want to work with. The easiest way to get there is by googling “cloudformation <resource name>”. The documentation is not always clear about which properties are required and which are optional, so it may require some trial and error when working with a new resource.

In the case of our VPC, we only define the property CidrBlock that defines the range of IP addresses available to any resources within the VPC that need an IP address. The value 10.0.0.0/16 means that we’re creating a network with an IP

address range from 10.0.0.0 through 10.0.255.255 (the 16 leading bits 10.0 are fixed, the rest is free to use).

We could deploy the CloudFormation stack with only this single resource, but we need some more infrastructure for deploying our application. Here's a list of all the resources we deploy with a short description for each. You can look them up in the `network.yml` file to see their configuration:

- **PublicSubnet:** A public subnet in one of the availability zones of the region we're deploying into. We make this subnet public by setting `MapPublicIpOnLaunch` to true and attaching it to an internet gateway.
- **InternetGateway:** An internet gateway to allow inbound traffic from the internet to resources in our public subnet and outbound traffic from the subnet to the internet.
- **GatewayAttachment:** This resource of type `VpcGatewayAttachment` attaches our subnet to the internet gateway, making it effectively public.
- **PublicRouteTable:** A `RouteTable` to define routes between the internet gateway and the public subnet.
- **PublicSubnetRouteTableAssociation:** Some boilerplate to link the route table with our public subnet.
- **PublicRoute:** The actual route telling AWS that we want to allow traffic from our internet gateway to any IP address within our public subnet.
- **ECSCluster:** A container for running ECS tasks. We'll deploy an ECS task with our Docker image later in the service stack (`service.yml`).
- **ECSSecurityGroup:** A security group that we can later use to allow traffic to the ECS tasks (i.e. to our Docker container). We'll refer to this security group later in the service stack (`service.yml`).
- **ECSSecurityGroupIngressFromAnywhere:** A security group rule that allows traffic from anywhere to any resources attached to our `ECSSecurityGroup`.

- **ECSRole:** A role that attaches some permissions to the `ecs-service` principal. We're giving the ECS service some permissions to modify networking stuff for us.
- **ECSTaskExecutionRole:** A role that attaches some permissions to the `ecs-tasks` principal. This role will give our ECS tasks permissions to write log events, for example.

That's quite some resources we need to know about and configure. Creating CloudFormation templates quickly becomes a trial-and-error marathon until you get it configured just right for your use case. Later in the book, we'll have a look at the AWS Cloud Development Kit (CDK) which takes some of that work from our shoulders.

In case you wondered about the special syntax used in some places of the YAML file, let's quickly run through it:

- **Fn::Select / !Select:** Allows us to select one element from a list of elements. We use it to select the first availability zone of the region we're working in.
- **Fn::GetAZs / !GetAZs:** Gives us a list of all availability zones in a region.
- **Fn::Ref / !Ref:** Allows us to reference another resource by the name we've given to it.
- **Fn::Join / !Join:** Joins a list of strings to a single string, with a given delimiter between each.
- **Fn::GetAtt / !GetAtt:** Resolves an attribute of a resource we've defined.

All functions have a long form (`Fn::...`) and a short form (`!...`) which behave the same but look a bit different in YAML. In a nutshell, we can use the short form for single-line expressions and the long form for longer expressions that we might want to split over several lines.

Finally, at the bottom of `network.yml`, we see an `Outputs` section:

```
Outputs:  
  ClusterName:  
    Description: The name of the ECS cluster  
    Value: !Ref 'ECSCluster'  
    Export:  
      Name: !Join [ ':', [ !Ref 'AWS::StackName', 'ClusterName' ] ]  
  ... (more outputs)
```

Each output describes a parameter that we want to export from the stack to be used in other stacks.

For example, we export the name of the ECS Cluster under the name `<NETWORK_STACK_NAME>:ClusterName`. In other stacks, like our service stack, we now only need to know the name of the network stack to access all of its output parameters.

Let's have a look at the service stack now to see how we deploy our application.

The Service Stack

The service stack is defined in `service.yml`. We call it “service stack” because it describes an ECS task and an ECS service that spins up Docker containers and does some magic to make them available via the internet.

Different from the network stack, the service stack starts with a `Parameters` section:

AWSTemplateFormatVersion: '2010-09-09'

Description: Deploys a Docker container within a previously created VPC.
Requires a running network stack.

Parameters:

NetworkStackName:

Type: String

Description: The name of the networking stack that these resources are put into.

ServiceName:

Type: String

Description: A human-readable name for the service.

ImageUrl:

Type: String

Description: The url of a docker image that will handle incoming traffic.

ContainerPort:

Type: Number

Default: 80

Description: The port number the application inside the docker container is binding to.

ContainerCpu:

Type: Number

Default: 256

Description: How much CPU to give the container. 1024 is 1 CPU.

ContainerMemory:

Type: Number

Default: 512

Description: How much memory in megabytes to give the container.

DesiredCount:

Type: Number

Default: 1

Description: How many copies of the service task to run.

...

Within the Parameters section, we can define input parameters to a stack. We're passing the name of an existing network stack, for example, so that we can refer to its output parameters. Also, we pass in a URL pointing to the Docker image we want to deploy and some other information that we might want to change from one deployment to another.

The service stack deploys merely three resources:

- **LogGroup:** A container for the logs of our application.
- **TaskDefinition:** The definition for an ECS task. The task will pull one or more Docker images from URLs and run them.
- **Service:** An ECS service that provides some logic around a task definition, like how many instances should run in parallel and if they should be assigned public IP addresses.

In several instances, you'll see references to the network stack's outputs like this one:

```
Fn::ImportValue:  
  !Join [':', [!Ref 'NetworkStackName', 'ClusterName']]
```

`Fn::ImportValue` imports an output value exported by another stack. Since we have included the network stack name in the name of its outputs, we need to join the network stack name with the output parameter name to get the right value.

So, we've looked at over 200 lines of YAML configuration describing the infrastructure we want to deploy. Later, we'll see how to use CDK to achieve this in Java instead of YAML, making it more reusable and easier to handle in general.

Inspecting the Deployment Scripts

Let's deploy our app to the cloud! We'll need the scripts `create.sh` and `delete.sh` from the `cloudformation` folder in the [GitHub repository](#).

Go ahead and run the `create.sh` script now, if you want. While you're waiting for the script to finish (it can take a couple of minutes), we'll have a look at the script itself.

The script starts with calling `aws cloudformation create-stack` to create the network stack:

```
aws cloudformation create-stack \  
  --stack-name stratospheric-ecs-basic-network \  
  --template-body file://network.yml \  
  --capabilities CAPABILITY_IAM  
  
aws cloudformation wait stack-create-complete \  
  --stack-name stratospheric-ecs-basic-network
```

We're passing the name for the stack, the path to our `network.yml` stack template, and the capability `CAPABILITY_IAM` to allow the stack to make changes to IAM (Identity and Access Management) roles.

Since the `create-stack` command executes asynchronously, we call `aws cloudformation wait stack-create-complete` afterward to wait until the stack is up and running.

Next, we're doing the same for the service stack:

```
aws cloudformation create-stack \  
  --stack-name stratospheric-ecs-basic-service \  
  --template-body file://service.yml \  
  --parameters \  
    ParameterKey=NetworkStackName,ParameterValue=stratospheric-ecs-basic-network \  
    ParameterKey=ServiceName,ParameterValue=todo-app-v1 \  
    ParameterKey=ImageUrl,ParameterValue=docker.io/stratospheric/todo-app-v1:latest \  
    ParameterKey=ContainerPort,ParameterValue=8080  
  
aws cloudformation wait stack-create-complete \  
  --stack-name stratospheric-ecs-basic-service
```

With `--parameters`, we're passing in all the parameters that we want to be different from the defaults. Specifically, we're passing `docker.io/stratospheric/todo-app-v1:latest` into the `ImageUrl` parameter to tell AWS to download our Docker image and run it.

After both stacks are up and running, we're using some AWS command-line magic to extract the public IP address of the running application:

```
CLUSTER_NAME=$(
  aws cloudformation describe-stacks \
    --stack-name stratospheric-ecs-basic-network \
    --output text \
    --query 'Stacks[0].Outputs[?OutputKey=='ClusterName'].OutputValue | [0]'
)
echo "ECS Cluster:      " $CLUSTER_NAME

TASK_ARN=$(
  aws ecs list-tasks \
    --cluster $CLUSTER_NAME \
    --output text --query 'taskArns[0]'
)
echo "ECS Task:        " $TASK_ARN

ENI_ID=$(
  aws ecs describe-tasks \
    --cluster $CLUSTER_NAME \
    --tasks $TASK_ARN \
    --output text \
    --query 'tasks[0].attachments[0].details[?name=='networkInterfaceId'].value'
)
echo "Network Interface: " $ENI_ID

PUBLIC_IP=$(
  aws ec2 describe-network-interfaces \
    --network-interface-ids $ENI_ID \
    --output text \
    --query 'NetworkInterfaces[0].Association.PublicIp'
)
echo "Public IP:        " $PUBLIC_IP

echo "You can access your service at http://$PUBLIC_IP:8080"
```

We're using different AWS commands to get the information we want. First, we output the network stack and extract the name of the ECS cluster. With the cluster name, we get the ARN (Amazon Resource Name) of the ECS task. With

the task ARN, we get the ID of the network interface of that task. And with the network interface ID, we finally get the public IP address of the application so we know where to go.

All commands use the AWS CLI to output the results as text and we extract certain information from that text with the `--query` parameter.

The output of the script should look something like that:

```
StackId: arn:aws:cloudformation:.../stratospheric-ecs-basic-network/...
StackId: arn:aws:cloudformation:.../stratospheric-ecs-basic-service/...
ECS Cluster:      stratospheric-ecs-basic-network-ECSCluster-qqX6Swdw54PP
ECS Task:         arn:aws:ecs:.../stratospheric-ecs-basic-network-...
Network Interface: eni-02c096ce1faa5ecb9
Public IP:        13.55.30.162
You can access your service at http://13.55.30.162:8080
```

Go ahead and copy the URL at the end into your browser and you should see the text “Welcome to the Todo application” on your screen.

Hooray! We’ve just deployed an app and all the infrastructure it needs to the cloud with a single CLI command! We’re going to leverage that later to create a fully automated continuous deployment pipeline.

But first, let’s inspect the infrastructure and application we’ve deployed.

Inspecting the AWS Console

The AWS console is the cockpit for all things AWS. With our browser, we can view the status of all the resources we’re using, interact with them, and provision new resources.

We could have done everything we’ve encoded into the CloudFormation templates above by hand using the AWS console. But manually setting up infras-

structure is error-prone and not repeatable, so we're not going to look at how to do that.

However, the AWS console is a good place to view the resources we've deployed, to check their status, and to kick off debugging if we need it.

Go ahead and log in to the [AWS console](#) and let's take a quick tour!

After logging in, type "CloudFormation" into the "Find Services" box and select the CloudFormation service.

You should see a list of your CloudFormation stacks with a status for each. The list should contain at least the stacks `stratospheric-ecs-basic-service` and `stratospheric-ecs-basic-network` in status `CREATE_COMPLETE`. Click on the network stack.

In the detail view of a stack, we get a host of information about the stack. Click on the "Events" tab first.

Here, we see a list of events for this stack. Each event is a status change of one of the stack's resources. We can see the history of events: In the beginning, a bunch of resources were in status `CREATE_IN_PROGRESS` and transitioned into status `CREATE_COMPLETE` a couple of seconds later. Then, when the resources they depend on are ready, other resources started their life in the same way. And so on. CloudFormation takes care of the dependencies between resources and creates and deletes them in the correct sequence.

The "Events" tab is the place to go when the creation of a stack fails for some reason. It will show which resource failed and will (usually) show an error message that helps us to debug the problem.

Let's move on to the "Resources" tab. It shows us a list of the network stack's resources. The list shows all the resources we've included in the `network.yml` CloudFormation template:

For some resources, we get a link to the resource in the “Physical ID” column. Let’s click on the ID of the `ECSCluster` resource to take a look at our application.

The link has brought us to the console of the ECS service. We can also get here by opening the “Services” dropdown at the top of the page and typing “ECS” into the search box.

The detail view of our ECS cluster shows that we have 1 service and 1 task running in this cluster. If we click on the “Tasks” tab, we see a list of running tasks, which should contain one entry only. Let’s click on the link in the “Task” column to get a detail view of the task.

The detail view shows a lot of information we’re not interested in, but it also shows the public IP address of the task. This is the IP address that we extracted via AWS CLI commands earlier. You can copy it into your browser, append the port 8080, and you should see the hello message again.

Below the general information is a section called “Containers”, which shows the container we’ve deployed with this task. Click on the little arrow on the left to expand it. In the “Log Configuration” section, click on the link “View logs in CloudWatch”.

CloudWatch is Amazon’s service for monitoring applications. In our service stack, we added a “LogGroup” resource and used the name of that log group in the logging configuration of the container definition. This is the reason why we can now see the logs of that app in CloudWatch.

After the “Events” tab in the CloudFormation UI, the logs are the second place to look at when (not if) something goes wrong.

This concludes our first experiment with AWS. Feel free to explore the AWS console a bit more to get a feel for how everything works. We’ll go into more detail about different services in the rest of this book.

When you're done, don't forget to run `delete.sh` to delete the stacks again, otherwise they will incur costs at some point. You can also delete the stacks via the CloudFormation UI.

2. An Overview of AWS Services

Throughout this book, we'll use a range of different AWS services. Depending on how you count, at the time of writing, AWS provides something between 150 and 300 different services for computing, file storage, networking and database access and management, and a host of other use cases. That number seems to be increasing every day.

The services we chose to include in this book and the sample application we're going to develop can only represent a small section of what AWS has to offer. For some areas, there are sometimes competing or seemingly overlapping services, too. At times, these might only differ in terms of specific requirements or even just nuances.

Our selection of services, therefore, has to be an opinionated choice. We chose those AWS services that met the requirements of our sample application. In case there was a choice between two competing services, we chose the more common one to provide more value for the reader.

Somebody else might arrive at a different selection of services, though, and that's perfectly fine. Where appropriate we'll be discussing alternatives to the services presented.

We believe that the AWS services we're going to cover in this book are suitable for addressing a wide range of common web application use cases. This will hopefully allow you to put the content of this book to good use in the wider context of web application development.

Let's have a very quick look at each of the AWS services we're going to talk about

in this book to give some context for the upcoming chapters.

AWS CloudFormation

With CloudFormation, we can describe all the infrastructure resources we need in a JSON or YAML file and CloudFormation will provision those resources for us. CloudFormation is the basis for automating the deployment of applications to AWS.

We have already used CloudFormation in the chapter [Getting Warm with AWS](#) to deploy a “Hello World” app.

Other than to get warm with AWS, we won’t be using CloudFormation directly anymore in this book. Instead, we’ll use CDK (Cloud Development Kit) to describe and deploy our infrastructure. Since CDK builds on top of the CloudFormation resources, we’ll learn a bit about CloudFormation anyway.

AWS Cloud Development Kit (CDK)

CDK builds on top of CloudFormation and allows us to describe the CloudFormation resources that we want to deploy in a programming language like Java or TypeScript. This way, we have a real “infrastructure as code” solution and don’t have to handle CloudFormation files in YAML or JSON anymore.

We’ll introduce CDK in the chapter [First Steps with CDK](#), learn more about it in the chapter [Designing a Deployment Project with CDK](#), and add more and more resources to our CDK project over the rest of this book. The goal is to deploy our sample application and the infrastructure it needs with only a few commands.

Amazon CloudWatch

Amazon CloudWatch Amazon's main observability service. With a log server, metrics server, dashboards, and alarms, it provides a wide range of observability features.

We will learn a lot about CloudWatch in Part III of the book, specifically in the chapters *Structured Logging with Amazon CloudWatch*, *Metrics with Amazon CloudWatch*, *Alerting with Amazon CloudWatch*, and *Synthetic Monitoring with Amazon CloudWatch*.

Amazon Cognito

Amazon Cognito provides user management capabilities. We can build user authentication and authorization features against it. Among other features, it provides interfaces for building social logins and custom logins via OAuth.

We will look into Cognito in more detail in the chapter *Building User Registration and Login with Cognito* where we'll be adding user registration and login features to our sample application.

Amazon DynamoDB

DynamoDB is Amazon's NoSQL solution promising "performance at any scale". It can be used as a key-value store or as a document store for billions of datasets if need be.

In the chapter *Tracing User Actions with DynamoDB*, we'll be using DynamoDB for keeping relatively unstructured, free-form documents in addition to our

relational data model.

Amazon Elastic Compute Cloud (EC2)

One of the oldest, originally available AWS services, EC2 provides virtual machines which we can use for any computation tasks that we might have. The term “elastic” comes from the fact that we can get as many compute instances as we want and that we can (almost) make them as big or small as we want them to be.

We’re not going to use EC2 directly in this book, but since EC2 is the underlying service for many other AWS services, it will come up now and then. We will instead use the higher-level ECS service to manage EC2 instances for us (see next section).

Amazon Elastic Container Registry (ECR)

ECR is Amazon’s managed Docker container registry. It provides functionality for storing and managing Docker images and it integrates with ECS.

In the chapter *Designing a Deployment Project with CDK*, we will learn how to create an ECR repository and how to publish Docker images of our sample application into this repository. We will then configure ECS to take one of the published images and deploy it into EC2 instances for us.

Amazon Elastic Container Service (ECS)

ECS is Amazon's flavor of a container orchestration service. On a very high level, it takes Docker images as input and deploys them on EC2 instances for us. It then manages this fleet of EC2 instances for us by scaling in and out, killing unhealthy instances, and starting fresh ones.

We will package our application into a Docker image and then let ECS manage the deployment of this image and the state of the resulting Docker containers. We already got a first glimpse at ECS in the chapter [Getting Warm with AWS](#). We'll go into more detail in the chapter [Designing a Deployment Project with CDK](#).

Amazon MQ

Amazon MQ is a service that manages Apache ActiveMQ and RabbitMQ message brokers for us.

We'll make use of Amazon MQ in the chapter [Push Notifications with Amazon MQ](#) but will also discuss some alternative AWS services like Amazon SNS, AWS IoT, or an AWS Lambda function.

We'll be using an ActiveMQ message broker as a relay for WebSocket connections because this will allow us to support publish-subscribe messaging in load-balanced environments.

Amazon Relational Database Service (RDS)

RDS is Amazon's managed database service. It takes care of managing, running, and updating our database instances for us.

We will introduce RDS with a PostgreSQL database for storing and retrieving data from our sample application in the chapter [Connecting to a Database with RDS](#).

Amazon Route 53

Route 53 is a cloud Domain Name System (DNS) service.

We'll use Route 53 for routing end users to our application via a user-friendly domain name in the chapter [Configuring HTTPS and a Custom Domain with Route 53 and ELB](#).

Amazon Simple Email Service (SES)

SES is what the name suggests: a simple email service. We can use it to, well, send emails.

We'll introduce SES in conjunction with SQS (see below) in the chapter [Sharing Todos with SQS and SES](#) to send emails when a user wants to share a todo with another user.

Amazon Simple Queue Service (SQS)

SQS is Amazon's bread-and-butter queuing service. It provides an API for sending and receiving messages at great scale and provides a lot of knobs and dials for robustly handling messages. It's the first choice if we want to decouple aspects of our application asynchronously.

We'll use SQS to queue deployments in the chapter [Building a Continuous Deployment Pipeline](#) and to queue email notifications in the chapter [Sharing Todos with SQS and SES](#).

Amazon Simple Storage Service (S3)

S3 is another one of Amazon's most commonly used services. S3 provides file and object storage at scale.

We will not use S3 directly in our sample application, but we will touch on it in the chapter [Synthetic Monitoring with Amazon CloudWatch](#), where it is used for uploading screenshots.

Amazon Virtual Private Cloud (VPC)

A VPC is a virtual network that can hold other AWS resources. With a VPC, we can define network boundaries between the public internet and our own infrastructure. We can even set boundaries within our infrastructure.

We make use of VPC for isolating our resources in their own private IP network. Within that network, servers can communicate with each other and access resources while the application will only be publicly available through a single entry point. This not only mitigates potential security issues but also vastly simplifies accessing resources.

We already learned a bit about VPC in the chapter [Getting Warm with AWS](#) and we'll go into more detail in the chapter [Designing a Deployment Project with CDK](#).

AWS Certificate Manager

Certificate Manager allows us to manage public and private Secure Sockets Layer/Transport Layer Security (SSL/TLS) certificates for use with AWS services and resources.

In the chapter [Configuring HTTPS and a Custom Domain with Route 53 and ELB](#), we'll provision and deploy an SSL certificate for the domain name we'll be using for our application through the AWS Certificate Manager.

AWS Identity and Access Management (IAM)

IAM provides mechanisms to manage users and user groups and permissions for those users and groups. It is intertwined with all other AWS services. We can use IAM to grant and deny access to resources a service provides.

We will introduce IAM in the chapter [Managing Permissions with IAM](#) and expand on it in other chapters throughout the book whenever we're creating resources that require authorization.

AWS Lambda

Lambdas are Amazon's solution to serverless computation. Instead of deploying a whole application that occupies a (virtual) server in EC2, we can deploy a single function that is executed on demand. Instead of paying a server per hour, we pay per invocation of the Lambda function.

We'll use AWS Lambda (and Amazon SQS) to queue deployments in the chapter [Building a Continuous Deployment Pipeline](#).

AWS Secrets Manager

AWS Secrets Manager is a service focused on secure storage and retrieval of secrets. It also provides features to improve security by rotating secrets.

During the deployment of CloudFormation stacks, we'll make use of the Secrets Manager to create and retrieve database connection secrets in the chapter [Connecting to a Database with RDS](#). This way, the database password will never leave the AWS servers.

AWS Systems Manager (SSM)

AWS Systems Manager is a service that helps with operating a large number of applications in the AWS cloud. In addition to some other aspects, it provides configuration management features, compliance features, and an inventory.

We'll introduce the "parameter store" feature of SSM in the chapter [Designing a Deployment Project with CDK](#). The parameter store allows us to store and retrieve configuration parameters. We'll use this to store some parameters with each CloudFormation stack we deploy. When we deploy another stack that depends on the first, we load the parameters again from the store, so that we don't have to manually pass these parameters around.

Elastic Load Balancing (ELB)

Elastic Load Balancing is a load balancing service that allows us to route and distribute traffic according to a set of rules.

Instead of directly exposing our application services to our users, in [Designing](#)

a Deployment Project with CDK we'll make use of an Application Load Balancer (ALB) provided through ELB for handling the load our application has to deal with.

In the chapter *Configuring HTTPS and a Custom Domain with Route 53 and ELB*, we'll also use a custom rule for redirecting from HTTP to HTTPS.

3. Managing Permissions with IAM

When deploying applications to a cloud service such as AWS, reliable security concepts are key. After all, we not only want to protect our users' data but also make sure that security within our organization isn't compromised.

With AWS Identity and Access Management (IAM), we're able to address these concerns for our cloud applications and in the larger context of security and access management within an organization.

Using AWS services requires both authentication and the appropriate permissions (or: *privileges*) for accessing a particular resource.

In this chapter, we'll have a look at basic access management concepts and how those are implemented with IAM. We'll give an overview of IAM terminology, tools, and techniques. We'll also outline best practices for managing permissions with AWS IAM. Finally, we'll delve into our sample application again and see how IAM works in the context of deploying and running a web application.

As we continue to develop our sample Todo application through the course of this book we'll occasionally need to add new permissions for accessing specific AWS resources or services. This chapter will equip you with the knowledge to do so.

If you're familiar with AWS IAM and your AWS account is set up already according to the guidelines laid out in the [AWS docs](#) you can skip ahead to the section *Defining Policies*.

Users, Groups, and Roles

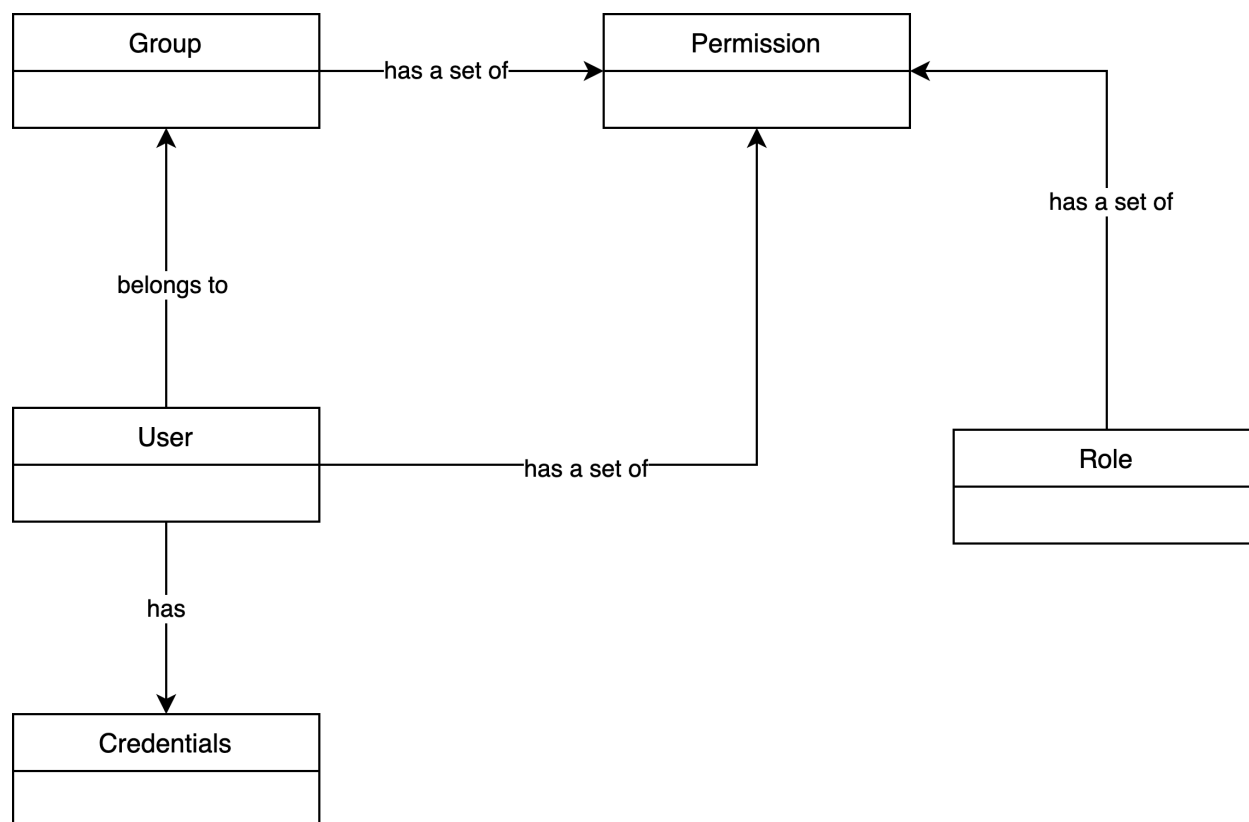
Access-control lists (ACLs) are a tried-and-true approach for granting and managing access to resources. ACLs essentially answer the question “Who is allowed to access a specific resource – and to what extent – at a specific point in time?”. This answer can be given in many ways, the simplest one being a single user with specific permissions for accessing a specific resource.

However, basing access control solely on specific users is not particularly scalable because sooner or later such an approach is bound to create plenty of duplication. If, for example, we grant read permissions to a file on a per-user basis but later on decide we want to withdraw those permissions again we have to do so for each user individually. Not only does this entail additional work but it’s also an error-prone process because the odd user or two might have been omitted and therefore still have the outdated permission.

To alleviate this problem, we can grant privileges based on a **group** a user belongs to or the **role** they assume at a given moment. While groups represent a set of users, roles combine a set of permissions that are necessary for fulfilling a certain function.

In computer security, an abstract entity that can be authenticated is often referred to as a **principal**.

This diagram gives an overview of how the main entities within IAM relate to each other:



Permissions are the core IAM concept. They can be granted to users and applications in a variety of ways.

In the context of AWS and IAM, a principal can be a person or an application identified by either user credentials or an associated role. Users are individuals with an identity and credentials (usually username and password) whereas roles don't have credentials but are usually assumed temporarily by either applications or already authenticated users. Roles, therefore, allow us to assign permissions without storing AWS user credentials with an application. Roles also allow already authenticated AWS users to switch between different sets of permissions based on their current task or context (think of a system administrator working for several departments within an organization, for example).

Allowing or restricting access to a resource is a two-part process consisting of authentication and subsequent authorization. First, a principal needs to identify through authentication. The most common way of doing so is via providing

both username and password. Keep in mind, though, that with IAM both users and roles can be principals, so a valid username/password combination is just one method of authentication. Once successfully authenticated, a principal can access the resources they are authorized for. For each request, AWS will check if the principal is permitted to perform the request action on the requested resource, as given by the resource's [Amazon Resource Name \(ARN\)](#).

In the following section, we'll take a look at how we can put these concepts to use with IAM.

Root Users vs. Individual Users

As a new AWS user, the first encounter with IAM occurs immediately upon registration.

When signing up for your AWS account we're also creating our first IAM principal. This **root** user will have all-encompassing privileges for every resource under your AWS account. For this reason, we should never use this root user for normal work or for accessing AWS resources. Rather, its only responsibility is to create our first administrator user and user group in the [IAM Console](#).

The IAM Console allows us to manage IAM users, groups, and policies (see the section [Defining Policies](#)) through a web application. Later on, we'll have a look at managing resources programmatically, too.

The [IAM docs](#) will guide you through the initial steps of creating an administrative user and an administrative group.

We also recommend creating non-administrative users and a corresponding non-administrative group for the people you collaborate with. Following the principle of least privilege, these individual users should only be granted the

minimum privilege each of them requires to do their work.

In contrast to the account's root user, individual users will need to supply the AWS account ID in addition to their username and password (and possibly an MFA one-time password) if activated for that particular principal). To save your users the effort of having to manually type in this account ID each time they log in you can provide them with this shortcut link:

<https://account-ID-or-alias.signin.aws.amazon.com/console>

Replacing *account-ID-or-alias* in this URL with your account ID will automatically fill the corresponding input field in the user login form.

Now that we've set up both administrative and normal users and their respective groups, we can continue with granting them privileges for accessing AWS resources.

Defining Policies

Once we've created our IAM user(s) for our daily work with AWS we can proceed to grant them privileges to AWS resources. Following the principle of least privilege, we should prefer adding permissions as needed instead of granting blanket permissions.

We can grant privileges by attaching so-called **policies** to a user, role, or group. We can revoke privileges by detaching the policies again. For many common access scenarios, AWS provides ready-to-use policies managed by AWS itself. Some example policies are:

- **AmazonEC2FullAccess:** All permissions required for creating and managing EC2 resources.

- **AmazonSQSReadOnlyAccess:** Read-only permissions to SQS resources.
- **SystemAdministrator:** All permissions required for common operations tasks.

If, for instance, we want our developers to have unrestricted access to EC2 we can grant them the *AmazonEC2FullAccess* policy. It's worth noting, however, that IAM has a [limit of 10 policies per user, role, or group](#).

Depending on our use cases it might be a good idea to create a more fine-grained group hierarchy that takes different roles and specializations into account. For example, some of our developers might predominantly work on the database infrastructure. This in turn might justify creating another group “Database Developers”, which could be granted *AmazonRDSFullAccess* privileges (RDS = Relational Database Service). Developers mainly working on web frontends on the other hand could be assigned to a “Frontend Developers” group that has *AmazonCognitoPowerUser* and *AmazonS3FullAccess* permissions.

This again contributes to a “least privilege” strategy by only granting specific privileges to only the groups that need them.

Apart from using predefined managed policies, we can also create custom policies tailored to our needs. For this, the IAM Console provides us with both a visual editor and a JSON editor for composing our own policies. Since IAM policies are just ordinary AWS resources they can be represented as JSON data structures.

This example shows the JSON representation of the managed *AmazonEC2FullAccess* policy:


```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Action": "ec2:*",
      "Effect": "Allow",
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "elasticloadbalancing:*",
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "cloudwatch:*",
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "autoscaling:*",
      "Resource": "*"
    },
    {
      "Effect": "Allow",
      "Action": "iam:CreateServiceLinkedRole",
      "Resource": "*",
      "Condition": {
        "StringEquals": {
          "iam:AWSServiceName": [
            "autoscaling.amazonaws.com",
            "ec2scheduled.amazonaws.com",
            "elasticloadbalancing.amazonaws.com",
            "spot.amazonaws.com",
            "spotfleet.amazonaws.com",
            "transitgateway.amazonaws.com"
          ]
        }
      }
    }
  ]
}

```

Policies consist of a collection of Statements, each of which has Action, Effect, and Resource properties plus an optional Condition property.

The Effect property can have a value of Allow or Deny. By default, access to resources is denied. When setting Effect to Allow we're granting access to the resource(s) specified by the Resource property of the statement. Assigning Effect a value of Deny overrides a previous Allow again.

The Resource property can either take a full ARN or a partial ARN with wildcard characters for matching multiple resources ("*" for any number of characters or "?" for any single character).

In the *AmazonEC2FullAccess* policy above, the Resource is set to * in all statements, giving access to all resources. If we want to restrict the resources that a statement is valid for, we could instead use `arn:aws:sqs:us-east-2:<ACCOUNT_ID>:mySqsQueue` to only grant access to a specific SQS queue, for example. We just need to know the ARNs of the resources we want to provide access to.

The Action property allows us to specify the exact action(s) to be allowed on a resource. An action consists of a namespace belonging to an AWS service and an action name under that namespace.

The first statement in the *AmazonEC2FullAccess* policy above, for example, grants permission to the actions `ec2:*`, i.e. all actions in the "ec2" namespace, while the last statement grants permission to the specific action `iam:CreateServiceLinkedRole` only.

A statement can also specify a list of actions:

```
"Action": [  
    "sqs:SendMessage",  
    "sqs:ReceiveMessage",  
    "ec2:StartInstances",  
    "iam:ChangePassword",  
    "s3:GetObject"  
]
```

Finally, the optional `Condition` property allows us to apply further constraints to when a policy takes effect.

In the *AmazonEC2FullAccess* policy above, the action `iam:CreateServiceLinkedRole` will only be allowed from one of the AWS services listed under the `StringEquals` attribute of the `Condition`.

Once we have created a policy, we can attach it to users, groups, and roles just as we would do with predefined policies.

Creating AWS Access Keys for Each User

To access resources programmatically through the CLI or the AWS API users have to create access keys. These keys allow authentication without providing a username and password. This is good security practice because it allows us to manage, revoke, and rotate API credentials independently of a user's password. More importantly, we can store these access keys (and their secrets) in a local configuration file and not worry about actual passwords lying around somewhere in plain text.

The default number of access keys per user is two, which allows users to regularly rotate their access keys by adding a new one and subsequently disabling the old one. Again, doing so regularly is considered a good security practice.

Each user can generate access keys individually. We need the resulting “AWS

Access Key ID” and “AWS Secret Access Key” to use the AWS CLI, for instance (see section [Installing the AWS CLI](#)).

Managing IAM Resources Programmatically

It’s worth noting that not only can we access IAM through the web console but also programmatically.

This is where the CLI and the AWS API come into the IAM picture. Every action that can be triggered manually by an IAM user can also be triggered via the [CLI](#) or the [API](#).

We can also use CDK or – alternatively – CloudFormation to create IAM resources during deployment.

One interesting use case for this is to create roles with all the permission that our application needs to function, and our application then assumes these roles during runtime. Unlike users, roles come with their own temporary credentials, which are entirely managed by IAM.

Since the credentials never leave the AWS servers, we can’t accidentally push credentials into version control, which would constitute a major security flaw.

Furthermore, we’re able to easily revoke permissions for a particular application, or even just a subset of those permissions.

We’ll cover permission management with CDK and CloudFormation in some detail in the chapter [Designing a Deployment Project with CDK](#).

Best Practices for Managing Permissions with IAM

Amazon have their own comprehensive guide on [Security best practices in IAM](#), some of which we've already talked about in this chapter. We suggest checking out this guide to get familiar with security measures for protecting your organization, its users, and its data.

4. The Evolution of Automated Deployments

In this book, we not only want to explore AWS services we can use in our Todo application but also create a continuous deployment pipeline that builds our app and deploys it to a production-like environment on AWS with every commit we push to the GitHub repository.

In fact, we'll build this pipeline before we even talk about adding any features to the app! In a software project, we want to deploy to production as soon and often as possible. Only if we deploy our application quickly and automatically will we learn about the features we've built. This enables us to pivot into the direction that provides the most value. This is a core tenet of both DevOps and Agile.

This chapter discusses the evolution of deployment options to set the stage for infrastructure-as-code deployments with AWS CDK that we'll look at in the following chapters.

An Anecdote about Manual Deployments

Let me (Tom) tell you a story from my early days as a software engineer. From the dark times of software deployment. Depending on when you joined the software industry you may or may not have experienced similar stories yourself. Feel free to skip this section if you don't need any motivation to build a continuous deployment pipeline.

Back when I still thought that you can only become a good software engineer through suffering, we inherited a 10-year-old codebase from another software shop. The client was no longer happy with that software shop and awarded the contract of maintaining and extending the product to us instead. We now had 350,000 lines of code we knew nothing about. And we didn't have unit tests or build scripts. The previous shop did have some unit tests and build scripts, but they didn't give them to us because someone forgot to mention them in the transition contract. We felt like adventurers in the worst sense of the word.

There was a transition period of a couple of months when the old software shop was supposed to finish and release some final features. In the meantime, we started to develop new features in parallel. So we put a copy of the codebase into our version control system and started to explore the code. The other shop was still working on their own copy of the code in their own version control system (which might or might not have been a network drive ...).

My first contact with deploying that beast of a software product was in the context of the last release by that other software shop. They used to release four times a year, on a weekend, to not disrupt the service for the users. Their last release before we took over involved the upgrade of the version of the JBoss server the software was running on.

I wasn't personally involved but I heard the stories about that particular deployment. I imagine the release went something like this (the following account may be tinged with the experience of some other software projects I was involved in):

- **Saturday, 08:00:** All people involved with the release check in during a teleconference: the project managers, testers, developers, sysadmins, and of course the managers of both the client and the software shop who call in from the golf resort where they're spending the weekend. The managers

open the teleconference to tell everybody how important it is to release the new version *this* weekend.

- **Saturday, 08:05:** In the teleconference, the project managers go through the 50-item spreadsheet that describes the steps they compiled for the release weekend. During the teleconference, they find out that the sysadmins have not been informed about the steps concerning some configuration changes to the production JBoss instance. But the project managers had distributed the spreadsheet to everybody for review beforehand and say that it's too late to complain now.
- **Saturday, 08:45:** They're done discussing the spreadsheet (for now).
- **Saturday, 09:00:** The sysadmins have successfully configured the load balancer to drain traffic from the production servers and serve the maintenance page instead. The developers are still busy building the new release on their local machines.
- **Saturday, 10:30:** There were some unexpected build errors but our hero developers have fixed them. They have encrypted the 500MB EAR file and are now uploading it to the secure platform shared with the sysadmins.
- **Saturday, 12:00:** Scheduled teleconference to report progress. The sysadmins could not open the file the developers have sent. It turns out the developers had used an old encryption key. The encryption key cycles every three months and the sysadmins refused to use the old key to decrypt the release because that is against security protocol. It took an hour to go through the process of creating a new encryption key because they had to call in the security person responsible for that. The project managers express their expectation that from now on, everything goes according to plan.
- **Saturday, 13:00:** The sysadmins have finally received a release they can decrypt. They have followed the release instructions provided by the developers to the letter but when they start the server, they get cryptic

NoSuchMethodErrors all over the place. The developers ask for log files.

- **Saturday, 13:30:** After the developers have found the right decryption key for the log files sent by the sysadmins, they can now view the logs.
- **Saturday, 14:00:** Scheduled teleconference to report progress. The developers are still analyzing the NoSuchMethodErrors. The project managers see that they are 20 steps behind in the spreadsheet and ask for the log files so that they can help to identify the root cause.
- **Saturday, 15:00:** The developers have spent an hour investigating a possible root cause suggested by one of the project managers (they knew it was a red herring, but the managers insisted). One of the developers sees that a sysadmin had sent an email an hour ago wondering why the release file still had the old version number in the file name. It turns out the developer who built the release had forgotten to update their local workspace from version control.
- **Saturday, 16:00:** The developers have successfully built, encrypted, and uploaded the new version.
- **Saturday, 16:15:** The sysadmins report different errors now and have uploaded the logs (without having been asked, this time).
- **Saturday, 16:30:** The developers ask the sysadmins if they have really followed all the steps in the release notes because it works on their machines. The sysadmins say that they are very much able to read and follow release notes.
- **Saturday, 17:00:** Final scheduled teleconference. The release should have been finished by now. The lead developer suggests postponing the go-live to tomorrow because people are becoming unfocused. The project managers reject because they have promised the upper managers the release for today.
- **Saturday, 21:00:** The developers and sysadmins have had a standing teleconference for two hours to compare configuration files one by one between

their local machines and the production machine to identify any difference. There are a lot of differences, of course, but they haven't found one that sounds like it would be responsible for the errors they're seeing. They've had a pizza in between and thoughts are getting incoherent. No one is particularly motivated because they're not getting paid for the overtime.

- **Saturday, 22:00:** Another teleconference to report progress. The project manager gives in and postpones to tomorrow, 08:00. Everybody goes home with a queasy feeling in their stomachs.
- **Sunday, 08:00:** Teleconference to kick off the day. One of the developers had an idea in the shower this morning. He proposes to just try with a more recent version of JBoss. No one actually believes it'll work but there's no other idea, so they go for it.
- **Sunday, 09:00:** The developers report that on their local machines, everything works fine with the latest JBoss version. They update the release notes and send them over to the sysadmins.
- **Sunday, 10:00:** The sysadmins report that the software starts up without errors (or rather, without any errors they're not used to). The testers start their work and go through their own spreadsheet with test cases to see if the new production release works.
- **Sunday, 12:00:** Scheduled teleconference to report progress. The testers have found a bug that blocks production deployment. The developers start working on a solution.
- **Sunday, 14:00:** The developers have fixed the bug and uploaded a new release. The sysadmins can decrypt it at the first attempt and update the production instance.
- **Sunday, 15:00:** The testers report that the bug has been fixed but another bug has been introduced. Still, the new bug isn't as severe as the previous one, so they're OK with releasing the current version (also, they don't want

to miss that birthday party they're invited to but they don't mention that).

- **Sunday, 16:00:** The sysadmins have worked through the final steps in the spreadsheet. No one except them understands what those steps are about, but nobody except them really cares, either. Finally, the load balancer is reconfigured to point to the application instead of the maintenance page and everybody is happy and goes home.
- **Monday, 08:00:** The first users report that they can't work with the software ...

In general, software releases went like this in the early days of my career. And note that the above was considered a *successful* release! A release was always something to be afraid of. Even when I was not planned to help during a release weekend, I felt nervous going home on Friday night, knowing that something will probably go wrong and I might be called in to help.

Manual deployments were no fun. We usually only had a couple of releases per year for practice. Usually, different people were involved each time, so that the learning we carried over from one release to the next was minimal.

Also, the long time between releases made them bigger, increasing the chance of failure. Features were crammed in because they couldn't wait for the next release. The pressure to succeed was greater, be it pressure from management or pressure we put on ourselves. And humans don't make good decisions under pressure.

Many manual steps could go wrong and communications between different departments could be misunderstood and end up in finger-pointing.

Due to these issues, I never want to experience manual deployments again. Automatically deploying the latest version with every commit to version control

solves most of these issues. Automated deployments bring their own complexity and set of problems, but that's exactly what we'll discuss throughout this and the next chapters of this book.

Self-Service Deployments with the AWS Console

Welcome to the world of the cloud, where we have unlimited options for how to deploy our application. Coming from the manual deployment described above, the next level of sophistication that AWS offers is the AWS Console.

The AWS Console is our main tool for observing and interacting with deployed applications and infrastructure elements. If you've gone through the exercise in the chapter [Getting Warm with AWS](#) you will have seen the AWS Console already.

It's a web-based user interface that gives us self-service access to all the AWS services at our disposal. We could, for example, use the AWS Console to create a VPC with subnets manually and then deploy a Docker container into these subnets to make them accessible from the internet.

Or, we could access the web UI of the CloudFormation service and upload the YAML files we created in the chapter [Getting Warm with AWS](#) via the web UI to achieve the same result.

Also, we could use the web UI of the [Elastic Beanstalk](#) service to just upload a WAR file and have AWS take care of provisioning the infrastructure it needs. There are more options for deploying an application via the AWS Console and it feels like yet more are added each week.

The AWS Console is a gateway to our cloud. We can use it to deploy resources and monitor the resources we have deployed already.

If we're using the AWS Console for deploying our application this is still a

manual process, though! However, if we use one of the higher-order services like CloudFormation or Beanstalk via the web console, this reduces the number of manual steps we have to follow and thus increases the chance of success compared with the manual deployment scenario from the previous section.

Self-service with the AWS Console is nice, and it gives us a much-needed overview of our cloud but that's not nearly enough. We want to automate our deployments to the very last step. So, let's go up another level of sophistication.

Scriptable Deployments with the AWS CLI

To automate deployments, we need to run all the commands from the command line. We can then combine these commands into one or multiple shell scripts and execute them on a remote build server with every pushed commit.

In the manual deployment scenario I outlined above, the sysadmins probably already used some shell scripts they prepared to make their lives easier. For example, a script that decrypts the file they got from the developers and automatically copies that file to the right place on the production server might be useful. Those scripts would still have to be called manually in-between other manual – and error-prone – steps, though.

This is where the AWS CLI comes into play. Pretty much everything that we can do via the web interface of the AWS Console, we can also achieve via the AWS CLI from the command line. The AWS CLI is a command-line utility that wraps the APIs of all AWS services in a single command-line tool.

In the chapter [Getting Warm with AWS](#), we've already seen the AWS CLI in action. We used `aws cloudformation` commands to deploy the resources described in a CloudFormation file and `aws ec2` commands to retrieve information about our EC2 instances after deployment.

Again, the AWS CLI allows us to use low-level APIs like those of the EC2 service to provision our servers. Or, we can use higher-order APIs like CloudFormation to reduce the number of steps.

No matter which APIs we choose, with the AWS CLI we now have the power to do everything we need from the command line. We can build a fully automated deployment pipeline!

Still, we're not satisfied, yet. We don't want to spend time writing shell scripts that are hard to test and even harder to debug.

Declarative Deployments with CloudFormation

If we use CloudFormation with the AWS CLI, we have already made another step toward repeatable and automated deployments.

As we have seen in the chapter [Getting Warm with AWS](#), CloudFormation allows us to declare all the resources we need in YAML or JSON template files. This is a declarative way of deployment, compared to the imperative style of using the AWS CLI to create the resources we need directly via the other AWS Services' APIs.

This declarative style offers a host of advantages over a collection of imperative shell scripts.

We declare “stacks” of resources – each stack in its own template file – and can create, update, or delete a stack with a single CLI command. We can also nest stacks – if need be – to create, update, or delete multiple nested stacks with a single command.

We can use the same template file with the same or different parameters to spin up a copy of the production environment for testing. Since both environments

declare the same resources, they will behave very similarly and make our lives easier when trying to reproduce bugs.

We no longer have to care (so much) about error handling during deployments. CloudFormation wraps all of its actions in a transaction. If one resource fails to be created, updated, or deleted, CloudFormation will automatically roll back all the changes that have already been made and revert to the last working state. We can then review the error message in the CloudFormation UI of the AWS Console and debug from there. No more searching through illegible shell scripts when something goes wrong.

We can share stacks across the organization so that many teams can use them to create their resources. This supports efforts for compliance and training. To create a continuous deployment pipeline, we no longer have to copy a host of shell scripts, but instead, we copy a well-defined template and a handful of CLI commands.

In summary, CloudFormation makes automated deployments a lot more manageable than a collection of shell scripts can ever be (except perhaps for people who speak Bash as their first language).

Still, we haven't yet reached the automation end game.

Programmable Deployments with CDK

If you consider yourself a programmer and not a manager of YAML and JSON files, you'll be happy to learn about the AWS Cloud Development Kit (CDK).

The CDK builds on top of CloudFormation and currently allows to describe cloud resources in Java, Javascript, Typescript, Python, or C#. It “synthesizes” the code we create in one of these languages into plain old CloudFormation

templates. We can then deploy them with the AWS CLI or the more specialized CDK CLI, which is a little easier to work with for us humans. A CDK “App” can contain one or more CloudFormation stacks that we can interact with separately via CLI commands.

When we say “programmable” in the title of this section, we are referring to using a programming language as opposed to creating JSON or YAML files. Putting too much programming logic into a CDK app is discouraged, though. With each conditional branch in a CDK app, the number of possible outcomes increases exponentially. And, since the goal of automated deployments is to create reproducible and predictable environments for our software applications, we want as little variation as possible.

So, in the end, a CDK app is still a mostly declarative approach to infrastructure-as-code. It’s just that we declare the resources we need in a programming language of our choice instead of in YAML or JSON. So, aside from using a different language, what are the benefits over plain CloudFormation templates?

The main benefit is that we can use the power of the programming language and its ecosystem to create and share reusable “constructs” of one or more CloudFormation resources. We can, for example, create a construct that contains all the infrastructure needed for running a Spring Boot app in the cloud, share this as a Maven module, and use the construct in different CDK apps that each deploy a different Spring Boot application.

While we could also share stacks with CloudFormation, we would need to solve the problem of distributing, versioning, and re-using the JSON or YAML files. With CDK, we can piggyback on whatever distribution mechanism the language’s ecosystem offers (Maven, in the case of Java). Also, with CloudFormation, we’re limited to sharing whole stacks. With CDK, the granularity of the constructs can be as coarse or as fine as we want. A CDK construct can

be a single pre-configured CloudFormation resource or a stack containing the infrastructure for a whole ecosystem of microservices. We can combine any number of such constructs into a single or multiple stacks, just as we see fit.

The CDK itself offers [constructs of different granularity](#):

- **Level 1 constructs:** these constructs are direct equivalents of CloudFormation resources. These are the most granular constructs. The names of level 1 constructs are prefixed with `Cfn` for “CloudFormation” and they translate 1:1 into their CloudFormation counterpart. We can refer to the CloudFormation docs to configure them. An example is the `CfnSecurityGroup` construct that creates a security group to restrict access to certain resources.
- **Level 2 constructs:** these are pre-configured groups of one or more CloudFormation resources. They often combine multiple resources and considerably reduce the amount of code we have to write. Also, they often provide a more convenient programming interface for configuring them. The `SecurityGroup` level 2 construct, for instance, offers methods for creating inbound and outbound access rules without having to create `CfnSecurityGroupIngress` or `CfnSecurityGroupEgress` constructs, which we would have to do if we used the level 1 construct `CfnSecurityGroup`. We’ll talk more about security groups later.
- **Level 3 constructs:** these constructs are the most high-level constructs. They are also called “patterns” because they usually represent certain architecture patterns like “deploy an app behind a load balancer”. The goal of the CDK team is to create a library of such constructs ready-to-use for common scenarios. However, most often we’ll create them ourselves tailored to our architecture. As with any abstraction, the downside of these powerful constructs is that they hide much of what they’re doing. Like a Matryoshka doll, a level 3 construct contains other constructs, which may

contain yet other constructs, and we might end up deploying resources that we don't need.

With CDK, we have reached the end of the deployment automation evolution. We can codify our entire infrastructure in code and deploy it with the click of a button. We're going hands-on with CDK in the next chapter.

5. First Steps with CDK

In the chapter [Getting Warm with AWS](#), we have already played around a bit with AWS CloudFormation. We have deployed a network stack that provides the network infrastructure we need, and a service stack that deploys a Docker image with our Spring Boot application into that network.

In this chapter, we'll do the same with the Cloud Development Kit (CDK) instead of CloudFormation. Instead of describing our stacks in YAML, however, we'll be using Java. Furthermore, we'll replace the AWS CLI with the CDK CLI which allows us to deploy and destroy our stacks with ease.

Under the hood, CDK will “synthesize” a CloudFormation file from our Java code and pass that file to the CloudFormation API to deploy our infrastructure. This means that with CDK, we describe the same resources as we would in a CloudFormation YAML file. But, having the power of a *real* programming language at our hands (in our case, Java), we can build abstractions on top of the low-level CloudFormation resources¹. These abstractions are called “constructs” in CDK lingo.

Let's create our first CDK app! Follow along the steps in this chapter to create a CDK app that deploys our “Hello World” application to the cloud.

Creating Our First CDK App

The unit of work in CDK is called an “app”. Think of an app as a project that we import into our IDE. In Java terms, this is a Maven project by default.

¹Most importantly, we don't have to worry about indentation.

In that app, we can define one or more stacks. And each stack defines a set of resources that should be deployed as part of that stack. Note that a CDK stack is the same concept as a CloudFormation stack.

Once we have an app in place, the CDK CLI allows us to deploy or destroy (undeploy) all stacks at the same time, or we can choose to interact with a specific stack only.

Before we can start, we have to get some prerequisites out of the way.

Installing Node

Even though we're using the Java CDK, the CDK CLI is built with Node.js. So, we need to install it on our machine.

If you don't have Node.js running, yet, you can download it from the [Node website](#) or use the package manager of your choice to install it. We have tested all the steps in this book with Node.js 16, which is the latest LTS version at the time of writing, but it will probably work with other versions as well.

You can check your Node.js version by calling `node -v`.

Installing the CDK CLI

Next, we want to install the CDK CLI.

Having Node.js installed, this is as easy as calling `npm install -g aws-cdk`. This will make the CDK CLI command `cdk` available globally on your system.

As with Node.js, you can check the version of your CDK CLI installation by calling `cdk --version`.



From version 1.4 of this book onwards, we'll be using the AWS CDK v2. Make sure to use a `cdk` version `>= 2.0.0` for all upcoming examples. For migrating your existing AWS CDK v1 infrastructure, follow the [official migration guide](#).

Creating the CDK App

Now we're ready to create our first CDK app!

Like many modern development CLIs, the CDK CLI provides the functionality to bootstrap a new project from scratch.

Let's create a new folder for our app, change into it, and run this command:

```
cdk init app --language=java
```

After CDK has created our app we're greeted with this message:

```
# Welcome to your CDK Java project!
```

This is a blank project for Java development with CDK.

The ``cdk.json`` file tells the CDK Toolkit how to execute your app.

It is a [Maven](<https://maven.apache.org/>) based project, so you can open this project with any Maven compatible Java IDE to build and run tests.

```
## Useful commands
```

```
* `mvn package`    compile and run tests
* `cdk ls`         list all stacks in the app
* `cdk synth`      emits the synthesized CloudFormation template
* `cdk deploy`     deploy this stack to your default AWS account/region
* `cdk diff`       compare deployed stack with current state
* `cdk docs`       open CDK documentation
```

Enjoy!

Aside from some useful commands, there is some important information in this message:

- the project relies on Maven to compile and package the code, and
- there's a file called `cdk.json` that tells the CDK how to run our app.

We'll make use of that information in the next section.

Making the CDK App Portable with the Maven Wrapper

Before we inspect the generated app in more detail, let's fix an issue with the auto-generated Maven setup.

The message above says that we need to run `mvn package` to compile and run the tests. That means Maven needs to be installed on our machine. Thinking a bit further, this also means that Maven needs to be installed on the build server once we decide to set up a continuous deployment pipeline.

While it's not an unsolvable problem to install Maven on a local or remote machine, we'll have a more self-contained solution if the build takes care of "installing" Maven itself.

The solution to this is the [Maven Wrapper](#). It's a script that downloads Maven if necessary. To install it we copy the folder `.mvn` and the files `mvnw` and `mvnw.cmd` from the [example project](#) into the main folder of our newly created CDK app.

Instead of calling `mvn package`, we can now call `./mvnw package` for the same effect, even if Maven is not installed on our machine.

But we're not completely done yet. Remember the message saying that the file `cdk.json` tells the CDK how to execute our app? Let's look into that file:

```
{  
  "app": "mvn -e -q compile exec:java",  
  "watch": {  
  },  
  "context": {  
  }  
}
```

The first line of this JSON structure is telling the CDK how to compile and then execute our CDK app. It's set up to call `mvn` by default. So, let's replace that with `./mvnw` and we're done.

Now, any time we call a command like `cdk deploy`, the CDK will call the Maven Wrapper instead of Maven directly to execute our CDK app.

Inspecting the Generated Source Code

With everything set up, let's have a look at the code that the CDK created for us. In the folder `src/main/java/com/myorg` we'll find the files `CdkApp` and `CdkStack`:

```
public class CdkApp {  
  public static void main(final String[] args) {  
    App app = new App();  
  
    new CdkStack(app, "CdkStack");  
  
    app.synth();  
  }  
}
```

```
public class CdkStack extends Stack {  
    public CdkStack(final Construct scope, final String id) {  
        this(scope, id, null);  
    }  
  
    public CdkStack(final Construct scope, final String id, final StackProps props) {  
        super(scope, id, props);  
  
        // The code that defines your stack goes here  
    }  
}
```

That's all the code we need for a working CDK app!

CdkApp is the main class of the app. It's a standard Java class with a standard `main()` method to make it executable. The `main()` method creates an App instance and a CdkStack instance and finally calls `app.synth()` to tell the CDK app to create CloudFormation files with all the CloudFormation resources it contains. These CloudFormation files will be written to the folder named `cdk.out`.

When we run CDK commands like `cdk deploy`, CDK will execute the main method of CdkApp to generate the CloudFormation files. The `deploy` command knows where to look for these files and then sends them to the CloudFormation API to deploy.

The CdkStack class represents a CloudFormation stack. As mentioned before, a CDK app contains one or more stacks. This stack is where we would add the resources we want to deploy. We'll add our own resources later in this chapter. For now, we'll leave it empty.

Bootstrapping the AWS Environment for a CDK Deployment

Before we can deploy AWS resources with custom CDK stacks, we may need to bootstrap our AWS environment.

The CDK requires a bootstrapped AWS environment in case we either deploy [Assets](#) as part of our CDK stack or generate AWS CloudFormation templates that exceed 50 kilobytes.

If we don't bootstrap our AWS environment and, for example, try to deploy a 100 kilobyte AWS CloudFormation template, the deployment will fail.

To avoid such scenarios and to complete all possible setup steps at least once, we're going to bootstrap our AWS environment.

When bootstrapping our AWS environment, the AWS CDK creates an S3 bucket, IAM roles, and an ECR for internal purposes. These resources may already result in minimal costs on our AWS bill. Those are negligible, however, and should be covered by the AWS free tier anyway.

Each environment (i.e. a tuple of an AWS account and a region) needs to be bootstrapped separately. This is a one-time effort, unless we migrate to a new major version of the CDK, in which case we'd have to "re-bootstrap" the environment.

To bootstrap our AWS environment, we need to run the following command once:

```
cdk bootstrap aws://ACCOUNT-NUMBER/REGION
```

An exemplary bootstrapping looks like the following:

```
cdk bootstrap aws://123456789/ap-southeast-2
  ☒ Bootstrapping environment aws://123456789/ap-southeast-2...
Trusted accounts for deployment: (none)
Trusted accounts for lookup: (none)
Using default execution policy of 'arn:aws:iam::aws:policy/AdministratorAccess'. Pass
s '--cloudformation-execution-policies' to customize.
CDKToolkit: creating CloudFormation changeset...

  ☒ Environment aws://123456789/ap-southeast-2 bootstrapped.
```

After running this command, we can navigate to the CloudFormation service in the AWS console in the browser and should see a stack called CDKToolkit.

That's everything we need for bootstrapping our AWS environment.

If we want to destroy the resources created by this bootstrap process, we can manually delete the stack inside the AWS console.

Deploying the Generated CDK App

Let's try to deploy the generated CDK app.

This is as easy as executing the `cdk deploy` command in the folder of the app. It will take a couple of seconds and we'll be rewarded with a success message like this one:

TestStack: deploying...

TestStack: creating CloudFormation changeset...

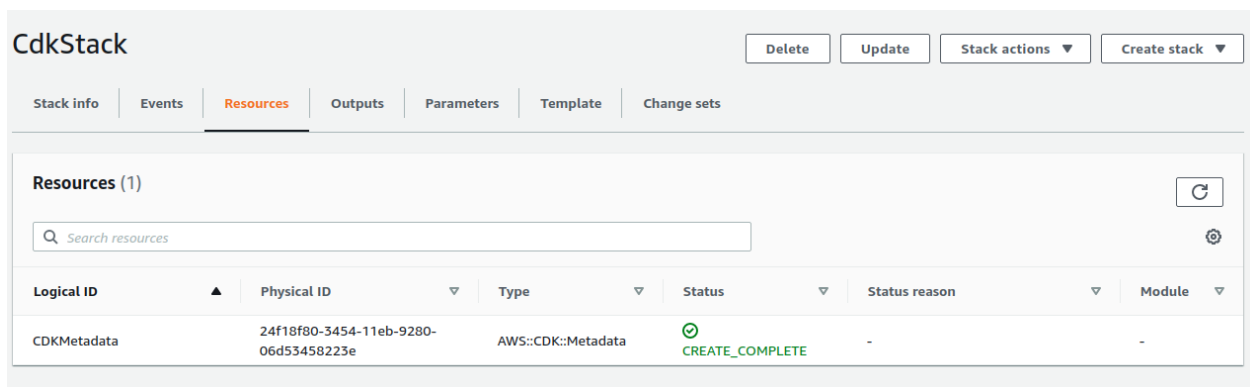
[=====] (2/2)

TestStack

Stack ARN:

arn:aws:cloudformation:ap-southeast-2:...

This means that CDK has successfully deployed the (empty) stack. If we log in to the AWS web console and navigate to the CloudFormation service, we should see a stack called “TestStack” deployed there:



The default CDK Stack in the CloudFormation web console.

The stack contains a single resource called CDKMetadata, which the CDK needs to work with that stack.

Before moving on, let’s destroy the stack again with `cdk destroy`.

Deploying a Spring Boot App with a CDK Construct

Now that we know the basic workings of CDK, let’s deploy a real application! The goal is to deploy the same network and ECS Cluster that we have deployed in the chapter [Getting Warm with AWS](#). We’ll deploy the same Docker image into

that cluster, namely the one containing our “Hello World” app (you can find the source code [on GitHub](#)).

As mentioned, the resources that we include in a CDK stack are called constructs. To show the power of CDK – and to keep it simple for now – we have prepared a construct with the name `SpringBootApplicationStack` that includes all the resources we need. All we need to do is to include this construct into our CDK stack.

Adding the Stratospheric Construct Library

To get access to the `SpringBootApplicationStack` construct, we need to include the `cdk-constructs` library in our project. We created this library to provide constructs that we’re going to use throughout the book.

Let’s add the following snippet to the `pom.xml` file in the CDK project:

```
<dependency>
  <groupId>dev.stratospheric</groupId>
  <artifactId>cdk-constructs</artifactId>
  <version>0.1.0</version>
</dependency>
```



From version 0.1.0 onwards, the `cdk-constructs` library only supports the AWS CDK v2.

You can check out the source code of this constructs library [on GitHub](#) and check for the latest version of it on [Maven Central](#).

Using the `SpringBootApplicationStack`

As you might expect from the name of the construct, `SpringBootApplicationStack` is a stack. It extends the `Stack` class of the CDK API. That means we can

use it to replace the generated `CdkStack` class.

So, we modify the generated `CdkApp` class to include a `SpringBootApplicationStack` instead of an empty `CdkStack`:

```
public class CdkApp {

    public static void main(final String[] args) {
        App app = new App();

        String accountId = (String) app
            .getNode()
            .tryGetContext("accountId");
        requireNonNull(accountId, "context variable 'accountId' must not be null");

        String region = (String) app
            .getNode()
            .tryGetContext("region");
        requireNonNull(region, "context variable 'region' must not be null");

        new SpringBootApplicationStack(
            app,
            "SpringBootApplication",
            makeEnv(accountId, region),
            "docker.io/stratospheric/todo-app-v1:latest");

        app.synth();
    }

    static Environment makeEnv(String account, String region) {
        return Environment.builder()
            .account(account)
            .region(region)
            .build();
    }
}
```

The first apparent change is that we’re now accepting two parameters. With `app.getNode().tryGetContext()`, we’re reading so-called “context variables” from the command line.

We can pass such parameters to the `cdk` command line with the `-c` parameter, for example like this:

```
cdk deploy -c accountId=123456789 -c region=ap-southeast-2
```

Why are we now passing the account ID and the AWS region into the app? The reason is to be more flexible. If not provided, the CDK CLI will always take the account and region that we have pre-configured with the AWS CLI. We'd have no way of deploying resources into other accounts and regions. We don't really need this flexibility yet but `SpringBootApplicationStack` uses more sophisticated constructs under the hood which need these parameters as input.

Next, we create a `SpringBootApplicationStack` instance. We pass in the app instance to let CDK know that this `SpringBootApplicationStack` is part of the app and should be included in the synthesized CloudFormation files.

The second parameter is an arbitrary (but unique) identifier for the construct within the app.

The third parameter combines the `accountId` and `region` parameters to create an `Environment` object. `Environment` is a CDK class that we're reusing here.

The final parameter is the URL to the Docker image that we want to deploy. We'll use the same image we have used before. We could also decide to make the URL a context variable to be passed from the outside to make the CDK app more flexible.

You might wonder why we're not doing anything with the `SpringBootApplicationStack` instance. When creating a construct, we always pass a parent construct or the parent app into the constructor. The construct will then register with the app so that the app knows which constructs to include in the synthesized CloudFormation stack when calling `app.synth()`.

Deploying the CDK App

Let's try out our shiny new CDK app! Let's run this command:

```
cdk deploy -c accountId=<ACCOUNT_ID> -c region=<REGION>
```

Replace ACCOUNT_ID and REGION with your AWS account number and region, respectively.

The CDK will show a list of “IAM Statement Changes” and “Security Group Changes” for you to confirm. This is a security measure to avoid unintended changes in the security configuration. After confirming, the console should show the deployment progress like this:

```
Do you wish to deploy these changes (y/n)? y
SpringBootApplication: deploying...
SpringBootApplication: creating CloudFormation changeset...
[=====] (7/46)

7:29:22 am | CREATE_IN_PROGRESS | AWS::CloudFormation::Stack | SpringBootAppli...
7:29:28 am | CREATE_IN_PROGRESS | AWS::EC2::InternetGateway | network/vpc/IGW
7:29:28 am | CREATE_IN_PROGRESS | AWS::EC2::VPC | network/vpc
7:29:29 am | CREATE_IN_PROGRESS | AWS::IAM::Role | Service/ecsTaskRole
7:29:29 am | CREATE_IN_PROGRESS | AWS::IAM::Role | Service/ecsTaskE...
```

Since the SpringBootApplicationStack contains a lot of resources under the hood, it will take a minute or two for the deployment to finish.

When it's done we should see an output like this in the console:

Outputs:

```
SpringBootApplication.loadbalancerDnsName = prod-loadbalancer-810384126.ap-southeast\
-2.elb.amazonaws.com
```

Stack ARN:

```
arn:aws:cloudformation:ap-southeast-2:494365134671:stack/SpringBootApplication/0b6b4\
410-3be9-11eb-b5d5-0a689720a8fe
```

This means the `SpringBootApplication` stack has been successfully deployed. CloudFormation stacks support the concept of “output parameters” and CDK prints any such output parameters after a successful deployment. The `SpringBootApplication` is built to expose the DNS name of its load balancer as an output parameter, which is why we see that DNS name in the console.

If we copy this URL into our browser, we should see our hello world application.

Inspecting the CloudFormation web console again, we should see a stack with a bunch of resources

Logical ID	Physical ID	Type	Status	Status reason	Module
CDKMetadata	5fae0c10-3b26-11eb-a1eb-0aae6094f03a	AWS::CDK::Metadata	CREATE_COMPLETE	-	-
ServicecsIngressFromLoadbalancer62209FDD	ServicecsIngressFromLoadbalancer62209FDD	AWS::EC2::SecurityGroupIngress	CREATE_COMPLETE	-	-
ServicecsIngressFromSelf8949B55F	ServicecsIngressFromSelf8949B55F	AWS::EC2::SecurityGroupIngress	CREATE_COMPLETE	-	-
ServicecsLogGroup1EA96AB9	prod-SpringBootApplication-logs	AWS::Logs::LogGroup	CREATE_COMPLETE	-	-
ServicecsSecurityGroup6D8F04D	sg-02a18e9c2ddfd6316	AWS::EC2::SecurityGroup	CREATE_COMPLETE	-	-
ServicecsService30915DD1	arn:aws:ecs:ap-southeast-2:494365134671:service/prod-ecsCluster/SpringBootApplication-ServicecsService30915DD1-S8QGc3DmRbE8	AWS::ECS::Service	CREATE_COMPLETE	-	-
ServicecsTaskExecutionRole0EBE7C5D	SpringBootApplication-ServicecsTaskExecutionRole0-S42R4B620PKR	AWS::IAM::Role	CREATE_COMPLETE	-	-
ServicecsTaskRole4E8999D2	SpringBootApplication-ServicecsTaskRole4E8999D2-1IRQVZDPLED9F	AWS::IAM::Role	CREATE_COMPLETE	-	-

The Spring Boot CDK Stack in the CloudFormation web console.

When done inspecting the stack don't forget to destroy it to avoid unnecessary costs:

```
cdk destroy -c accountId=<ACCOUNT_ID> -c region=<REGION>
```

Why Not Stop Here?

We have successfully deployed a Spring Boot application with about 20 lines of Java code with the help of AWS CDK. Before, in the chapter [Getting Warm with AWS](#), the same has taken us a couple of hundred lines of YAML configuration. That's quite an achievement!

So, why not stop here? Why is there another in-depth chapter about CDK coming up? Our `SpringBootApplicationStack` gives us everything we need to deploy a Spring Boot application, doesn't it?

The main reason is that our `SpringBootApplicationStack` construct is not very flexible. The only thing we have control over is the URL of the Docker image. Like any abstraction, the `SpringBootApplicationStack` hides a lot of details from us.

What if we need to connect our Spring Boot application to a database or SQS queues? What if the path to our application's health check is different from the default? What if our application needs more CPU power than the default 256 units? What if we prefer to use HTTPS rather than HTTP?

Also, imagine an environment with more than one application. We'd have one network for staging and another for production. We'd want to deploy multiple applications into each network. This doesn't work currently, because each `SpringBootApplicationStack` would try to create its own VPC (which would fail for the second application because it would try to use the same resource names).

This means our CDK project needs to be flexible enough to let us deploy additional resources as needed and give us a lot of knobs and dials to configure the infrastructure and our application. We want to have fine-grained control.

To get this control, we have to build our own stacks and constructs. And this is exactly what we're going to do in the next chapter.

6. Designing a Deployment Project with CDK

In the previous chapter, we have already deployed a Spring Boot application to AWS with the CDK. We used a pre-configured “black box” construct named `SpringBootApplicationStack`, passed in a few parameters, and wrapped it in a CDK app to deploy it with the CDK CLI.

In this chapter, we want to go a level deeper and answer the following questions:

- How can we create reusable CDK constructs?
- How do we integrate such reusable constructs in our CDK apps?
- How can we design an easy-to-maintain CDK project?

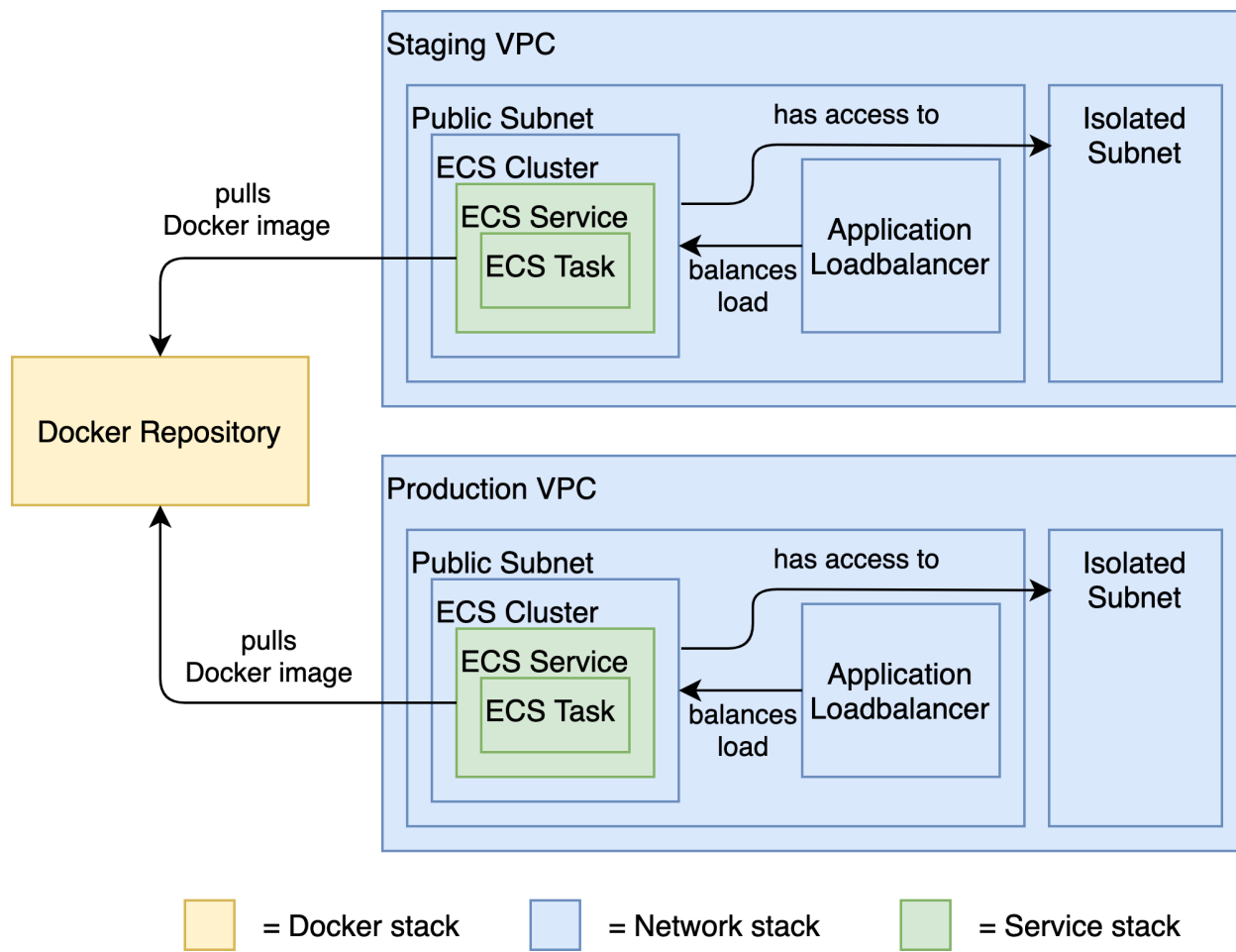
We’ll later build on that in the chapter [Building a Continuous Deployment Pipeline](#), where we’ll be creating an automated deployment pipeline with GitHub Actions.

Let’s dive in!

The Big Picture

The basic goal for this chapter is still the same as in the chapters [Getting Warm with AWS](#) and [First Steps with CDK](#): We want to deploy a simple “Hello World” Spring Boot application (in a Docker image) into a public subnet in our

own private virtual network (VPC). This time, however, we want to do it with reusable CDK constructs and we're adding some more requirements:



We want to deploy our application into a staging and a production environment.

The image above shows what we want to achieve. Each box is a CloudFormation resource (or a set of CloudFormation resources) that we want to deploy. This is a high-level view. So, there are actually more resources involved, but let's not worry about that, yet. Each color corresponds to a different CloudFormation stack. Let's go through each of the stacks one by one.

The **Docker Repository stack** creates – you guessed it – a Docker repository for our application's Docker images. The underlying AWS service we're using here is ECR – Elastic Container Registry. We can later use this Docker repository to

publish new versions of our application.

The **Network stack** is pretty much the same as the one we've already seen in the chapter [Getting Warm with AWS](#) but with a few twists. It deploys a VPC (Virtual Private Network) with a public subnet and an isolated (private) subnet. The public subnet now contains an Application Load Balancer (ALB) that forwards incoming traffic to an ECS (Elastic Container Service) Cluster – the runtime of our application. The isolated subnet is not accessible from the outside and is designed to secure internal resources such as our database.

The **Service stack** contains an ECS service and an ECS task. Remember that an ECS task is basically a Docker image with a few additional configurations, and an ECS service wraps one or several such tasks. In our case, we'll have exactly one task because we only have one application. In an environment with multiple applications, like in a microservice environment, we might want to deploy many ECS tasks into the same ECS service – one for each application. ECS (in its “Fargate” flavor) takes care of spinning up EC2 compute instances for hosting the configured Docker image(s). It even handles automatic scaling if we want it to.

ECS will pull the Docker image that we want to deploy as a task directly from our Docker repository.

Note that we'll deploy the Network stack and the Service stack twice: once for a staging environment and once for a production environment. This is where we take advantage of infrastructure-as-code: we will re-use the same CloudFormation stacks to create multiple environments. We'll use the staging environment for tests before we deploy changes to the production environment.

The Docker repository stack, on the other hand, we'll deploy only once. It will serve Docker images to both the staging and production environments. Once we've tested a Docker image of our application in staging we want to deploy ex-

actly the same Docker image to production, so we don't need a separate Docker repository for each environment. If we had more than one application, though, we would probably want to create a Docker repository for each application to keep the Docker images cleanly separated. In that case, we would re-use our Docker repository stack and deploy it once for each application.

Let's take a look at how we can build each of those three stacks with CDK in a manageable and maintainable way. We'll walk through each of the stacks and discuss how we can implement them with reusable CDK constructs.

Each stack lives in its own CDK app. While discussing each stack, we'll point out concepts that we applied when developing the CDK constructs and apps. These concepts helped us manage the complexity of CDK, and hopefully, they will help you with your endeavors, too.

Having said that, please don't take those concepts as a silver bullet – different circumstances will require different concepts. We'll discuss each of these concepts in its own section so they don't get lost in a wall of text.

Working with CDK

Before we get our hands dirty with CDK, though, some words about working with CDK.

Building hand-rolled stacks with CDK can require a lot of time, especially when you're not yet familiar with the CloudFormation resources that you want to use. Tweaking the configuration parameters of those resources and then testing them is a lot of effort because you have to deploy the stack each time to test it.

Also, CDK and CloudFormation will spout error messages at you every chance they get. Especially with the Java version, you will run into strange errors every

once in a while. These errors are hard to debug because the Java code uses a JavaScript engine ([JSii](#)) for generating the CloudFormation files. Its stack traces often come from somewhere deep in that JavaScript engine, with little to no information about what went wrong.

Another common source of confusion is the distinction between “synthesis time” errors (errors that happen during the creation of the CloudFormation files) and “deploy time” errors (errors that happen while CDK is calling the CloudFormation API to deploy a stack). If one resource in a stack references an attribute of another resource, this attribute will be just a placeholder during synthesis time and will be evaluated to the real value during deployment time. Sometimes, it can be surprising that a value is not available at synthesis time.

CDK has been originally written in TypeScript and then ported to other languages (e.g. C#, Python, and, of course, Java). This means that the Java CDK does not yet feel like a first-class citizen within the CDK ecosystem. There are not as many construct libraries around and it has some teething problems that the original TypeScript variant doesn’t have.

Having listed all those annoyances of the Java CDK, not all is bad. The community on GitHub is very active and there has been a solution or workaround for any problem we’ve encountered so far. The investment of time will surely pay off once you have built constructs that many teams in your company can use to quickly deploy their applications to AWS.

Now, finally, let’s get our hands dirty on building CDK apps!

The Docker Repository CDK App

We’ll start with the simplest stack – the Docker Repository stack. This stack will only deploy a single CloudFormation resource, namely an ECR repository.

You can find the code for the `DockerRepositoryApp` [on GitHub](#). Here it is in its entirety:

```
public class DockerRepositoryApp {

    public static void main(final String[] args) {
        App app = new App();

        String accountId = (String) app
            .getNode()
            .tryGetContext("accountId");
        requireNonEmpty(accountId, "accountId");

        String region = (String) app
            .getNode()
            .tryGetContext("region");
        requireNonEmpty(region, "region");

        String applicationName = (String) app
            .getNode()
            .tryGetContext("applicationName");
        requireNonEmpty(applicationName, "applicationName");

        Environment awsEnvironment = makeEnv(accountId, region);

        Stack dockerRepositoryStack = new Stack(
            app,
            "DockerRepositoryStack",
            StackProps.builder()
                .stackName(applicationName + "-DockerRepository")
                .env(awsEnvironment)
                .build());

        DockerRepository dockerRepository = new DockerRepository(
            dockerRepositoryStack,
            "DockerRepository",
            awsEnvironment,
            new DockerRepositoryInputParameters(applicationName, accountId));

        app.synth();
    }
}
```



```
static Environment makeEnv(String accountId, String region) {  
    return Environment.builder()  
        .account(accountId)  
        .region(region)  
        .build();  
}  
  
}
```

We'll pick it apart step by step in the upcoming sections. It might be a good idea to open the code in your browser to have it handy while reading on.

Parameterizing Account ID and Region

The first concept we're applying is to *always* pass in an account ID and region.

As discussed in [First Steps with CDK](#), we can pass parameters into a CDK app with the `-c` command-line parameter or by adding them to the context section in the `cdk.json` file. In the app, we read the parameters `accountId` and `region` like this:

```
String accountId = (String) app  
    .getNode()  
    .tryGetContext("accountId");  
  
String region = (String) app  
    .getNode()  
    .tryGetContext("region");
```

We're using these parameters to create an `Environment` object:

```
static Environment makeEnv(String accountId, String region) {  
    return Environment.builder()  
        .account(accountId)  
        .region(region)  
        .build();  
}
```

Then, we pass this `Environment` object into the stack we create via the `env()` method on the builder.

It's not mandatory to explicitly define the environment of our CDK stack. If we don't define an environment, the stack will be deployed to the account and region configured in our local AWS CLI. Remember executing the command `aws configure`? Whatever we typed in there as the account and region would then be used.

Using the default account and region depending on our local configuration state is not desirable. We want to be able to deploy a stack from any machine (including CI servers) into any account and any region, so we *always* parameterize them.

Sanity Checking Input Parameters

It should come as no surprise that we strongly recommend validating all input parameters. There are few things more frustrating than deploying a stack only to have CloudFormation complain 5 minutes into the deployment that something is missing.

In our code, we add a simple `requireNonEmpty()` check to all parameters:

```
String accountId = (String) app.getNode().tryGetContext("accountId");  
requireNonEmpty(accountId, "accountId");
```

The method `requireNonEmpty()` throws an exception with a helpful message if the parameter is null or an empty string.

That's enough to catch a whole class of errors early on. For most parameters, this simple validation will be enough. We don't want to do heavy validations like checking if an account or a region really exists, because CloudFormation is eager to throw errors in that case.

One Stack per App

Another concept we're advocating is that of a single stack per CDK app.

Technically, CDK allows us to add as many stacks as we want to a CDK app. When interacting with the CDK app we could then choose which stacks to deploy or destroy by providing a matching filter:

```
cdk deploy Stack1  
cdk deploy Stack2  
cdk deploy Stack*  
cdk deploy *
```

Assuming the CDK app contains many stacks, the first two commands would deploy exactly one stack. The third command would deploy all stacks with the prefix "Stack", and the last command would deploy all stacks.

There is a big drawback with this approach, however. CDK will create the CloudFormation files for *all* stacks, even if we want to deploy a single stack only. This means that we have to provide the input parameters for *all* stacks, even if we only want to interact with a single stack.

Different stacks will most probably require different input parameters, so we'd have to provide parameters for a stack that we don't care about at the moment!

This can be mitigated by putting all input parameters into the `context` section of a shared `cdk.json` file, so that we don't have to pass them to the `cdk` command using the `-c` parameter anymore. But this still means that the CDK stacks are coupled via this `cdk.json` file.

It might make sense to group certain strongly coupled stacks into the same CDK app, but in general, we want our stacks to be loosely coupled (if at all). So, we chose to wrap each stack into its own CDK app to decouple the stacks.

In the case of our `DockerRepositoryApp`, we're creating exactly one stack:

```
Stack dockerRepositoryStack = new Stack(
    app,
    "DockerRepositoryStack",
    StackProps.builder()
        .stackName(applicationName + "-DockerRepository")
        .env(awsEnvironment)
        .build());
```

One input parameter to the app is the `applicationName`, i.e. the name of the application for which we want to create a Docker repository. We're using the `applicationName` to prefix the name of the stack, so we can identify the stack quickly in CloudFormation.

The DockerRepository Construct

Let's have a look at the `DockerRepository` construct, now. This construct is the heart of the `DockerRepositoryApp`:

```
DockerRepository dockerRepository = new DockerRepository(
    dockerRepositoryStack,
    "DockerRepository",
    awsEnvironment,
    new DockerRepositoryInputParameters(applicationName, accountId));
```

DockerRepository is another of the constructs from our [constructs library](#).

We're passing in the previously created dockerRepositoryStack as the scope argument, so that the construct will be added to that stack.

The DockerRepository construct expects an object of type DockerRepositoryInputParameters as a parameter, which bundles all input parameters the construct needs into a single object. We use this approach for all constructs in our library because we don't want to handle long argument lists and make it very explicit what parameters need to go into a specific construct.

Let's take a look at the [code of the construct](#) itself:

```
public class DockerRepository extends Construct {

    private final IRepository ecrRepository;

    public DockerRepository(
        final Construct scope,
        final String id,
        final Environment awsEnvironment,
        final DockerRepositoryInputParameters dockerRepositoryInputParameters) {
        super(scope, id);

        this.ecrRepository = Repository.Builder.create(this, "ecrRepository")
            .repositoryName(dockerRepositoryInputParameters.dockerRepositoryName)
            .lifecycleRules(singletonList(LifecycleRule.builder()
                .rulePriority(1)
                .maxImageCount(dockerRepositoryInputParameters.maxImageCount)
                .build()))
            .build();

        // grant pull and push to all users of the account
```

```
        ecrRepository.grantPullPush(  
            new AccountPrincipal(dockerRepositoryInputParameters.accountId));  
    }  
  
    public IRepository getEcrRepository() {  
        return ecrRepository;  
    }  
}
```

DockerRepository extends Construct, which makes it a custom construct. The main responsibility of this construct is to create an ECR repository with Repository.Builder.create() and pass in some of the parameters that we previously collected in the DockerRepositoryInputParameters.

Repository is a level 2 construct, meaning that it doesn't directly expose the underlying CloudFormation attributes, but instead offers an abstraction over them for convenience. One such convenience is the method grantPullPush(), which we use to grant all users of our AWS account access to pushing and pulling Docker images to and from the repository, respectively.

In essence, our custom DockerRepository construct is just a glorified wrapper around the CDK's Repository construct with the added responsibility of taking care of permissions. It's a bit over-engineered for the purpose, but it's a good candidate for introducing the structure of the constructs in our cdk-constructs library.

Wrapping CDK Commands with NPM

With the above CDK app we can now deploy a Docker repository with this command:

```
cdk deploy \  
  -c accountId=... \  
  -c region=... \  
  -c applicationName=...
```

That will work as long as we have a single CDK app, but as you might suspect by now, we're going to build multiple CDK apps – one for each stack. As soon as there is more than one app on the classpath, CDK will complain because it doesn't know which of those apps to start.

To work around this problem, we use the `--app` parameter:

```
cdk deploy \  
  --app "./mvnw -e -q compile exec:java \  
    -Dexec.mainClass=dev.stratospheric.todoapp.cdk.DockerRepositoryApp" \  
  -c accountId=... \  
  -c region=... \  
  -c applicationName=...
```

With the `--app` parameter, we can define the executable that CDK should call to execute the CDK app. By default, CDK calls `mvn -e -q compile exec:java` to run an app (this default is configured in `cdk.json`, as discussed in [First Steps with CDK](#)).

Having more than one CDK app in the classpath, we need to tell Maven which app to execute, so we add the `exec.mainclass` system property and point it to our `DockerRepositoryApp`.

Now we've solved the problem of having more than one CDK app but we don't want to type all that into the command line every time we want to test a deployment, do we?

To make it a bit more convenient to execute a command with many arguments, we outsource non-sensitive configuration parameters into the `cdk.json` file:

```
{  
  "context": {  
    "accountId": "221875718260",  
    "region": "eu-central-1",  
    "applicationName": "todo-app"  
  }  
}
```

Furthermore, we will wrap the CDK call into an NPM package. For this, we create a `package.json` file that contains a script for each command we want to run:

```
{  
  "name": "stratospheric-cdk",  
  "version": "0.1.0",  
  "private": true,  
  "scripts": {  
    "repository:deploy": "cdk deploy --app ...",  
    "repository:destroy": "cdk destroy --app ..."  
  },  
  "devDependencies": {  
    "aws-cdk": "2.5.0"  
  }  
}
```

Once we've run `npm install` to install the CDK dependency (and its transitive dependencies, for that matter), we can deploy our Docker repository stack with a simple `npm run repository:deploy`.

We will add scripts for all our CDK apps into this `package.json` file over the course of this book.

The only parameter we have to pass to the `cdk` command is the `--app` parameter, since this is different for each CDK app.

Should the need arise, we can override a parameter in the command line with:

```
npm run repository:deploy -- -c applicationName=...
```

Arguments after the `--` will override any arguments defined in the `package.json` script or the `cdk.json` file. We can use this mechanism to

pass secrets and passwords to our apps and avoid committing any sensitive information to our GitHub repository.

With this `package.json` file in place, we now have a central location where we can look up the commands we have at our disposal for deploying or destroying CloudFormation stacks. Moreover, we don't have to type a lot to execute one of the commands. We'll later add more commands to this file and enrich our `cdk.json` file with new configuration parameters. You can have a peek at the complete file with all three stacks [on GitHub](#).

The Network CDK App

The next stack we're going to look at is the Network stack. The CDK app containing that step is the `NetworkApp`. You can find its code [on GitHub](#):

```
public class NetworkApp {

    public static void main(final String[] args) {
        App app = new App();

        String environmentName = (String) app
            .getNode()
            .tryGetContext("environmentName");
        requireNonEmpty(environmentName, "environmentName");

        String accountId = (String) app
            .getNode()
            .tryGetContext("accountId");
        requireNonEmpty(accountId, "accountId");

        String region = (String) app
            .getNode()
            .tryGetContext("region");
        requireNonEmpty(region, "region");

        Environment awsEnvironment = makeEnv(accountId, region);
```

```
Stack networkStack = new Stack(
    app,
    "NetworkStack",
    StackProps.builder()
        .stackName(environmentName + "-Network")
        .env(awsEnvironment)
        .build());

Network network = new Network(
    networkStack,
    "Network",
    awsEnvironment,
    environmentName,
    new Network.NetworkInputParameters());

app.synth();
}

static Environment makeEnv(String account, String region) {
    return Environment.builder()
        .account(account)
        .region(region)
        .build();
}
}
```

It's built in the same pattern as the `DockerRepositoryApp`. First, we have some input parameters, then we create a stack, and finally, we add a `Network` construct to that stack.

Let's explore this app in a bit more detail.

Managing Different Environments

The first difference from the `DockerRepositoryApp` is that we now expect an `environmentName` as an input parameter.

Remember that one of our requirements is the ability to deploy our application into different environments like staging or production. We introduced the `environmentName` parameter for precisely that purpose.

The environment name can be an arbitrary string. We use it in the `stackName()` method to prefix the name of the stack. Later, we'll see that we use it within the `Network` construct as well to prefix the names of some other resources. This separates the stack and the other resources from those deployed in another environment.

Once we've deployed the app with, say, the environment name "staging", we can deploy it again with the environment name "prod" and a new stack will be deployed. If we use the same environment name, CDK will recognize that a stack with the same name has already been deployed and update that stack instead of trying to create a new one.

With this simple parameter, we now have the power to deploy multiple networks that are completely isolated from each other.

The Network Construct

Let's take a look into the `Network` construct. This is another construct from our construct library, and you can find the full code [on GitHub](#). Here's an excerpt:

```
public class Network extends Construct {

    // fields omitted

    public Network(
        final Construct scope,
        final String id,
        final Environment environment,
        final String environmentName,
        final NetworkInputParameters networkInputParameters) {

        super(scope, id);

        this.environmentName = environmentName;

        this.vpc = createVpc(environmentName);

        this.ecsCluster = Cluster.Builder.create(this, "cluster")
            .vpc(this.vpc)
            .clusterName(prefixWithEnvironmentName("ecsCluster"))
            .build();

        createLoadBalancer(vpc, networkInputParameters.getSslCertificateArn());

        createOutputParameters();
    }

    // other methods omitted
}
```

It's structured very similar to the network stack we created in the chapter *Getting Warm with AWS*. We create a VPC and an ECS cluster to later host our application with. Additionally, we're now creating a load balancer and connecting it to the ECS cluster. This load balancer will distribute requests between multiple nodes of our application.

The load balancer takes the VPC and an optional SSL certificate as input. We have omitted an SSL certificate for now, so the load balancer will allow only

plain HTTP calls. If you want to play around with an SSL certificate, create one in the AWS Certificate Manager, copy its ARN, and pass it into the `NetworkInputParameters`, as we have done in the `NetworkApp` [on GitHub](#).

There are about 100 lines of code hidden in the `createVpc()` and `createLoadBalancer()` methods that create level 2 constructs and connections between them. But that's way better than a couple of hundred lines of YAML code, don't you think?

We won't go into the details of this code, however, because it's best looked up in the CDK and CloudFormation docs to understand which resources to use and how to use them. If you're interested, feel free to browse the code of the `Network` construct [on GitHub](#) and open up the [CDK docs](#) in a second browser window to read up on each of the resources. If the CDK docs don't go deep enough you can always search for the respective resource in the [CloudFormation docs](#).

Sharing Output Parameters via SSM

We *are*, however, going to investigate the method `createOutputParameters()` called in the last line of the constructor: What's that method doing?

Our `NetworkApp` creates a network in which we can later place our application. Other stacks – such as the Service stack, which we're going to look at next – will need to know some parameters from that network, so they can connect to it. The Service stack will need to know into which VPC to put its resources, to which load balancer to connect, and into which ECS cluster to deploy the Docker container, for example.

The question is: How does the Service stack get these parameters? We could, of course, look up these parameters by hand after deploying the Network stack, and then pass them manually as input parameters when we deploy the Service

stack. That would require manual intervention, though, which we're trying to avoid.

We could automate it by using the AWS CLI to get those parameters after the Network stack is deployed, as we did in the chapter [Getting Warm with AWS](#), but that would require lengthy and brittle shell scripts.

We opted for a more elegant solution that is easier to maintain and more flexible: When deploying the Network stack, we store any parameters that other stacks need in the [SSM parameter store](#).

And that's what the method `createOutputParameters()` is doing. For each parameter that we want to expose, it creates a `StringParameter` construct with the parameter value:

```
private void createOutputParameters(){
    StringParameter vpcId=StringParameter.Builder.create(this,"vpcId")
        .parameterName(createParameterName(environmentName,PARAMETER_VPC_ID))
        .stringValue(this.vpc.getVpcId())
        .build();

    // more parameters
}
```

An important detail is that the method `createParameterName()` prefixes the parameter name with the environment name to make it unique, even when the stack is deployed into multiple environments at the same time:

```
private static String createParameterName(  
    String environmentName,  
    String parameterName) {  
    return environmentName + "-Network-" + parameterName;  
}
```

A sample parameter name would be `staging-Network-vpcId`. The name makes it clear that this parameter contains the ID of the VPC that we deployed with the Network stack in staging.

With this naming pattern, we can read the parameters we need when building other stacks on top of the Network stack.

To make it convenient to retrieve the parameters again, we added static methods to the Network construct that retrieve a single parameter from the parameter store:

```
private static String getVpcIdFromParameterStore(  
    Construct scope,  
    String environmentName) {  
  
    return StringParameter.fromStringParameterName(  
        scope,  
        PARAMETER_VPC_ID,  
        createParameterName(environmentName, PARAMETER_VPC_ID))  
        .getStringValue();  
}
```

This method uses the same `StringParameter` construct to read the parameter from the parameter store again. To make sure we're getting the parameter for the right environment, we're passing the environment name into the method.

Finally, we provide the public method `getOutputParametersFromParameterStore()` that collects all output parameters of the Network construct and combines them into an object of type `NetworkOutputParameters`:

```
public static NetworkOutputParameters getOutputParametersFromParameterStore(  
    Construct scope,  
    String environmentName) {  
  
    return new NetworkOutputParameters(  
        getVpcIdFromParameterStore(scope, environmentName),  
        // ... other parameters  
    );  
}
```

We can then invoke this method from other CDK apps to get all parameters with a single line of code.

We pass the stack or construct from which we're calling the method as the scope parameter. The other CDK app only has to provide the `environmentName` parameter and will get all the parameters it needs from the Network construct for this environment.

The parameters never leave our CDK apps, which means we don't have to pass them around in scripts or command-line parameters!

You might remember the `Outputs` section in the CloudFormation template in chapter [Getting Warm with AWS](#) and wonder why we're not using the feature of CloudFormation output parameters. With the `CfnOutput` level 1 construct, CDK actually supports CloudFormation outputs.

These outputs, however, are tightly coupled with the *stack* that creates them, while we want to create output parameters for *constructs* that can later be composed into a stack. Also, the SSM store serves as a welcome overview of all the parameters that exist across different environments, which makes debugging configuration errors a lot easier.

Another reason for using SSM parameters is that we have more control over them. We can name them whatever we want and we can easily access them using the pattern described above. That allows for a convenient programming

model.

That said, SSM parameters have the downside of incurring additional AWS costs with each API call to the SSM parameter store. In our example application, this is negligible but in a big infrastructure it may add up to a sizeable amount.

In conclusion, we could have used CloudFormation outputs instead of SSM parameters – as always, it's a game of tradeoffs.

The Service CDK App

Let's look at the final CDK app for this chapter: `ServiceApp`. Here's most of the code. Again, you can find the complete code [on GitHub](#):

```
public class ServiceApp {

    public static void main(final String[] args) {
        App app = new App();

        String environmentName = (String) app
            .getNode()
            .tryGetContext("environmentName");
        requireNonEmpty(environmentName, "environmentName");

        String applicationName = (String) app
            .getNode()
            .tryGetContext("applicationName");
        requireNonEmpty(applicationName, "applicationName");

        String accountId = (String) app
            .getNode()
            .tryGetContext("accountId");
        requireNonEmpty(accountId, "accountId");

        String springProfile = (String) app
            .getNode()
            .tryGetContext("springProfile");
        requireNonEmpty(springProfile, "springProfile");
    }
}
```

```
String dockerImageUrl = (String) app
    .getNode()
    .tryGetContext("dockerImageUrl");
requireNonEmpty(dockerImageUrl, "dockerImageUrl");

String region = (String) app
    .getNode()
    .tryGetContext("region");
requireNonEmpty(region, region);

Environment awsEnvironment = makeEnv(accountId, region);

ApplicationEnvironment applicationEnvironment = new ApplicationEnvironment(
    applicationName,
    environmentName
);

Stack serviceStack = new Stack(
    app,
    "ServiceStack",
    StackProps.builder()
        .stackName(applicationEnvironment.prefix("Service"))
        .env(awsEnvironment)
        .build());

DockerImageSource dockerImageSource =
    new DockerImageSource(dockerRepositoryName, dockerImageTag);

NetworkOutputParameters networkOutputParameters =
    Network.getOutputParametersFromParameterStore(
        serviceStack,
        applicationEnvironment.getEnvironmentName());

ServiceInputParameters serviceInputParameters =
    new ServiceInputParameters(
        dockerImageSource,
        environmentVariables(springProfile))
        .withHealthCheckIntervalSeconds(30);

Service service = new Service(
    serviceStack,
    "Service",
```

```
        awsEnvironment,  
        applicationEnvironment,  
        serviceInputParameters,  
        networkOutputParameters);  
  
    app.synth();  
}
```

Again, its structure is very similar to that of the CDK apps we've discussed before. We extract a bunch of input parameters, create a stack, and then add a construct from our construct library to the stack – this time the `Service` construct.

Some new things are happening here, though. Let's explore them.

Managing Different Environments

In the `Network` stack, we already used an `environmentName` parameter to be able to create multiple stacks for different environments from the same CDK app.

In the `ServiceApp`, we go a step further and introduce the `applicationName` parameter.

From these two parameters, we create an object of type `ApplicationEnvironment`:

```
ApplicationEnvironment applicationEnvironment = new ApplicationEnvironment(  
    applicationName,  
    environmentName  
);
```

We use this `ApplicationEnvironment` object to prefix the name of the stack we're creating. The `Service` construct also uses it internally to prefix the names of the resources it creates.

While for the network stack it was sufficient to prefix stacks and resources with the `environmentName`, we now need the prefix to contain the `applicationName`, as well. After all, we might want to use the Service stack to deploy multiple applications into the same network.

So, given the `environmentName` “staging” and the `applicationName` “todoapp”, all resources will be prefixed with `staging-todoapp-` to account for the deployment of multiple Service stacks, each with a different application.

Accessing Output Parameters from SSM

We’re also using the `applicationEnvironment` for accessing the output parameters of a previously deployed Network construct:

```
NetworkOutputParameters networkOutputParameters =  
    Network.getOutputParametersFromParameterStore(  
        serviceStack,  
        applicationEnvironment.getEnvironmentName());
```

The static method `Network.getOutputParametersFromParameterStore()` we discussed earlier loads all the parameters of the Network construct that was deployed with the given `environmentName`. If no parameters with the respective prefix are found, CloudFormation will complain during deployment and stop deploying the Service stack.

We then pass these parameters into the Service construct so that it can use them to bind the resources it deploys to the existing network infrastructure.

Later in the book, we’ll make more use of this mechanism when we’ll be creating more stacks that expose parameters that the application needs, like a database URL or password parameters.

Pulling a Docker Image

The `Service` construct exposes the class `DockerImageSource`, which allows us to specify the source of the Docker image that we want to deploy:

```
DockerImageSource dockerImageSource =  
    new DockerImageSource(dockerImageUrl);
```

The `ServiceApp` shouldn't be responsible for defining where to get a Docker image from, so we're delegating that responsibility to the caller by expecting an input parameter `dockerImageUrl`. We're then passing the URL into the `DockerImageSource` and later pass the `DockerImageSource` to the `Service` construct.

The `DockerImageSource` also has a constructor that expects a `dockerRepositoryName` and a `dockerImageTag`. The `dockerRepositoryName` is the name of an ECR repository. This allows us to easily point to the Docker repository we have deployed earlier using our `DockerRepository` stack. We're going to make use of that constructor when we're building a continuous deployment pipeline later.

Managing Environment Variables

A Spring Boot application (or any application, for that matter), is usually parameterized for the environment it is deployed into. The parameters may differ between the environments. Spring Boot supports this through configuration profiles. Depending on the value of the environment variable `SPRING_PROFILES_ACTIVE`, Spring Boot will load configuration properties from different YAML or properties files.

If the `SPRING_PROFILES_ACTIVE` environment variable has the value `staging`, for example, Spring Boot will first load all configuration parameters from the

common `application.yml` file and then add all configuration parameters from the file `application-staging.yml`, overriding any parameters that might have been loaded from the common file already.

The `Service` construct allows us to pass in a map with environment variables. In our case, we're adding the `SPRING_PROFILES_ACTIVE` variable with the value of the `springProfile` variable, which is an input parameter to the `ServiceApp`:

```
static Map<String, String> environmentVariables(String springProfile) {  
    Map<String, String> vars = new HashMap<>();  
    vars.put("SPRING_PROFILES_ACTIVE", springProfile);  
    return vars;  
}
```

We'll add more environment variables in later chapters as our infrastructure grows.

The Service Construct

Finally, let's have a quick look at the `Service` construct. The code of that construct is a couple of hundred lines strong, which makes it too long to discuss in detail here. Let's discuss some of its highlights, though.

The scope of the `Service` construct is to create an ECS service within the ECS cluster that is provided by the `Network` construct. For that, it creates a lot of resources in its constructor (see the full code [on GitHub](#)):

```

public Service(
    final Construct scope,
    final String id,
    final Environment awsEnvironment,
    final ApplicationEnvironment applicationEnvironment,
    final ServiceInputParameters serviceInputParameters,
    final Network.NetworkOutputParameters networkOutputParameters){
    super(scope,id);

    CfnTargetGroup targetGroup=...
    CfnListenerRule httpListenerRule=...
    LogGroup logGroup=...

    ...
}

```

It accomplishes quite a bit more than the service stack from the chapter [Getting Warm with AWS](#):

- It creates a `CfnTaskDefinition` to define an ECS task that hosts the given Docker image.
- It adds a `CfnService` to the ECS cluster previously deployed in the `Network` construct and adds the tasks to it.
- It creates a `CfnTargetGroup` for the load balancer deployed in the `Network` construct and binds it to the ECS service.
- It creates a `CfnSecurityGroup` for the ECS containers and configures it so the load balancer may route traffic to the Docker containers.
- It creates a `LogGroup` so the application can send logs to CloudWatch.

You might notice that we're mainly using level 1 constructs here, i.e. constructs with the prefix `Cfn`. These constructs are direct equivalents to the CloudFormation resources and provide no abstraction over them. Why didn't we use higher-level constructs that would have saved us some code?

The reason is that the existing higher-level constructs did things we didn't want them to. They added resources we didn't need and didn't want to pay for. Hence, we decided to create our own higher-level `Service` construct out of exactly those low-level CloudFormation resources we need.

This highlights a potential downside of high-level constructs: Different software projects need different infrastructure, and high-level constructs are not always flexible enough to serve those different needs. The construct library we created for this book, for example, will probably not serve all of the needs of your next AWS project.

We could, of course, create a construct library that is highly parameterized and flexible for many different requirements. This might make the constructs complex and error-prone, though. Another option is to expend the effort to create your own construct library tailored for your project (or organization).

It's tradeoffs all the way down.

Accessing the Service

Once deployed, the service is accessible through the load balancer.

You can open the service URL from the Elastic Container Service in the AWS Console if you navigate to the *Networking* tab under “Clusters > YOUR_CLUSTER > Services > YOUR_SERVICE”.

Under “DNS names”, you'll find the URL of the load balancer. Click on “open address” to open the service in your browser:



Accessing the deployed service

Playing with the CDK Apps

If you want to play around with the CDK apps we've discussed above, feel free to clone the [GitHub repo](#) and navigate to the `cdk` folder. Then:

- run `npm install` to install the dependencies
- look into the `package.json`
- change the parameters inside the `cdk.json` (most importantly, set the account ID to your AWS account ID)
- run `npm run repository:deploy` to deploy a Docker repository
- run `npm run network:deploy` to deploy a network
- run `npm run service:deploy` to deploy the “Hello World” Todo app

Then, have a look around in the AWS Console to see the resources those commands created.

Don't forget to delete the stacks afterward, either by deleting them in the CloudFormation console, or by calling the `npm run *:destroy` scripts as otherwise, you'll incur additional costs.



As part of the [appendix](#), you'll find a detailed deployment guide for deploying the entire Todo application that we're about to create in the upcoming chapters.

7. Building a Continuous Deployment Pipeline

Deployment frequency is one of the four DORA² metrics measuring software delivery performance. Software delivery performance is highly correlated with organizational performance, so we'll want to deploy often. That means we don't want to trigger a deployment manually but instead implement a continuous deployment pipeline that pushes each change into production as quickly as possible.

To deploy quickly without manual intervention (which is error-prone) we automate the deployment. Each change in our codebase should trigger a deployment. In this chapter, we'll use the CDK project we've built in the previous chapter to build an automated continuous deployment pipeline.

While we're using GitHub Actions as the tool for building this pipeline, the concepts apply to any other CI/CD tool as well. Since we hide most of the work behind `npm` commands in our CDK project, the pipeline will just be a thin configuration layer on top of GitHub Actions and should be easily transferable to other CI/CD tools.

You might notice that with CodeDeploy and CodePipeline AWS provide their own tools to deploy applications into their cloud. We decided to use GitHub Actions instead because it allows us to create a pipeline from a config file in the code repository whereas CodeDeploy and CodePipeline both take a rather proprietary approach. Configuring a pipeline from a config file is the de-facto

²DevOps Research and Assessments. Also see "Accelerate" by Nicole Forsgren, Jez Humble, and Gene Kim.

standard these days. Thus, this approach is easier to transfer to other tools like Jenkins, CircleCI, or Bitbucket Pipelines, if necessary.

In this chapter, we'll first talk about GitHub Actions before we'll build self-serve workflows that we can use to spin up a whole new application environment, if needed. Then, we'll build a continuous deployment workflow. Finally, we'll discuss how to support continuous deployment with a codebase that gets a lot of commits.

Let's start by talking about the GitHub Actions vocabulary.

GitHub Actions Concepts

There are four main GitHub Actions concepts that we'll be using: workflows, jobs, steps, and workflow runs. The terms have been chosen quite well by the GitHub team because they are largely self-explanatory.

A **step** is the smallest unit within a CI/CD pipeline built with GitHub Actions. Ideally, it executes a single command like checking out the source code, compiling the code, or running the tests. We should aim to make each step as simple as possible to keep it maintainable. We compose multiple steps into a job.

A **job** groups multiple steps into a logical unit and puts them into a sequence. While steps within a job are executed in sequence, multiple jobs within a workflow run in parallel by default. If a job depends on the results of another job, we can mark it as dependent on that other job and GitHub Actions will run them in sequence. While all steps within a job run in the same container and filesystem, a job always starts fresh, and we will have to take care of transporting any build artifacts from one job to another, if needed.

A **workflow**, in turn, groups multiple jobs into a logical unit. While steps and

jobs are internal concepts, a workflow can be triggered by external events like a push into a repository or a webhook from some tool. A workflow can contain many jobs that run in sequence or in parallel or a mixture of both, as we see fit. GitHub Actions will show a nice visualization of the jobs within a workflow and how they depend on each other when a workflow is running.

A **workflow run**, finally, is an instance of a workflow that has run or is currently running. We can look at previous runs in the GitHub UI and see if any steps or jobs failed and look at the logs of each job.

That's enough vocabulary, for now, let's look at some real workflows!

Bootstrapping a New Environment

Let's create a workflow that bootstraps and prepares our AWS environment. You can browse the code of all workflows we're discussing [on GitHub](#):

```
name: 01 - Manually Bootstrap the CDK Environment

on:
  workflow_dispatch

env:
  AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
  AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
  AWS_DEFAULT_REGION: ${ secrets.AWS_REGION }

jobs:
  deploy:
    runs-on: ubuntu-20.04
    name: Bootstrap CDK
    if: github.ref == 'refs/heads/main'
    steps:
      - name: Checkout code
        uses: actions/checkout@v2
```

- **name:** Set up JDK 17
uses: actions/setup-java@v2
with:
 - distribution:** 'temurin'
 - java-version:** 17
 - cache:** 'maven'
- **name:** NPM install
working-directory: cdk
run: npm install
- **name:** Deploy CDK bootstrap stack
working-directory: cdk
run: npm run bootstrap
- **name:** Deploy Docker registry
working-directory: cdk
run: npm run repository:deploy

This workflow has a single responsibility: Preparing our AWS account for infrastructure and application deployments with the CDK. This includes bootstrapping the AWS environment and creating a Docker container registry with ECR.

To get to the point where we can run CDK commands, our job requires a couple of preparatory steps: checking out the code, installing Java, and running `npm install`.

We're passing in the AWS credentials as environment variables from GitHub's `secrets` context. These AWS credentials belong to a technical IAM user with admin-like privileges. This user needs sufficient permissions to create and destroy infrastructure in our AWS account. We've created this user in advance in the AWS Management console and copied over the access key ID and secret.



Visit the [appendix](#) for a list of the required permissions.

The GitHub Actions `secrets` context will make all secrets available that we have

configured in the settings of our GitHub repository. Secrets are “write-only” and cannot be viewed again after storing them. They also won’t be logged in GitHub Actions logs. Even if one of our jobs printed them to the console, they’d be masked with asterisk characters in the logs.

It’s worth noting that we’ve configured this workflow to be triggered by a `workflow_dispatch` event, which means it has to be triggered manually. GitHub Actions will show a button in the UI that allows us to trigger this workflow.

This workflow needs to be run only once for an AWS environment. Automating these steps with GitHub Actions, we now have a self-service capability for all developers in the team to bootstrap new environments.

Deploying a Shared Network

Before we can deploy our application, we need to deploy the infrastructure. We could do that manually, of course, because it is a one-off task. But wouldn’t it be great if we could make this a self-serve task for developers who want to create their own testing environment?

The first element of an environment for our app is the network stack, which can be shared across multiple applications (or instances of the same application). So, let’s create a workflow that deploys a network:

```
name: 02 - Manually create a shared environment

on:
  workflow_dispatch:
    inputs:
      environmentName:
        description: 'The name of the environment to create.'
        required: true

# ... environment variables

jobs:
  deploy-network-stack:
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    name: Deploy the network stack
    steps:
      # ... preparatory steps

      - name: Deploy network stack
        working-directory: cdk
        run: |
          npm run network:deploy -- \
            -c environmentName=${{ github.event.inputs.environmentName }}
```

The main step calls the `network:deploy` command of the CDK project we've built in the chapter [Designing a CDK Project](#).

As with bootstrap workflow before, we're using the `workflow_dispatch` event to manually trigger this workflow. Since we define an additional input parameter `environmentName` the UI will also show us an input field for defining the name of the environment we want to create a network for. We pass that input parameter into the `environmentName` parameter of the `network:deploy` command.

Any developer in the team can trigger this workflow to either create or update their own network infrastructure. They just need to navigate to the workflow in their browser and click a button to deploy their own network, should they need

it. We can also use this workflow to update an existing network, for example, if we have changed something in the network stack in our CDK project. We just have to pass in the correct `environmentName` and GitHub Actions. CDK and CloudFormation will do the work for us.

Deploying an Application Environment

Our Todo application needs more infrastructure than just a network, though. In later chapters, we'll integrate a database, messaging systems, and authentication infrastructure as well. Other applications may require their own database instance or other stateful infrastructure.

To create this application-specific infrastructure, we build a workflow that is very similar to the shared network workflow in the previous section:

```
name: 03 - Manually create the Todo-App environment

on:
  workflow_dispatch:
    inputs:
      environmentName:
        description: 'The name of the environment to deploy the resources to.'
        required: true

jobs:
  deploy-messaging-stack:
    ...
  deploy-database-stack:
    ...
  deploy-cognito-stack:
    ...
```

Same as before, this workflow is triggered manually and takes an `environmentName` as an input parameter.

This workflow has a couple of jobs. Each job contains steps very similar to the steps of the workflow in the previous section, but each with a different command. One job might call `npm run deploy:database` to deploy the database, and another might call `npm run deploy:messaging` to spin up a stack with an ActiveMQ instance. We'll create the CDK apps behind these commands in later chapters of this book. Since these stacks are independent of each other, we can deploy them in parallel. Hence, we put each stack into its own job.

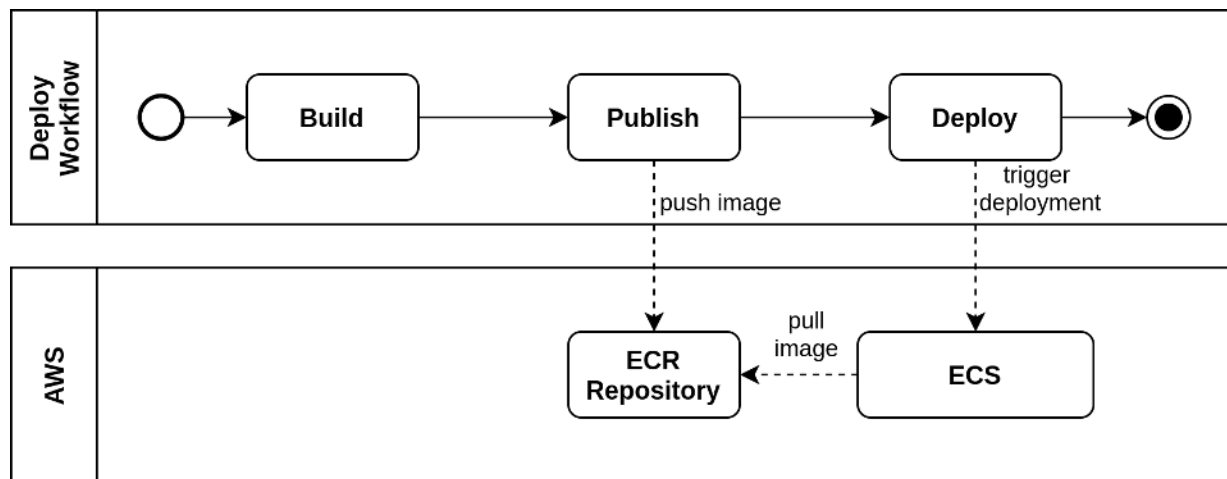
With the two workflows we have built so far, a developer can spin up their own environment to deploy their own instance of the Todo application into. Since workflows are triggered manually, the environment won't update automatically when we change something in the corresponding CDK apps. However, we can easily modify the workflows to be triggered by a change in the CDK code to automatically redeploy the infrastructure in the staging and production environments, for example. We have chosen not to do that, though, because the infrastructure tends not to change very often – and if it does change, we want to have a human involved to assess the impact.

With the infrastructure up and running, we can finally create a continuous deployment pipeline that deploys the application with every code change.

Building a Continuous Deployment Workflow

Let's say we've executed all the workflows we've discussed so far and created a complete environment. Now it's time to deploy our Todo app into this environment.

We'll now build a "Deploy" workflow, which will look like this on paper:



The “Deploy” workflow builds, publishes, and deploys the application.

The workflow will contain:

- a “Build” step that compiles the code and runs the tests,
- a “Publish” step that creates a Docker image and publishes it to an ECR repository, and
- a “Deploy” step that calls the CDK project to deploy that Docker image into the infrastructure we’ve created earlier.

Here’s the skeleton of our continuous deployment GitHub Actions workflow:

```
name: 04 - Publish Todo-App
```

```
on:
```

```
  push:
```

```
    paths:
```

- ```
 - 'application/**'
 - 'cdk/**/*Service*'
 - 'cdk/pom.xml'
```

```
 workflow_dispatch:
```

```
jobs:
```

```
 build-and-deploy:
```

```
 ...
```

It's not triggered by a `workflow_dispatch` event as the previous workflows are, but instead by a push to the codebase. However, we don't want to trigger a deployment if a README file changes (or any other file in the repository that is not part of the application). Every deployment blocks the deployment pipeline for a couple of minutes and we don't want to block it unnecessarily. So, we configure the workflow to only trigger when something changes within the application folder and or in some select files in our `cdk` folder.

The workflow has only one job named `build-and-deploy`, which implements all three steps from the diagram above: "Build", "Publish", and "Deploy". We could have built a separate job for each of the steps, but since we want to share the output of one step with the following steps, this would have made the workflow a bit more complicated.

Let's have a look at each of the three steps.

## The "Build" Step

The "Build" step is rather straightforward:

```
- name: Build application
 working-directory: application
 run: ./gradlew build --stacktrace
```

We execute the Gradle build task inside the `application` folder. Gradle takes care of running the tests and packaging our Spring Boot application into a JAR file.

## The "Publish" Step

Once the application has been packaged, the next step in the workflow is to create and publish a Docker image:

```

- name: Create Docker image tag
 id: dockerImageTag
 run: echo "tag=$(date +%Y%m%d%H%M%S')-${GITHUB_SHA}" >> $GITHUB_OUTPUT

- name: Publish Docker image to ECR registry
 if: github.ref == 'refs/heads/main'
 env:
 DOCKER_IMAGE_TAG: ${ steps.dockerImageTag.outputs.tag }
 working-directory: application
 run: |
 docker build -t todo-app .
 docker tag todo-app our.ecr.amazonaws.com/todo-app:${DOCKER_IMAGE_TAG}
 docker tag todo-app our.ecr.amazonaws.com/todo-app:latest
 aws ecr get-login-password --region ${AWS_REGION} \
 | docker login --username AWS --password-stdin our.ecr.amazonaws.com
 docker push our.ecr.amazonaws.com/todo-app:${DOCKER_IMAGE_TAG}
 docker push our.ecr.amazonaws.com/todo-app:latest

```

These are actually two steps: One for creating a tag for the Docker image, and one for creating and publishing the Docker image.

The “Create Docker image tag” step just builds a string from the current date and time and the Git commit SHA hash. We can use that string to uniquely tag our Docker image.

The “Publish Docker image to ECR registry” step then calls a bunch of different commands that:

- create a Docker image from the `Dockerfile` in the `application` folder,
- tag that image with the Docker image tag created in the previous step,
- also tag that image with the `latest` tag,
- get the login credentials for our ECR repository,
- use these credentials to have Docker login to that ECR repository, and finally,
- push the image and both tags to the repository.

If this step has finished successfully, we now have an up-to-date Docker image in our ECR repository, ready to deploy.

The required AWS environment variables are configured once at the top of the workflow and hence implicitly available for each job execution:

```
env:
 AWS_ACCOUNT_ID: ${ secrets.AWS_ACCOUNT_ID }
 AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
 AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
 AWS_DEFAULT_REGION: ${ secrets.AWS_REGION }
```

So far, this is a *continuous delivery* workflow. With every change to the code, we deliver a new version of the application to a Docker repository.

To have a *continuous deployment* workflow, the “deploy” part is missing, yet.

## The “Deploy” Step

Thanks to our CDK project, the “Deploy” step is also rather straightforward:

```
- name: Deploy service stack
 if: github.ref == 'refs/heads/main'
 working-directory: cdk
 run: |
 npm run service:deploy -- \
 -c environmentName=staging \
 -c applicationName=todo-app \
 -c dockerImageTag=${GITHUB_RUN_NUMBER}
```

It merely calls the `npm run service:deploy` command which we have created in the chapter [Designing a CDK Project](#) to update the service stack with the latest Docker image.

## Reviewing the Continuous Deployment Pipeline

With the “Build”, “Publish”, and “Deploy” steps in place, we now have a working continuous deployment pipeline. Each time we push a change into the `main` branch of our GitHub repository, it will trigger the pipeline and deploy a new version of the application to the cloud. Changes to other branches than `main` will merely run the “Build” part of the pipeline because we don’t want to publish new Docker images for branch builds.

The “Deploy” step, in particular, will take some time to finish in each workflow run, though. When we update the service stack, the ECS cluster will automatically start new nodes with the new application version and then remove the nodes with the old version. Depending on the configuration, this may take up to 15 minutes.

What happens, however, if we push two or more commits to the code repository within a 15-minute time frame? Both commits will trigger the GitHub Actions workflow and successfully run the “Build” and “Publish” steps. But the workflow run that was triggered second will fail in the “Deploy” step because CDK/CloudFormation will detect that another deployment is currently in progress. The second workflow run will have to be manually triggered again once the deployment has finished.

So, while this solution is probably totally fine for many projects, it is not suited for projects with a high frequency of commits. A lot of workflow runs will fail and there is no guarantee that a commit will be deployed promptly.

In the next section, we’ll discuss a solution to this problem.

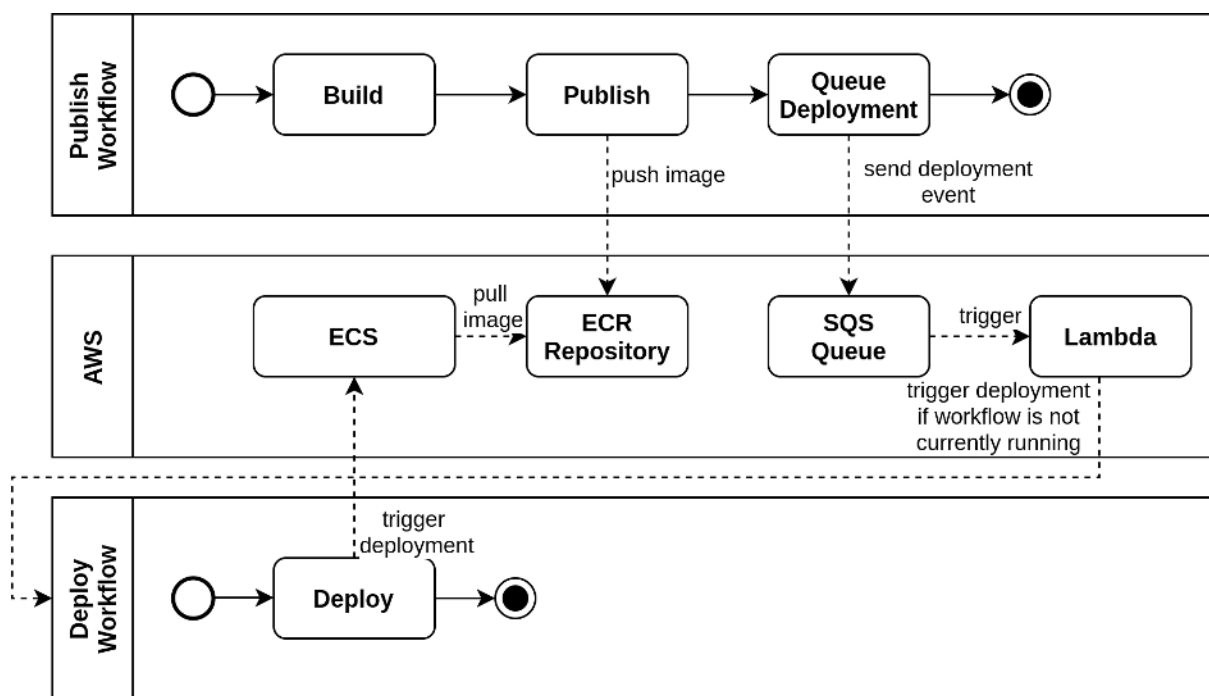
## Supporting High-Frequency Deployments with Amazon SQS and AWS Lambda

Let's try and extend the continuous deployment pipeline we've built so far to fix the problem with frequent commits. While a deployment is currently running, we don't want to start a second deployment because it would fail anyway. Instead, we want to wait until the current deployment is finished, and then start a new deployment to deploy the latest version of the application that was published to the Docker registry in the meantime.

GitHub Actions provides a [“Concurrency” feature](#) for this use case. This allows us to assign a “concurrency group” to each job. All jobs within the same concurrency group will be queued if another job with the same concurrency group is already running. Currently queued jobs will be cancelled if a new job is queued. This feature does exactly what we want.

However, not all CI tools provide a concurrency feature like this. Anyway, we want to show how we can plug together SQS, Lambda functions, and third-party APIs to solve a problem. So, we're going to build our own queuing solution that is usable with any CI tool.

This is what we're going to build:



Deploying only the latest version of a Docker image with the help of SQS and Lambda.

Instead of only one GitHub workflow, we now have two: a “Publish” workflow, and a “Deploy” workflow.

The “Publish” workflow is very similar to before, except for the last step. Instead of calling our CDK app to deploy the service stack, we send an event to an Amazon SQS queue that is dedicated to receiving deployment requests.

Each time a deployment request arrives in the queue, it will be processed by an AWS Lambda function we’re going to develop. This Lambda function will then trigger a deployment only if there is currently no deployment running, effectively sequencing the deployments one after another, even if there were many commits in a short period.

The Lambda will trigger the “Deploy” workflow via the GitHub API, which finally uses our CDK app to deploy the service stack.

In the following sections, we’ll look at all those new elements.



## Building a Sequencer Lambda Function

Let's start with building the Lambda function. We chose TypeScript as the programming language and Node.js as the Lambda runtime. We could have chosen any other language supported by AWS Lambda as well, but we want to showcase the flexibility we have with Lambdas.

Our AWS Lambda handler function processes all incoming SQS messages like this:

```
export const handler = async (e: SqsEvent): Promise<any> => {
 const queueUrl = process.env.QUEUE_URL as string;
 const region = process.env.REGION as string;
 const githubToken = process.env.GITHUB_TOKEN as string;
 const event = new SqsEventWrapper(e);
 const latestDeploymentEvent: DeploymentEvent = event.getLatestDeploymentEvent();
 const github = new GitHub(githubToken);
 const queue = new DeploymentQueue(queueUrl, region);

 console.log(`Received event: ${JSON.stringify(latestDeploymentEvent)}`);

 // If there are more events in the queue: finish and wait for the next event.
 if (await queue.hasWaitingEvents()) {
 console.log(
 "Skipping this event because there are more events waiting in the queue!"
);
 return;
 }

 // If the GitHub workflow is currently running: retry this event later
 if (await github.isWorkflowCurrentlyRunning(latestDeploymentEvent)) {
 console.log(
 "GitHub workflow is currently running - retrying at a later time!"
);
 throw "retrying later!";
 }

 // Triggering the GitHub workflow.
 await github.triggerWorkflow(latestDeploymentEvent);
};
```

We're not going into the details of all the functions we're calling but you can browse the code in its entirety [on GitHub](#).

A Lambda must always declare exactly one handler function which is invoked when the Lambda is activated. In our case, the Lambda will be activated each time there is a new deployment event in our deployment queue. In Typescript/JavaScript, this handler function is simply a function that we export in our main file (`index.ts`).

The handler function takes in an `SqsEvent`, which is just a JSON structure for which we declared a type ourselves. This type describes the shape of an incoming SQS event. Most importantly, an SQS event has the field `Records`, which contains one or more events, each with a body that contains the actual payload from an SQS event. We expect the body of an event to have this shape:

```
{
 "commitSha": "674e5044b3b269ccb8f4530193cc144f2d6a5ae6",
 "ref": "main",
 "owner": "stratospheric-dev",
 "repo": "stratospheric",
 "workflowId": "05-update-todo-app-in-staging.yml",
 "dockerImageTag": "20210320051451-674e5044b3b269ccb8f4530193cc144f2d6a5ae6"
}
```

Each event contains the name of the Docker image that needs to be deployed.

At the top of our handler function, we read some parameters from environment variables like the URL of the SQS queue and an API token to access the GitHub API. We'll see later how to configure them.

The main code of the handler function consists of just three steps:

First, we check if the SQS queue has more events waiting in line. If there is at least one other event waiting, we just skip the current event. The idea is that in this case, we don't want to deploy the Docker image of this event since

there already is a more recent Docker image waiting to be deployed in the queue. Instead, we just deploy that more recent Docker image. An important detail for this to work is that the SQS queue has to be configured as a FIFO queue (first in first out). This means that the order of the events reaching the Lambda function is guaranteed to be the order in which the events were received by the SQS queue. For more details about how SQS works, have a look at the chapter [Sharing Todos with SQS and SES](#).

If there is no other event waiting in line we check if there is a current deployment running. We're calling the GitHub API to see if the "Deploy" workflow is currently running. We use the `ref`, `owner`, `repo`, and `workflowId` fields of the event and the `GITHUB_TOKEN` environment variable to build a request to GitHub. If the workflow is currently running we throw an error. If the Lambda invocation exits with an error it signals to the SQS queue that the event has not been processed successfully, and it will be tried again after some time.

Finally, if there is no other event waiting in line, and the "Deploy" workflow is currently not running, we trigger the "Deploy" workflow via the GitHub API.

## Provisioning the Queue and Lambda with CDK

Now that we have the code for the Lambda ready, we need to deploy the Lambda and the SQS queue. We could do that manually, of course, but since we already have a CDK project up and running, we can just as well add another stack to it to make it all automatic and repeatable.

So, we're adding a `DeploymentSequencerApp` to our CDK project which creates a `DeploymentSequencerStack`. Here's the whole code for the stack:

```

public DeploymentSequencerStack(
 final Construct scope,
 final String id,
 final Environment awsEnvironment,
 final String applicationName,
 final String githubToken) {

 super(scope, id, StackProps.builder()
 .stackName(applicationName + "-Deployments")
 .env(awsEnvironment).build());

 this.deploymentsQueue = Queue.Builder.create(this, "deploymentsQueue")
 .queueName(applicationName + "-deploymentsQueue.fifo")
 .fifo(true)
 .build();

 SqsEventSource eventSource = SqsEventSource.Builder.create(deploymentsQueue)
 .build();

 this.deploymentsLambda = LambdaFunction.Builder.create(new Function(
 this,
 "deploymentSequencerFunction",
 FunctionProps.builder()
 .code(Code.fromAsset("./deployment-sequencer-lambda/dist/lambda.zip"))
 .runtime(Runtime.NODEJS_12_X)
 .handler("index.handler")
 .reservedConcurrentExecutions(1)
 .events(singletonList(eventSource))
 .environment(Map.of(
 "GITHUB_TOKEN", githubToken,
 "QUEUE_URL", deploymentsQueue.getQueueUrl(),
 "REGION", awsEnvironment.getRegion()
))).build()
).build();
}

```

We create an SQS queue with `fifo` set to `true`. Note that the name of a FIFO queue always needs to have the suffix “.fifo”, otherwise there will be an error during deployment.

Then, we create a `LambdaFunction` with the code from our Lambda project. The code of the Lambda is packaged within the `lambda.zip` archive in the `deployment-sequencer-lambda/dist` folder. We have to make sure to build a new `.zip` archive each time before we're deploying the `DeploymentSequencerStack`.

For the runtime, we choose Node 12 and as the handler we provide `index.handler` because the handler function is called `handler` and it's located in the file `index.ts` (or rather `index.js` after it has been transpiled from TypeScript into JavaScript).

Importantly, we set `reservedConcurrentExecutions` to 1. We do this because we don't want multiple instances of the Lambda processing SQS events at the same time. We want to have a single instance that controls the sequencing of the deployment events. If we had multiple instances processing deployment events concurrently, the logic in the Lambda code wouldn't work.

Next, we add an `SqsEventSource` to the Lambda. This connects the SQS queue with the Lambda. Every event in the queue will now trigger the Lambda function.

Finally, we add some environment variables that the Lambda code needs to work. We've already seen how the Lambda function reads these environment variables in the Lambda code above.

That's all the resources we need to support our idea of sequencing deployments.

To make it convenient to deploy the new stack, we add new scripts to the `package.json` of our CDK project:

```
{
 ...
 "scripts": {
 ...
 "deployment-sequencer:deploy": "cdk deploy \"*\" --app ...",
 "deployment-sequencer:destroy": "cdk destroy \"*\" --app ..."
 }
}
```

Then, we can simply run `npm run deployment-sequencer:deploy` and `npm run deployment-sequencer:destroy` from the command line or from a CI workflow to deploy the deployment queue and Lambda.

You can find the complete code of the `DeploymentSequencerApp` [on GitHub](#).

## Splitting Up the “Publish” Workflow

The missing piece now is how the deployment events end up in the deployment SQS queue. For this, we have to modify our existing “Deploy” workflow.

Previously, we had one workflow with the steps “Build”, “Publish”, and “Deploy”. We’re going to move the “Deploy” step into its own workflow so it can be triggered by our Lambda. The new “Deploy” workflow looks like this:

```

name: 05 - Update the Todo-App in staging

on:
 workflow_dispatch:
 inputs:
 docker-image-tag:
 description: "The Docker image to deploy"
 required: true

env:
 AWS_ACCOUNT_ID: ${ secrets.AWS_ACCOUNT_ID }
 AWS_ACCESS_KEY_ID: ${ secrets.AWS_ACCESS_KEY_ID }
 AWS_SECRET_ACCESS_KEY: ${ secrets.AWS_SECRET_ACCESS_KEY }
 AWS_DEFAULT_REGION: ${ secrets.AWS_REGION }

jobs:
 deploy-app:
 runs-on: ubuntu-20.04
 name: Deploy Todo App
 steps:
 - name: Checkout code
 uses: actions/checkout@v2

 - name: NPM install
 working-directory: cdk
 run: npm install

 - name: Create or update service stack
 if: github.ref == 'refs/heads/main'
 working-directory: cdk
 run: |
 npm run service:deploy -- \
 -c environmentName=staging \
 -c applicationName=todo-app \
 -c dockerImageTag=${ github.event.inputs.docker-image-tag }

```

This workflow is now triggered exclusively via `workflow_dispatch`, which means that we can trigger it manually or via the GitHub API. In our case, the Lambda will trigger the workflow via the GitHub API. It will pass the input parameter `docker-image-tag` so that the workflow knows which Docker image

to deploy.

The rest of the workflow looks very similar to what we had before. In the call to our CDK app, we're now using the special GitHub Actions variable `github.event.inputs` to pass the Docker image tag to the CDK app.

Now, all that's left to do is to modify the previous "Deploy" workflow to become the new "Publish" workflow. Instead of deploying the Docker image, the "Publish" workflow will now just send a deployment request to the SQS queue after publishing a Docker image:

```
name: 04 - Publish Todo-App

on:
 push:
 paths:
 - 'application/**'
 - 'cdk/**/*Service*'
 - 'cdk/pom.xml'
 workflow_dispatch:
jobs:
 build-and-publish:
 runs-on: ubuntu-20.04
 name: Build and publish Todo App
 steps:
 - name: Create Docker image tag
 id: dockerImageTag
 run: echo "tag=$(date +%Y%m%d%H%M%S')-${GITHUB_SHA}" >> $GITHUB_OUTPUT

 - name: Publish Docker image to ECR registry
 if: github.ref == 'refs/heads/main'
 ...

 - name: Sending deployment event to queue
 if: github.ref == 'refs/heads/main'
 env:
 DOCKER_IMAGE_TAG: ${ steps.dockerImageTag.outputs.tag }
 run: |
 export EVENT_PAYLOAD="\
```



```
{\"commitSha\": \"${GITHUB_SHA}\", \\
 \"ref\": \"main\", \\
 \"owner\": \"stratospheric-dev\", \\
 \"repo\": \"stratospheric\", \\
 \"workflowId\": \"05-update-todo-app-in-staging.yml\", \\
 \"dockerImageTag\": \"${DOCKER_IMAGE_TAG}\"}
aws sqs send-message \
 --queue-url=https://.../todo-app-deploymentsQueue.fifo \
 --message-group-id default \
 --message-deduplication-id $GITHUB_SHA \
 --message-body \"$EVENT_PAYLOAD\"
```

We’ve added the step “Sending deployment event to queue” which first creates an environment variable with the name `EVENT_PAYLOAD` containing a JSON structure with the information the Lambda needs to process a deployment event. Then, we use the AWS CLI to send this event to the deployment SQS queue we provisioned previously.

That’s it! Every time the publish workflow has successfully published a Docker image with a new version of our application, it will now send the name of that Docker image to the deployment queue, which will forward it to our Lambda, which in turn will decide if and when to call our deployment workflow with a given Docker image tag.

## Reviewing the Continuous Deployment Pipeline

You might think it’s overkill to create a hand-rolled solution as we did above to make sure that deployments are executed in sequence. And you’re right if you have access to tools that can do it for you. If not, however, investing a couple of days of work into this kind of automation will avoid manual triggering and re-triggering of failing deployments for the whole team and will pay off in the long run.

With the power of CDK, we can also package the solution into a construct library

and make it available to all teams in the company, paying off the investment of time even quicker.

Please note that the solution described here is not foolproof. The queue and the Lambda make sure that deployments are triggered in the same sequence in which they arrive in the queue. If two builds triggered by two commits are running at roughly the same time, and the build triggered by the earlier commit takes longer than the build triggered by the later commit, then the Lambda function might drop the deployment request for the Docker image that contains the later commit and will only deploy the Docker image that contains the earlier commit. To solve this, some kind of persistent data store would be needed for storing the commit sequence, which then could be looked up each time a deployment event is received.

# Addendum: Configuring HTTPS and a Custom Domain with Route 53 and ELB

Once deployed, we can access our application via HTTP and an IP address. Remember chapter [Getting Warm with AWS](#), where we used the AWS CLI to get the public IP address of our deployed application? Using this approach, the URL of our application might look something like this:

```
http://13.55.30.162
```

Alternatively, we can use the address of the load balancer that AWS automatically generates for us, like this one:

```
https://staging-loadbalancer-1376672807.eu-central-1.elb.amazonaws.com/
```

However, while this might be fine for development or testing purposes, no user wants to type this URL into their browser. We usually want to serve our production application via a custom domain name like `app.stratospheric.dev`.

A memorable (sub)domain name is better in terms of user experience, branding, marketing, and search engine optimization (SEO).

Additionally, we want to keep our users' data safe. That means we want to use HTTPS and Transport Layer Security (TLS)/Secure Sockets Layer (SSL) instead of plain HTTP. This is pretty much a standard nowadays and users expect it.

In this chapter, we'll add a custom domain to our application and create and install an SSL certificate for that domain.

We'll also enforce HTTPS so that even when the app is accessed via plain HTTP, the request is redirected to the HTTPS URL.

AWS allows us to manage DNS settings, SSL certificates, and HTTP redirect rules by using Amazon Route 53, AWS Certificate Manager, and Elastic Load Balancing, respectively.

Before we dive into how to configure all this with AWS, however, let's learn a bit about DNS to put everything in context.

## Domain Name System (DNS)

Having originated in 1983, the Domain Name System (DNS) is one of the core systems and protocols that make the internet work like we're used to. It's the service that allows us to access websites via memorable domain names such as `stratospheric.dev` rather than IP addresses.

While naming machines and services on the internet certainly is the most common use case for DNS, it can also be used for naming entities on private TCP/IP-based networks such as company intranets.

Although it's such a vital part of network infrastructure, we often take DNS for granted. We don't want to go into too much detail here but provide a short recap of how DNS works.

DNS is a hierarchical, distributed system for naming resources. On the internet, there are thirteen root name servers, which are responsible for the network's root domain. Each top-level domain (TLD) such as `.com` or `.dev` has a TLD name server. Each second-level domain (for example `stratospheric.dev`) in turn has an authoritative name server associated with it.

Clients each use a DNS resolver software, usually provided by the client's

operating system, for mapping domain names to IP addresses.

When a user requests a resource under a specific second-level domain name, say `stratospheric.dev`, that domain name is resolved like this:

1. The client's DNS resolver queries a root name server for the TLD `.dev`.
2. The DNS resolver then issues a request to that TLD name server to obtain the authoritative name server for `stratospheric.dev`.
3. Finally, the DNS resolver asks this authoritative name server for the IP address of the machine serving resources for the domain `stratospheric.dev`. That machine can be a single server or a gateway/reverse proxy, which itself routes requests to multiple machines (as is the case with the VPC and the internet gateway for our sample Todo application).

With this information assembled, the client is now able to retrieve resources from a network address without requiring the user to enter that address.

There are various types of DNS entries, for example:

- **A records:** Address records for mapping domains to IP addresses like in the example above.
- **CNAME records:** Aliases for mapping from one domain to another.
- **TXT records:** Arbitrary string resources commonly used for mail exchange policies such as SPF, DKIM, and DMARC.

Amongst other uses, TXT records provide a convenient approach for verifying ownership of a second-level domain. Whoever controls the DNS entries for a domain ultimately controls the resources served from that domain and thus can prove their ownership by changing a TXT record.

We will later make use of this for verifying our identity when requesting an SSL certificate for a domain.

## HTTPS and Transport Layer Security (TLS)

Though not quite as old as a DNS, SSL (first publicly released by Netscape in 1995), its successor TLS, and the HTTPS protocol it enables are indispensable ingredients of the modern internet.

TLS – and its predecessor SSL – is a cryptographic protocol for enabling secure communication over a computer network. Even though it’s largely agnostic in terms of the communication protocols it can be applied to, its most widely known application is with Hypertext Transfer Protocol Secure (HTTPS). HTTPS in turn is an extension of HTTP that allows us to encrypt data in transit.

More specifically, HTTPS can be described as **HTTP over TLS**: Communication still is the responsibility of the HTTP we know and love while TLS enables encryption and cryptographic verification of identity.

Through TLS, HTTPS provides security via asymmetric encryption and public-private key cryptography.

Generally, as the name suggests, these algorithms work through key pairs of public and private keys. In the most basic incarnation of public-private key encryption, when encrypting a message for a receiver the sender uses the receiver’s public key in a trapdoor function. A trapdoor function is a function that – similar to a real-life trapdoor – provides an “easy way out” for those who know that there is a trapdoor (and where that trapdoor is located). A trapdoor function’s result (i.e., the encrypted message) is easy to compute, whereas its inverse (i.e., the decrypted message) is difficult to compute without additional information (the receiver’s secret key). A message encrypted in that manner is hard to decrypt without knowledge of the corresponding private key. If that private key is known, however, decryption becomes trivially easy. Hence, to decrypt the encrypted message and obtain the original plaintext message, the

receiver simply uses a complementary private key in an inverse operation.

Only public keys ever are publicly exchanged (hence the name). Since knowledge of the public key alone doesn't allow an attacker unauthorized access this doesn't compromise security.

Although asymmetric encryption, in general, has been mathematically proven to be a secure means of transport (given sufficient key lengths) one gnarly issue remains: How do we verify that the entity we encrypted a message for actually is the entity we believe it to be? In other words: How do we verify identity? If we omitted to do so this would make us susceptible to man-in-the-middle (MITM) attacks. With this category of attacks, an attacker eavesdrops on the communication by pretending to be the legitimate receiver of a message.

If, for example, the machines serving our sample Todo application were compromised, a malicious actor could pretend to be the legitimate entity behind `app.stratospheric.dev` and try to siphon off user credentials.

This is where trusted **certificate authorities** (CAs) come into play. Within TLS, these CAs are licensed to issue cryptographically signed certificates confirming the ownership of a particular public key. Alongside the public key, these certificates contain information associated with that key such as the domain name in question and the contact details of the person the certificate has been issued to. Such certificates are commonly known as **SSL certificates**. Ownership can be confirmed in various ways. As mentioned above, a popular way for proving ownership for a domain is to modify a DNS TXT record.

While it's possible to have self-signed certificates, i.e., certificates that are cryptographically signed by their owner instead of an independent certificate authority, this would leave us open to MITM attacks. Anyone can self-sign an SSL certificate and try to claim an identity or the ownership of a domain. The only way to verify that claim is through an independent and trusted third party.

Nevertheless, self-signed certificates tend to be used quite often in company-internal settings, for example on intranets. In those cases, the company itself becomes the trusted authority for internal resources. For those use cases, this might be acceptable depending on the exact requirements and additional security measures for preventing intrusion. For the internet at large, though, that isn't an option since the parties involved don't necessarily know or trust each other.

With this background, we've now laid the groundwork for working with DNS, HTTPS, and SSL/TLS certificates. So, let's dive into configuring the DNS and SSL settings for our application's domain name.

## Registering or Transferring a Domain

Since a valid domain name is a prerequisite for a verified SSL certificate, we first need to create Domain Name System (DNS) records for our custom domain. The tool to do this within the AWS ecosystem is Route 53.

The name "Route 53" is a play on the UDP port 53, which is the default port for the DNS protocol.

With Route 53, we can either register a new domain or [transfer an existing one](#) from a different domain registrar like GoDaddy, Namecheap or Hetzner. The [Route 53 console](#) provides wizards for guiding us through either of those processes.

Once our desired custom domain has been registered with or transferred to Route 53, it will provide a **hosted zone** for us to configure.



The screenshot shows the AWS Route 53 console for a hosted zone named `stratospheric.dev`. The interface includes buttons for `Delete zone`, `Test record`, and `Configure query logging`. Below the zone details, there are tabs for `Records (18)`, `DNSSEC signing`, and `Hosted zone tags (0)`. The `Records (18)` tab is active, showing a table of DNS records. The table has columns for `Record name`, `Type`, `Routin...`, `Differ...`, and `Value/Route traffic to`. The records listed are:

| Record name       | Type | Routin... | Differ... | Value/Route traffic to                                                                               |
|-------------------|------|-----------|-----------|------------------------------------------------------------------------------------------------------|
| stratospheric.dev | A    | Simple    | -         | 185.199.108.153<br>185.199.109.153<br>185.199.110.153<br>185.199.111.153                             |
| stratospheric.dev | MX   | Simple    | -         | 10 inbound-smtp.eu-west-1.amazonaws.com                                                              |
| stratospheric.dev | NS   | Simple    | -         | ns-1967.awsdns-53.co.uk.<br>ns-853.awsdns-42.net.<br>ns-362.awsdns-45.com.<br>ns-1305.awsdns-35.org. |
| stratospheric.dev | SOA  | Simple    | -         | ns-1967.awsdns-53.co.uk. awsdns-hostmaster.amazon.com. 1 7200 900 120                                |

### Configuring a hosted zone in Route 53.

A hosted zone is a container for the DNS records of a specific domain within Route 53. In the screenshot above we can see the records that belong to the domain `stratospheric.dev`. These records define the routing behavior for our domain beyond our internet gateway. For example, the MX – or mail exchanger – records specify how email traffic to and from our domain is supposed to be routed.

## Creating an SSL Certificate with CDK

We now have an overview of how DNS and SSL work. We have seen how these technologies can be used to assign a domain name to our application and at the same time encrypt the entire incoming and outgoing traffic for our application.

We can use the hosted zone mentioned above to apply routing and HTTPS-related settings to our application.

Usually, these configuration options have to be applied only once during the

lifetime of an application.

However, automating this process still is a good idea because codifying a process implicitly documents a process and makes that process reproducible across different environments.



For the upcoming example, it's mandatory to have the domain controlled by Route53 for the automatic DNS validation to work. For creating an SSL certificate for a domain that is not part of Route53, please follow the official AWS documentation and [manually validate the SSL certificate](#). As soon as you have the ARN for your verified SSL certificate, continue with [the next section](#).

So, let's have a look at how to automate – and document – making the necessary settings for our use case with CDK.

For this task, we create a new CDK app called `CertificateApp`:

```
public class CertificateApp {

 public static void main(final String[] args) {
 App app = new App();

 // ...

 String hostedZoneDomain = (String) app
 .getNode()
 .tryGetContext("hostedZoneDomain");
 String applicationDomain = (String) app
 .getNode()
 .tryGetContext("applicationDomain");

 // ...

 new CertificateStack(
 app,
 "certificate",
 awsEnvironment,
```

```
 applicationEnvironment,
 applicationDomain,
 hostedZoneDomain
);

 app.synth();
}
}
```

This app itself doesn't do a lot besides initializing a `CertificateStack` instance.

It does take the parameters `hostedZoneDomain` and `applicationDomain`, though. These parameters allow us to flexibly set the hosted zone as well as the desired domain for the application, so we don't have to hardcode it into our infrastructure code.

The `hostedZoneDomain` refers to the name of the hosted zone within Route53. For our example, that's `stratospheric.dev`. With `applicationDomain`, we specify the domain name for which we want to create the SSL certificate. This will be the domain our users will access our application by later on, for example, `app.stratospheric.dev`.

We supply these parameters within our `cdk.json` file and add two new scripts in our CDK project's `package.json`: `certificate:deploy` and `certificate:destroy`

```
{
 "scripts": {
 "certificate:deploy": "cdk deploy --app ... ",
 "certificate:destroy": "cdk destroy --app ..."
 }
}
```

To be able to automate Route53 and Certificate Manager actions we need to add the relevant dependencies to our CDK project's `pom.xml`:

```
<!-- Both imports can be omitted when using cdk-constructs -->
<dependency>
 <groupId>software.amazon.awscdk</groupId>
 <artifactId>aws-cdk-lib</artifactId>
 <version>${aws-cdk-lib.version}</version>
</dependency>
<dependency>
 <groupId>software.constructs</groupId>
 <artifactId>constructs</artifactId>
 <version>${constructs.version}</version>
</dependency>
```

When including the `cdk-constructs` dependency in our project, we can omit the dependency imports above as both dependencies will be transitively available via the `cdk-constructs` library.

In our new `CertificateStack` we use these dependencies for creating and automatically verifying the SSL certificate.

First, we retrieve the hosted zone we created while registering or transferring our domain:

```
IHostedZone hostedZone = HostedZone.fromLookup(
 scope,
 "HostedZone",
 HostedZoneProviderProps.builder()
 .domainName(hostedZoneDomain)
 .build()
);
```

Remember that a hosted zone is a container for all the DNS records belonging to a domain. We use the `hostedZoneDomain` parameter we provided as an input parameter to our `CertificateApp` to look it up.

Next, we create a new SSL certificate validated via DNS (a DNS-validated certificate):

```
DnsValidatedCertificate websiteCertificate = DnsValidatedCertificate.Builder
 .create(this, "WebsiteCertificate")
 .hostedZone(hostedZone)
 .region(awsEnvironment.getRegion())
 .domainName(applicationDomain)
 .build();
```

A DNS-validated certificate (also known as a **domain-validated certificate**) makes use of the fact that only the owner of a domain has control over its DNS entries. Using this approach, the issuer of the certificate sends us a unique string. By showing that we're able to create a DNS TXT record for our domain with that unique value, we can prove that we have control over the DNS settings for that domain.

Note how this complex, multi-step process requires just one high-level construct from the Certificate Manager construct library. This construct does all the heavy lifting such as exchanging cryptographic keys or creating DNS TXT records for us.

As soon as we now deploy this CDK stack with `npm run certificate:deploy`, we'll get the ARN of the SSL certificate printed to the console:

```
❏ certificate (staging-todo-app-Certificate)
```

Outputs:

```
certificate.sslCertificateArn = arn:aws:acm:eu-central-1:...:certificate/...
```

This output parameter will be important for the following steps as we're about to create an HTTPS listener for our load balancer.

## Creating an HTTPS Listener Using the Network App

So far we've deployed the `NetworkApp` without any information about our SSL certificate.

We've designed the underlying `Network` construct in a way that it only creates an HTTP listener for the ELB if no SSL certificate is passed. AWS doesn't provide SSL certificates for their default ELB domains. Whenever we want to create an HTTPS listener, we have to provide a valid SSL certificate.

As soon as we're passing an `sslCertificateArn` to the `NetworkApp`, the `Network` construct will create two ELB listeners. The default listener for port 80 (HTTP) and one for port 443 (HTTPS) using the SSL certificate:

```
// ... creating an HTTP listener by default

if (sslCertificateArn.isPresent()) {
 IListenerCertificate certificate = ListenerCertificate
 .fromArn(sslCertificateArn.get());

 httpsListener = loadBalancer.addListener("httpsListener",
 BaseApplicationListenerProps.builder()
 .port(443)
 .protocol(ApplicationProtocol.HTTPS)
 .certificates(Collections.singletonList(certificate))
 .open(true)
 .build()
);

 httpsListener.addTargetGroups("https-defaultTargetGroup",
 AddApplicationTargetGroupsProps.builder()
 .targetGroups(Collections.singletonList(dummyTargetGroup))
 .build());
}
```

We first fetch the `ListenerCertificate` based on the `sslCertificateArn` parameter, then pass it to the construction of our HTTPS listener. As we did when creating the HTTP listener, we add a `dummyTargetGroup` for the default target group. The actual target group that points to our ECS service will be added as soon as we redeploy our `ServiceApp`.

Serving our application with this setup from both port 80 and 443 might have some users access our application via HTTP. To avoid any insecure traffic, we should therefore enforce the secure option and only allow access via HTTPS. For this to work, we automatically redirect any unencrypted traffic to HTTPS:

```
ListenerAction redirectAction = ListenerAction.redirect(
 RedirectOptions.builder()
 .protocol("HTTPS")
 .port("443")
 .build()
);

ApplicationListenerRule applicationListenerRule = new ApplicationListenerRule(
 this,
 "HttpListenerRule",
 ApplicationListenerRuleProps.builder()
 .listener(httpListener)
 .priority(1)
 .conditions(List.of(ListenerCondition.pathPatterns(List.of("*")))
 .action(redirectAction)
 .build()
```

We attach the additional `ApplicationListenerRule` to our existing HTTP listener. Using priority 1, this will overrule any existing listener rule and hence enforce HTTPS communication.

To apply these changes for our already deployed `NetworkApp`, we add the SSL certificate ARN to the context section of our `cdk.json` using the `sslCertificateArn` key.

What's left to do is to trigger a redeployment of our network and service apps:

```
npm run network:deploy
npm run service:deploy
```

After this redeployment, our load balancer listens for traffic on both port 80 and 443. Any HTTP traffic will be automatically redirected to port 443 to ensure encrypted communication for our users.

So far, any traffic from and to our application is secured with TLS. However, our users still have to use the default ELB domain and can't access the application from our custom domain `app.stratospheric.dev`.



Let's fix this next.

## Associating a Custom Domain For the ELB

Up until now, users won't be able to access our application when entering `app.stratospheric.dev` in the browser. Route53 isn't aware (so far) of our app subdomain and our intent to route the traffic to our load balancer.

What's missing is a DNS A record for this subdomain that points to the ELB.

Let's see how we can create this A record using the CDK.



The upcoming CDK app works for any domain that is managed within Route53. When using another domain registrar, you have to create this record manually.

As our goal is to automate as much of our infrastructure setup as possible, we create a new CDK app called `DomainApp`. Similar to the `CertificateApp`, the `DomainStack` depends on the `applicationDomain` and the `hostedZoneName`:

```
public class DomainApp {

 public static void main(final String[] args) {
 App app = new App();

 // ...

 String hostedZoneDomain = (String) app
 .getNode()
 .tryGetContext("hostedZoneDomain");
 String applicationDomain = (String) app
 .getNode()
 .tryGetContext("applicationDomain");

 // ...
 }
}
```

```
 new DomainStack(
 app,
 "domain",
 awsEnvironment,
 applicationEnvironment,
 hostedZoneDomain,
 applicationDomain
);

 app.synth();
}

}
```

For a convenient deployment and cleanup, we extend our `package.json` with two new commands for this stack:

```
{
 "scripts": {
 "domain:deploy": "cdk deploy --app ... ",
 "domain:destroy": "cdk destroy --app ..."
 }
}
```

Inside the `DomainStack`, we first retrieve our ELB instance using the parameters stored in the SSM parameter store by our previously redeployed `Network` construct:

```
Network.NetworkOutputParameters networkOutputParameters =
 Network.getOutputParametersFromParameterStore(
 this,
 applicationEnvironment.getEnvironmentName()
);

IApplicationLoadBalancer applicationLoadBalancer = ApplicationLoadBalancer
 .fromApplicationLoadBalancerAttributes(
 this,
 "LoadBalancer",
 ApplicationLoadBalancerAttributes.builder()
 .loadBalancerArn(
 networkOutputParameters.getLoadBalancerArn()
)
 .securityGroupId(
 networkOutputParameters.getLoadBalancerSecurityGroupId()
)
 .loadBalancerCanonicalHostedZoneId(
 networkOutputParameters.getLoadBalancerCanonicalHostedZoneId()
)
 .loadBalancerDnsName(
 networkOutputParameters.getLoadBalancerDnsName()
)
 .build()
);
```

We have to use the method `fromApplicationLoadBalancerAttributes()` for retrieving our existing load balancer. There is another method `fromLookup()` but it requires actual string values instead of the placeholder tokens returned via CDK `StringParameters`, so we can't use it in our case.

Finally, we create a DNS A record (mapping a domain name to an IP address) for our application domain and point it to our application's ALB via the ALB's ARN:

```
ARecord aRecord = ARecord.Builder.create(this, "ARecord")
 .recordName(applicationDomain)
 .zone(hostedZone)
 .target(
 RecordTarget.fromAlias(new LoadBalancerTarget(applicationLoadBalancer)))
 .build();
```

What remains to be done is to run `npm run domain:deploy` to create an A record for our subdomain app within Route53. Our application might not be immediately accessible, as the DNS change might take some minutes until it's propagated to DNS servers around the world.

With these two CDK apps, that is `CertificateApp` and `DomainApp`, we've now set up our DNS and SSL certificate configuration as code. This way, the infrastructure related to our application's domain configuration can be reproduced and regenerated, if need be.

From now on, the sample Todo application is accessible from our custom domain using HTTPS: `https://app.stratospheric.dev`.

With this production-ready infrastructure in place, now it's time to implement some features!

# Part II: Spring Boot & AWS

Now that we have a deployment pipeline set up we can start implementing an application on top of it!

In this part of the book, we'll introduce our example Todo application and get ready for local development.

Then, we'll implement some features that integrate with AWS services.

We'll start with setting up user registration and login with Cognito and integrate that into our Spring Boot application.

Then, we'll talk about RDS, Amazon's service for relational databases, and use it to create a PostgreSQL database for our application.

Two important use cases for any application are queueing and sending emails, so we'll integrate with Amazon SQS and SES to implement a collaboration feature that notifies a user when a Todo has been shared with them.

To showcase a more advanced use case, we will implement push notifications on top of Amazon MQ and WebSocket that notify a user in real time about a collaboration request in the Todo app – without reloading the page!

Finally, we'll introduce Amazon DynamoDB as a NoSQL store and use it to trace user actions in our application.

In each chapter, we'll extend the CDK project we've created in Part I so that in

the end, we'll have a continuous deployment pipeline that creates all the AWS infrastructure we need with the click of a button.

## 8. The Sample Todo Application

To explore the various AWS features and best practices we're going to create a sample Spring Boot web application.

Todo applications are something of a gold standard for comparing language or framework features. The business logic of such an application is simple, which allows us to focus on the language or framework at hand.

Hence, we decided to create yet another todo web application, which will serve as a continuous example as we move along.

The actual business logic of the application doesn't really matter, in the end. It's just a means for presenting various AWS features and how we can use them in the context of a real-world Spring Boot application.

We suggest you look at the application's code [on GitHub](#) in parallel to reading this chapter.

### Features

To start, let's get an overview of the Todo application's features and the AWS services we'll use to implement them in an AWS cloud environment.

### Registration and Login

The sample application is a multi-user application that allows each user to have their own set of todos. Since users would usually want to keep their todos

private, we need a means for separating todos by their respective owners. We will use Spring Security in conjunction with Amazon Cognito to achieve this and to perform both user authentication and authorization.

Users will be identified by their email addresses. The user data itself will be stored in a Cognito user pool. Our application will use OIDC (OpenID Connect, an authentication framework on top of OAuth2) for retrieving and maintaining the user's Cognito session.

### **CRUD: Viewing, Adding, and Deleting Todos**

Any application dealing with user-editable data, in this case, todos, has to provide some sort of CRUD (Create, Read, Update, Delete) functionality.

Users can create todos and add text notes to those todos. They can see their todos on the screen and edit or delete them.

For storing the data, we'll be using a PostgreSQL database running on Amazon's Relational Database Service (RDS).

### **Sharing Todos and Email Notifications**

Sometimes users might not just want to work on their tasks alone but to collaborate with other users as well. Therefore, the sample application allows users to share their todos with others through email notifications sent via Amazon Simple Email Service (SES) and Amazon Simple Queue Service (SQS).

### **Push Notifications**

When sharing Todos and collaborating with others, we want to stay up-to-date and be notified about changes in real time. To show that kind of functionality



the sample application uses WebSockets and a managed [Apache ActiveMQ](#) message broker running on Amazon MQ to notify a todo owner right in the browser once a collaboration request for a todo has been accepted.

## Application Architecture

The application is a simple Spring Boot application that tries to follow common Spring and Spring Boot patterns. This book first and foremost is about running a Spring Boot application on AWS, after all. We don't want to get lost in technical details or design considerations that are not relevant to this book's topic.

This chapter will give you an understanding of the application we're going to deploy to AWS, so we don't get caught up in details as we make progress.

The `TodoApplication` class serves as the entry point.

## Configuration

There's a `dev.stratospheric.config` package containing multiple configuration classes. One example is the `WebSecurityConfig` class that configures Spring Security.

Complementing these configuration classes are the default Spring Boot YAML configuration files in the `resources` folder, which contain the basic settings required to run the application. These YAML configuration files come for both the local development environment (`application-dev.yml`) and an AWS environment (`application-aws.yml`) with a default `application.yml` for those configuration attributes that are common to both environments.

## Features

For the features themselves, the application follows a package-by-feature structure. Hence, the feature folders `collaboration`, `person`, `registration`, and `todo` contain the code artifacts related to these respective features. These code artifacts encompass controllers, service interfaces (and their implementations), Spring Data JPA repositories, and data model classes.

## User Interface

Since Thymeleaf is very well integrated with Spring Boot, we decided to use it for rendering the views to the user. The Thymeleaf HTML templates and static files can be found in the `resources` folder, as is common practice.

## Storage

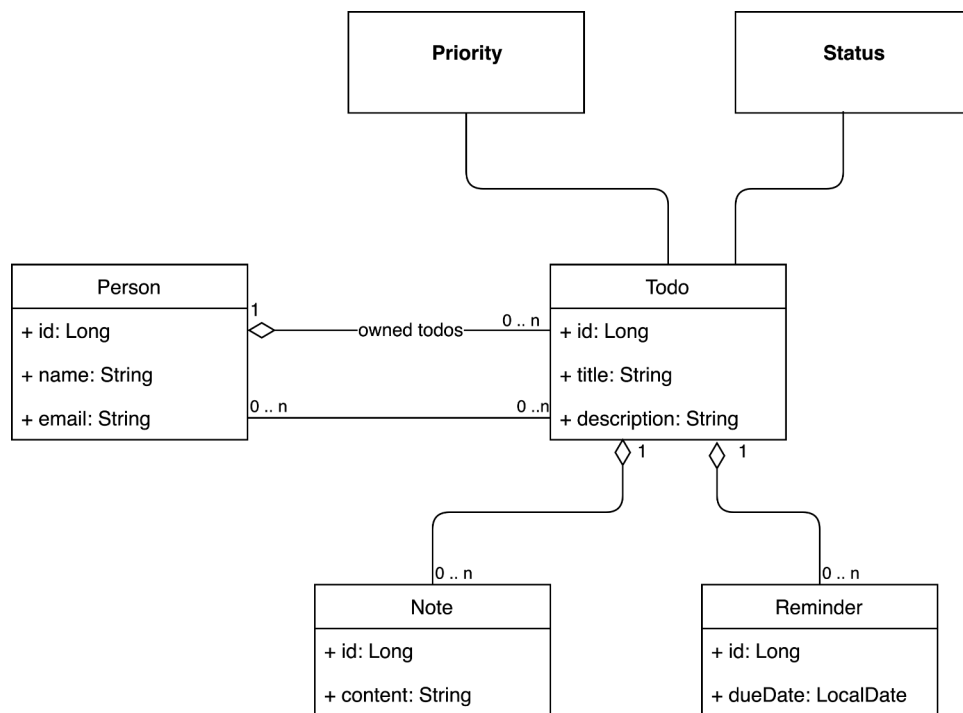
The application uses Spring Data JPA to store data in a PostgreSQL database. We use Flyway for setting up and subsequently migrating the application database to the state required by the current application source code.

The `resources` folder contains a sub-directory named `db/migration/postgresql` with the Flyway migration SQL scripts for setting up the application database.

## Domain Model

Let's have a look at the domain model, which we kept rather simple.

The domain model is structured around the “Todo” entity, as shown in this diagram:



**The domain model of the Todo application.**

A todo item has a priority and a status, so that we can handle them differently depending on those attributes. We can add text notes and reminders to a todo item, and a person can own todos.

## Skeleton Application Setup

Let's take a look at the technical foundation of our application's architecture and features. We'll first explain the bare minimum setup to get our Todo application running on AWS and serving one Thymeleaf view.

### Core Dependencies

What do all excellent Spring Boot projects have in common? They were all bootstrapped from [start.spring.io](https://start.spring.io). So was our Todo application. Let's take a look

at the configuration and the core dependencies of our project skeleton.

Apart from choosing Gradle as the project's build tool and Java 17 as the programming language, our project includes the following Spring Boot/Cloud starters:

```
implementation 'org.springframework.boot:spring-boot-starter-web'
implementation 'org.springframework.boot:spring-boot-starter-thymeleaf'
implementation 'org.springframework.boot:spring-boot-starter-validation'
implementation 'io.awspring.cloud:spring-cloud-aws-starter'
```

Those starters have the following purposes:

- **Spring Boot Starter Web:** The core of our application. It includes and auto-configures Tomcat as an embedded Servlet container for our Spring MVC application.
- **Spring Boot Starter Thymeleaf:** This starter includes all relevant Thymeleaf dependencies and auto-configures the ViewResolver to render Thymeleaf templates.
- **Spring Boot Starter Validation:** As of Spring Boot 2.4, the validation component is no longer part of spring-boot-start-web. Hence, we have to include it explicitly whenever we want to validate incoming payloads using Bean Validation.
- **Spring Cloud Starter AWS:** The main integration point for integrating our Spring Boot application with AWS.

The Spring Cloud AWS project takes care of aligning the AWS Java SDK version (v2 since Spring Cloud AWS 3.0, previously v1) for us by using the software.amazon.awssdk:bom internally. We can override the AWS Java SDK version but we would then have to ensure its compatibility ourselves:

```
dependencyManagement {
 imports {
 // explicitly define the AWS Java SDK v2 version
 mavenBom "software.amazon.awssdk:bom:2.20.4"
 }
}
```

Even though we only have one Spring Cloud dependency (so far), we're using the BOM of Spring Cloud to align the dependency versions.



A BOM (Bill of Materials) is a Maven concept to align dependency versions for efficient dependency management. It's a special kind of POM that defines dependency versions at a central place. The main goal is to avoid incompatibilities that may occur when we include different versions of related dependencies. In case we're using a BOM, we can include a dependency to our project without specifying its version explicitly. Many Java projects/libraries/frameworks that publish multiple artifacts use this concept (for example Spring Cloud, AWS Java SDK, JUnit 5, and Testcontainers).

The Spring Cloud AWS BOM ensures that all (upcoming) dependencies that are part of the Spring Cloud AWS umbrella share the same version:

```
ext {
 set('awsSpringVersion', '3.0.0-RC1')
}

dependencyManagement {
 imports {
 mavenBom "io.awspring.cloud:spring-cloud-aws-dependencies:${awsSpringVersion}"
 }
}
```



On the 17th of April 2020, [the Spring Cloud Team announced](#) that Spring Cloud AWS will no longer be part of the Spring Cloud umbrella and the corresponding release train. Spring Cloud AWS found a new home at [awspring](#) and is now a community project driven by core committers [Maciej Walkowiak](#), [Eddú Meléndez](#) and [Matej Nedic](#).

Apart from the Spring Boot Starter and Spring Cloud AWS, we're including the following frontend-related dependencies:

```
implementation 'nz.net.ultraq.thymeleaf:thymeleaf-layout-dialect:2.5.3'
implementation 'org.webjars:webjars-locator-core'
implementation 'org.webjars:bootstrap:4.6.1'
implementation 'org.webjars:font-awesome:5.15.3'
```

The `thymeleaf-layout-dialect` dependency allows creating reusable layouts for our Thymeleaf views. We'll take a look at our basic layout in one of the next sections.

As our frontend needs proper styling, we're using so-called [WebJars](#) that bundle web libraries (e.g. Bootstrap's CSS or JavaScript) into JAR files. We can then serve these static resources as part of our Spring Boot application.

With each upcoming feature of our Todo application, we'll add more dependencies to this setup. As our core dependencies are now in place, we can continue configuring the relevant AWS configuration for our application.

## AWS-Specific Configuration

First, we need to configure access to our AWS account. The [AWS SDK for Java](#) already offers several solutions for this, such as using environment variables, a property file, or loading them from the [Amazon EC2 Instance Metadata Service](#). Technically speaking, they are implementations of the `AwsCredentialsProvider` interface that are part of the `aws-java-sdk-core` dependency.

With Spring Cloud AWS, we can also configure our AWS credentials in the “Spring Boot way”. That is, we can store the credentials inside our `application.yml` by defining these properties:

- `spring.cloud.aws.credentials.secret-key`, and
- `spring.cloud.aws.credentials.access-key`.

However, the Elastic Container Service (ECS) populates an environment variable (`AWS_CONTAINER_CREDENTIALS_RELATIVE_URI`) for all our running containers. This path points to our credential’s location for the IAM role attached to our ECS task. The AWS SDK ships with a provider (`ContainerCredentialsProvider`) that can retrieve the credentials for this setup.

As this comes with the least configuration overhead, we’ll favor this approach. We have no additional configuration effort as Spring Cloud AWS configures a `DefaultCredentialsProvider`. Upon application start, the AWS Java SDK will traverse this provider chain to search for AWS credentials.

This provider chain will detect the underlying ECS runtime and configure access using the `AwsCredentials` interface. In case the provider chain can’t find AWS credentials at any lookup site, our application won’t start.

What's left is to configure the AWS region we deploy our application to. Spring Cloud AWS can automatically detect this based on our environment using the `DefaultAwsRegionProviderChain`. This provider chain tries to resolve the AWS region from various locations:

- a system property called `aws.region`
- an environment variable called `AWS_REGION`
- a configuration file within `~/.aws`
- the EC2 Metadata Service

As we're using ECS on Fargate (meaning we don't specify our own EC2 instances, because Fargate does that for us), the [metadata retrieval works slightly different](#). Unfortunately, this is not supported by Spring Cloud AWS.

Apart from the provider chain mentioned before, Spring Cloud AWS also offers a static region configuration option as part of our `application.yml` file. For our use case, that's sufficient as we deploy our application to a dedicated region:

```
spring:
 cloud:
 aws:
 region:
 static: eu-central-1
```

For a more flexible deployment and region agnostic setup, we can also populate the `AWS_REGION` environment variable as part of our Service CDK construct.

## Our First Thymeleaf View

Our skeleton application already has one public endpoint that exposes a Thymeleaf view:



```
@Controller
public class IndexController {

 @GetMapping
 public String getIndex() {
 return "index";
 }

}
```

This Spring MVC @Controller resolves the index view located inside src/main/resources/templates:

```
<!DOCTYPE html>
<html lang="en"
 xmlns:th="http://www.thymeleaf.org"
 xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
 layout:decorate="~{layout/layout}"
 th:with="activeMenuItem='start', headline='Welcome to the Todo Application!'">
<head>
 <title>Start</title>
</head>
<section class="section" layout:fragment="page-content">
 <div class="container">
 <p>There's not much to see here (yet).</p>
 </div>
</section>
</html>
```

There's not much to explain for this index view as we're merely rendering a static message. What's worth mentioning, though, is the usage of the layout namespace. As we want to reuse the same page structure for all our views we're going to define a general layout.

The `th:with` defines variables that we can refer to in our Thymeleaf fragments or layout.

Our layout, located inside `src/main/resources/templates/layout`, defines the basic structure of every upcoming page:

```

<!DOCTYPE html>
<html lang="en"
 xmlns="http://www.w3.org/1999/xhtml"
 xmlns:layout="http://www.ultraq.net.nz/thymeleaf/layout"
 xmlns:th="http://www.thymeleaf.org">

 <!--/*@thymesVar id="headline" type="java.lang.String"*/-->

 <head>
 <meta charset="UTF-8">
 <title layout:title-pattern="$CONTENT_TITLE | $LAYOUT_TITLE">
 Todo Application
 </title>

 <meta content="IE=edge" http-equiv="X-UA-Compatible"/>
 <meta content="width=device-width, initial-scale=1" name="viewport"/>

 <link rel="icon" th:href="@{/favicon.svg}">

 <link rel="stylesheet" th:href="@{/webjars/bootstrap/css/bootstrap.min.css}">
 <link rel="stylesheet" th:href="@{/webjars/font-awesome/css/all.css}">
 <link rel="stylesheet" th:href="@{/styles.css}">

 <script th:src="@{/webjars/jquery/jquery.min.js}"></script>
 <script th:src="@{/webjars/popper.js/umd/popper.min.js}"></script>
 <script th:src="@{/webjars/bootstrap/js/bootstrap.min.js}"></script>
 </head>
 <body>
 <div>
 <div th:replace="fragments/header :: header"></div>

 <div th:replace="fragments/toast :: toast"></div>

 <div class="container has-text-centered">
 <h1 th:text="${headline}" class="title text-center"></h1>
 <div th:replace="fragments/messages :: messages"></div>
 <div layout:fragment="page-content"></div>
 </div>

 <div th:replace="fragments/modals :: confirmDeletionModal"></div>
 <div th:replace="fragments/footer :: footer"></div>
 </div>
 </body>

```

```
</html>
```

We include CSS and JavaScript resources that are required for each view and structure the page with a set of fragments. These fragments have several purposes: display the top navigation bar, the footer, render info messages, and a confirmation modal.

Those fragments are part of `src/main/resources/templates/fragments`.

Furthermore, we're referencing the `${headline}` variable that each view defines to render a headline for the page.

With `layout:fragment`, we then specify where the actual content of the view is supposed to be displayed.



For a great Spring Boot & Thymeleaf reference with real-world examples (pagination, internationalization, testing, security, etc.) and in-depth explanation, take a look at Wim Deblauwe's [Taming Thymeleaf](#) book.

## Docker Image

As we deploy our Todo application to a container runtime on AWS, we have to create a Docker image for our application. The `Dockerfile` from the chapter [Getting Warm with AWS](#) is our foundation:

```
FROM eclipse-temurin:17-jre

ARG JAR_FILE=build/libs/*.jar
COPY ${JAR_FILE} app.jar

ENTRYPOINT ["java", "-jar", "/app.jar"]
```



The AdoptOpenJDK project [moved to the Eclipse Foundation](#). Among the Eclipse Adoptium umbrella, they're now publishing Java runtimes called Temurin. Except the name, this Docker image has no relationship with the Eclipse IDE.

## GitHub Repository Overview

We manage all source code examples for this book as part of the [Stratospheric GitHub repository](#). The repository contains:

- the CDK apps for deploying the AWS infrastructure (in the `cdk` folder)
- the continuous deployment pipeline (in the folder `.github/workflows`)
- the code of the Todo application in its final state (in the `application` folder)
- the code of the Todo application's state for each chapter (in the `chapters` folder) and their CloudFormation and CDK setup, respectively

It also contains some CloudFormation templates for the AWS infrastructure that we may use as an alternative to the CDK apps (in the `cloudformation` folder).

Go ahead and check out the code. Make sure to have everything from the [Prerequisites](#) chapter installed.

## Building the App

To build and test the app, change to the application directory in your command line and run this default Gradle command:

```
./gradlew build
```

This will automatically download the appropriate version of Gradle and then run Gradle to build and test the app.

If you see `BUILD SUCCESSFUL` after some time, you're good to go!

## Running the App in Dev Mode

The application connects to a PostgreSQL database and several AWS services. To start the application on a local machine, we somehow need access to those services locally, too.

That's a non-trivial task, and there are several solutions for this. In the upcoming chapter [Local Development](#) we take a closer look at how local development works. For now, it's enough to know that we use Docker Compose.

In the application folder, run `docker-compose up` to spin up every infrastructure component we need.

Then, to start our app locally, run this Gradle command, also from the application folder:

```
./gradlew bootRun
```

This command makes use of the Spring Boot plugin for Gradle and starts the Spring Boot application.

You can have a look at the Gradle configuration in the `build.gradle` file.

## Build Setup and Continuous Deployment

The folder `.github/workflows` contains files for the GitHub Actions workflows we created in the chapter *Building a Continuous Deployment Pipeline* that deploy the application to AWS with each code change. The application will only be deployed to AWS if there's a new Git push to the main branch. Pushing to a feature branch will merely result in an application build run (including tests).

## 9. Local Development

In this chapter, we'll discuss how we can develop and start the Todo application locally. Having a convenient local development setup allows every developer to get immediate feedback for code changes and everyone to play with the application to get used to it.

As the main topic of this book is AWS, we can expect our application to integrate multiple AWS services. Whenever we start the application on our local machines, we need to decide how to interact with these AWS services.

Are we going to mock the services for local development? Are we calling the real services? Can we somehow run them locally? Probably not, because the whole AWS ecosystem is too big to fit on our local machines.

Our main goal is to make local development as convenient as possible. Having a big list of setup steps with multiple scripts to execute is not only bad for developer experience but also requires keeping the list up-to-date. We all know the frustration of working with a new project, following the instructions in the README, and then realizing that they are outdated.

### The Challenges of Local Cloud Development

Let's take a look at different options for making local development against AWS services as convenient as possible.

First, we could connect to real AWS services. This requires our entire AWS setup to be up and running. With this option, we would be as close to production as

possible, and our application wouldn't need any modification.

However, there are some major drawbacks to this approach. If we only have one AWS environment, developers could interfere with the production runtime, and real users could see funny (or not so funny) test data. We can fix this by providing a separate test environment or, ideally, an on-demand environment for each developer. This incurs additional costs for the usage of AWS services. It'd also be inconvenient for developers because they'd need an AWS account and be required to spend some AWS money just to start the application locally.

Another approach would be to disable parts of our application that require communication with AWS (for example, with feature toggles). That, in turn, would mean we wouldn't be able to subscribe to a queue, read from a DynamoDB table, or access any other AWS resources when running the application with the development profile. While this is technically possible and requires no additional setup, our local application would be different from the production runtime with this approach. New features that involve AWS services couldn't be tested locally and would require blind commits.

Furthermore, we'd end up with additional complexity to our application as most of our features would have two flavors: one for production runtime in AWS and one for local development. We also wouldn't be able to see the full beauty of the Todo application locally because not all features would work. We don't want this.

Let's see if we can combine the two options we just discussed.

We somehow need an AWS mock that allows starting and using most of the Todo application features. This should not involve significant cost and fit on our machines. The approach ideally needs little to no manual setup and is available on all operating systems. We can also live with the fact that a small subset of our features works slightly differently. Sending emails is a good example of



this. While we could configure a local SMTP server, we can probably do without delivering emails for local development. Fortunately, there is a solution to this: **LocalStack**.

## LocalStack - Our Local AWS Cloud

LocalStack is ...

“A fully functional local AWS cloud stack.”

This project was formerly owned by Atlassian but is now a standalone and open-source project. The main goal of LocalStack is to provide an easy-to-use test and mocking framework for developing cloud applications.

With LocalStack, we can spin up core AWS services on our local machine to test our application without any interaction with the actual AWS cloud.

There are multiple ways to install and start LocalStack, but we are going to use their official [Docker image](#).

The following command starts LocalStack and exposes the AWS S3 service on port 4566

```
docker run -p 4566:4566 -e SERVICES=s3 localstack/localstack:0.14.4
```

To now make use of this local AWS S3 while working with the AWS CLI, we have to pass `localhost:4566` as the `--endpoint-url`:

```
aws s3api create-bucket \
 --bucket local-s3-bucket \
 --endpoint-url http://localhost:4566

aws s3api put-object \
 --bucket local-s3-bucket \
 --key any_file.pdf \
 --body any_file.pdf \
 --endpoint-url http://localhost:4566
```

With the two AWS CLI commands above, we first create an S3 bucket and then upload a file to this local S3 bucket. We don't need to configure any specific AWS credentials as LocalStack doesn't use the IAM authentication mechanism. Every `AWS_ACCESS_KEY` and `AWS_SECRET_KEY` will be accepted (or rather: ignored) by LocalStack.

This implies that we have to override our Java AWS SDK clients to connect to LocalStack rather than AWS when starting the application locally. With Spring Cloud AWS, we can conveniently override the endpoint URLs and regions for all AWS SDK clients:

```
spring:
 cloud:
 aws:
 endpoint: http://localhost:4566
 region:
 static: eu-central-1
```

When it comes to configuring the AWS credentials for the local setup, we hard-code the AWS credentials. Otherwise, the application wouldn't start since the `DefaultCredentialsProvider` wouldn't be able to find any AWS credentials as we're not simulating a local ECS runtime:

```
spring:
 cloud:
 aws:
 # above configuration
 credentials:
 secret-key: foo
 access-key: bar
```

By default, LocalStack bypasses any AWS client authentication. Hence, any secret and access key combination is accepted.

What's left is to create our resources (for example, an S3 bucket or SQS queue) once LocalStack is up and running. This has to happen before starting the Todo application. The LocalStack Docker container executes any shell script that is part of the `/docker-entrypoint-initaws.d` folder once all services are available.

We can make use of this feature and create a script to initialize our infrastructure:

```
#!/bin/sh

awslocal sqs create-queue --queue-name stratospheric-todo-sharing

... more infrastructure to setup
```

The `awslocal` binary is a wrapper around the real AWS CLI that is part of the LocalStack container. This tiny wrapper is already configured to point to the mocked AWS services inside the container.

We'll also make heavy use of LocalStack when it comes to testing our application. More about this in a later chapter.

## Local Amazon RDS & Amazon Cognito

LocalStack works great for the majority of AWS services and their basic features. There are AWS services that are not part of the [LocalStack community edition](#), however. Our application, for example, will use an RDS PostgreSQL for persistence and Amazon Cognito as an identity provider, both of which are only supported by the [pro version](#) of LocalStack. As we can't expect every reader to spend money on the pro version of LocalStack, we'll choose an alternative (even though you still should support this project if you are using it).

Replacing RDS is simple. We can use an official PostgreSQL Docker image as we won't use many RDS-specific features.

For Amazon Cognito, we are going to choose a different identity provider. As we are going to use OIDC (OpenID Connect, an identity layer on top of OAuth 2.0 protocol – more in the upcoming chapter [Building User Registration and Login with Cognito](#)) we are making use of a standardized protocol. The good thing about standards is that we can replace an existing provider with another compatible solution.

Therefore, we'll use [Keycloak](#), an open-source identity and access management solution, for local development. We'll pre-configure the Keycloak instance with users, clients, and other settings. Hence, there's no manual setup involved.

## Bringing It All Together

As we don't want to start every Docker container one by one with `docker run`, we'll use `docker-compose`. With some upcoming chapters, we'll extend our `docker-compose.yml` file with additional containers or settings.

When developing, you only need to execute `docker-compose up` before starting the application with `./gradlew bootRun`.

In the end, we'll have two application profiles:

- `aws`: for running the application on AWS in production mode.
- `dev`: for local development, connecting to local Docker containers, and overriding the AWS clients to point to `localhost`.

With Spring Boot, we can separate both configurations with two files: `application-aws.yml` and `application-dev.yml`. In addition to this, we'll have an `application.yml` that configures defaults for both profiles.

For the JavaScript and CSS libraries that our frontend requires, we are using [Webjars](#). This means that there's no need to download any external resource from a CDN as everything is bundled and served by our application.

This setup allows working with the Todo application in “airplane mode”. No internet connection is required anymore after an initial pull of all dependencies and Docker images.

With this approach, we recognize that our local setup is not 100% mirroring the real AWS setup. There might still be differences, especially when it comes to load and performance. However, this setup is “good enough” and easy to set up and maintain.

# 10. Building User Registration and Login with Amazon Cognito

With this chapter, we're adding the first features to our Todo application: user registration and login. Managing user pools is a common task of every enterprise application. Whenever we deal with such security concerns, we're usually better off using an off-the-shelf solution rather than reinventing the wheel.

To take this sensitive task off our hands, we chose an AWS service that we can seamlessly integrate with our Spring Boot application.

Before we jump right into configuring our Todo application, we'll first discuss important terminology and the formal specifications (OAuth 2.0 & OpenID Connect 1.0) that make all of this work.

## What's OAuth2?

[OAuth 2.0](#) (Open Authorization) is ...

“... the industry-standard protocol for authorization. OAuth 2.0 focuses on client developer simplicity while providing specific authorization flows for web applications, desktop applications, mobile phones, and living room devices.”



While *authentication* is the process of verifying someone's identity (for example by validating credentials), *authorization* is about determining which level of access a user has for specific resources or operations (for example only admin users are allowed to create new users).

This standard currently exists in two versions: OAuth 1.0 and OAuth 2.0. Both address the same idea, but version 2 simplifies the integration and offers more flexibility. An in-depth comparison of both versions is available on the [OAuth 2.0 Simplified](#) blog.

As this book's focus is on Spring Boot and AWS, we'll only cover the parts of OAuth that are required to understand the integration between our app and Amazon Cognito. There are excellent resources [available online](#) to dive deeper into the topic. Marco Behler, for example, has put together [a comprehensive guide](#) that focuses on OAuth 2.0 and Spring Security.

Most of us probably have seen OAuth 2.0 in action at some point in time when using modern SaaS applications. Often, it's used behind the scenes when we grant access to a third-party application's resources or operations.

A good example is the CI/CD platform [Travis CI](#), which we can use to build and deploy code from our GitHub projects. After signing up for a Travis CI account, we are asked to grant Travis access to our GitHub repositories for Travis to build our projects on every push. For this to work, we'll be redirected to GitHub, where we then have to enter our GitHub credentials. Afterwards, we have to confirm that we'll be granting Travis CI access to a set of operations/information that Travis CI can now perform on our behalf. Once granted, we'll see our GitHub projects popping up in the Travis web interface.

To understand how the scenario we just described works without giving Travis access to our GitHub credentials (which would be a significant security flaw), we first have to define and explain some OAuth 2.0 terms.

## OAuth 2.0 Terminology

The OAuth 2.0 protocol defines four roles:

- **Resource Owner:** Typically an end user (us) that owns resources in a third-party application (e.g., repositories on GitHub).
- **Resource Server:** Most of the time, a server (GitHub's API) that exposes protected resources.
- **Client:** An application (e.g., Travis CI) that wants access to protected resources on behalf of the *Resource Owner*.
- **Authorization Server:** A server that authenticates the *Resource Owner* (end user) and issues access tokens after obtaining authorization.

How a *Client* gets a valid access token to request protected resources of a *Resource Owner* depends on the use case and the application's architecture. OAuth2.0 defines different workflows (so-called "grant types") on how an application can access protected resources. The most commonly used grant types are the following:

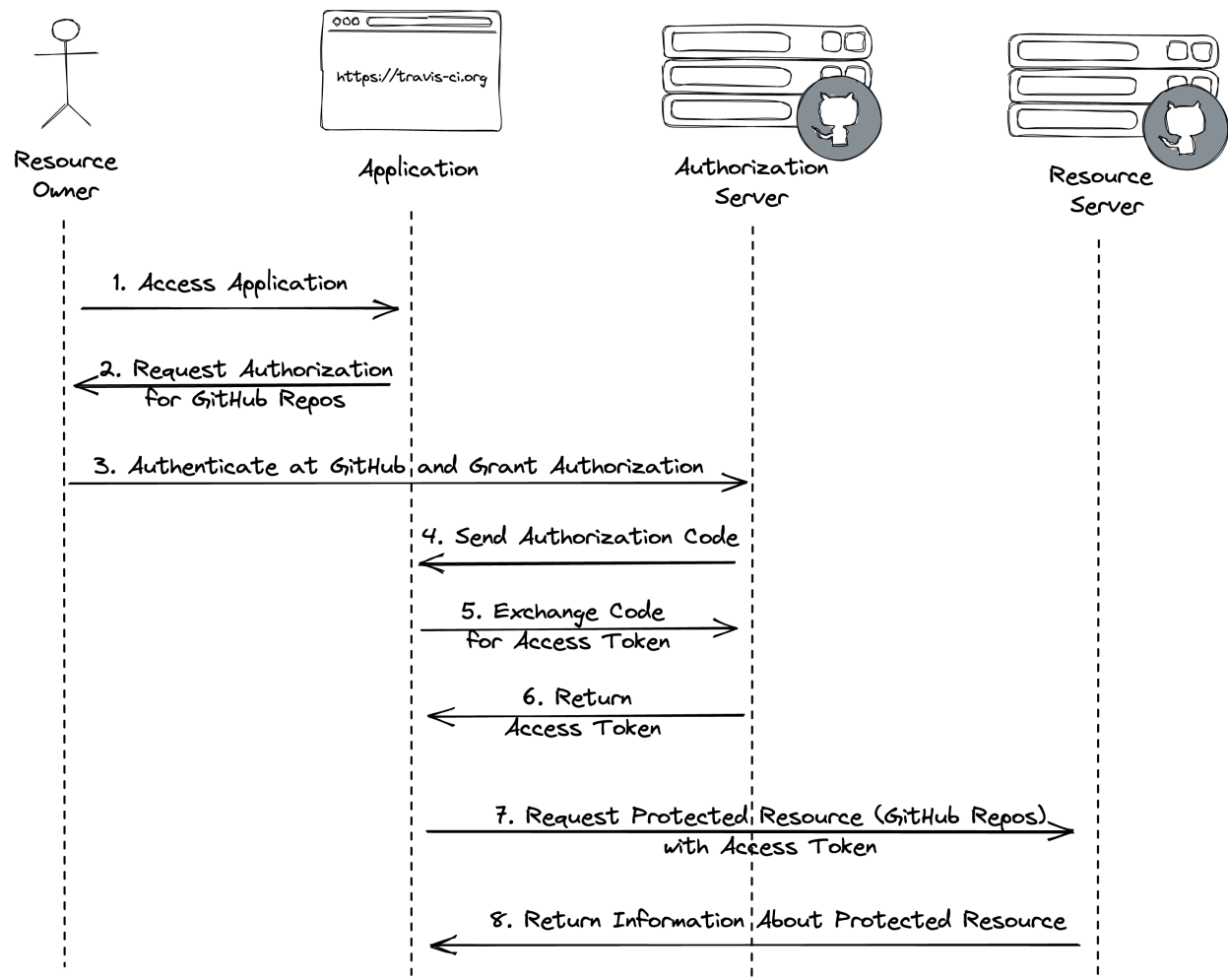
- **Authorization Code:** Most commonly used for web and mobile applications. Requires launching a browser to begin the flow.
- **Client Credentials:** Suitable for server-to-server communication without user interaction.
- **Device Code:** Used by input-constrained devices (e.g., smart TV sets) to request an access token.
- **Refresh Token:** The client can frequently exchange the refresh token for a new access token without requiring the user to be redirected each time the access token expires.



In our Travis example, we've used the *Authorization Code* grant to allow access to our GitHub repositories. Travis needs a valid access token to access GitHub's APIs and retrieve information about our GitHub repositories. Using the *Authorization Code* grant, the procedure works like this on a high level:

1. The application (the *Client*, Travis in this case) sends us (the *Resource Owner*) to the *Authorization Server* (GitHub's login page).
2. The *Resource Owner* (us) enters the credentials for the *Authorization Server* and approves the request to access data on their behalf.
3. The user is redirected to the application (Travis) with an authorization code in the query string.
4. The application (Travis) exchanges this authorization code for an access token to fetch data from the *Resource Server* (GitHub's API).

The following diagram visualizes this flow:



OAuth 2.0 Authorization Code Grant Flow

With the help of so-called “scopes” a client defines which operations and data it needs access to. While there are different operations and resources available [on GitHub](#) (e.g., “read profile”, “see public repositories”, “manage an organization”, ...), Travis doesn’t require all of these. Defining a subset of scopes allows Travis to only perform those operations it requires.

While OAuth 2.0 is all about authorization, can we somehow use these concepts to enable authentication? This is where OpenID Connect 1.0 comes into play.

## OpenID Connect 1.0 (OIDC)

While OAuth 2.0 focuses on *authorization* (granting Travis CI access to your GitHub repositories), there is an additional protocol that builds on top of OAuth 2.0 to solve *authentication*: [OpenID Connect 1.0](#) (short OIDC).

In a nutshell, OIDC is a ...

“... simple identity layer on top of the OAuth 2.0 protocol. It allows Clients to verify the identity of the End-User based on the authentication performed by an Authorization Server, as well as to obtain basic profile information about the End-User in an interoperable and REST-like manner.”

With OIDC, the end user entity now becomes the protected resource instead of our GitHub repositories or Gmail contacts, for instance. This authentication mechanism works with several OAuth 2.0 grant types, *Authorization Code* being the one that is most commonly used.

The process is similar to what we already described for the Travis CI and GitHub example. The main difference is that the client (our application) is now called the “Relying Party” (RP). Furthermore, the scope `openid` is mandatory, and the end user becomes the protected resource our application requests access to.

Whenever you see “Login with GitHub” or “Login with Google” on the login page of a particular website or app, that app or website is making use of this OpenID Connect protocol.

For the following chapters, it’s enough to understand that OIDC is an identity layer on top of OAuth 2.0 that enables user login with a third-party account (e.g., Google, GitHub, Twitter). This frees us from having to implement user

management ourselves. You can read through the official [specification](#) to get more information in greater detail.

AWS provides the “Cognito” service, which implements the OAuth 2.0 and OpenID Connect specifications and which we’ll integrate into our sample application.

## Alternatives to OAuth2 & OpenID Connect

As an alternative to OAuth2 and OIDC, our application could also implement its own user management. Spring Security provides a set of authentication mechanisms out-of-the-box: Basic Authentication, Form-based authentication, etc. Moreover, the Spring team recently introduced the [Authorization Server](#) project.

With all of those solutions, we would have to store sensitive information about our users (such as passwords) and provide features like password recovery or confirmation emails. That’s interesting to implement (at least once) but the additional maintenance effort and security concerns may not be worth it.

As security is a crucial subject, we prefer experts to solve this for us. If neglected at the beginning of a project, adding security-related features in a rush right before going live is not beneficial. Having it solved at the beginning of a project brings peace of mind and improves our time-to-market as we can concentrate on our features.

On the other hand, we now have an additional dependency that is crucial for our application to work: We’re relying on the uptime of the identity provider. Hence, our application would be unusable (at least its protected areas) whenever there is an outage.

## Using Amazon Cognito for User Management

With the upcoming sections, we'll learn more about Cognito, its vocabulary, and how to configure and integrate it with our Todo application.

### Introduction to Amazon Cognito

[Amazon Cognito](#) is a managed service that provides authentication, authorization, and user management for web and mobile applications. It supports OAuth 2.0, OpenID Connect 1.0, and various other protocols. Using this service, we'll delegate user management to AWS. This provides us with the following features:

- secure and simple user management
- pay-per-active-user model
- social logins via Facebook, Google, etc.
- optional multi-factor authentication
- SDKs for various programming languages (Java, JavaScript, Python, etc.)
- customizable web sign-in page

Similar off-the-shelf solutions that we could use instead of Amazon Cognito are [Okta](#), [Keycloak](#), or [Auth0](#).

Let's take a look at the terminology of Cognito.

### Amazon Cognito Terminology

A **User Pool** acts as a user directory where we can store and manage user information. Each *User Pool* comes with sign-up and sign-in functionality. This

includes a web interface to sign in that we can customize and configure, for example with additional social logins (Google, GitHub, etc.) or multi-factor authentication.

We'll create a single *User Pool* for our application and store all our users there. It's not required to have a separate *User Pool* per application as we might want to log in with the same credentials across multiple applications.

As part of the *User Pool*, we can also configure our password policy, define required and optional user attributes, enable password recovery mechanisms, and customize the email notifications.

A **User Pool App Client** is associated with a *User Pool* and has permission to call unauthenticated API operations like signing in or registering users. Therefore, every *App Client* requires a client ID and an optional secret.

As part of creating the *App Client*, we configure the valid callback/logout URLs and define which OAuth 2.0 flows and scopes this client can use. We'll register our Todo application as a *User Pool App Client* to enable user login via OIDC and the OAuth 2.0 *Authorization Code Grant* flow.

With an **Identity Pool**, we can map a user from an *Identity Provider* to an IAM role. This allows us to give users access to AWS resources based on their IAM permissions. Since the users of our Todo app don't need IAM roles to use the app, an *Identity Pool* is not relevant for us.

For an in-depth comparison of *Identity Pools* and *User Pools* take a look at Jake Bennett's blog post [Understanding Amazon Cognito User and Identity Pools for Serverless Apps](#). Not only does this article explain their difference but also showcases how we could use both concepts together.

Like all other AWS resources that we provide for our application, we prefer infrastructure-as-code over repetitive manual efforts in the AWS Console.

Let's see how we can create our Cognito resources with CDK.

## The Amazon Cognito CDK App

As per our convention from the chapter *Designing a Deployment Project with CDK*, we'll create a new CDK app for our Cognito instance:

```
public class CognitoApp {
 public static void main(final String[] args) {
 App app = new App();

 // omitted standard configuration values like the AWS region and sanity checks

 String applicationUrl = (String) app
 .getNode()
 .tryGetContext("applicationUrl");

 String loginPageDomainPrefix = (String) app
 .getNode()
 .tryGetContext("loginPageDomainPrefix");

 Environment awsEnvironment = makeEnv(accountId, region);

 ApplicationEnvironment applicationEnvironment = new ApplicationEnvironment(
 applicationName,
 environmentName
);

 new CognitoStack(
 app,
 "cognito",
 awsEnvironment,
 applicationEnvironment,
 new CognitoStack.CognitoInputParameters(
 applicationName,
 applicationUrl,
 loginPageDomainPrefix));

 app.synth();
 }
}
```

```
}
```

You can find the entire source code for the CognitoApp on [GitHub](#).

Apart from our standard input parameters (`environmentName`, `applicationName`, `accountId`, `region`), this CDK app depends on the following additional parameters:

- **applicationUrl:** As part of configuring the `UserPoolClient`, we need to define the valid callback and logout URLs. This parameter allows us to pass the final base URL of our application. An example URL would be `https://app.stratospheric.dev`.
- **loginPageDomainPrefix:** Each user pool provides a customizable web UI for users to log in. We can either provide a custom domain or pass a prefix to use an Amazon Cognito domain that will look something like this: `https://<prefix>.auth.<region>.amazoncognito.com`.

For the upcoming stack definition, we include the AWS CDK Cognito module in our CDK project. The AWS CDK construct library already ships stable level 1 and level 2 constructs for *User Pools*:



```

<!-- Both imports can be omitted when using cdk-constructs -->
<dependency>
 <groupId>software.amazon.awscdk</groupId>
 <artifactId>aws-cdk-lib</artifactId>
 <version>${aws-cdk-lib.version}</version>
</dependency>
<dependency>
 <groupId>software.constructs</groupId>
 <artifactId>constructs</artifactId>
 <version>${constructs.version}</version>
</dependency>

```

Our AWS CDK CognitoStack defines three resources: a UserPool, one UserPoolClient, and a UserPoolDomain. Let's start with the configuration of the UserPool.

### Creating the UserPool

The UserPool acts as the user directory and is the first resource we create as part of our CognitoStack (see the code on [GitHub](#)):

```

public class CognitoStack extends Stack {

 // fields omitted

 public CognitoStack(
 final Construct scope,
 final String id,
 final Environment awsEnvironment,
 final ApplicationEnvironment applicationEnvironment,
 final CognitoInputParameters inputParameters) {

 super(scope, id, StackProps.builder()
 .stackName(applicationEnvironment.prefix("Cognito"))
 .env(awsEnvironment).build());

 this.applicationEnvironment = applicationEnvironment;

 this.userPool = UserPool.Builder.create(this, "userPool")

```

```
.userPoolName(inputParameters.applicationName + "-user-pool")
.selfSignUpEnabled(false)
.standardAttributes(StandardAttributes.builder()
 .email(StandardAttribute
 .builder()
 .required(true)
 .mutable(false)
 .build())
 .build())
.signInAliases(SignInAliases
 .builder()
 .username(true)
 .email(true)
 .build())
.signInCaseSensitive(true)
.autoVerify(AutoVerifiedAttrs
 .builder()
 .email(true)
 .build())
.mfa(Mfa.OFF)
.accountRecovery(AccountRecovery.EMAIL_ONLY)
.passwordPolicy(PasswordPolicy.builder()
 .requireLowercase(true)
 .requireDigits(true)
 .requireSymbols(true)
 .requireUppercase(true)
 .minLength(12)
 .tempPasswordValidity(Duration.days(7))
 .build())
.build();

// further Cognito-related resources

}
}
```

In contrast to the approach of creating our own CDK Constructs that we followed in the chapter *Designing a Deployment Project with CDK*, we're using a slightly different approach here: We're creating a CDK Stack that composes official Cognito CDK level 2 constructs instead of our own building blocks. This

gives us more flexibility as we're able to fine-tune the highly customizable Cognito setup.

Most of the [attributes](#) of this resource are self-explanatory. We disable `self-SignUp` to only allow admin users to add new users to our user pool. Otherwise, users would be able to register themselves.

Our Todo application will act as such an admin and create our users. For this to work, we have to extend the IAM role for our ECS task and allow our application to perform all operations related to the identity provider.

The `Service` construct of our `cdk-constructs` library now takes a list of `PolicyStatement` objects, which are needed for configuring the access to internal AWS resources for our application. We'll add the following object to this list (see `ServiceApp` on [GitHub](#)) to allow all operations for `cognito-idp`:

```
.withTaskRolePolicyStatements(List.of(
 PolicyStatement.Builder.create()
 .sid("AllowCreatingUsers")
 .effect(Effect.ALLOW)
 .resources(
 List.of(String.format("arn:aws:cognito-idp:%s:%s:userpool/%s", region,
 accountId, cognitoOutputParameters.getUserPoolId()))
)
 .actions(List.of("cognito-idp:AdminCreateUser"))
 .build()
))
```



When working with IAM roles for Cognito, make sure to use `cognito-idp` as a prefix when targeting relevant actions for a User Pool. The `cognito-identity` prefix refers to Identity Pools.

To keep the amount of user information to a minimum, we only require a username and email address during the sign-up process. The username is

available by default. Hence, we don't need to add it when configuring the `standardAttributes`.

Users can use both their email and username to log in (`signInAliases`). Both are case-sensitive. We automatically approve the user's email by adding email to the `autoVerify` attributes.

Multi-factor authentication (MFA) is disabled as we set `mfa` to `OFF`. Users can only use their email to recover their accounts in case they forgot their passwords (`accountRecovery`).

As the last step, we adjust the `passwordPolicy` to a solid security standard by requesting users to enter a password with at least twelve mixed (symbols, digits, uppercase, lowercase) characters.

By default, this level 2 construct configures Amazon Cognito as the email delivery service (for sending the password or a recovery email, for example) instead of Amazon SES. However, with this setup, we have to keep in mind that Amazon Cognito comes with a daily email limit. We should use Amazon SES in the long run (as is the suggested best practice).

### **Creating the `UserPoolClient` and `UserPoolDomain`**

Next, we add a `UserPoolClient` for our `Todo` application. If we had other clients that required access to Cognito (for example a mobile app) we would have to duplicate the configuration below and adjust the values:

```
this.userPoolClient = UserPoolClient.Builder.create(this, "userPoolClient")
 .userPoolClientName(inputParameters.applicationName + "-client")
 .generateSecret(true)
 .userPool(this.userPool)
 .oAuth(OAuthSettings
 .builder()
 .callbackUrls(asList(
 String.format(
 "%s/login/oauth2/code/cognito",
 inputParameters.applicationUrl),
 "http://localhost:8080/login/oauth2/code/cognito"
))
 .logoutUrls(asList(
 inputParameters.applicationUrl,
 "http://localhost:8080"))
 .flows(OAuthFlows
 .builder()
 .authorizationCodeGrant(true)
 .build())
 .scopes(asList(
 OAuthScope.EMAIL,
 OAuthScope.OPENID,
 OAuthScope.PROFILE))
 .build())
 .supportedIdentityProviders(singletonList(
 UserPoolClientIdentityProvider.COGNITO))
 .build();
```

We define the client's name (`userPoolClientName`) to avoid random client names. For security reasons, AWS generates the secret for this client for us (`generateSecret`). Afterwards, we can use the Cognito SDK to retrieve this auto-generated value.

By setting `userPool` to the Java object that was returned when creating the `UserPool` we connect this client to our *User Pool*.

Next, for both the callback and logout URLs, we're adding two URLs. One URL is the final production URL, while the other one is used for local development and troubleshooting. The path `/login/oauth2/code/cognito` is the standard

callback URI for Spring Security.

With the remaining attributes, we allow this user pool client to use the OAuth 2.0 *Authorization Code Grant* flow with a pre-defined set of OAuth 2.0 scopes.

The `supportedIdentityProviders()` method expects a list of identity providers that our users can use to sign with this client. Apart from COGNITO, we could add FACEBOOK, AMAZON, GOOGLE, or a custom identity provider here.

The last resource of this stack is the `UserPoolDomain`:

```
this.userPoolDomain = UserPoolDomain.Builder.create(this, "userPoolDomain")
 .userPool(this.userPool)
 .cognitoDomain(CognitoDomainOptions
 .builder()
 .domainPrefix(inputParameters.loginPageDomainPrefix)
 .build())
 .build();
```

As we are not using a custom domain for the sign-in page, we're configuring the prefix for the Amazon Cognito domain. If we deploy this stack to the region `eu-central-1` and use "stratospheric" as the `LoginPageDomainPrefix`, we will get the following sign-in URL: `https://stratospheric.auth.eu-central-1.amazonaws.com`.

We could also re-use the `applicationName` input parameter as the domain prefix, but we would have to ensure that this prefix is unique for the AWS region. The additional parameter gives us the flexibility to pick another prefix if the application name is already taken.

### Amazon Cognito Output Parameters

To configure Spring Security for our Todo application later on, we need to expose our Cognito setup's dynamic attributes:

```
StringParameter.Builder.create(this, "userPoolId")
 .parameterName(createParameterName(
 applicationEnvironment,
 PARAMETER_USER_POOL_ID))
 .stringValue(this.userPool.getUserPoolId())
 .build();

StringParameter.Builder.create(this, "userPoolClientId")
 .parameterName(createParameterName(
 applicationEnvironment,
 PARAMETER_USER_POOL_CLIENT_ID))
 .stringValue(this.userPoolClient.getUserPoolClientId())
 .build();

StringParameter.Builder.create(this, "logoutUrl")
 .parameterName(createParameterName(
 applicationEnvironment,
 PARAMETER_USER_POOL_LOGOUT_URL))
 .stringValue(this.logoutUrl)
 .build();

StringParameter.Builder.create(this, "providerUrl")
 .parameterName(createParameterName(
 applicationEnvironment,
 PARAMETER_USER_POOL_PROVIDER_URL))
 .stringValue(this.userPool.getUserPoolProviderUrl())
 .build();
```

AWS generates random values for both the `userPoolId` and `userPoolClientId`. We need to expose both IDs to later fetch the secret of the OAuth 2 client.

The `logoutUrl` is required for fully logging out the end user. More about this at the end of this chapter. This URL contains our `loginDomainPrefix` and the AWS region, which will look something like this:

```
https://stratospheric.auth.eu-central-1.amazoncognito.com/logout
```

We need to expose the `providerUrl` to configure Spring Security to discover the OAuth 2.0 relevant endpoints. This URL contains the AWS region and our user pool identifier and will look like this:

[https://cognito-idp.eu-central-1.amazonaws.com/eu-central-1\\_pD8flsXa](https://cognito-idp.eu-central-1.amazonaws.com/eu-central-1_pD8flsXa)

To retrieve this auto-generated secret, we call the `getUserPoolClientSecret()` method of our `UserPoolClient`. We then unwrap the secret value returned by that method into a string, which we then store inside the AWS Parameter Store:

```
this.userPoolClientSecret = this.userPoolClient.getUserPoolClientSecret().unsafeUnwrap();

StringParameter userPoolClientSecret =
 StringParameter.Builder.create(this, "userPoolClientSecret")
 .parameterName(createParameterName(
 applicationEnvironment,
 PARAMETER_USER_POOL_CLIENT_SECRET))
 .stringValue(this.userPoolClientSecret)
 .build();
```

In summary, we need the following output values to integrate our application with Cognito properly:

- User Pool provider URL
- User Pool logout URL
- User Pool client name
- User Pool client ID
- User Pool client secret

As our Spring Boot application expects these values on application startup, we have to deploy or update the Cognito stack before triggering a new ECS deployment of our Todo app.

Finally, we extend our `package.json` with two scripts to conveniently create, update, or destroy the Cognito CDK app:



```
npm run cognito:deploy
npm run cognito:destroy
```

### A Note on Secure Parameters

You might notice that we stored the `userPoolClientSecret` parameter as a plain `StringParameter` in the AWS Parameter Store, meaning it's not encrypted as a secret. Later, in the chapter [Connecting to a Database with RDS](#), we will see how to store and retrieve an encrypted secret in AWS Secrets Manager.

By itself, this encryption isn't worth much, however, since we'll be passing all secrets as plain-text environment variables into the Docker container with our Spring Boot application. That means they will be available in plain text during runtime. Also, all environment variables are displayed in plain text in the ECS web console.

Another option to handle secret parameters is to store them as secrets in AWS Secrets Manager and just inject the name of the parameter as an environment variable into our application. The application can then use the parameter name to load the secret from the secret store when it needs it. This way, the parameter wouldn't be accessible as an environment variable during the entire runtime of the application. It would only be accessible for admins with permissions to the Secrets Manager.

Spring AWS [integrates the AWS Secrets Manager](#) to make secrets available as application properties. We opted for simplicity over security in this case and used a plain environment variable for the Cognito user pool client secret.

As an alternative, we could also store sensitive configuration values as `SecureStrings` inside the AWS Parameter Store and use Spring Cloud AWS to [fetch the values upon application start](#).

## Using Amazon Cognito as an Identity Provider With Spring Security

Now, let's see how we can integrate the Cognito User Pool and Client with our Spring Boot application.

Spring Security already supports both OAuth 2.0 and OpenID Connect 1.0 out-of-the-box. As we're building on top of an industry-standard protocol there's a common procedure happening behind the scenes that Spring Security implements for us.

The relevant security components are part of the following two Spring Boot starters:

```
dependencies {
 implementation 'org.springframework.boot:spring-boot-starter-security'
 implementation 'org.springframework.boot:spring-boot-starter-oauth2-client'
}
```

These starters already do a lot of configuration work for us. What's left is to configure information about the OpenID Connect 1.0 Provider – which is Cognito in our case.

Spring Security needs to know a set of URIs about the external provider – the `authorization-uri`, `token-uri`, `user-info-uri`, etc. – to perform the OAuth 2.0 *Authorization Code Grant* flow. While we could specify all of these attributes on our own, there is an even simpler solution available.

The OpenID Connect specification requires every compatible provider to expose a discovery endpoint for retrieving configuration values. This endpoint must be available at the following specified path: `FULL_URI_OF_THE_PROVIDER/.well-known/openid-configuration`.

When invoking this endpoint (via an HTTP GET request) for our Cognito user pool we get the following result:

```
curl -v https://cognito-idp.eu-central-1.amazonaws.com/eu-central-1_pXOUNokLO/.well-known/openid-configuration
```

```
{
 "authorization_endpoint": "https://dev101...amazoncognito.com/oauth2/authorize",
 "id_token_signing_alg_values_supported": [
 "RS256"
],
 "issuer": "https://...amazonaws.com/eu-central-1_pXOUNokLO",
 "jwks_uri": "https://...amazonaws.com/eu-central-1_pXOUNokLO/.well-known/jwks.json",
 "response_types_supported": [
 "code",
 "token"
],
 "scopes_supported": [
 "openid",
 "email",
 "phone",
 "profile"
],
 "subject_types_supported": [
 "public"
],
 "token_endpoint": "https://...amazoncognito.com/oauth2/token",
 "token_endpoint_auth_methods_supported": [
 "client_secret_basic",
 "client_secret_post"
],
 "userinfo_endpoint": "https://...amazoncognito.com/oauth2/userInfo"
}
```

By specifying the OpenID Discovery endpoint of our identity provider, Spring Security automatically retrieves all relevant information on application startup.

Putting it all together, the relevant Spring Security configuration in our `application-aws.yml` file will look like this:

```
spring:
 security:
 oauth2:
 client:
 registration:
 cognito:
 clientId: ${COGNITO_CLIENT_ID}
 clientSecret: ${COGNITO_CLIENT_SECRET}
 scope: openid, profile, email
 clientName: stratospheric-client
 provider:
 cognito:
 issuerUri: ${COGNITO_PROVIDER_URL}
```

The `${}` placeholders will be replaced with environment variables that we specify when deploying our Todo application with ECS (using SpEL – Spring’s powerful expression language).

## User Registration with Amazon Cognito

An empty *User Pool* is good for our AWS bill but not for our application’s reputation. It’s time to fill it with actual users of our Todo application.

While configuring the Cognito *User Pool* we specified that only admin users are allowed to add new users to the pool.

For communicating with Cognito we’re using the Java SDK. To align all AWS Java SDK versions Spring Cloud AWS already specifies the AWS Java SDK BOM. Hence, we only have to add the following dependency to our project without specifying a version:

```
dependencies {
 implementation 'software.amazon.awssdk:cognitoidentityprovider'
}
```

Next, we define a Spring Bean of type `CognitoIdentityProviderClient` as this client is not auto-configured by Spring Cloud AWS for us:

```
@Configuration
public class AwsConfig {

 @Bean
 @ConditionalOnProperty(prefix = "custom",
 name = "use-cognito-as-identity-provider", havingValue = "true")
 public CognitoIdentityProviderClient cognitoIdentityProviderClient(
 AwsRegionProvider regionProvider,
 AwsCredentialsProvider awsCredentialsProvider) {
 return CognitoIdentityProviderClient.builder()
 .credentialsProvider(awsCredentialsProvider)
 .region(regionProvider.getRegion())
 .build();
 }
}
```

As outlined in the chapter [Local Development](#), we don't want to connect to the real Cognito service when we're working locally, so this Spring bean will not be activated if our custom property `use-cognito-as-identity-provider` is set to `false`.

Our users will be able to register their accounts as part of the Todo application. To avoid bots from auto-generating accounts, we add a layer of protection and require an invitation code on each signup. To that end, let's create a public Thymeleaf view that contains the relevant information for new sign-ups.

The model for our view contains the following information:

```
public class Registration {

 @NotBlank
 private String username;

 @Email
 private String email;

 @ValidInvitationCode
 private String invitationCode;

 // getters & setters

}
```

We're using Bean Validation to validate the incoming data on form submit. A basic Spring MVC controller endpoint exposes this view and instantiates an empty Registration object to be filled by the user:

```
@Controller
@RequestMapping("/register")
public class RegistrationController {

 private final RegistrationService registrationService;

 public RegistrationController(RegistrationService registrationService) {
 this.registrationService = registrationService;
 }

 @GetMapping
 public String getRegisterView(Model model) {
 model.addAttribute("registration", new Registration());
 return "register";
 }
}
```

The relevant Thymeleaf part for the HTML form binds the model attributes to input fields and displays an error message whenever the validation fails:

```

<form th:action="@{/register}" th:object="${registration}" method="post">
<div class="form-group">
 <label for="username">Username</label>
 <input
 type="text"
 th:field="*{username}"
 class="form-control"
 id="username"
 required>
</div>
<div class="form-group">
 <label for="email">Email address</label>
 <input
 type="email"
 th:field="*{email}"
 class="form-control"
 id="email"
 required
 aria-describedby="emailHelp">
 <small id="emailHelp" class="form-text text-muted">
 We'll never share your email with anyone else.
 </small>
</div>
<div class="form-group">
 <label for="invitationCode">Invitation code</label>
 <input
 th:field="*{invitationCode}"
 th:classappend="${#fields.hasErrors('invitationCode')}? 'is-invalid'"
 type="text"
 class="form-control"
 id="invitationCode"
 aria-describedby="invitationCodeHelp"
 required>
 <div th:if="${#fields.hasErrors('invitationCode')}"
 th:text="${#strings.listJoin(#fields.errors('invitationCode'), ', ')}"
 class="invalid-feedback">
 </div>
 <small id="invitationCodeHelp" class="form-text text-muted">
 Enter the invitation code you received to register.
 </small>
</div>
<button type="submit" class="btn btn-primary">Submit</button>
</form>

```



We won't go into much detail about Thymeleaf and explain every bit of our views as the main subject of this book is AWS and Spring Boot. If you are looking for an excellent resource on developing Spring Boot applications with Thymeleaf, we highly recommend Wim Deblauwe's [Taming Thymeleaf](#) book.

Our users will see the following registration page:

# Register

Username

Email address

We'll never share your email with anyone else.

Invitation code

Enter the invitation code you received to register.

Submit

Stratospheric registration page.

For validating the submitted invitation code, we're defining a custom `ConstraintValidator` that checks the code against a set of valid invitation codes (see `InvitationCodeValidator` on [GitHub](#)). We can configure these codes with an environment variable or store it in the AWS SSM Parameter Store.



Whenever the user hits the submit button, the browser performs an HTTP POST against `/register` and includes the registration information as payload.

To be able to process this request, we have to add a new endpoint mapping to our `RegistrationController`:

```
@PostMapping
public String registerUser(
 @Valid Registration registration,
 BindingResult bindingResult,
 Model model, RedirectAttributes redirectAttributes) {
 if (bindingResult.hasErrors()) {
 model.addAttribute("registration", registration);
 return "register";
 }

 try {
 registrationService.registerUser(registration);

 redirectAttributes.addFlashAttribute("message",
 "You successfully registered for the Todo App. " +
 "Please check your email inbox for further instructions."
);
 redirectAttributes.addFlashAttribute("messageType", "success");

 return "redirect:/";
 } catch (CognitoIdentityProviderException exception) {

 model.addAttribute("registration", registration);
 model.addAttribute("message", exception.getMessage());
 model.addAttribute("messageType", "danger");

 return "register";
 }
}
```

First, we ensure that our model matches our validation rules (e.g., no empty email or username). Even though we also have some basic client-side validation for them, we never know how clients reach our endpoint and hence should always validate the payload.

If there are validation errors, `bindingResult.hasErrors()` call returns true, and we are displaying the registration page to the user again with additional hints. For example, this might happen if a user enters the wrong validation code.

# Register

Username

duke

Email address

duke@stratospheric.dev

We'll never share your email with anyone else.

Invitation code

WRONG\_CODE 

Invalid invitation code. Please check it again or contact the person who invited you.

Enter the invitation code you received to register.

Submit

**Stratospheric registration page with binding error.**

We delegate the actual registration to the `RegistrationService` and pass the submitted values. Our `RegistrationService` is an interface as we'll have two flavors of it depending on which environment our application runs in:

```
public interface RegistrationService {
 void registerUser(Registration registration);
}
```

For the production runtime on AWS, we'll create a new user as part of our Cognito User Pool:

```
@Service
@ConditionalOnProperty(prefix = "custom",
 name = "use-cognito-as-identity-provider", havingValue = "true")
public class CognitoRegistrationService implements RegistrationService {

 private final CognitoIdentityProviderClient cognitoIdentityProviderClient;
 private final String userPoolId;

 public CognitoRegistrationService(
 CognitoIdentityProviderClient cognitoIdentityProviderClient,
 @Value("${COGNITO_USER_POOL_ID}") String userPoolId) {
 this.cognitoIdentityProviderClient = cognitoIdentityProviderClient;
 this.userPoolId = userPoolId;
 }

 @Override
 public void registerUser(Registration registration) {
 AdminCreateUserRequest registrationRequest = new AdminCreateUserRequest()
 .withUserPoolId(userPoolId)
 .withUsername(registration.getUsername())
 .withUserAttributes(
 new AttributeType()
 .withName("email")
 .withValue(registration.getEmail()),
 new AttributeType()
 .withName("email_verified")
 .withValue("true")
)
 .withDesiredDeliveryMediums(DeliveryMediumType.EMAIL)
 .withForceAliasCreation(Boolean.FALSE);

 cognitoIdentityProviderClient.adminCreateUser(registrationRequest);
 }
}
```

The Amazon Cognito Java SDK provides the relevant classes for performing an `AdminCreateUserRequest`. There isn't much data we pass alongside the registration, as we only require a username and an email from the user.

Up until now, the user did not select any password. That's because we configured our Cognito *User Pool* to send a temporary password upon registration to the user's inbox.

## Login with Amazon Cognito

Right now, all endpoints of our application are protected by the auto-configuration of Spring Security. Our `Todo` application should have both a protected and public area, though. Only logged-in users should be allowed to create, edit, delete, and share todos.

However, the previous view for the registration process has to be available without any authentication. Furthermore, the landing page of our application and the `/health` endpoint have similar requirements. The same is true of any static resource (JavaScript, CSS, etc.) that our Spring Boot application serves to the browser.

Any other part of the application should be protected and only accessible for users who logged in via OIDC and their Cognito credentials.

Let's see how we can configure our application to meet these requirements.

Spring Security allows us to define our own security rules by providing a `SecurityFilterChain` bean. We can inject the application's `HttpSecurity` instance to specify our security restrictions using a fluent builder:

```
@Configuration
public class WebSecurityConfig {

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity)
 throws Exception {
 httpSecurity
 .csrf()
 .and()
 .oauth2Login()
 .and()
 .authorizeRequests()
 .requestMatchers(PathRequest.toStaticResources().atCommonLocations())
 .permitAll()
 .requestMatchers("/", "/health", "/register").permitAll()
 .anyRequest().authenticated();

 return httpSecurity.build();
 }
}
```



With Spring Security 5.7.0-M2, the Spring team deprecated the `WebSecurityConfigurerAdapter` class. Previously, we extended this class to specify our Spring Security configuration. The Spring development team now encourages us to move towards a component-based security configuration. Find more information about this deprecation [here](#).

With the configuration above, we first enable CSRF protection (to prevent a Cross-Site-Request-Forgery attack). Due to the excellent integration of Thymeleaf, Spring MVC, and Spring Security, all our HTML forms are already protected by default and include a hidden CSRF token.

Next, we configure the authentication support for OIDC. With `oauth2Login`, Spring Security refers to either OAuth 2.0 and/or OpenID Connect 1.0 authentication. As we follow the default conventions and since we've already specified

the relevant configuration properties inside `application-aws.yml`, there's nothing more to configure in regard to OIDC.

The remaining part of the security configuration permits any request to our static resources and public endpoints/views. Everything else requires authentication.

As part of our application header, we can now include a button to allow users to log in. We'll only show this button whenever an unauthenticated visitor loads our application.

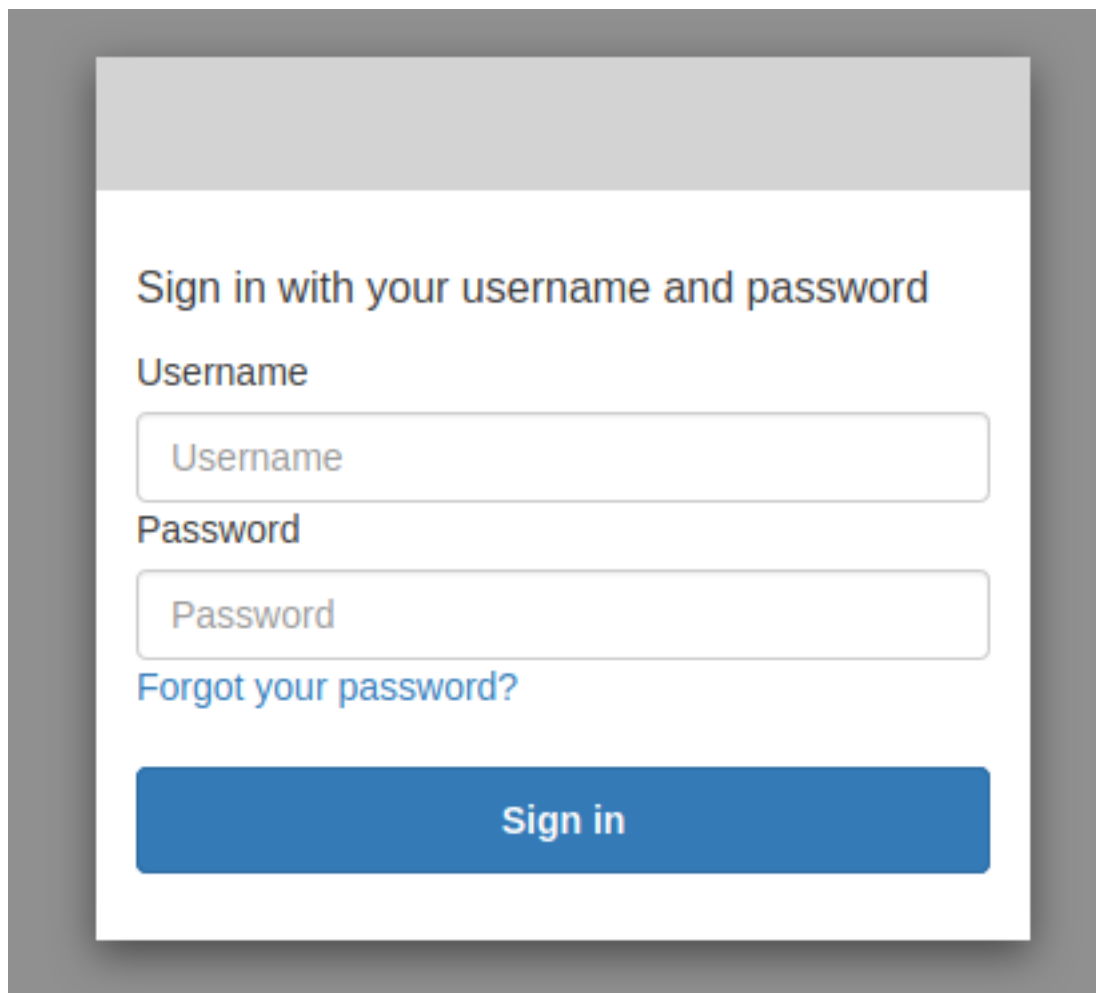
We control this with the `isAnonymous()` method that is part of the `thymeleaf-extras-springsecurity5` dependency:

```
<li class="nav-item" sec:authorize="isAnonymous()">

 Login


```

The login location contains the name of our OAuth 2.0 client/provider: `cognito`. Spring Security takes care of redirecting the user to our Cognito *User Pool* login dialog.

The image shows a screenshot of the Amazon Cognito hosted sign-in form. It features a white card with a light gray header bar at the top. The card is set against a dark gray background. The form contains the following elements: a heading "Sign in with your username and password" in a bold, dark blue font; a label "Username" in a dark blue font above a white text input field with a light gray border; a label "Password" in a dark blue font above a white text input field with a light gray border; a link "Forgot your password?" in a blue font below the password field; and a large, solid blue button with the text "Sign in" in white font at the bottom.

**Amazon Cognito's hosted sign-in form.**

After the user has entered the correct credentials, they'll be redirected back to our application, and Spring Security will create a valid session for this user. Technically speaking, Spring Security creates a `Principal` in the form of an `OidcUser`. We can then inject that `Principal` into our Spring MVC controllers:

```

@GetMapping
public String getIndex(Model model, @AuthenticationPrincipal OidcUser user) {
 // extract attributes from the user object

 return "index";
}

```

Or we can get it from the SecurityContextHolder:

```

OidcUser user = (OidcUser) SecurityContextHolder
 .getContext()
 .getAuthentication()
 .getPrincipal();

```

We can even check the authentication status as part of our Thymeleaf views. If we bind the XML namespace `http://www.thymeleaf.org/extras/spring-security` to the `sec` prefix, we can use it to conditionally render parts of a view based on security expressions:

```

<!-- rendered for unauthenticated users -->
<div sec:authorize="isAnonymous()">
 <p>Seems like you are not logged-in yet. Please login first to see your Todos.</p>

 Login

</div>

<!-- rendered for authenticated users -->
<div sec:authorize="isAuthenticated()">
 <p>Welcome to the protected area!</p>
 <p>Your email: [[${email}]]</p>
 <p>Your claims:</p>

 th:each="claim : ${claims}"
 th:text="${claim.key} + ': ' + ${claim.value}"

</div>

```



Furthermore, we can also access the Spring Security authentication object by referring to the expression utility object `#authentication`.

```
<li class="nav-item" sec:authorize="isAuthenticated()">
 Howdy, [[${#authentication.principal.attributes.name}]]!

```

## Shortcomings when Scaling Out

While this procedure works smoothly when starting the application locally or with just one instance, we run into issues once we scale out to multiple instances. As our final AWS infrastructure makes sure to run at least two instances of our Todo application, we're affected by this issue.

Having a stateless application allows scaling out and distributing traffic without any problems. Unfortunately, two technical details prevent us from being fully stateless with our Todo application. Let's take a look at the first one.

A sample OAuth 2.0 Authorization Code Grant request looks like this:

```
https://authorization-server.com/oauth/authorize
?client_id=a17c21ed
&response_type=code
&state=5ca75bd30
&redirect_uri=https%3A%2F%2Fexample-app.com%2Fauth
&scope=openid
```

The specification defines the `state` parameter as a recommended field. This parameter is used to transfer state and also protect from CSRF attacks. Spring Security creates a random string for this value and verifies that the same value is returned after the user is authenticated.

This validation fails whenever a callback is routed to a Spring Boot instance that didn't initiate this call and therefore is unaware of this state. Having multiple

nodes running our application in parallel, the likelihood that this validation fails is high.

One solution is to configure sticky sessions as part of the AWS Elastic Load Balancer. This ensures that users are always routed to the same instance they were assigned to on their first request.

We can configure sticky sessions for the `ApplicationTargetGroup` as part of the `ServiceInputParameters` of our `Service` CDK construct:

```
new Service(
 serviceStack,
 "Service",
 awsEnvironment,
 applicationEnvironment,
 new Service.ServiceInputParameters(...)
 .withStickySessionsEnabled(true)
);
```

The second issue is the creation of in-memory HTTP sessions. Without further configuration, the session is only known to the Spring Boot instance that created it. If a request arrives at a different instance, Spring Security will reject the request. With our sticky session configuration, this won't happen, as our clients are always routed to the same instance.

With this approach, we lose the benefit of an unnoticed fail-over. Whenever one of our instances terminates, the user gets redirected to the other instance and has to log in again. That might not be the best user experience, but it's also not too big a concern.

As an alternative to configuring sticky sessions for our load balancer, Spring provides a sub-project called [Spring Session](#). With this sub-project, we can enable clustered sessions for our Spring application by using a persistent storage like Hazelcast, MongoDB, or an RDBMS to store session information.

The persistent storage acts as a single point of truth for all session information and is resilient to re-deployments and application failures. Any Spring Boot instance will recognize an incoming JSESSIONID even though the session was created at a different instance. As Spring Security stores the state parameter for the OAuth 2.0 Authorization Code Grant as a session attribute, the callback will also be accepted at any instance.

Both sticky sessions and Spring Session support scaling out our authentication and authorization setup. To reduce the additional setup complexity, we're going forward with sticky sessions.

## Logout Process

Whenever an authenticated user enters our application, we display a logout button as part of our main header:

```
<li class="nav-item" sec:authorize="isAuthenticated()">
 <form th:action="@{/logout}" method="post">
 <input class="btn btn-danger" type="submit" value="Logout">
 </form>

```

The `/logout` route is the default location of Spring Security for handling the logout process.

As our Spring Boot application creates and stores a session for each of our users, the first step is to invalidate this session once the user decides to log out. This will make the current user unknown to our application. Subsequently, all `isAnonymous()` parts of our Thymeleaf views will evaluate to `true`.

However, the user still has an active session at the identity provider. With every subsequent attempt to re-login to our application, the users don't have to

provide their credentials for Cognito (unless the session expires) and are logged in automatically.

OIDC has a standardized procedure for also logging out the end user at the identity provider: [OpenID Connect RP \(Relying Party\)-Initiated Logout 1.0](#). Unfortunately, this specification is currently a draft and hence not a strict requirement to be compatible.

In short: the authorization server has to expose an `end_session_endpoint` that accepts a pre-defined list of parameters for logging the user out and then redirecting them back to the application.

Spring Security is already capable of following this standard procedure for fully logging out the user. We enable this by specifying the `OidcClientInitiatedLogoutSuccessHandler` as part of the `logoutSuccessHandler` for our `WebSecurityConfig`:

`@Configuration`

```
public class WebSecurityConfig {
```

```
 private final ClientRegistrationRepository clientRegistrationRepository;
```

```
 public WebSecurityConfig(
 ClientRegistrationRepository clientRegistrationRepository) {
 this.clientRegistrationRepository = clientRegistrationRepository;
 }
```

```
 private OidcClientInitiatedLogoutSuccessHandler oidcLogoutSuccessHandler() {
 OidcClientInitiatedLogoutSuccessHandler successHandler =
 new OidcClientInitiatedLogoutSuccessHandler(clientRegistrationRepository);
 successHandler.setPostLogoutRedirectUri("{baseUrl}");
 return successHandler;
 }
```

`@Bean`

```
 public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity)
 throws Exception {
 httpSecurity
```

```
 .csrf()
 .and()
 .oauth2Login()
 .and()
 .authorizeRequests()
 .requestMatchers(PathRequest
 .toStaticResources()
 .atCommonLocations())
 .permitAll()
 .requestMatchers("/", "/health", "/register")
 .permitAll()
 .anyRequest().authenticated()
 .and()
 .logout()
 .logoutSuccessHandler(oidcLogoutSuccessHandler());

 return httpSecurity.build();
}
```

This works fine for identity providers that already support this draft. The sad news is that Cognito [doesn't follow this specification](#) (yet). However, Cognito exposes an [endpoint](#) that we can use to log out an end user. For this to work, we have to implement our own `LogoutSuccessHandler`.

As the Cognito logout endpoint expects a simple HTTP GET request with two query parameters, we can use Spring Security's `SimpleUrlLogoutSuccessHandler`.

What's left todo is to override the `determineTargetUrl()` method and create the URL accordingly:

```
public class CognitoOidcLogoutSuccessHandler
 extends SimpleUrlLogoutSuccessHandler {

 private final String logoutUrl;
 private final String clientId;

 public CognitoOidcLogoutSuccessHandler(
 String logoutUrl,
 String clientId) {
 this.logoutUrl = logoutUrl;
 this.clientId = clientId;
 }

 @Override
 protected String determineTargetUrl(
 HttpServletRequest request,
 HttpServletResponse response,
 Authentication authentication) {

 UriComponents baseUrl = UriComponentsBuilder
 .fromHttpUrl(UrlUtils.buildFullRequestUrl(request))
 .replacePath(request.getContextPath())
 .replaceQuery(null)
 .fragment(null)
 .build();

 return UriComponentsBuilder
 .fromUri(URI.create(logoutUrl))
 .queryParam("client_id", clientId)
 .queryParam("logout_uri", baseUrl)
 .encode(StandardCharsets.UTF_8)
 .build()
 .toUriString();
 }
}
```

We make both the `logoutUrl` (representing the logout endpoint of our Cognito instance, e.g., `https://stratospheric...amazoncognito/logout`) and the `clientId` (the app client id of our Todo application) configurable, as these depend on the actual Cognito instance.

The `logout_uri` parameter is the URL that Cognito will redirect the end user to after the logout. This has to be a valid URL that was configured as part of the LogoutURLs of the app client. For our application, we redirect all users to the base URL of the Todo application.

With this custom `LogoutSuccessHandler` in place, we can now define it as a bean for when `custom.use-cognito-as-identity-provider` is set to `true` and inject the relevant credentials and region to configure the handler:

```
@Configuration
public class LogoutSuccessHandlerConfig {

 @Bean
 @ConditionalOnProperty(prefix = "custom",
 name = "use-cognito-as-identity-provider", havingValue = "true")
 public CognitoIdentityProviderClient cognitoIdentityProviderClient(
 AwsRegionProvider regionProvider,
 AwsCredentialsProvider awsCredentialsProvider) {

 return CognitoIdentityProviderClient.builder()
 .credentialsProvider(awsCredentialsProvider)
 .region(regionProvider.getRegion())
 .build();
 }
}
```

Finally, we have to make our `HttpSecurity` configuration aware of this logout handler:

```
@Configuration
public class WebSecurityConfig {

 private final LogoutSuccessHandler logoutSuccessHandler;

 public WebSecurityConfig(LogoutSuccessHandler logoutSuccessHandler) {
 this.logoutSuccessHandler = logoutSuccessHandler;
 }

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity)
 throws Exception {
 httpSecurity
 // ...
 .logout()
 .logoutSuccessHandler(logoutSuccessHandler);

 return httpSecurity.build();
 }
}
```

The only drawback that comes with not implementing a custom `LogoutSuccessHandler` is that after a user signs out from our application, the session at Cognito is still valid. For every subsequent login attempt, our users will be immediately signed in. That's not a problem per se, but it makes switching users within the same browser (without manually clearing cookies, that is) more complicated, for example.

## Enabling Local Development

As already outlined in the chapter [Local Development](#), Cognito is not part of the free tier of LocalStack. Therefore, for local development, we're using Keycloak as a compatible OIDC identity provider instead.

To start and stop a local Keycloak instance we are using Docker Compose. As Keycloak is the only infrastructure component that our Todo application requires at



this point of the book, our `docker-compose.yml` file is quite straightforward:

```
version: '3.3'

services:
 keycloak:
 image: quay.io/keycloak/keycloak:18.0.0-legacy
 ports:
 - 8888:8080
 environment:
 - KEYCLOAK_USER=keycloak
 - KEYCLOAK_PASSWORD=keycloak
 - DB_VENDOR=h2
 - JAVA_OPTS=\
 -Dkeycloak.migration.action=import \
 -Dkeycloak.migration.provider=singleFile \
 -Dkeycloak.migration.file=/tmp/stratospheric-realm.json
 volumes:
 - ./src/test/resources/keycloak/stratospheric-realm.json:\
 /tmp/stratospheric-realm.json
```

The `stratospheric-realm.json` file that we are mapping to the inside of the Docker container contains the configuration for Keycloak. To avoid any manual setup steps (define users, configure the realm, etc.), we import this config during Keycloak startup (see the file `stratospheric-realm.json` on [GitHub](#)).

This config pre-populates three users and registers our Todo application as a known client. All that remains to be done is to specify the relevant Spring Security properties as part of our `application-dev.yml` file:

```

spring:
 security:
 oauth2:
 client:
 registration:
 cognito:
 clientId: spring-boot-application
 clientSecret: 27b07baf-53ba-42c6-b11f-6384769cada3
 scope: openid
 provider:
 cognito:
 issuerUri: http://localhost:8888/auth/realms/stratospheric

```

Since we're always using the same Keycloak configuration we can hardcode the client secret. We are still using cognito as the provider key to avoid further complexity in our setup.

The only feature that's not working locally is user registration. We'll still see a success message when trying to register but internally we're doing nothing:

```

@Service
@ConditionalOnProperty(prefix = "custom",
 name = "use-cognito-as-identity-provider", havingValue = "false")
public class LocalRegistrationService implements RegistrationService {

 @Override
 public void registerUser(Registration registration) {
 // no registration as we use a local Keycloak instance
 // with a pre-defined set of users
 }
}

```

We can still log in to the Todo application by using one of the pre-defined users:

- tom with password stratospheric
- bjoern with password stratospheric
- philip with password stratospheric

There is good news for the OIDC logout feature mentioned above: Keycloak already supports this draft. This means that whenever we start the application locally we'll be able to fully log out.

This is made possible by switching the actual LogoutSuccessHandler per profile. If running locally, we instantiate Spring Security's OidcClientInitiatedLogoutSuccessHandler as Keycloak already supports the RP-initiated logout specification:

```
@Configuration
public class LogoutSuccessHandlerConfig {

 @Bean
 @ConditionalOnProperty(prefix = "custom", name = "use-cognito-as-identity-provider",
 havingValue = "false")
 public LogoutSuccessHandler oidcLogoutSuccessHandler(ClientRegistrationRepository \
clientRegistrationRepository) {
 OidcClientInitiatedLogoutSuccessHandler successHandler = new OidcClientInitiated\
LogoutSuccessHandler(clientRegistrationRepository);
 successHandler.setPostLogoutRedirectUri("{baseUrl}");
 return successHandler;
 }
}
```

# 11. Connecting to a Database with Amazon RDS

As soon as we want to provide our users with a way of persisting their data across sessions we need some sort of data storage. Our sample Todo application is no different in that regard because it will allow users to create, edit, and share todos.

Web applications commonly use *relational database management systems* (RDBMS) for storing data. Due to the language generally used for interacting with them, these database systems are also more commonly known as SQL (Structured Query Language) databases.

Relational databases store and manage data as rows in tables and relationships between those tables. This approach, devised by E. F. Codd in 1970, has been the standard way of dealing with persistent data in web applications for a long time.

It was only with the advent of Web 2.0 applications and the NoSQL approach promoted in that context that alternatives such as document stores or object databases were widely considered again. Nowadays, such non-relational databases are often used in conjunction with relational databases. In the chapter *Tracing User Actions with DynamoDB* we'll talk more about NoSQL databases in the context of AWS.

Relational databases aren't going anywhere, though, because they are still useful for common CRUD (Create, Read, Update, Delete) operations or time-

series data, for example. So, we'll use such a database for our sample Todo application, too.

In the sample Todo application, we'll be using a database for storing todos as well as for managing users and sharing todos, and for that we've opted for a PostgreSQL RDBMS managed by Amazon Relational Database Service (RDS).

While setting up the database involves AWS specifics, using it in our Spring-based application works just as one would expect from an ordinary, non-cloud, non-AWS database.

But before going into the details of how to use the database from our Spring Boot application, let's explore what Amazon RDS offers and how we can automate the provisioning of a database for our application.

## Introduction to AWS Relational Database Service (RDS)

Amazon Relational Database Service (RDS) is the AWS service for running and managing relational databases.

Apart from PostgreSQL, MySQL, MariaDB, Oracle Database, and Microsoft SQL Server, RDS also supports Amazon's own [Aurora](#) database technology. Aurora is a MySQL- and PostgreSQL-compatible RDBMS specifically designed with requirements of highly scalable cloud applications in mind.

RDS allows us to create and manage relational databases on AWS using its common tools and techniques such as the AWS CLI, IAM, and CDK.

In addition to using the above tools to integrate the database into our AWS environment, we can manage the database through the [Amazon RDS Management Console](#).

As usual with AWS services, RDS provides a scalable infrastructure that grows

according to our application's and our users' needs. The underlying environment is managed entirely by AWS, which frees us from maintenance headaches. Furthermore, [RDS supports SSL for encrypting data in transit](#) and with [AWS Key Management Service \(KMS\)](#) we can encrypt our data at rest as well. RDS integrates with [AWS Config](#) to provide us with a way of enforcing compliance by recording and auditing changes to database configurations.

## Setting up IAM Permissions

At this point, we assume that the IAM basics have already been set up as described in the chapter [Managing Permissions with IAM](#). To access RDS resources during development we simply need to attach the AWS-managed `AmazonRDS-FullAccess` policy to those IAM groups our application developers are in.

This access is needed to run `deploy` a new database or make changes to an existing database instance via the AWS Console or via the CDK app that we're going to build for that purpose. Once the database is deployed and ready to use, it's best to remove full access again, at least for production-like environments.

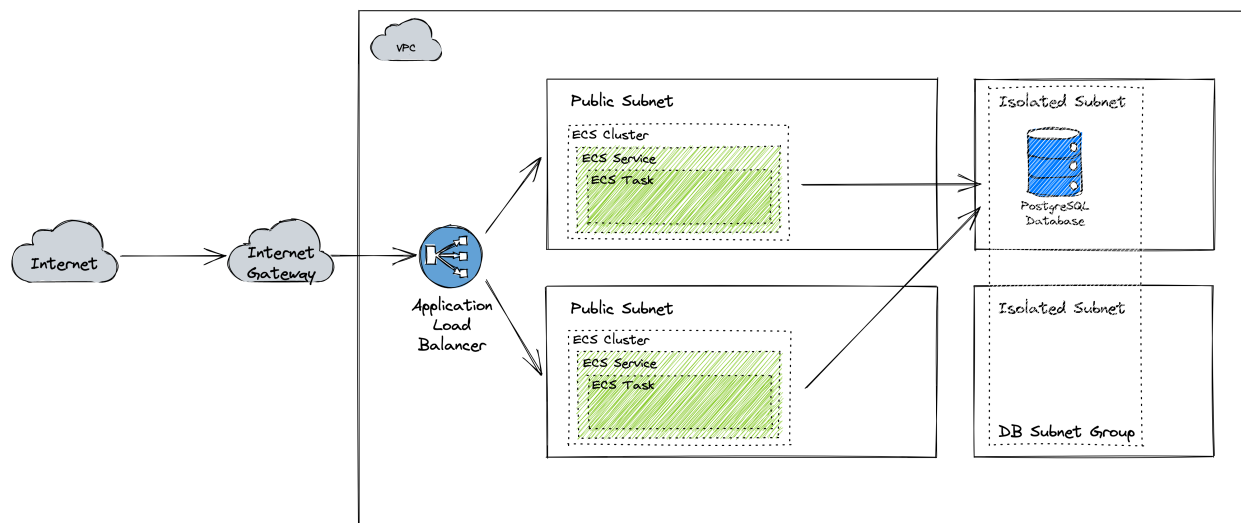
The application itself also needs access to the database, but we will include the permissions for that in the CDK app, so it's completely decoupled from developer IAM permissions.

## Creating a Database CDK App

As we did before for the Cognito resources, we'll now create a CDK app that allows us to deploy a database with a simple CLI command.

The following diagram gives an overview of the additional infrastructure we're

going to create for our database:



Deploying a PostgreSQL instance into our AWS infrastructure.

That is, in addition to the network and service stacks that we already have, we'll create a new CDK stack for the database. This stack will place a PostgreSQL database instance into the private subnets provided by our network stack. The application running in our service stack will then connect to the database.

## The PostgresDatabase CDK Construct

To create the necessary environment for our database we'll use the PostgresDatabase construct from our constructs library. You can browse its code on [GitHub](#).

The network stack we created earlier takes care of creating all the basic resources we need to run our Spring Boot application and our database. When deployed, it writes a few parameters to the SSM parameter store, including information about the isolated subnets it created for the database. The PostgresDatabase construct loads these parameters from the parameter store using the helper method `Network.getOutputParametersFromParameterStore()`:

```
public class PostgresDatabase extends Construct {

 // ...

 public PostgresDatabase(
 final Construct scope,
 final String id,
 final Environment awsEnvironment,
 final ApplicationEnvironment applicationEnvironment,
 final DatabaseInputParameters databaseInputParameters) {

 // ...

 Network.NetworkOutputParameters networkOutputParameters =
 Network.getOutputParametersFromParameterStore(
 this,
 applicationEnvironment.getEnvironmentName());

 // ...

 }
}
```

In case these parameters are not present in the parameter store (i.e., when the network stack hasn't been deployed before), the `PostgresDatabase` construct will fail to deploy.

The `PostgresDatabase` construct also takes an object of type `DatabaseInputParameters` as a parameter, which contains some configuration parameters it needs to set up the database.

Let's go through the code of the `PostgresDatabase` to see what it's doing with all these parameters.

### Database Security Group

First, the construct creates a *database security group* into which we'll later put the database. Also, it adds a *database subnet group*, which combines a set of subnets



into a group to be used by database instance:

```
CfnSecurityGroup databaseSecurityGroup = CfnSecurityGroup.Builder.create(
 this,
 "databaseSecurityGroup")
 .vpcId(networkOutputParameters.getVpcId())
 .groupDescription("Security Group for the database instance")
 .groupName(applicationEnvironment.prefix("dbSecurityGroup"))
 .build();

CfnDBSubnetGroup subnetGroup = CfnDBSubnetGroup.Builder.create(
 this,
 "dbSubnetGroup")
 .dbSubnetGroupDescription("Subnet group for the RDS instance")
 .dbSubnetGroupName(applicationEnvironment.prefix("dbSubnetGroup"))
 .subnetIds(networkOutputParameters.getIsolatedSubnets())
 .build();
```

We will later pass both the subnet group's name and the security group's ID into the `CfnDBInstance` construct that creates our database instance.

### Secret for Database Authentication

Next, we'll create a `Secret` called `databaseSecret`, which will be used as a password for the database:

```
ISecret databaseSecret = Secret.Builder.create(this, "databaseSecret")
 .secretName(applicationEnvironment.prefix("DatabaseSecret"))
 .description("Credentials to the RDS instance")
 .generateSecretString(SecretStringGenerator.builder()
 .secretStringTemplate(String.format(
 "{\"username\": \"%s\"}",
 username))
 .generateStringKey("password")
 .passwordLength(32)
 .excludeCharacters("@/\\\" ")
 .build())
 .build();
```

The `secretStringTemplate()` method's argument specifies a JSON structure with the user name. The argument of the `generateStringKey()` method defines that the generated password be added to this JSON structure in the password field. The resulting JSON string will look like this:

```
{
 "username": "<value of DBUserName parameter>",
 "password": "<generated password>"
}
```

Using the `excludeCharacters()` method we're excluding some characters from the password creation because they are not allowed in PostgreSQL RDS instances. We'd get an error message saying that "Only printable ASCII characters besides '/', '@', '', ' ' may be used" if the password contained any of these characters.

## Database Instance

The core of a database stack is, of course, the database instance:

```
String username = sanitizeDbParameterName(applicationEnvironment.prefix("dbUser"));

CfnDBInstance dbInstance = CfnDBInstance.Builder.create(this, "postgresInstance")
 .allocatedStorage(String.valueOf(databaseInputParameters.storageInGb))
 .availabilityZone(networkOutputParameters
 .getAvailabilityZones()
 .get(0))
 .dbInstanceClass(databaseInputParameters.instanceClass)
 .dbName(sanitizeDbParameterName(applicationEnvironment.prefix("database")))
 .dbSubnetGroupName(subnetGroup.getDbSubnetGroupName())
 .engine("postgres")
 .engineVersion(databaseInputParameters.postgresVersion)
 .masterUsername(username)
 .masterUserPassword(databaseSecret
 .secretValueFromJson("password")
 .toString())
 .publiclyAccessible(false)
 .vpcSecurityGroups(singletonList(databaseSecurityGroup.getAttrGroupId()))
 .build();
```

We pass the previously created `subnetGroup`, `databaseSecurityGroup`, and `databaseSecret` into the DB instance configuration.

The rest of the configuration parameters we either set statically – like the `publiclyAccessible` parameter, which we set to `false` – or we read them from the `databaseInputParameters` or the `networkOutputParameters`.

### Secret Attachment

Finally, we attach the secret to the database:

```
CfnSecretTargetAttachment.Builder.create(this, "secretTargetAttachment")
 .secretId(databaseSecret.getSecretArn())
 .targetId(dbInstance.getRef())
 .targetType("AWS::RDS::DBInstance")
 .build();
```

This associates the secret with the database, so we can take advantage of the [secret rotation feature](#) provided by the AWS Secrets Manager.

## Output Parameters

Finally, the `PostgresDatabase` construct exports some resources from the database stack, so we can use them from other stacks like our service stack:

```
StringParameter endpointAddress =
 StringParameter.Builder.create(this, "endpointAddress")
 .parameterName(createParameterName(
 this.applicationEnvironment,
 PARAMETER_ENDPOINT_ADDRESS))
 .stringValue(this.dbInstance.getAttrEndpointAddress())
 .build();

StringParameter endpointPort =
 StringParameter.Builder.create(this, "endpointPort")
 .parameterName(createParameterName(
 this.applicationEnvironment,
 PARAMETER_ENDPOINT_PORT))
 .stringValue(this.dbInstance.getAttrEndpointPort())
 .build();

StringParameter databaseName =
 StringParameter.Builder.create(this, "databaseName")
 .parameterName(createParameterName(
 this.applicationEnvironment,
 PARAMETER_DATABASE_NAME))
 .stringValue(this.dbInstance.getDbName())
 .build();

StringParameter securityGroupId =
 StringParameter.Builder.create(this, "securityGroupId")
```

```
.parameterName(createParameterName(
 this.applicationEnvironment,
 PARAMETER_SECURITY_GROUP_ID))
.stringValue(this.databaseSecurityGroup.getAttrGroupId())
.build();

StringParameter secret =
 StringParameter.Builder.create(this, "secret")
.parameterName(createParameterName(
 this.applicationEnvironment,
 PARAMETER_SECRET_ARN))
.stringValue(this.databaseSecret.getSecretArn())
.build();
```

We'll need the `endpointAddress`, `endpointPort`, `databaseName`, `securityGroupId`, and `secret` parameters in the service stack to connect our Spring Boot application to the database.

Note that parameters stored in the parameter store are not encrypted. That means they are visible for anyone with access to the parameter store. If that is not sufficient for your security purposes, make sure to store these parameters in the [AWS Secrets Manager](#) instead.

However, we keep the database username and password in a `Secret`, rather than as plaintext values. We only store the `Secret`'s ARN in the parameter store. Therefore, no sensitive information is shared through the parameter store.

## The Database CDK App

Finally, to be able to deploy our `PostgresDatabase` construct, we wrap it in a CDK app called `DatabaseApp` (the full code is available on [GitHub](#)):

```
public class DatabaseApp {

 public static void main(final String[] args) {
 App app = new App();

 String environmentName = (String) app
 .getNode()
 .tryGetContext("environmentName");

 String applicationName = (String) app
 .getNode()
 .tryGetContext("applicationName");

 String accountId = (String) app
 .getNode()
 .tryGetContext("accountId");

 String region = (String) app
 .getNode()
 .tryGetContext("region");

 Environment awsEnvironment = makeEnv(accountId, region);

 ApplicationEnvironment applicationEnvironment = new ApplicationEnvironment(
 applicationName,
 environmentName
);

 Stack databaseStack = new Stack(
 app,
 "DatabaseStack",
 StackProps.builder()
 .stackName(applicationEnvironment.prefix("Database"))
 .env(awsEnvironment)
 .build());

 new PostgresDatabase(
 databaseStack,
 "Database",
 awsEnvironment,
 applicationEnvironment,
 new PostgresDatabase.DatabaseInputParameters());
 }
}
```

```
 app.synth();
}

// ...
}
```

The app creates a Stack and adds a `PostgresDatabase` construct to it using the default `DatabaseInputParameters`. If we wanted to make any of the parameters in `DatabaseInputParameters` configurable, we could pass them into the app and then into the `DatabaseInputParameters` from there.

## Deploying the Database Stack

To interact with the new database stack, we add the scripts `database:deploy` and `database:destroy` to the `package.json` file in our CDK project.

We can now run these scripts from the command line to deploy a database stack:

```
npm run database:deploy
npm run database:destroy
```

## Modifying the Service Stack

What's left is to tell our Spring Boot application to use the new database. For this purpose, we modify the existing service stack that's responsible for deploying the Docker container our application resides in. We add the default environment variables Spring Boot uses for defining the database connection:

- `SPRING_DATASOURCE_URL`,
- `SPRING_DATASOURCE_USERNAME`, and
- `SPRING_DATASOURCE_PASSWORD`.

Spring Boot will automatically read these environment variables and create a data source pointing to our new database.

To this end, in the `ServiceApp` we first load a `NetworkOutputParameters` object from the `PostgresDatabase` construct:

```
PostgresDatabase.DatabaseOutputParameters databaseOutputParameters =
 PostgresDatabase.getOutputParametersFromParameterStore(
 parametersStack,
 applicationEnvironment);
```

The method `getOutputParametersFromParameterStore()` is a convenience method we built into the database construct. This method loads all the output parameters we discussed above into a single object.

Then, we pass these parameters into the `environmentVariables()` method, which creates a map of all environment variables that should be injected into the Docker container of our Spring Boot app:

```
public class ServiceApp {

 // ...

 static Map<String, String> environmentVariables(
 Construct scope,
 PostgresDatabase.DatabaseOutputParameters databaseOutputParameters,
 String springProfile) {

 Map<String, String> vars = new HashMap<>();

 String databaseSecretArn = databaseOutputParameters.getDatabaseSecretArn();
 ISecret databaseSecret = Secret.fromSecretCompleteArn(
 scope,
 "databaseSecret",
 databaseSecretArn);

 vars.put("SPRING_DATASOURCE_URL",
 String.format("jdbc:postgresql://%s:%s/%s",
 databaseOutputParameters.getEndpointAddress(),
```



```
 databaseOutputParameters.getEndpointPort(),
 databaseOutputParameters.getDbName()));

vars.put("SPRING_DATASOURCE_USERNAME",
 databaseSecret.secretValueFromJson("username").toString());

vars.put("SPRING_DATASOURCE_PASSWORD",
 databaseSecret.secretValueFromJson("password").toString());

// ...

return vars;
}

// ...
}
```

We combine the parameters `EndpointAddress`, `EndpointPort`, and `DBName` to create a valid JDBC URL of this format:

```
jdbc:postgresql://<EndpointAddress>:<EndpointPort>/<DBName>
```

We load the username and password from the `Secret` we created in the database stack. Note that this secret never leaves the AWS servers! We don't need to put it into a configuration file anywhere!

Also note that, since the service stack now depends on the output parameters of the database stack, we need to deploy the database stack before the service stack.

At this point, our AWS-specific database infrastructure work is done! We now can reap the rewards of our efforts: We can use our newly created database stack and the PostgreSQL database it contains from our Spring Boot application like any other PostgreSQL database.

## Strategies for Initializing the Database Structure

Now that the database has been deployed and is up and running, we need to create a database schema for our application. There are various ways to approach this task. Let's look at some of those.

### Creating the Database Structure Manually

The most basic, framework-agnostic technique would be to simply trigger the execution of an initialization script after the application start, for example by using the `@PostConstruct` annotation. This approach provides us with more control over the initialization process but it also tends to be more brittle and error-prone because manually created DDL code might become outdated or out-of-sync with the application source code. Moreover, because such DDL code is not managed by the framework itself this can lead to unwanted results. For example, scripts intended for development environments might be inadvertently executed in production, too!

### Standard DDL Scripts: `schema.sql`, `data.sql`

Spring Boot provides a standard way of creating database schemas and importing data. It does so via SQL scripts in the root classpath that follow specific naming conventions. A `schema.sql` script located under the root classpath will create the required database schema whereas a `data.sql` will use SQL `INSERTS` to fill that schema with actual data.

This approach, while standardized and more agnostic than the previous one, comes with similar disadvantages: While with this technique the files for generating the database structure and populating the database with initial data are

managed by Spring Boot, keeping these files in sync with the application source code can still become an issue over time.

## DDL Generation with JPA and Hibernate

Database APIs such as *JPA* and *ORM* (object-relational mapping) frameworks such as Hibernate can generate or modify a database schema automatically upon application start. The advantage of this approach is that the generated database structure will automatically correspond to our model classes and application code in general. Therefore, we don't need to manually maintain separate SQL scripts in this case. However, (re)initializing the database in this way might not always yield the exact structure we want because these frameworks come with conventions. While the default behavior can be customized, this can result in complex, hard-to-understand configuration. Another huge downside to this technique is that it only manages a single, authoritative state for our database structure. This means that at any point there's only ever one definition of our database structure (the current one), and maintaining a version history of the database structure quickly becomes cumbersome.

## Database Migration Tools: Liquibase and Flyway

This is where database migration tools such as [Liquibase](#) or [Flyway](#) come into play. These tools allow us to maintain different versions of our database structure that correspond to our application source code at any given time. In a way, these tools are version control systems for database schemas.

Spring Boot integrates with both of these tools. We chose Flyway for its simplicity and for being the de-facto default approach for Spring Boot applications. Rather than an XML-based language for defining migrations, such as the one

used by Liquibase, Flyway uses plain old SQL scripts for applying migrations. While not as RDBMS-agnostic, this approach is much simpler in that doesn't incur the additional complexity that comes with yet another language we need to learn to maintain our application.

However, subject to your requirements, Liquibase can still be a valid choice. For example, depending on the environment, you might want to support multiple different RDBMS like PostgreSQL or Oracle Database without having to maintain individual migration scripts for each SQL dialect that comes with those. In that case, due to its more agnostic approach, Liquibase might have its merits.

## **Configuring the Database in the Todo App**

Having chosen Flyway as our tool for initializing and managing the database schema, let's now dive into the code of our sample application to see how we can configure the new database. If you're already familiar with configuring a Spring Boot app to use a SQL database, you can safely skip this section.

### **Connecting to the Database**

To connect to the PostgreSQL database instance our sample application needs a few additional dependencies:

```
dependencies {
 // ...
 implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
 runtimeOnly 'org.postgresql:postgresql'
 // ...
}
```

`org.postgresql:postgresql` contains the JDBC driver for connecting to the PostgreSQL database whereas `org.springframework.boot:spring-boot-starter-data-jpa` provides us with [Spring Data JPA](#) (and its dependencies), a widely used library for accessing databases from within Spring Boot applications.

Now, once we've started the service stack, our Spring Boot application will automatically connect to the PostgreSQL database using the standard `SPRING_DATASOURCE_URL`, `SPRING_DATASOURCE_USERNAME`, and `SPRING_DATASOURCE_PASSWORD` parameters we set before.

## Initializing the Database

Because our database is now up and running and our sample Todo application is connected to it, we're now able to use any of the strategies for initializing our database discussed above.

Since we've decided to use Flyway for that purpose, we add this dependency to our `build.gradle` file:

```
dependencies {
 // ...
 implementation 'org.flywaydb:flyway-core'
 // ...
}
```

We can now add SQL migration files to the default folder used by Flyway for PostgreSQL RDBMS. This folder is located under `db/migration/postgresql` in the application's `src/main/resources` folder.

Any `.sql` files we place in that folder that follow the Flyway naming conventions will automatically be executed upon application start.

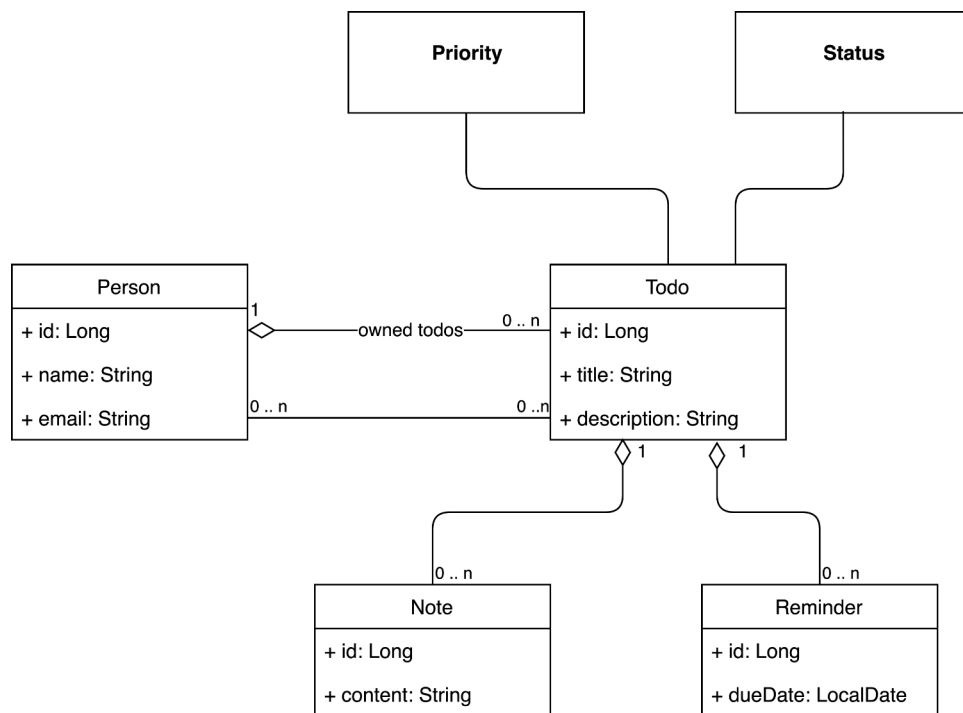
We can now start to implement business logic that requires persistent data storage for our todo entities! We can do this in the same way as we would for any Spring- and JPA-based application, for example with JPA's `@Entity` annotation, JPA-based repositories, or Spring's `@Repository` stereotype.

## Using the Database for Storing and Retrieving Todos

Now that we've prepared the surrounding infrastructure and our Spring Boot app is configured to use our new PostgreSQL database, we can finally put it to use in our application code.

If you're familiar with using JPA in a Spring Boot application, you may safely skip this section.

The domain model of our application is structured around the “Todo” entity, as shown in this diagram:



Domain model of the Todo app.

As mentioned in the chapter [The Sample Todo Application](#), our application follows a package-by-feature structure. This means that the feature folders `collaboration`, `person`, `registration`, and `todo` contain the code artifacts related to each of those features. These packages include controllers, service interfaces (and their implementations), Spring Data JPA repositories, and data model classes.

We'll have a closer look at some of the classes from the `dev.stratospheric.todo` package to examine how we use the newly created PostgreSQL database for our application.

We'll focus on the `Todo` class and how it's used. This class is annotated with the `@Entity` annotation from the `jakarta.persistence` package (the API provided by *JPA / Jakarta Persistence*). This annotation marks the class as a database entity. By default, the unqualified class name (as opposed to its *fully-qualified name*, which would include the package name) is used as the entity's database table name.

As mentioned previously, we use Flyway for database initialization and migration. Therefore, we need to provide an SQL *DDL* migration script that creates this database table upon application start. As per Flyway conventions we've named this script `V001__INIT_DATABASE.sql` and placed it in the `/src/main/resources/db/migration/postgresql` folder.

The SQL script contains this statement:

```
-- ...

create table TODO
(
 ID BIGSERIAL not null primary key,
 DESCRIPTION VARCHAR(255),
 DUE_DATE DATE,
 PRIORITY INTEGER,
 STATUS VARCHAR(255),
 TITLE VARCHAR(255),
 OWNER_ID BIGINT,
 constraint FK_TODO_OWNER
 foreign key (OWNER_ID) references PERSON (ID)
);

-- ...
```

This statement will create the required database table with the columns needed for storing the information for each todo. When taking a closer look at the `Todo` class we can see that these columns are reflected in the class's attributes:



```
// ...

@Entity
public class Todo {

 @Id
 @GeneratedValue(strategy = GenerationType.IDENTITY)
 private Long id;

 @NotBlank
 @Size(max = 30)
 private String title;

 @Size(max = 100)
 private String description;

 private Priority priority;

 @NotNull
 @Future
 @DateTimeFormat(pattern = "yyyy-MM-dd")
 private LocalDate dueDate;

 @Enumerated(EnumType.STRING)
 private Status status;

 @ManyToOne
 @JoinColumn(name = "owner_id")
 private Person owner;

 // ...
}
```

Again, as with the entity name and the table name, the attribute names by default match the column names. Most of those attributes are annotated with one or more annotations. These annotations allow us to further specify rules and constraints that should apply to an attribute.

## Primary Key and Object Identity

The `id` attribute is annotated with `@Id` from the `jakarta.persistence` package marking this attribute as the entity's unique identifier (or: *primary key*).

The `@GeneratedValue` annotation (also from the `jakarta.persistence` package) denotes that the attribute's value is generated automatically. The `strategy = GenerationType.IDENTITY` argument further specifies that this value is provided through an identity column in the database table. This behavior in turn is enabled by this column definition from the SQL migration script, specifically the `BIGSERIAL` and `primary key` keywords:

```
-- ...

ID BIGSERIAL not null primary key

-- ...
```

## Constraints and Validation

Some columns are annotated with annotations from the package `jakarta.validation.constraints`, like `@NotBlank`, `@Size`, or `@Future`. Such annotations allow us to define the rules – or: *constraints* – we'd like to apply to each of the attributes and their value. For example, the `todo`'s title is supposed to contain a non-empty (*not blank*) string with 30 characters at most.

## Storing and Retrieving Information

Now that we have defined our database entity we can use it for storing and retrieving information. With Spring Data JPA, the abstraction for doing so is the `JpaRepository` interface from the

`org.springframework.data.jpa.repository` package. This interface provides us with a set of methods for manipulating and retrieving data:

```
public interface JpaRepository<T, ID> extends PagingAndSortingRepository<T, ID>, QueryByExampleExecutor<T> {

 @Override
 List<T> findAll();

 @Override
 List<T> findAll(Sort sort);

 @Override
 List<T> findAllById(Iterable<ID> ids);

 @Override
 <S extends T> List<S> saveAll(Iterable<S> entities);

 // ...
}
```

To use this for a specific database entity such as our `Todo` class we have to extend this interface with our own `TodoRepository` interface:

```
public interface TodoRepository extends JpaRepository<Todo, Long> {
 List<Todo> findAllByOwnerEmailOrderByIdAsc(String email);
}
```

This interface specifies a `JpaRepository` that's responsible for persisting `Todo` entities with a `Long`-typed ID. Moreover, we've added a `findAllByOwnerEmailOrderByIdAsc()` method that allows us to find all `Todos` whose owner has the email address given by the method's `email` argument. Spring Data JPA uses a query generation mechanism that derives the required SQL query from the method name (see [Defining Query Methods](#) from the Spring Data JPA documentation for more information).

The `TodoRepository` then can be injected into other classes, like our `TodoService`, via Spring's dependency injection:

```
// ...

@Service
public class TodoService {

 private final TodoRepository todoRepository;
 private final PersonRepository personRepository;

 public TodoService(
 TodoRepository todoRepository,
 PersonRepository personRepository) {
 this.todoRepository = todoRepository;
 this.personRepository = personRepository;
 }

 public Todo saveNewTodo(Todo todo) {
 // ...

 return todoRepository.save(todo);
 }
}
```

Spring will automatically inject the `TodoRepository` into the constructor of `TodoService`. With this dependency, we now can insert a new row into the database table by calling `todoRepository.save()`.

The `TodoService`, in turn, is injected into the `TodoController`, again using constructor injection, where it is used for creating, retrieving, updating, and deleting `Todos` in various controller methods. These controller methods are mapped to HTTP paths within our application that can be accessed through HTTP GET (in case of the methods annotated with `@GetMapping`) and POST (for those methods annotated with `@PostMapping`) requests.

Finally, the Thymeleaf template in our application's [resources/templates](#) folder, specifically `dashboard.html`, `show.html`, and `edit.html`, make use of these controller methods to allow the user to display and edit todos.

## Enabling Local Development

One thing is still missing in our setup: In most cases, we wouldn't want to wait until our entire AWS infrastructure has been redeployed through our continuous deployment pipeline after a code change. Instead, we want to test changes locally.

That's where a local database instance comes into play. We can use Docker to spin one up by adding a service to the `docker-compose.yml` file located in our application's root directory:

```
services:
 postgres:
 image: postgres:12.9
 ports:
 - 5432:5432
 environment:
 - POSTGRES_USER=stratospheric
 - POSTGRES_PASSWORD=stratospheric
 - POSTGRES_DB=stratospheric
 # ...
```

With these lines added to the `docker-compose.yml` file, we can now run `docker-compose up` from the command line in our application's root directory to start a PostgreSQL instance alongside other services already defined in our `docker-compose.yml`.

Now, all we have to do is add these lines to a `application-dev.yml` properties file in our application's `src/main/resources` folder:

```
spring:
 datasource:
 url: jdbc:postgresql://localhost:5432/stratospheric
 username: stratospheric
 password: stratospheric
 # ...
```

Since our `build.gradle` file already configures Gradle to use `dev` as Spring Profile (by using `-Dspring.profiles.active=dev` as a JVM argument) these settings will be picked up automatically when running `./gradlew bootrun`:

```
// ...
bootRun {
 jvmArgs = [
 "-agentlib:jdwp=transport=dt_socket,server=y,suspend=n,address=5005",
 "-Dspring.profiles.active=dev",
]
}
```

## 12. Sharing Todos with Amazon SQS and Amazon SES

With this chapter, we'll implement the next feature for our Todo application and integrate two new AWS services: SQS (Simple Queue Service) and SES (Simple Email Service).

So far, the users of our application only work on their todos by themselves. As this seems rather boring, we'd like to foster some collaboration among our user base.

The upcoming feature allows the owner of a todo to share it with other users of our application. This way, they can collaboratively work on the task at hand. Similar to the workflow of typical collaborative applications, the user a todo has been shared with gets a notification and has to accept the collaboration first.

Technically speaking, we're going to decouple this feature into two operations. Whenever a user decides to share one of their todos, we'll first store this request in an SQS queue. The queue acts as a buffer, and another part of our application will then handle the incoming requests and send out emails.

This chapter has two parts that iteratively build the sharing feature. We'll start with the messaging component and learn how to send and receive SQS messages. Next comes the email sending part, where we integrate Amazon SES.

Let's get started and first understand how the fully managed messaging service from AWS (SQS) can help us build resilient and decoupled applications.

## Using Amazon SQS for Asynchronous Workloads

Before we jump into integrating Amazon SQS with our Spring Boot application, let's take a look at how this AWS service works.

### Introduction to Amazon SQS

Amazon SQS is a fully managed messaging service. We can use this service to pass messages between different parts of our system in a highly scalable fashion. It allows point-to-point communication where only one receiver will handle the message at a given time. Amazon SQS can further help us integrate or decouple components in a distributed architecture.

The messages that we process with SQS can have a payload with a maximum size of 256 KB. It allows server-side encryption of our messages, and we can control access to it. We interact with SQS via a generic HTTPS API, similar to other AWS services. Depending on the configuration, SQS can persist our message for a “retention period” of up to 14 days.

Scalability, durability, and reliability of our processing with SQS depend on the queue type. AWS offers two different SQS types: FIFO and Standard.

With the *Standard* queue type, messages will get delivered with a best-effort ordering. SQS won't ensure a strict message order for this queue type, and occasionally the messages are delivered out of order. Furthermore, the messages are delivered at least once. Our receiver might consume the same message more than once. Despite having these downsides, this type of queue offers nearly unlimited throughput (according to the [AWS documentation](#)). It's the cheapest option, too.

As the name implies, the *FIFO* queue type follows a first-in-first-out delivery



model. This type of queue guarantees to deliver messages in the same order they were sent to the queue. Each message will be processed exactly once, avoiding that our consumer receives duplicates. The strict consistency results in a 25% higher AWS bill compared to the *Standard* queue type. When it comes to scalability, *FIFO* queues will handle up to 3,000 messages per second. We can increase this limit by getting in contact with AWS support.

The [pricing model](#) of Amazon SQS has no upfront costs. After we exceed the first free one million requests per month, we're charged for the number of requests and the outbound data transfer.

The basic operations we perform against the SQS API are `SendMessage`, `ReceiveMessage`, and `DeleteMessage`. Every message will remain in the queue until a consumer actively deletes it from the queue. In other words, Amazon SQS doesn't automatically remove a message when it's consumed, but the consumer has to acknowledge the successful processing of the message by deleting it.

SQS clients have to proactively perform the `ReceiveMessage` operation regularly to consume and check for new messages. Amazon SQS won't push messages to a particular client.

To avoid parallel message consumption, Amazon SQS has the concept of a [visibility timeout](#). This timeout defines the period after which a message will be made available again to all consumers after one consumer has received it. Whenever a consumer doesn't delete the message within the given timeout period (due to slow processing, for example), SQS will replay the message by including it in the response of a following `ReceiveMessage` API call. Hence, we should configure this value to the maximum amount of time it takes to process a single message. We can define the visibility timeout for both the entire queue or on each `ReceiveMessage` request individually.

To avoid receiving the same unprocessable message over and over again (for

example due to an invalid payload or a downtime of a remote system), Amazon SQS supports the concept of so-called Dead-Letter Queues (DLQ for short).

### **Dead-Letter Queues**

Each Amazon SQS message has a `ReceiveCount` attribute that stores a counter and tracks how often the message has been consumed already. As part of the queue configuration, we can define a `maxReceiveCount` to specify how many processing attempts our messages have until they're moved to a dead-letter queue.

A dead-letter queue is just another Amazon SQS queue that stores unprocessable messages. The type of this queue has to match the type of the source queue. In other words, a FIFO queue can only target a FIFO queue as a dead-letter queue. DLQs are optional, so we can define Amazon SQS queues without specifying a DLQ.

However, it's a general best practice to create a DLQ for each of our processing queues due to several reasons:

- It helps to analyze and debug error scenarios.
- We isolate problematic messages for further investigations.
- We reduce the load on our system if there are multiple (if not thousands) of unprocessable messages.
- We get improved observability, as we can define alarms for our dead-letter queues to detect failures early.
- We don't block the message processing by consuming the same faulty message over and over.

The strategy for handling these dead messages highly depends on the use case and error scenario. In case there was a temporary hiccup (for example a remote

system we depend on was not available), we can move the messages back to our source queue for processing.

### **Amazon SQS vs. AWS SNS vs. Amazon MQ**

Amazon SQS is not the only messaging service in the AWS service portfolio. While Amazon SQS is a queuing service (point-to-point), with AWS SNS (Simple Notification Service), we can inform multiple receivers for each message. This is known as the publish-subscribe pattern that internally works with topics that several consumers subscribe to.

Both Amazon SQS and AWS SNS are highly scalable and require zero setup as AWS fully manages them.

In contrast to that, with Amazon MQ we can bootstrap a managed Apache ActiveMQ or RabbitMQ instance. Both are message brokers that support multiple traditional APIs (JMS) as well as a variety of messaging protocols (AMQS, MQTT, STOMP, etc.).

We'll discuss Amazon MQ and ActiveMQ, together with other messaging and notification capabilities of AWS, in greater detail as part of the upcoming chapter [\*Push Notifications with Amazon MQ\*](#).

That's enough theory for now. Let's dive right into implementing the sharing feature. As a first step, we'll create the required AWS resources with AWS CDK.

### **Creating the Amazon SQS Setup with CDK**

As a quick reminder, we'll create the Amazon SQS queue to technically decouple the todo sharing feature and send a message to SQS whenever a user decides to invite a collaborator for their todo. Sending the actual invitation email then is the responsibility of the SQS message consumer.

For our use case, the standard Amazon SQS queue fits our purpose. We don't need strict message ordering because it doesn't really matter which email we send first. Also, we don't care much if we receive a message twice, because we could store in the database who we sent an email to already, so that the same user doesn't get the email twice.

We'll create a new CDK app for the messaging-related parts of our application. You can find the code for this app on [GitHub](#). For this `MessagingApp` we don't need any additional input parameters except the default ones:

```
public class MessagingApp {

 public static void main(final String[] args) {
 App app = new App();

 // omitted standard configuration values like the AWS region and sanity checks

 Environment awsEnvironment = makeEnv(accountId, region);

 ApplicationEnvironment applicationEnvironment = new ApplicationEnvironment(
 applicationName,
 environmentName
);

 new MessagingStack(
 app,
 "messaging",
 awsEnvironment,
 applicationEnvironment);

 app.synth();
 }
}
```

As the Amazon SQS CDK module comes with stable level 2 constructs, we can conveniently create our Amazon SQS resources and don't have to work with low-level CloudFormation constructs. Inside the `MessagingStack`, we create both our Amazon SQS processing and dead-letter queues:

```
this.todoSharingDlq = Queue.Builder.create(this, "todoSharingDlq")
 .queueName(applicationEnvironment.prefix("todo-sharing-dead-letter-queue"))
 .retentionPeriod(Duration.days(14))
 .build();

this.todoSharingQueue = Queue.Builder.create(this, "todoSharingQueue")
 .queueName(applicationEnvironment.prefix("todo-sharing-queue"))
 .visibilityTimeout(Duration.seconds(30))
 .retentionPeriod(Duration.days(14))
 .deadLetterQueue(DeadLetterQueue.builder()
 .queue(todoSharingDlq)
 .maxReceiveCount(3)
 .build())
 .build();
```

For both queues, we set the `retentionPeriod` to the maximum duration of 14 days. This gives us enough time to analyze the underlying issue even if a first alarm gets overlooked, or our development team is on Christmas vacation.

We set the `visibilityTimeout` to 30 seconds, which should give our Spring Boot application plenty of time to store the collaboration request in the database and send an email. We can always tweak this value as soon as we have a good understanding of our application's behavior in production.

We connect the main processing queue with the dead-letter queue by passing the field `todoSharingDlq` to the `DeadLetterQueue` builder. With the `maxReceiveCount` set to 3, Amazon SQS will move the message to the DLQ if our application fails to delete the message from the queue at the fourth processing attempt.

Unless we call the method `fifo(true)` on the construct, the created SQS queue will be a standard SQS queue. In case we opt-in for a FIFO queue type, the queue name must end with the `.fifo` suffix.

Next, we have to expose the name of the SQS queue our application will connect to:

```
StringParameter.Builder.create(this, "todoSharingQueueName")
 .parameterName(createParameterName(
 applicationEnvironment,
 PARAMETER_TODO_SHARING_QUEUE_NAME))
 .stringValue(this.todoSharingQueue.getQueueName())
 .build();
```

This stores the name of the queue in the SSM parameter store so we can later use it to configure our Spring Boot application. Spring Cloud AWS accepts both the logical and physical names for a queue. Internally, it will fetch the queue URL as the AWS SDK for SQS requires this URL for receiving messages.

We won't connect to the dead-letter queue from our application. Messages might end up in the DLQ for several reasons. We can't really implement an automatic procedure to understand what went wrong. Therefore, manual inspection is the only feasible approach here. For now, though, it's enough to isolate unprocessable messages. Later on, we can create an AWS CloudWatch alarm to get notified as soon as the first message arrives in this queue.

With the Amazon SQS resource definition in place, our application now requires access to the processing queue. Similar to how we have configured access to Amazon Cognito in the chapter [Building User Registration and Login with Cognito](#), we add a new `PolicyStatement` as part of our `Service` construct:

```

.withTaskRolePolicyStatements(List.of(
 // ... skipping other PolicyStatements,
 PolicyStatement.Builder.create()
 .sid("AllowSQSAccess")
 .effect(Effect.ALLOW)
 .resources(List.of(
 String.format("arn:aws:sqs:%s:%s:%s", region, accountId,
 messagingOutputParameters.getTodoSharingQueueName())
))
 .actions(Arrays.asList(
 "sqs:DeleteMessage",
 "sqs:GetQueueUrl",
 "sqs:ListDeadLetterSourceQueues",
 "sqs:ListQueues",
 "sqs:ListQueueTags",
 "sqs:ReceiveMessage",
 "sqs:SendMessage",
 "sqs:ChangeMessageVisibility",
 "sqs:GetQueueAttributes"))
 .build()
))
))

```

This gives our application the necessary permissions to send, receive, and delete messages for any Amazon SQS queue as we've used the `*` wildcard. We can now start to integrate Amazon SQS into our Todo application.

## Using Amazon SQS for Our Todo Application

As a first step, we have to add a new dependency to our project:

```

dependencies {
 // omitted other dependencies
 implementation 'io.awspring.cloud:spring-cloud-starter-aws-messaging'
}

```

The version of this dependency is managed by the Spring Cloud BOM that we already included as part of the core dependencies in the chapter [\\* The Sample](#)

**Todo Application\***. The Spring Cloud AWS Messaging module comes with the following features for both Amazon SQS and AWS SNS:

- Annotation-driven listener/notification endpoints.
- Integration with the Spring Messaging API (fully for SQS, partially for SNS).
- Serialization support for messages (which SQS only knows as strings).
- Convenient access via `SqsTemplate` (SQS) and `NotificationMessagingTemplate` (SNS).

In the upcoming sections, we won't cover any AWS SNS-specific features of Spring Cloud AWS and focus on Amazon SQS.

To set our Spring Boot application up for SQS, there's only one thing left to do. The `SqsTemplate` that we'll use to conveniently access Amazon SQS is not auto-configured by default. In favor of this abstraction and to avoid any interaction with the low-level AWS Java SDK, we add a bean of type `SqsTemplate` to our application context:

```
@Configuration
public class MessagingTemplateConfig {

 @Bean
 public SqsTemplate sqsTemplate(
 SqsAsyncClient sqsAsyncClient) {
 return SqsTemplate.newTemplate(sqsAsyncClient);
 }
}
```

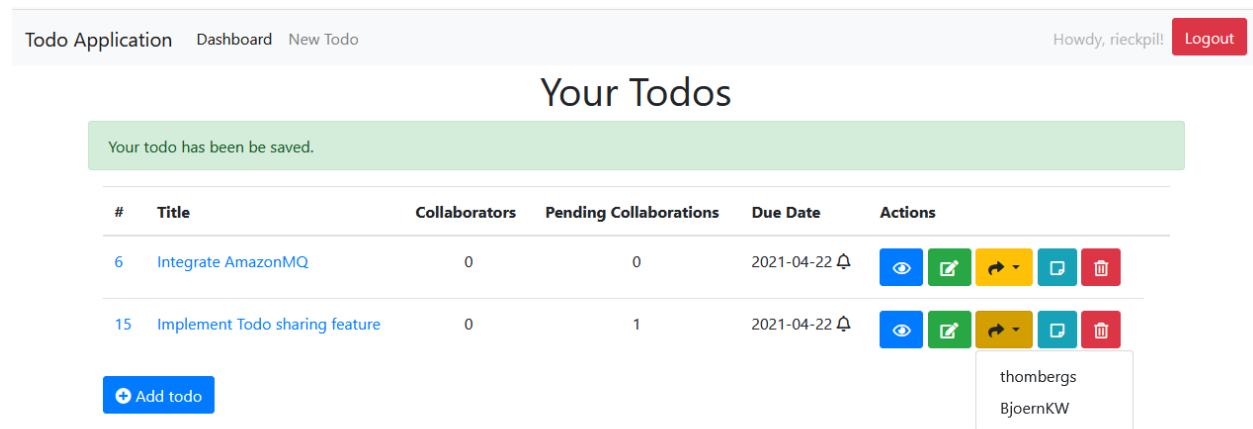
Spring Cloud AWS auto-configures the Amazon SQS Java SDK `AmazonSQSAsync` for us.

Let's start with the first step and send a message to SQS whenever a user decides to invite a collaborator to one of their todos.



## Sending Messages to Amazon SQS

To allow users to add collaborators, we enhance the action items of a todo with a drop-down element. Inside the drop-down, our users will find a list of all available collaborators to share their tasks with. This new element is displayed for each todo that the user owns:



### Sharing a todo with a collaborator.

We create this element as part of the `<table>` that visualizes each todo inside the `dashboard.html` view:

```
<div class="dropdown-menu" aria-labelledby="dropdownMenuLink">

 No collaborator available

 <form th:method="POST"
 th:each="collaborator : ${collaborators}"
 th:action="@{/todo/{todoId}/collaborations/{collaboratorId}
 (todoId=${todo.id}, collaboratorId=${collaborator.id})}">
 <button
 th:text="${collaborator.name}"
 type="submit"
 name="submit"
 class="dropdown-item">
 </button>
 </form>
</div>
```

The `{collaborators}` attribute is populated as part of the Spring MVC controller endpoint for the dashboard view and contains a list of all available collaborators. Users will be able to share their todo with any user of our application except themselves and only if they are the todo owner.

The `TodoCollaborationController` takes incoming collaboration requests and passes them to the `TodoCollaborationService` to start the sharing process:

```
@Controller
@RequestMapping("/todo")
public class TodoCollaborationController {

 private final TodoCollaborationService todoCollaborationService;

 public TodoCollaborationController(
 TodoCollaborationService todoCollaborationService) {
 this.todoCollaborationService = todoCollaborationService;
 }

 @PostMapping("/{todoId}/collaborations/{collaboratorId}")
 public String shareTodoWithCollaborator(
 @PathVariable("todoId") Long todoId,
 @PathVariable("collaboratorId") Long collaboratorId,
 RedirectAttributes redirectAttributes
) {

 String collaboratorName = todoCollaborationService
 .shareWithCollaborator(todoId, collaboratorId);

 // add success message

 return "redirect:/dashboard";
 }
}
```

In this service, we first verify that both the collaborator and the todo exist in our database. Next, we ensure that the user is the owner of the Todo and no equivalent collaboration request is currently active.

We store each new collaboration request in our PostgreSQL database. The `TodoCollaborationRequest` JPA entity contains the relevant information about the invite. For an additional layer of security, we create a random token for each collaboration request that is required for accepting the invite:

```
@Service
@Transactional
public class TodoCollaborationService {

 // more fields (Spring Data JPA repositories, etc.)

 private final SqsTemplate sqsTemplate;
 private final String todoSharingQueueName;

 public String shareWithCollaborator(Long todoId, Long collaboratorId) {

 // check if todo and collaborator exists
 // return if there's already an active collaboration request

 TodoCollaborationRequest collaboration = new TodoCollaborationRequest();
 collaboration.setToken(UUID.randomUUID().toString());
 collaboration.setCollaborator(collaborator);
 collaboration.setTodo(todo);

 todo.getCollaborationRequests().add(collaboration);

 todoCollaborationRequestRepository.save(collaboration);

 sqsTemplate.send(todoSharingQueueName,
 new TodoCollaborationNotification(collaboration));

 return collaborator.getName();
 }
}
```

To transmit the collaboration information, we populate a `TodoCollaborationNotification` object. This acts as a data transfer object (DTO) and contains the relevant information about the collaboration request.

Instead of including a full `TodoCollaborationNotification` object in the SQS

message, we could make the message contain only the ID of the `TodoCollaborationRequest` entity. In this scenario, the consumer of a message would have to use this ID to load the request from the database again. We opted for a “flattened” approach that includes all information necessary to send an email. This further decouples the email sending feature from the rest of our application. There’s one downside to this solution, however: The invited person might see outdated information about the todo since the owner can update the todo between sending the collaboration request and waiting for the confirmation. However, we decided that this is acceptable.

As the last step, we use the `SqsTemplate` to send a message to a specified SQS queue. The `send()` method will serialize the Java object to a JSON string before sending it to SQS.

With this setup, new messages will now queue up inside our SQS queue. In case we don’t consume the messages within 14 days (the `retentionPeriod` of our queue), they’ll be gone. To avoid this scenario and unhappy users, we now need to consume the messages. Let’s connect to the SQS queue as a consumer to further process the collaboration requests.

### **Receiving Amazon SQS Messages with Spring Cloud AWS**

The responsibility of our consumer is to inform the invited collaborator via email. For demonstration purposes and to avoid bootstrapping a new service, we’ll self-consume the messages in the same application.

Why do we then go the extra mile and send a message to SQS – couldn’t we just send out the email right inside the `TodoCollaborationService`? Of course we could, but this would couple the email sending with the rest of our application. By using an SQS queue in between, we decouple the inviting logic from the email sending logic and gain the flexibility to easily move these features around within our codebase (or even into another codebase).

Also, the SQS queue between these two operations acts as a buffer. This gives us an implicit retry mechanism because, with our `maxReceiveCount`, we try processing the message up to four times. As we're building a distributed system and have to integrate with a cloud provider, literally everything can go wrong. Hence, we should be prepared for failure. On top of this, we can control the email sending throughput by throttling the number of messages we consume. This helps with staying within the boundaries of potential email sending limits. Spring Cloud AWS provides two ways of consuming messages:

- using an annotation-driven listener, and
- using the `receive()` method of the `SqsTemplate`.

While calling `receive()` explicitly is useful for on-demand message consumption, with an annotated listener class, we're continuously polling messages with a background thread and can process messages almost in real time. We can also directly interact with the SQS client (`AmazonSQS`) via the AWS Java SDK but in that case, we'd lose the Spring Messaging abstraction and have to deal with the low-level API instead.

The actual listener class is a standard Spring bean. Any public method of this class that uses the `@SqsListener` annotation acts as an SQS handler method. This annotation expects a list of logical or physical SQS queue names. For our use case, we only listen to one queue, but we could also define the same handler for multiple SQS queues:

```
@Component
public class TodoSharingListener {

 @SqsListener(value = "${custom.sharing-queue}")
 public void listenToSharingMessages(TodoCollaborationNotification payload) {

 // upcoming implementation to notify collaborator via email and AWS ES
 LOG.info("Incoming todo sharing payload: {}", payload);

 }
}
```

As part of the listener's method argument, we can define the expected Java object. Spring Cloud AWS, and in this particular case its Spring Messaging integration, is responsible for extracting the message payload and resolving it using a `PayloadMethodArgumentResolver`. By default, this makes use of the `MappingJackson2MessageConverter` for deserializing the incoming JSON message (of type `String`) to a Java object using Jackson's `ObjectMapper`.

### Acknowledging Amazon SQS Messages with Spring Cloud AWS

So far, we consume the incoming SQS message, extract its payload and convert it to a Java object. One key responsibility of the SQS listener is still missing: deleting the message after processing it.

Recall that a message will remain inside SQS until the consumer explicitly deletes it. The process of receiving a message won't delete it from the queue. Hence, after (successfully) processing the message, we have to acknowledge it by deleting the message from the queue.

The AWS Java SQS SDK provides a low-level method for this purpose: `deleteMessage()`. This method expects the message's receipt handle. This handle is up to 1024 characters long and identifies a particular message consumption event.

Spring Cloud AWS provides an abstraction on top of this so that we don't have to keep track of all receipt handles and manage the message deletion on our own. For annotation-driven SQS listeners, we can specify a deletion policy. In case we're using the `receive()` message of the `SqsTemplate`, the message will be deleted right away.

Spring Cloud AWS defines the following four deletion policies as part of the `SqsMessageDeletionPolicy` enum:

- **ALWAYS:** The outcome of the processing is irrelevant (success or exception) as the message will be deleted every time.
- **NEVER:** We're in charge of deleting the message.
- **NO\_REDRIIVE:** Deletes the message on success and in case of an exception only if no re-drive policy is configured for the queue. This is the default.
- **ON\_SUCCESS:** Deletes the message unless an exception is thrown during the processing.

Since, with our dead-letter queue, we've configured a re-drive policy for our SQS setup, the default `NO_REDRIIVE` deletion policy is sufficient. Whenever our message processing finishes without an exception, Spring Cloud AWS will delete the message. If our message processing throws an exception, the message will remain inside the queue, and we'll retry processing the message after the `visibilityTimeout` has elapsed, until it lands in the dead-letter queue after too many failed attempts.

Nevertheless, let's see what a deletion policy of `NEVER` would look like. We can configure the `deletionPolicy` per SQS listener as part of the `@SqsListener` annotation:

```
@SqsListener(value = "${custom.sharing-queue}", deletionPolicy = NEVER)
public void listenToSharingMessages(
 TodoCollaborationNotification payload,
 Acknowledgment acknowledgment) {

 // perform processing
 if(successfulProcessing) {
 acknowledgment.acknowledge();
 }
}
```

With this deletion policy, Spring Cloud AWS will never automatically delete a message for us. We can inject an Acknowledgement object and take full control of when to acknowledge (i.e. delete) the message.

Please note that the `acknowledge()` method returns a Java Future, and we might want to wait for its result to resolve. The message deletion itself internally is just another HTTP request to AWS, which can fail anytime.

On top of this, we can resolve headers of an SQS message in case they're important for our message processing:

```
@SqsListener(value = "${custom.sharing-queue}")
public void listenToSharingMessages(
 TodoCollaborationNotification payload,
 @Headers Map<String, Object> headers) {

}
```

It's also possible to resolve a specific header using the `@Header("HEADER_NAME")` annotation.

With our SQS message listener in place, it's now time to inform the collaborator via email. Let's see how Amazon SES can help us here and how we can integrate this particular AWS service with our Spring Boot backend in the following section.



## Sending Emails with Amazon SES

The second part of this chapter implements the missing email notification for our collaboration feature to be finished. We'll integrate Amazon Simple Email Service (SES) with our sample Todo application to inform a collaborator about a shared Todo.

### Introduction to Amazon SES

Amazon SES (Simple Email Service) is an [easy-to-set-up](#) email sending and receiving service. Managing an on-premise email infrastructure is a complex topic. With Amazon SES, we take advantage of years of email infrastructure experience at Amazon.

We can interact with Amazon SES via the AWS SDK, the [Amazon SES SMTP interface](#) or the [Amazon SES API](#) directly.

The possible use cases for Amazon SES range from classic transactional emails (like sign-up confirmations) to marketing and newsletter-like email correspondence. We're billed based on the number of emails we send and receive. There are no upfront costs for licensing or server resources.

Furthermore, Amazon SES is [available in several regions](#). Each AWS account has sandbox access for Amazon SES in the available regions by default. We'll find out more about this sandbox access in the following section.

Amazon SES comes with email sending quotas to prevent fraud and maintain a good IP address reputation. These quotas define how many emails we can send per day and the maximum number of emails we can send per second. We can configure those quotas per region by getting in contact with the AWS Support Center.

The default quota depends on the Amazon SES instance type, which we're about to explore in the next section.

## Creating the Amazon SES Instance

Unlike with other AWS services like AWS S3 or Amazon SQS, there's no need to create an Amazon SES instance as all new AWS accounts are placed in the Amazon SES sandbox by default. We won't find any CloudFormation resources or CDK constructs to create the Amazon SES instance for a given region as it's already available.

However, there are Amazon SES-specific resources that we can bootstrap with infrastructure-as-code (either [CloudFormation](#) or [CDK](#)).

With `AWS::SES::Template`, for example, we can configure reusable email templates. In case we want to define rules for incoming emails (for example to store them in an S3 bucket), we can create an `AWS::SES::ReceiptRule` resource. We won't use any of these resources for our sample Todo application as we only need the bare minimum feature of Amazon SES: sending emails.

There's one minor adjustment for our existing CDK infrastructure setup. As we're about to send emails from our Spring Boot application, the ECS task role requires sufficient permissions for the Amazon SES API. We'll add a new `PolicyStatement` to our [ServiceApp](#) AWS CDK construct:

```
.withTaskRolePolicyStatements(List.of(
 // ... existing policy statements
 PolicyStatement.Builder.create()
 .sid("AllowSendingEmails")
 .effect(Effect.ALLOW)
 .resources(
 List.of(String.format(
 "arn:aws:ses:%s:%s:identity/stratospheric.dev", region, accountId))
)
 .actions(List.of(
 "ses:SendEmail",
 "ses:SendRawEmail"
))
 .build()
))
```

When working with Amazon SES, it's crucial to understand the implications of sandbox access. As the name applies, it comes with several restrictions:

- we can only send emails to verified email addresses,
- we can only send emails from verified domains and email addresses,
- we can only send 200 emails within 24 hours, and
- we can only send one email per second.

These restrictions might be acceptable for development and testing purposes with minimal and foreseeable email correspondence, but not for production. Before we can effectively use Amazon SES, we therefore first have to move out of the sandbox environment and request production access.

### Requesting Amazon SES Production Access

The Amazon SES sandbox access we're assigned to by default limits the usage for production. Not only are the sending quotas low but we also have to verify each and every email address before we even contact the user for the first time. This prevents seamless operation.

That's why we should request production access as soon as our application is about to go live for the first time. Requesting production access is a one-time manual effort by way of a support ticket.

To request Amazon SES production access, we have to file an AWS support ticket. As part of this ticket, we have to specify our website's URL, our use case, additional contact information, and which type of emails we're about to send. For our collaboration feature, the correct type is TRANSACTIONAL (for emails like order confirmations), as the alternative would be PROMOTIONAL (for marketing-related emails).

This ticket triggers a verification process on the AWS side and can take up to 24 hours. In case we're operating in several AWS regions, we have to request Amazon SES production access for each region once.



For an up-to-date and visual step-by-step guide on how to request production access, please take a look at the [Amazon SES documentation](#). As the Amazon SES console is currently transitioning from v1 to v2, screenshots might become outdated quite soon.

Once the production access is enabled, we can request an increase (if needed) of our Amazon SES sending quotas (sending rate and number of emails per day). This step is optional and can be accomplished with an additional [support ticket](#). With the Amazon SES production access in place, we first need to verify our domain before sending emails from our Spring Boot application.

### Verifying a Domain

To send emails with Amazon SES, we have to verify each identity (domain name) that we're going to use as part of "From", "Source", "Sender", or "Return-Path" to avoid unauthorized usage. Our goal is to send confirmation emails from `noreply@stratospheric.dev` for implementing our todo sharing feature.

The process to verify a domain identity is a one-time effort inside the AWS Management Console.



For an up-to-date guide on verifying domain identities, please follow the steps as described in the [AWS documentation](#).

During this process, we're asked if we want to create [DKIM \(DomainKeys Identified Mail\)](#) settings for this domain identity. By signing our email message with the DKIM private key, we can prove that the email originates from our domain. DKIM signatures are stored in our domain's DNS records. Receivers can verify such a signature to ensure the message wasn't modified in transit. As a general recommendation, we should opt-in for the DKIM settings to improve our email delivery rates.

To verify that we're the owner of the domain, we need to update the DNS settings of our domain. At the end of the domain identity creation process, we can download a `.csv` file with the required record set changes.

In case we're using [Route 53](#) as our DNS provider, Amazon's own DNS service, our DNS settings are updated automatically. What's left is to wait until the Amazon SES console reports our domain as verified.

If we're using a different DNS provider (GoDaddy, Namecheap, Cloudflare, you name it), we have to update the DNS settings of our domain ourselves. As we're dealing with changes to our DNS settings, the propagation to all other DNS servers can take up to 48 hours. Most of the time, however, the propagation is a matter of hours.

With `stratospheric.dev` being a verified Amazon SES domain, we can start implementing the email confirmation.

## Using Amazon SES for Our Todo Application

Both the Spring framework and Spring Cloud AWS provide excellent support for sending emails from our Java backend. Spring defines two central interfaces for doing so: `MailSender` and `JavaMailSender`. The latter inherits from the former and adds support for sending MIME messages via email.

In general, Spring provides [utility methods](#) and a convenient abstraction on top of the JavaMail standard API (nowadays Jakarta Mail).

While Spring's email support is agnostic to the underlying mailing system, Spring Cloud AWS provides an implementation of both interfaces that use Amazon SES as the underlying transport system.

To get started, we add the following Spring Cloud AWS dependency to our `build.gradle` file:

```
dependencies {
 implementation 'io.awspring.cloud:spring-cloud-aws-starter-ses'
}
```

An instance of the Amazon SES SDK client `SesClient` is auto-configured for us. We can either interact with the SDK client directly or use the already mentioned `MailSender` or `JavaMailSender`, for which Spring Cloud AWS auto-configures an implementation, which in turn uses Amazon SES internally.

With this setup in place, we can start to send emails to our users.

### Sending Emails to Invite Collaborators

We'll continue with the implementation where we left off at the end of the [previous section](#). We already have the `@SqsListener` in place to consume our Amazon SQS collaboration request messages. What's missing is to inform the collaborator by sending an email.

Spring's `MailSender` is sufficient for our use case as we're not sending complex emails with attachments or more involved formatting that would require the usage of `JavaMailSender`. We can inject the auto-configured `MailSender` into our `TodoSharingListener` and create the email message based on the incoming payload:

```
@Component
public class TodoSharingListener {

 private final MailSender mailSender;

 public TodoSharingListener(MailSender mailSender) {
 this.mailSender = mailSender;
 }

 @SqsListener(value = "${custom.sharing-queue}", deletionPolicy = ON_SUCCESS)
 public void listenToSharingMessages(TodoCollaborationNotification payload) {

 SimpleMailMessage message = new SimpleMailMessage();
 message.setFrom("noreply@stratospheric.dev");
 message.setTo(payload.getCollaboratorEmail());
 message.setSubject("A todo was shared with you");
 message.setText("Nicely formatted text including the confirmation link");

 mailSender.send(message);
 }
}
```

As we've confirmed the domain identity for our `stratospheric.dev` domain, we can use `noreply@stratospheric.dev` as the email sender. With Amazon SES production access enabled, we can also send emails to any destination and don't have to confirm every target manually.

The actual email body formatting is omitted for brevity's sake in the code example above (but available on [GitHub](#)). It contains metadata about the todo (title, due date, description, etc.) and, most notably, the confirmation link with a short-lived token.

Before carrying on, let's recall the implications of using the AWS default SQS queue type. When working with the default queue type (instead of FIFO), we have to keep in mind that messages are delivered at least once. This would mean our application could send the same invitation email multiple times in our concrete example. While the likelihood of this is relatively rare and the business impact might be acceptable (we'd just *annoy* the collaborator ...), we can mitigate that risk.

To avoid duplicate actions or inconsistent data when receiving the same message twice, we can design the message processing in an idempotent manner.



An operation is idempotent if it produces the same result for the same input even when it's executed multiple times. In our context, to have idempotent processing, consuming the same Amazon SQS message twice shouldn't result in a second email notification. For a further explanation of Idempotency (with cows instead of Amazon SQS messages), take a look at [this article](#).

For our use case, we could keep track of whom we already notified. For every incoming SQS message, we would check first if this collaboration request is new and otherwise drop the message as the person has already been informed. For our sample Todo application, we'll live with the fact that we might send out the same invitation twice.

As soon as we're calling `mailSender.send()`, the email is handed over to Amazon SES. Now it's the responsibility of AWS to deliver that email. Whenever we dispatch emails from our application, it's crucial to monitor our delivery rates. It's not unlikely that other email servers might reject our messages or mark them as spam. That's something we'll pick up in the chapter [Metrics with Amazon CloudWatch](#).

Once the invited person receives our email and clicks on the invitation link, it's



our area of responsibility again. Let's see how our Spring Boot backend handles these confirmations in the following section.

## Accepting Confirmations

The link we include inside the invitation email has the following structure:

```
<domain>/todo/{todoId}/collaborations/{collaboratorId}/confirm?token={token}
```

It targets an endpoint of our Spring Boot backend to confirm the collaboration. This endpoint requires authentication and hence the collaborator needs to be logged in. With this additional layer of security, we ensure that only the invited person can confirm the collaboration.

The following Spring MVC endpoint handles the confirmation part of the collaboration feature:

```
@Controller
@RequestMapping("/todo")
public class TodoCollaborationController {

 // ... existing endpoint to create a collaboration

 @GetMapping("/{todoId}/collaborations/{collaboratorId}/confirm")
 public String confirmCollaboration(
 @PathVariable("todoId") Long todoId,
 @PathVariable("collaboratorId") Long collaboratorId,
 @RequestParam("token") String token,
 @AuthenticationPrincipal OidcUser user,
 RedirectAttributes redirectAttributes
) {

 if(todoCollaborationService
 .confirmCollaboration(user.getEmail(), todoId, collaboratorId, token)) {

 redirectAttributes.addFlashAttribute("message",
 "You've confirmed that you'd like to collaborate on this todo.");
 redirectAttributes.addFlashAttribute("messageType", "success");
 }
 }
}
```

```

 }else {
 redirectAttributes.addFlashAttribute("message",
 "Invalid collaboration request.");
 redirectAttributes.addFlashAttribute("messageType", "danger");
 }

 return "redirect:/dashboard";
}
}

```

As part of the `confirmCollaboration()` method of the `TodoCollaborationService`, we either confirm the collaboration or reject it:

```

public class TodoCollaborationService {

 // other methods and fields

 public boolean confirmCollaboration(String authenticatedUserEmail,
 Long todoId, Long collaboratorId, String token) {

 Person collaborator = personRepository
 .findByEmail(authenticatedUserEmail)
 .orElseThrow(() -> new IllegalArgumentException());

 if (!collaborator.getId().equals(collaboratorId)) {
 return false;
 }

 TodoCollaborationRequest collaborationRequest = collaborationRequestRepository
 .findByTodoIdAndCollaboratorId(todoId, collaboratorId);

 if (collaborationRequest == null ||
 !collaborationRequest.getToken().equals(token)) {
 return false;
 }

 Todo todo = todoRepository
 .findById(todoId)
 .orElseThrow(() -> new IllegalArgumentException());

 todo.addCollaborator(collaborator);
 }
}

```

```

 collaborationRequestRepository.delete(collaborationRequest);

 return true;
 }
}

```

We reject the confirmation for one of the following reasons:

- the logged-in user tries to confirm a collaboration request for another user,
- no collaboration request for this todo and/or user exists, or
- the confirmation token is invalid.

Depending on the outcome of the confirmation process, we return a boolean value to display either a success or error message to the user.

We're registering the invited user as a collaborator for the todo when calling `todo.addCollaborator(collaborator)`. We've modeled this relationship with `@ManyToMany` as a todo can have multiple collaborators, and one person can collaborate on numerous todos:

```

@Entity
public class Todo {

 // ... more fields

 @ManyToMany
 @JoinTable(name = "todo_collaboration",
 joinColumns = @JoinColumn(name = "todo_id"),
 inverseJoinColumns = @JoinColumn(name = "collaborator_id"))
 private List<Person> collaborators = new ArrayList<>();

 public void addCollaborator(Person person) {
 this.collaborators.add(person);
 person.getCollaborativeTodos().add(this);
 }
}

```

As this `confirmCollaboration()` method runs within a transaction, Hibernate will determine the necessary SQL statements with its dirty-checking mechanism, and we don't have to explicitly call `save()` to persist this change.

As an additional housekeeping task, we're removing this particular collaboration request from our database as we no longer have to keep track of it.

Once the invited person successfully confirms the collaboration, we'll inform the todo owner with a push notification. We're going to implement this feature in the upcoming chapter [Push Notifications with Amazon MQ](#).

## Enabling Local Development

What's left is to update our local development setup. We're integrating two new AWS services to our existing tech stack with the collaboration feature: Amazon SQS and Amazon SES. When developing and starting the application locally, we need access to both AWS services.

As already outlined in the chapter [Local Development](#), we're using Docker and LocalStack to simulate a local AWS cloud. To that end, we're extending our existing `docker-compose.yml` file with a new service definition:

```
version: '3.3'
```

```
services:
```

```
... existing Docker container definitions
```

```
localstack:
```

```
 image: localstack/localstack:0.14.4
```

```
 ports:
```

```
 - 4566:4566
```

```
 environment:
```

```
 - SERVICES=sqs,ses
```

```
 - DEFAULT_REGION=eu-central-1
```

```
 - USE_SINGLE_REGION=true
```

```
 volumes:
```

```
 - ./src/test/resources/localstack/local-aws-infrastructure.sh:\
 /docker-entrypoint-initaws.d/init.sh
```

As part of the Docker container definition, we're instructing LocalStack which AWS services to start using the `SERVICES` environment variable. Furthermore, we define the default AWS region. There's no need to configure any specific credentials for our AWS SDK clients as LocalStack permits the use of any credentials.

To initialize and set up our local AWS cloud, we mount a script inside the Docker container (`local-aws-infrastructure.sh`). LocalStack will run any script inside the `docker-entrypoint-initaws.d` folder during container initialization. As part of our initialization script, we're creating our Amazon SQS queue and verify several email identities for Amazon SES:

```
#!/bin/sh
```

```
awslocal sqs create-queue --queue-name stratospheric-todo-sharing

awslocal ses verify-email-identity --email-address noreply@stratospheric.dev
awslocal ses verify-email-identity --email-address info@stratospheric.dev
awslocal ses verify-email-identity --email-address tom@stratospheric.dev
awslocal ses verify-email-identity --email-address bjoern@stratospheric.dev
awslocal ses verify-email-identity --email-address philip@stratospheric.dev
```

The `awslocal` CLI tool is a wrapper for the `aws` binary that targets LocalStack instead of the real AWS cloud.



Starting with LocalStack version v0.11.0, all AWS services are accessible via a single edge service on port 4566. With previous versions of LocalStack, each AWS service allocated a dedicated port.

Having the LocalStack container up and running, we need to configure our application's AWS SDK clients to point to our local AWS cloud. We achieve this by overriding the endpoint URLs as part of our `application-dev.yml`:

```
spring:
 cloud:
 aws:
 endpoint: http://localhost:4566
 region:
 static: eu-central-1
 credentials:
 secret-key: foo
 access-key: bar
```

With this configuration, the auto-configured AWS SDK clients point to LocalStack.

There's one caveat with LocalStack's Amazon SES service. It won't send any email to the collaborator's inbox. LocalStack acts as a sandbox and will store

the received emails in memory. Nevertheless, our application will receive a successful response whenever we use Spring's MailSender.

To make the entire collaboration journey work, we can auto-confirm collaborations locally. With a boolean property like `custom.auto-confirm-collaborations`, we can control this behavior. For all applications profiles except dev this value is set to false. Our `TodoSharingListener` checks for this property at the end of the processing:

```
@Component
public class TodoSharingListener {

 // ...

 private final boolean autoConfirmCollaborations;

 public TodoSharingListener(
 @Value("${custom.auto-confirm-collaborations}")
 boolean autoConfirmCollaborations) {
 this.autoConfirmCollaborations = autoConfirmCollaborations;
 }

 @SqsListener(value = "${custom.sharing-queue}", deletionPolicy = ON_SUCCESS)
 public void listenToSharingMessages(TodoCollaborationNotification payload) {

 // ...

 if (autoConfirmCollaborations) {
 LOG.info("Auto-confirm collaboration request");
 todoCollaborationService.confirmCollaboration(
 payload.getCollaboratorEmail(), payload.getTodoId(),
 payload.getCollaboratorId(), payload.getToken());
 }
 }
}
```

The LocalStack Amazon SQS service works just as expected: We're able to publish and consume SQS messages when running our application locally.

## 13. Push Notifications with Amazon MQ

As discussed in the chapter [Sharing Todos with SQS and SES](#) we now can share our todos with others. That's already quite helpful for working on tasks collaboratively. Receiving real-time feedback whenever a collaborator makes changes to a todo, however, would be what product managers call a “delighter” – a feature that excites our users beyond their expectations.

This is where push notifications come into play. In this chapter, we'll explore push notifications, create the required AWS infrastructure, and implement them in our Spring Boot application.

### What Are Push Notifications Anyway?

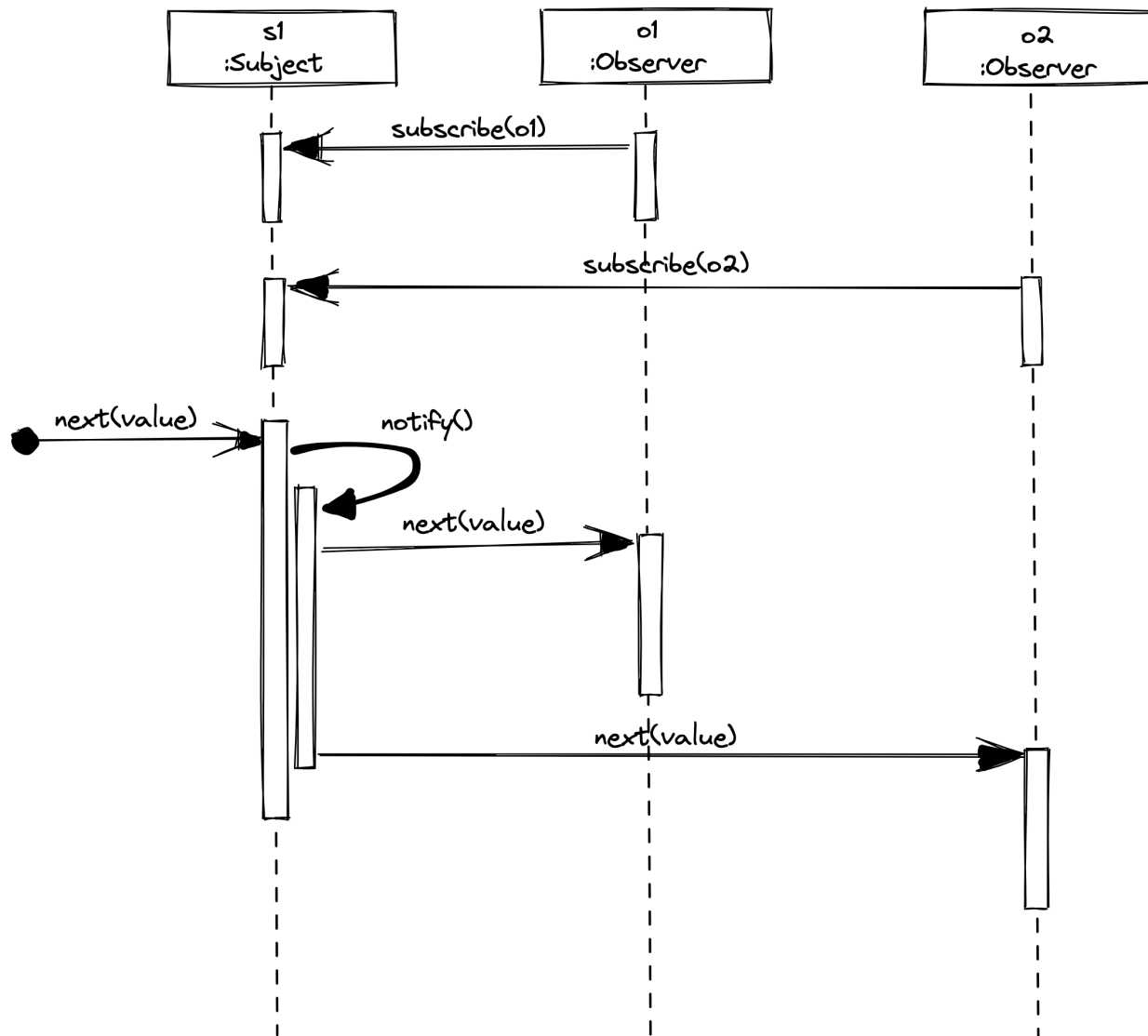
The term *push notifications* originating in the mobile app space in 2009. It usually refers to a backend server pushing information to the user interface of a mobile application rather than the application itself initiating a request – or pulling – to get information from the server.

The main benefit of this approach is that, rather than continuously polling for new information – and hence using a lot of network and system resources – a client only ever gets notified when there's an update that's relevant to that specific client.

While the term was originally coined for mobile apps, the general pattern applies to any kind of distributed client-server application, especially if there are multiple clients involved, which in turn might need to communicate with each other.



This pattern comes in several flavors and implementations. A common design pattern for implementing such behavior is the observer pattern as outlined in this sequence diagram:

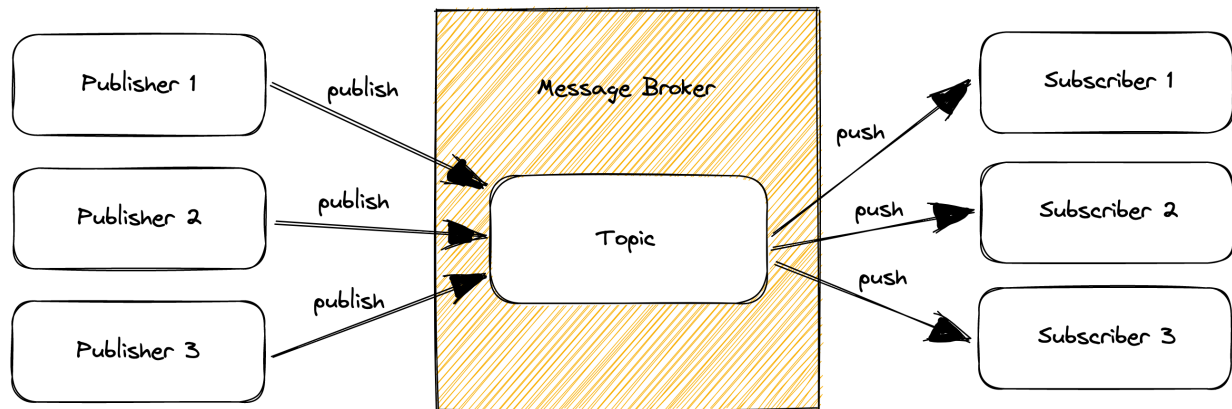


An observer subscribes to a subject and gets notified by the subject when something happens.

With this design pattern multiple **observers** subscribe to a **subject** or **observable**, which in turn will notify each of its subscribers once there's an update available.

To take both scalability and easier management of subscribers into account,

message brokers such as Apache ActiveMQ generalize this idea following the **publish-subscribe pattern**. This pattern allows us to have an n:m relationship between **publishers** and **subscribers** with multiple publishers publishing messages to a **topic** or **channel** and multiple subscribers receiving messages from that topic:



A message broker connects publishers with subscribers via topics that a subscriber can subscribe to.

We will apply this general concept of a server notifying its clients whenever there's a relevant update to a todo in our application. In our case, we will be notifying users in their browser window when another user has accepted their request to collaborate on a todo.

To do so, we'll be using WebSocket and STOMP as protocols on top of an ActiveMQ message broker running on [Amazon MQ](#). While implementing a publish-subscribe pattern is certainly possible with other protocols such as HTTP, WebSocket, and STOMP particularly lend themselves to this scenario since WebSocket – in contrast to HTTP – allows for bi-directional, full-duplex communication.

Although technically not required, a bi-directional communication protocol greatly simplifies implementing a publish-subscribe pattern. There are alternative approaches such as HTTP Polling, HTTP Streaming, or Server-Sent Events,

each of which comes with its own upsides and tradeoffs.

Still, WebSocket not only fits the bill in terms of what we need for our particular use case but it also sports better performance and lower latency than HTTP.

## Push Notifications for Live Updates

In the chapter *Sharing Todos with SQS and SES* we've seen how we can communicate and collaborate with other users with Amazon Simple Queue Service (SQS) and Amazon Simple Email Service (SES).

So far, however, that communication has been one-way only: We allow a user to share their todos with other users but the todo's owner won't receive a notification yet in case their collaborators made progress on a shared todo. To amend that situation, we'll be implementing a notification feature that will inform a user of such updates.

While using Spring's *embedded in-memory message broker* might seem like an obvious – and simple – solution, this doesn't work here because we want to run (at least) two instances of our Spring Boot application for reliability. In this setting, only clients connected to a specific instance would be able to exchange messages with each other. Hence, to communicate across instances we need a stateful service that enables message sharing across application instances.

We'll employ an AWS service for that stateful service. Within the AWS ecosystem, there are quite a few services that allow us to implement notifications with a publish-subscribe pattern. Each of those services targets different use cases, so let's have a look at them.

## AWS Services for Implementing Push Notifications

Let's have a quick glance at each of these potential alternatives. Each one comes with advantages and downsides. Deciding which one to use is a matter of tradeoffs.

One of the tradeoffs we need to consider is the question of “Do I want to go serverless or am I fine with running an always-on instance of a service?”. Answering that question most importantly has an impact on our cost structure and the payment model used. Some services are *pay-per-use* while others are *pay-per-hour*.

### Amazon Pinpoint

*“[Amazon Pinpoint](#) is a flexible and scalable outbound and inbound marketing communications service.”*

While that might not sound all that useful immediately, Pinpoint indeed offers mobile push notifications as one of its communications.

Our application is a web application, though, and this keeps us from using Pinpoint for our purposes. Besides, Pinpoint has a lot more to offer in terms of marketing communications and user segmentation which is irrelevant to our use case at best and adds pointless complexity and features we don't need at worst.

For these reasons, we won't be using Pinpoint.

## Amazon IoT (Core)

*“[Amazon IoT Core](#) lets you connect IoT devices to the AWS cloud without the need to provision or manage servers.”*

Amazon IoT is another one of those industry-specific services whose feature set might at first sound like it has nothing to do with our requirements.

While Amazon IoT is clearly targeted at the use case of allowing physical objects to communicate with each other on the *Internet of Things* one of the protocols used to do so indeed is WebSocket.

WebSocket in turn, applies to our use case. In theory, there's nothing that prevents us from using Amazon IoT even though we don't have physical objects communicating with each other but rather browser clients and the people using these clients.

However, using infrastructure designed for IoT purposes and the massive scale that implies also seems like massive over-engineering for our – rather simple – use case.

Therefore, we won't be using Amazon IoT either.

## Amazon SNS

According to Amazon's own description,

*[Amazon Simple Notification Service \(Amazon SNS\)](#) is a fully managed messaging service for both application-to-application (A2A) and application-to-person (A2P) communication. It's a managed service that provides message delivery from publishers to subscribers”.*

This actually sounds pretty much like what we need for our purposes, doesn't it? In fact, it does. SNS allows publishers to send messages to topics, which subscribers in turn can listen to. However, there's one small detail that keeps us from employing SNS for our purposes: It doesn't support WebSocket as a protocol!

SNS only allows us to send messages to these subscriber endpoint types:

- HTTP/HTTPS
- email
- [Amazon Kinesis Data Firehose](#)
- Amazon SQS
- a custom AWS Lambda function
- a platform application endpoint
- SMS

So, unfortunately, WebSocket isn't supported by SNS. We could of course still use SNS and provide an HTTP endpoint in our Todo application and relay any communication to a WebSocket endpoint we provide from our Todo application.

However, not only would this needlessly increase the complexity of our architecture but it would also defeat the purpose of using a service such as SNS in the first place: Why use such a service if we don't really benefit from doing so?

Hence, SNS regrettably isn't a viable option for our scenario either.

## Amazon SQS

What about SQS, though? After all, we've already been using it for sharing todos. So, why not use it for communicating a todo's status as well?

To reiterate the purpose of SQS:

*“[Amazon Simple Queue Service \(SQS\)](#) is a fully managed message queuing service that enables you to decouple and scale microservices, distributed systems, and serverless applications”.*

Again, this sounds like something that might lend itself to our case. As we’ve seen when we implemented our todo sharing feature, a subscriber can listen to an SQS queue and send an email for each incoming message.

This could be used for informing subscribers about incoming updates to their todos, too.

Unfortunately, though, SQS doesn’t support WebSocket as a protocol. While consuming SQS messages from [browser clients is possible](#), this requires using [long polling via HTTP](#), which we’ve decided against for our purposes.

## Amazon MQ

So, have we run out of options? Fortunately, no, we haven’t. With Amazon MQ, there’s still one option left that might help us with our publish-subscribe scenario:

*“[Amazon MQ](#) is a managed message broker service for Apache ActiveMQ and RabbitMQ that makes it easy to set up and operate message brokers on AWS.”*

Amazon MQ is different from the services discussed before in that it doesn’t provide a high-level, serverless API for specific use cases but rather a traditional service instance, on which to run our own message broker. The benefit of this approach is that we’re not constrained by a particular API’s or platform’s limitations but rather we’re able to use the full range of features those message brokers provide, particularly when it comes to the WebSocket protocol, which both ActiveMQ and RabbitMQ support. A demerit of this is that setting up the

requisite infrastructure is more involved than with the AWS services mentioned before.

In contrast to the other publish-subscribe options provided by AWS Amazon MQ operates under a pay-per-hour business model, which means that it'll incur costs even when we're not actively using it.

While both the potentially higher costs and the more complex infrastructure setup can be considered major downsides, Amazon MQ is the only service of those mentioned that natively supports the WebSocket protocol.

Hence, it's the only one that meets our specific requirements, which is why we'll be using Amazon MQ for implementing our "push notifications" feature.

## Setting up a Message Broker with CDK

Having settled for Amazon MQ, the next decision to make is the one between the two message broker options available: [Apache ActiveMQ](#), or [RabbitMQ](#).

Both come with a similar set of features and a roughly similar performance profile. Therefore, deciding which to use boils down to specific requirements for your particular project or environment.

However, ActiveMQ not only natively supports the WebSocket and STOMP protocols but also the Java Message Service (JMS) API and the MQTT and AMQP protocols as well. So, it seems like the more versatile solution, particularly in a Java-based environment. Should the need arise to directly connect the message broker to our Spring Boot application via JMS we could do so without swapping the underlying message broker technology.

For these reasons, we'll be using ActiveMQ for our push notifications feature.

So, let's get started by setting up the required Amazon MQ infrastructure. As



usual, we'll create a new stack in our CDK project – this one called `ActiveMqStack`. Here's the skeleton of that stack:

```
public class ActiveMqStack extends Stack {

 // ...

 public ActiveMqStack(
 final Construct scope,
 final String id,
 final Environment awsEnvironment,
 final ApplicationEnvironment applicationEnvironment,
 final String username
) {

 // ...

 this.applicationEnvironment = applicationEnvironment;
 this.username = username;
 this.password = generatePassword();

 // ...
 }

 // ...
}
```

## Creating an ActiveMQ User

First, we need to create a list of users that should gain access to our ActiveMQ instance:

```
List<User> userList = new ArrayList<>();
userList.add(new User(
 username,
 password
));
```

We'll later pass the `userList` (containing exactly one user in our case) into the `ActiveMQ` construct to create access credentials. We could make access available for anonymous users, of course, but this is not a particularly good idea in terms of security and resource usage because it would potentially open up our message queue to the world and make it vulnerable to DDoS attacks.

Since we're going to place our `ActiveMQ` instance into our private VPC, a DDoS attack in this case is more of a theoretical rather than an actual issue. Still, it's good practice to protect our resources. Especially if this comes at no cost as in this case. If, for example, we mistakenly placed our `ActiveMQ` instance on a public subnet we'd still have another layer of protection.

The username is passed into the stack via its constructor while the password is generated using a `PasswordGenerator` with a few rules that make sure our generated password will consist of 32 characters with both at least 5 lower-case and 5 upper-case characters as well as 5 digits:

```
public class ActiveMqStack extends Stack {
 // ...

 private String generatePassword() {
 PasswordGenerator passwordGenerator = new PasswordGenerator();
 CharacterData lowerCaseChars = EnglishCharacterData.LowerCase;
 CharacterRule lowerCaseRule = new CharacterRule(lowerCaseChars);
 lowerCaseRule.setNumberOfCharacters(5);
 CharacterData upperCaseChars = EnglishCharacterData.UpperCase;
 CharacterRule upperCaseRule = new CharacterRule(upperCaseChars);
 upperCaseRule.setNumberOfCharacters(5);
 CharacterData digitChars = EnglishCharacterData.Digit;
 CharacterRule digitRule = new CharacterRule(digitChars);
 digitRule.setNumberOfCharacters(5);
 return passwordGenerator
 .generatePassword(32, lowerCaseRule, upperCaseRule, digitRule);
 }

 // ...
}
```

## Defining a Security Group

Next, we create a specific security group for our ActiveMQ stack using CDK's `CfnSecurityGroup.Builder`. We supply this security group with the ID from our already existing VPC (retrieved from a previously deployed Network stack via the `NetworkOutputParameters` class). This will make sure our ActiveMQ instance resides within the same VPC as our main application:

```
CfnSecurityGroup amqSecurityGroup = CfnSecurityGroup.Builder.create(
 this,
 "amqSecurityGroup")
 .vpcId(networkOutputParameters.getVpcId())
 .groupDescription("Security Group for the Amazon MQ instance")
 .groupName(applicationEnvironment.prefix("amqSecurityGroup"))
 .build();
```

## Creating the Message Broker

We collect the parameters we've created so far and supply those as input parameters to CDK's `CfnBroker.Builder` class for creating our ActiveMQ broker:

```
CfnBroker broker = CfnBroker.Builder
 .create(this, "amqBroker")
 .brokerName(applicationEnvironment.prefix("stratospheric-amq-message-broker"))
 .securityGroups(Collections.singletonList(this.securityGroupId))
 .subnetIds(Collections.singletonList(
 networkOutputParameters.getIsolatedSubnets().get(0)))
 .hostInstanceType("mq.t2.micro")
 .engineType("ACTIVEMQ")
 .engineVersion("5.16.2")
 .authenticationStrategy("SIMPLE")
 .encryptionOptions(
 CfnBroker.EncryptionOptionsProperty
 .builder()
 .useAwsOwnedKey(true)
 .build()
)
 .users(userList)
 .publiclyAccessible(false)
 .autoMinorVersionUpgrade(true)
 .deploymentMode("SINGLE_INSTANCE")
 .build();
```

Because for now we want to save costs, we choose the smallest and cheapest possible [instance type](#) here with `mq.t2.micro`.

Since we opted for using ActiveMQ, we define `ACTIVEMQ` as our engine type.

We'll use 5.16.2 as the ActiveMQ engine version, which as of writing is the latest version supported by Amazon MQ.

Furthermore, we choose the default `SIMPLE` (password-based) authentication strategy and provide our `userList` from above as the list of valid logins. Another option would have been to use LDAP here but since that would add significant complexity we decided against using that option.

We also make sure that our ActiveMQ won't be `publiclyAccessible`. Moreover, Amazon MQ will automatically take care of minor version upgrades because we have set `autoMinorVersionUpgrade` to `true`.

We'll set the broker's deployment mode to `SINGLE_INSTANCE`. This is the most simple setup possible, with just a single ActiveMQ instance running for our entire application. Other options would have been:

- **ACTIVE\_STANDBY\_MULTI\_AZ:** A pair of active/standby brokers in multiple availability zones for both high availability and automatic failover capabilities.
- **CLUSTER\_MULTI\_AZ:** An entire cluster of brokers in multiple availability zones.

We're using just a small set of options available with Amazon MQ here. See the official guide "[How Amazon MQ works](#)" and the [CloudFormation API reference documentation](#) for a full list of options available when configuring a message broker with Amazon MQ.

## Exporting Output Parameters

After that, we export some output parameters, which we will later pass to our service stack for configuring our Spring Boot application to connect with the ActiveMQ instance:

```
StringParameter.Builder.create(this, PARAMETER_USERNAME)
 .parameterName(createParameterName(
 applicationEnvironment,
 PARAMETER_USERNAME))
 .stringValue(username)
 .build();

StringParameter.Builder.create(this, PARAMETER_PASSWORD)
 .parameterName(createParameterName(
 applicationEnvironment,
 PARAMETER_PASSWORD))
 .stringValue(password)
 .build();

StringParameter.Builder.create(this, PARAMETER_STOMP_ENDPOINT)
 .parameterName(createParameterName(
 applicationEnvironment,
 PARAMETER_STOMP_ENDPOINT))
 .stringValue(Fn.select(0, this.broker.getAttrStompEndpoints()))
 .build();

StringParameter.Builder.create(this, PARAMETER_SECURITY_GROUP_ID)
 .parameterName(createParameterName(
 applicationEnvironment,
 PARAMETER_SECURITY_GROUP_ID))
 .stringValue(this.securityGroupId)
 .build();
```

We'll use the username and the password later to connect to the broker from the Spring Boot application backend while we'll use the STOMP endpoint URL for connecting from our Thymeleaf frontend via JavaScript and WebSocket.

One thing to note here, in particular, is the security group ID, which we'll use to refer to our broker's security group when allowing ingress access from our application.

## The ActiveMq CDK App

All that's left to do now is instantiate our `ActiveMqStack` and have CDK synthesize a CloudFormation from that stack:

```
public class ActiveMqApp {

 public static void main(final String[] args) {
 App app = new App();

 // ...

 new ActiveMqStack(
 app,
 "activeMq",
 awsEnvironment,
 applicationEnvironment,
 username);

 app.synth();
 }

 // ...
}
```

## Connecting the Application to ActiveMQ

To configure our application to access the broker we have to modify our `ServiceApp` a bit. First, we need to provide network-level access from the ECS-managed container to the message broker. For this, we add the message broker's security group to the list of groups from which ingress is allowed:

```

public class ServiceApp {

 public static void main(final String[] args) {
 // ...

 List<String> securityGroupIdsToGrantIngressFromEcs = Arrays.asList(
 databaseOutputParameters.getDatabaseSecurityGroupId(),
 activeMqOutputParameters.getActiveMqSecurityGroupId()
);

 // pass the list of security groups into the Service construct
 }
}

```

Our Service construct internally uses this list for setting up the necessary ingress rules.

Also, we add some of the output parameters of the ActiveMqStack as environment variables:

```

public class ServiceApp {

 // ...

 static Map<String, String> environmentVariables(
 Construct scope,
 PostgresDatabase.DatabaseOutputParameters databaseOutputParameters,
 CognitoStack.CognitoOutputParameters cognitoOutputParameters,
 MessagingStack.MessagingOutputParameters messagingOutputParameters,
 ActiveMqStack.ActiveMqOutputParameters activeMqOutputParameters,
 String springProfile
) {

 Map<String, String> vars = new HashMap<>();

 // ...

 vars.put(
 "WEB_SOCKET_RELAY_ENDPOINT",
 activeMqOutputParameters.getStompEndpoint());
 vars.put(

```



```
 "WEB_SOCKET_RELAY_USERNAME",
 activeMqOutputParameters.getActiveMqUsername());
vars.put(
 "WEB_SOCKET_RELAY_PASSWORD",
 activeMqOutputParameters.getActiveMqPassword());

 // ...
}
}
```

This way, the Spring Boot application can read those environment variables and connect to the message broker.

## Implementing Push Notifications in the Todo App

In the following, we'll finally implement the push notifications feature on top of the messaging infrastructure we have created.

First, however, we'll have a quick look at protocols we're going to use for implementing this feature.

### Protocols: WebSocket and STOMP

**WebSocket** is a protocol that – unlike HTTP – allows for full-duplex communication, i.e. parties can communicate with each other simultaneously with both parties being able to initiate a communication flow. Before the advent of WebSocket, if we wanted to display updates in an HTML client we had to initiate a client request and wait for the server's response. Depending on the number of updates, we either had to resort to frequent polling or long-polling approaches such as [Comet](#). Neither performed particularly well, both in terms of network throughput and latency.

To alleviate these issues, WebSocket provides an always-on connection between client and server. The server can send messages to the client on its own without any explicit request from the client. The same is true the other way around.

A WebSocket connection is initiated by the client via HTTP. If the server is capable of communicating via WebSocket that HTTP connection is upgraded to a WebSocket connection using an [HTTP upgrade header](#).

Once a WebSocket connection has been established, the server can send updates to the client whenever they occur. This model has proven much more efficient than the previous (long-)polling models.

**STOMP** ([Simple Text Oriented Messaging Protocol](#)) is a messaging protocol that provides messaging semantics with actions such as SUBSCRIBE, UNSUBSCRIBE, or SEND. While WebSocket allows us to establish full-duplex connections it is agnostic as to how data is transmitted via these connections. To communicate using a publish-subscribe pattern as described above we need to define a protocol, commands, and payload formats. STOMP provides us with a simple set of commands, headers, and network frames, so we don't have to reinvent this particular wheel.

## Relaying WebSocket Connections

As mentioned above, ActiveMQ natively supports WebSocket and STOMP protocols. However, to not expose our ActiveMQ to the public internet and to control the message flow to some extent, we'll use our Spring Boot application as a relay between our HTML frontend and ActiveMQ.

To implement this relay, we first have to add these dependencies to our application's `build.gradle` Gradle configuration:

```
implementation 'org.springframework.boot:spring-boot-starter-websocket'
implementation 'org.springframework.boot:spring-boot-starter-activemq'
```

These will provide us with the necessary libraries for connecting to an ActiveMQ server and exposing WebSocket connectivity to our users.

Next, we create a new Spring configuration called `WebSocketConfig`:

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

 private final Endpoint websocketEndpoint;
 private final String websocketUsername;
 private final String websocketPassword;
 private final boolean websocketUseSsl;

 public WebSocketConfig(
 @Value("${custom.web-socket-relay-endpoint:#{null}}")
 String websocketRelayEndpoint,
 @Value("${custom.web-socket-relay-username:#{null}}")
 String websocketUsername,
 @Value("${custom.web-socket-relay-password:#{null}}")
 String websocketPassword,
 @Value("${custom.web-socket-relay-use-ssl:#{false}}")
 boolean websocketUseSsl
) {
 this.websocketEndpoint = Endpoint.fromEndpointString(websocketRelayEndpoint);
 this.websocketUsername = websocketUsername;
 this.websocketPassword = websocketPassword;
 this.websocketUseSsl = websocketUseSsl;
 }

 @Override
 public void configureMessageBroker(MessageBrokerRegistry registry) {
 registry
 .enableStompBrokerRelay("/topic")
 .setAutoStartup(true)
 .setClientLogin(this.websocketUsername)
 .setClientPasscode(this.websocketPassword)
 .setSystemLogin(this.websocketUsername)
```

```
 .setSystemPasscode(this.websocketPassword);
 }

 // ...

 @Override
 public void registerStompEndpoints(StompEndpointRegistry registry) {
 registry
 .addEndpoint("/websocket")
 .withSockJS();
 }

 // ...
}
```

This configuration makes use of the `@EnableWebSocketMessageBroker` annotation to enable WebSocket communication. Furthermore, it implements the `WebSocketMessageBrokerConfigurer` interface, which provides us with the `configureMessageBroker()` and `registerStompEndpoints()` methods. These methods allow us to configure the connection to our ActiveMQ message broker:

In our `registerStompEndpoints()` we add a `/websocket` endpoint with [SockJS](#) support enabled. SockJS is a library for WebSocket connectivity and emulation that serves as a fallback option for when WebSocket support isn't available on either the client or the server.

The `websocketRelayEndpoint`, `websocketUsername`, and `websocketPassword` constructor parameters are taken from the `ActiveMqStack`'s output parameters that we made available to the application via environment variables. These environment variables are picked up by Spring through our [application-aws.yml](#) configuration file:

**custom:**

```
...
web-socket-relay-endpoint: ${WEB_SOCKET_RELAY_ENDPOINT}
web-socket-relay-username: ${WEB_SOCKET_RELAY_USERNAME}
web-socket-relay-password: ${WEB_SOCKET_RELAY_PASSWORD}
```

Finally, we have to make sure that WebSocket connections to our relay don't require the user of our sample Todo application to sign in first. To this end, we add a `/websocket/**` path matcher to the `ignoringRequestMatchers` call in our [WebSecurityConfig](#):

```
@Configuration
public class WebSecurityConfig {

 // ...

 @Bean
 public SecurityFilterChain securityFilterChain(HttpSecurity httpSecurity)
 throws Exception {

 httpSecurity
 .csrf()
 .ignoringRequestMatchers(
 "/stratospheric-todo-updates/**",
 "/websocket/**"
)
 .and()
 // ...

 return httpSecurity.build();
 }
}
```

Upon startup, our Spring Boot application will now open up a connection to our ActiveMQ instance and act as a WebSocket relay under the path prefix `/topic`.

## Connecting to a STOMP Server

Ideally, that would have been all we had to do on the server side for providing WebSocket and STOMP functionality through our Spring Boot application.

Unfortunately, the default TCP client implementation used by [Spring Reactor](#), and conversely [Netty](#), misses two vital features: SSL support and a working DNS resolver (see the corresponding [issue](#) on GitHub).

Since Amazon MQ instances are only – and rightfully so – available via connections encrypted through SSL as well as domain names rather than IP addresses (which also is pretty much a prerequisite for using SSL) we have to resort to creating our TCP client based on Reactor’s [ReactorNettyTcpClient](#):

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

 // ...

 private ReactorNettyTcpClient<byte[]> getCustomTcpClientWithSSLSupport() {
 return new ReactorNettyTcpClient<>(configurer -> configurer
 .host(this.websocketEndpoint.host)
 .port(this.websocketEndpoint.port)
 .resolver(DefaultAddressResolverGroup.INSTANCE)
 .secure(), new StompReactorNettyCodec());
 }

 private ReactorNettyTcpClient<byte[]> getCustomTcpClientWithoutSSLSupport() {
 // similar but not SSL support to connect to a local ActiveMQ
 }

 // ...
}
```

The builder method call `secure()` enables SSL connectivity depending on the value of the variable `websocketUseSsl`, which is controlled by the `web-socket-`

relay-use-ssl property in application.yml configuration file. Calling the `resolver(...)` provides us with the working DNS resolver that allows us to resolve domain names in our VPC.

This custom TCP client then is used by calling `setTcpClient(tcpClient)` when we register our STOMP relay:

```
@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

 // ...

 @Override
 public void configureMessageBroker(MessageBrokerRegistry registry) {
 ReactorNettyTcpClient<byte[]> customTcpClient =
 this.websocketUseSsl ?
 getCustomTcpClientWithSSLSupport() :
 getCustomTcpClientWithoutSSLSupport();

 registry
 .enableStompBrokerRelay("/topic")
 //...
 .setTcpClient(customTcpClient);
 }
}
```

Although we're only running a single ActiveMQ instance rather than an active/-standby set of instances, our custom TCP client would also support multiple hosts in a round-robin fashion:

```

@Configuration
@EnableWebSocketMessageBroker
public class WebSocketConfig implements WebSocketMessageBrokerConfigurer {

 // ...

 private ReactorNettyTcpClient<byte[]>
 createRoundRobinTcpClient(Endpoint endpoint) {
 final List<InetSocketAddress> addressList = new ArrayList<>();

 for (String hostURI : endpoint.activeStandbyHosts) {
 String[] hostAndPort = hostURI.split(":");
 addressList.add(
 new InetSocketAddress(hostAndPort[0], Integer.parseInt(hostAndPort[1])));
 }

 final RoundRobinList<InetSocketAddress> addresses =
 new RoundRobinList<>(addressList);

 return new ReactorNettyTcpClient<>(builder ->
 builder
 .remoteAddress(addresses::get)
 .secure()
 .resolver(DefaultAddressResolverGroup.INSTANCE),
 new StompReactorNettyCodec()
);
 }

 // ...
}

```

One final thing to note regarding our `WebSocketConfig` is how we process our ActiveMQ endpoint URL when calling `Endpoint.fromEndpointString()`. This static method in turn removes the “stomp+ssl://” protocol part returned with instance URLs by Amazon MQ (`replace("stomp+ssl://", "")`). This is required because rather than a complete URL including the protocol the `ReactorNettyTcpClient` expects only the host and the port as parameters.



## Adding Push Notifications to the Frontend

Now that we've set up WebSocket connectivity on the server side we can start with the fun part and finally implement our actual push notification feature on the client side.

First, we add these WebJars dependencies to our `build.gradle` Gradle configuration:

```
implementation 'org.webjars:sockjs-client:1.1.2'
implementation 'org.webjars:stomp-websocket:2.3.3-1'
```

These JavaScript libraries provide us with the SockJS WebSocket client and STOMP support on the client side.

Then, since we'll be using JavaScript for connecting to our WebSocket relay we add `/js/todo-updates.js` to our Thymeleaf [layout.html](#) layout for loading the JavaScript file responsible for exchanging messages with our WebSocket server:

```
<!-- ... -->

<head>
 <meta charset="UTF-8">
 <title layout:title-pattern="$CONTENT_TITLE | $LAYOUT_TITLE">
 Todo Application
 </title>
 <!-- ... -->
 <script th:src="@{/js/todo-updates.js}" sec:authorize="isAuthenticated()"></script>
</head>
<body>
<div>

 <!-- ... -->

 <script sec:authorize="isAuthenticated()">
 connectToWebSocketEndpoint('[[${#authentication.principal.attributes.email}]]');
 </script>
```

```
</div>
</body>
</html>
```

In case the user is logged in (since in our use case it only makes sense for authenticated users to receive messages on the progress of their todos) we call the function `connectToWebSocketEndpoint()` from that JavaScript file with the authenticated user's email address as an argument:

```
<script sec:authorize="isAuthenticated()">
 connectToWebSocketEndpoint('[[${#authentication.principal.attributes.email}]]');
</script>
```

The email address here serves as an identifier, so we can tell which messages to display to which user.

The `connectToWebSocketEndpoint()` function from [todo-updates.js](#) creates a STOMP client using the SockJS library:

```
let stompClient = null;

function connectToWebSocketEndpoint(email) {
 const socket = new SockJS('/websocket');

 stompClient = Stomp.over(socket);
 stompClient.connect({}, () => {
 stompClient.subscribe('/topic/todoUpdates', function (message) {
 $('#message').html(message.body);
 $('#toast').toast('show');
 });

 if (email) {
 stompClient.subscribe('/topic/todoUpdates/' + email, function (message) {
 $('#message').html(message.body);
 $('#toast').toast('show');
 });
 }
 });
}
```

```
}

function disconnectFromWebSocketEndpoint() {
 if (stompClient !== null) {
 stompClient.disconnect();
 }
}

$(document).ready(function () {
 $('#toast').toast({delay: 5000});
});
```

The client subscribes to both a channel for general updates and a specific channel for updates relevant to the authenticated user (identified by that user's email). Whenever there's a new message on either of these channels a [toast](#) message will be displayed for 5 seconds.



If WebSocket isn't supported by the browser, SockJS will degrade gracefully to HTTP. However, these days this is a largely theoretical concern since [pretty much every modern browser now supports WebSocket](#).

When opening the sample Todo application in our browser and after having logged in, our browser console should display log messages like these, meaning that connecting to our relay via WebSocket has been successful and we're now subscribed to the update channels:

Opening Web Socket...

stomp.min.js:8 Web Socket Opened...

stomp.min.js:8 >>> **CONNECT**

accept-version:1.1,1.0

heart-beat:10000,10000

stomp.min.js:8 <<< **CONNECTED**

server:ActiveMQ/5.16.2

heart-beat:10000,10000

**session**:ID:b-6df78131-e12f-4ffd-94ab-1.mq.eu-central-1.amazonaws.com-39383-3:547

version:1.1

**user**-name:4a275ce6-c775-4651-8da7-f2a7f60fc667

stomp.min.js:8 connected **to** server ActiveMQ/5.16.2

stomp.min.js:8 send PING **every** 10000ms

stomp.min.js:8 **check** PONG **every** 10000ms

todo-updates.js:8 **Connection to** WebSocket endpoint was **successful**: **CONNECTED**

**user**-name:4a275ce6-c775-4651-8da7-f2a7f60fc667

version:1.1

**session**:ID:b-6df78131-e12f-4ffd-94ab-1.mq.eu-central-1.amazonaws.com-39383-3:547

heart-beat:10000,10000

server:ActiveMQ/5.16.2

stomp.min.js:8 >>> **SUBSCRIBE**

id:sub-0

destination:/topic/todoUpdates

stomp.min.js:8 >>> **SUBSCRIBE**

id:sub-1

destination:/topic/todoUpdates/test@example.com

The toast message is defined in the [toast.html](#) fragment in the src/main/resources/templates/fragments folder:

```

<!DOCTYPE html>
<html lang="en"
 xmlns:th="http://www.thymeleaf.org">
<head>
</head>
<body>
<div th:remove="tag" th:fragment="toast">
 <div
 id="toast"
 class="toast"
 role="alert"
 aria-live="assertive"
 aria-atomic="true">
 <div class="toast-header">
 <strong class="mr-auto">Collaboration confirmed.
 <button
 type="button"
 class="ml-2 mb-1 close"
 data-dismiss="toast"
 aria-label="Close">
 ×
 </button>
 </div>
 <div id="message" class="toast-body">
 Message
 </div>
 </div>
</div>
</body>
</html>

```

This fragment, in turn, is loaded by adding this tag to our Thymeleaf layout:

```

<!-- ... -->
<div th:replace="fragments/toast :: toast"></div>
<!-- ... -->

```

Now, whenever our `TodoCollaborationService`'s `confirmCollaboration()` method is called we'll send a new message to the todo's owner using `SimpMessagingTemplate`:

```
@Service
@Transactional
public class TodoCollaborationService {

 // ...

 public TodoCollaborationService(
 @Value("${custom.sharing-queue}") String todoSharingQueueName,
 TodoRepository todoRepository,
 PersonRepository personRepository,
 TodoCollaborationRequestRepository todoCollaborationRequestRepository,
 SqsTemplate sqsTemplate,
 SimpMessagingTemplate simpMessagingTemplate) {
 this.todoRepository = todoRepository;
 this.personRepository = personRepository;
 this.todoCollaborationRequestRepository = todoCollaborationRequestRepository;
 this.sqsTemplate = sqsTemplate;
 this.todoSharingQueueName = todoSharingQueueName;
 this.simpMessagingTemplate = simpMessagingTemplate;
 }

 // ...

 public boolean confirmCollaboration(
 Long todoId, Long collaboratorId, String token) {

 TodoCollaborationRequest collaborationRequest =
 collaborationRequestRepository
 .findByTodoIdAndCollaboratorId(todoId, collaboratorId);

 if (collaborationRequest != null
 && collaborationRequest.getToken().equals(token)) {
 // existing logic for confirming the collaboration
 // ...

 String name = collaborationRequest.getCollaborator().getName();
 String subject = "Collaboration confirmed.";
 String message = "User "
 + name
 + " has accepted your collaboration request for todo #"
 + collaborationRequest.getTodo().getId()
 + ".";
 String ownerEmail = collaborationRequest.getTodo().getOwner().getEmail();
```

```

 simpMessagingTemplate.convertAndSend(
 "/topic/todoUpdates/" + ownerEmail, subject + " " + message);

 return true;
 }

 return false;
}
}

```

A bean of type `SimpMessagingTemplate` is available in the application context via the dependency to [Spring Messaging](#), which was loaded transitively by the `spring-boot-starter-websocket` and `spring-boot-starter-activemq` dependencies. We'll use that instance for sending messages to channel subscribers.

When a user who received a collaboration request via email confirms that request by clicking on the link in that email the `confirmCollaboration()` method from our `TodoCollaborationService` will be called from the [TodoCollaborationController](#):

```

@Controller
@RequestMapping("/todo")
public class TodoCollaborationController {

 private final TodoCollaborationService todoCollaborationService;

 public TodoCollaborationController(
 TodoCollaborationService todoCollaborationService) {
 this.todoCollaborationService = todoCollaborationService;
 }

 // ...

 @GetMapping("/{todoId}/collaborations/{collaboratorId}/confirm")
 public String confirmCollaboration(
 @PathVariable("todoId") Long todoId,

```

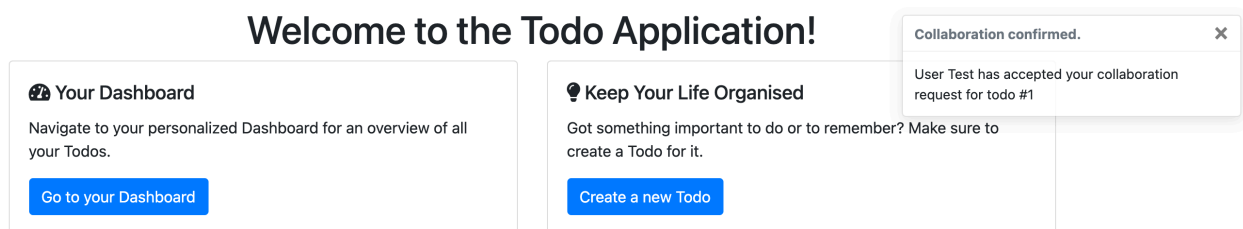
```

@PathVariable("collaboratorId") Long collaboratorId,
@RequestParam("token") String token,
RedirectAttributes redirectAttributes
) {
 if (todoCollaborationService.confirmCollaboration(
 todoId,
 collaboratorId,
 token)) {
 redirectAttributes.addFlashAttribute(
 "message",
 "You've confirmed that you'd like to collaborate on this todo.");
 redirectAttributes.addFlashAttribute(
 "messageType",
 "success");
 } else {
 redirectAttributes.addFlashAttribute(
 "message",
 "Invalid collaboration request.");
 redirectAttributes.addFlashAttribute(
 "messageType",
 "danger");
 }

 return "redirect:/dashboard";
}
}

```

Finally, we'll display a toast message like the one in the top right corner of this screenshot to the owner of todo when this message is received from the owner's /topic/todoUpdates channel:



Toast message



## Enabling Local Development

For a smooth development process, there's one small thing still missing: An ActiveMQ instance running locally, so we don't have to connect to a remote instance, which would be cumbersome and error-prone.

Hence, using the same approach as in the chapter [Connecting to a Database with RDS](#) we'll run a local ActiveMQ instance with Docker.

To accomplish this, we'll add an `activemq` section to our already existing `docker-compose.yml` file located in our application's root directory:

```
version: '3.3'

services:
 # ...
 activemq:
 image: stratospheric/activemq-docker-image
 ports:
 - 5672:5672
 - 61613:61613
 - 61614:61614
 - 61616:61616
 # ...
```

Now, when running `docker-compose up` in that directory, an ActiveMQ instance will be started in addition to other services (such as Keycloak and PostgreSQL), which have already been present before. Although multiple ports are mapped here (5672, 61613, 61614, 61616), for our purposes we'll only use port 61613 for connecting to the local ActiveMQ instance through WebSocket.

To point our local application to the local ActiveMQ instance, we configure the endpoint in the configuration file `application-dev.yml`:

```
custom:
 # ...
 web-socket-relay-endpoint: localhost:61613
 web-socket-relay-username: admin
 web-socket-relay-password: admin
 web-socket-relay-use-ssl: false
```

As we're running locally, we disable the SSL support of the underlying Tcp-Client by setting `web-socket-relay-use-ssl` to `false`. For obtaining the connection, we use the default ActiveMQ admin user.

The source code for the [stratospheric/activemq-docker-image](#) ActiveMQ Docker image used in the `docker-compose.yml` file is available on [GitHub](#).

# 14. Tracing User Actions with Amazon DynamoDB

In the chapter [Connecting to a Database with RDS](#), we have used a *relational database management system (RDBMS)* for storing data in our sample Todo application.

As mentioned before, relational databases aren't going anywhere and can still be considered the cornerstone of data management when it comes to web applications.

However, with the rise of Web 2.0 applications and a renewed interest in non-relational databases since around 2010, these days relational databases often are accompanied by and complemented with so-called *NoSQL databases*.

The term (originally coined by Carlo Strozzi) was reintroduced in 2009, when Johan Oskarsson, who was a software developer at [Last.fm](#) at the time, organized an event about “open-source distributed, non-relational databases”.

While on the surface the term looks like it's advocating against the use of SQL databases it simply stands for “non-SQL” or “non-relational” databases that are often used in tandem with more traditional relational databases to meet specific data management needs.

NoSQL databases come in a variety of flavors, such as:

- key-value stores (e.g. Amazon DynamoDB, Redis, or MemcacheDB)
- wide-column stores or column-oriented databases (e.g. Apache Cassandra)

- document stores or document-oriented databases (e.g. Amazon DynamoDB, MongoDB, or CouchDB)
- graph databases (e.g. Neo4j)

These are quite different from each other, both in terms of the use cases they support and regarding their implementation. About the only thing they have in common is that they don't use *SQL* as a query language or relational concepts for managing and normalizing data.

That said, there are some similarities between different categories of NoSQL databases. In particular, NoSQL databases are commonly used for relatively unstructured data or for data where the exact structure isn't known beforehand. A good example of this are document stores, which are built around the premise of complex, nested, loosely structured collections of data rather than tabular data with strictly defined relations between different tables and entities.

In the following, we'll have a more detailed look at [Amazon DynamoDB](#), a NoSQL database service provided by AWS. DynamoDB can be used as both a key-value store and a document database. It's particularly suited to use cases with a large number and high frequency of events to be stored in a database. DynamoDB is a scalable database service that, according to AWS, provides "single-digit millisecond performance".

## Use Case: Tracing User Actions

Processes that generate a huge amount of events at a high frequency are ideal use cases for DynamoDB. One such use case in the context of our sample Todo application would be tracing the user's journey through our application, for example, to generate data for optimizing the user experience. By knowing which links are clicked the most and which features are used most frequently, we

can make informed decisions about which parts of our application to optimize rather than making guesses about which areas of our application first deserve our attention.

Using [Spring Events](#), we'll generate this data by emitting events whenever a user executes a relevant action in our application. We'll use an `@EventListener` to store this data in a DynamoDB table mapped to an entity in our application.

There's one important caveat regarding this feature, though: Depending on the personally identifiable information you store about individual users (e.g. the users' IP addresses) this might violate their privacy and laws and regulations like the [General Data Protection Regulation \(GDPR\)](#). So, for both your users' and your own sake, you might want to exercise some caution when actually applying a user tracing feature in a production application.

With that word of warning out of the way, let's start with implementing our user tracing feature.

## Amazon RDS vs. Amazon DynamoDB

Let's recap why we're using DynamoDB in the first place rather than our existing relational database running on Amazon RDS. After all, adding another component and service to the mix introduces additional complexity to our architecture. So, if we're doing so it'd better be worth it.

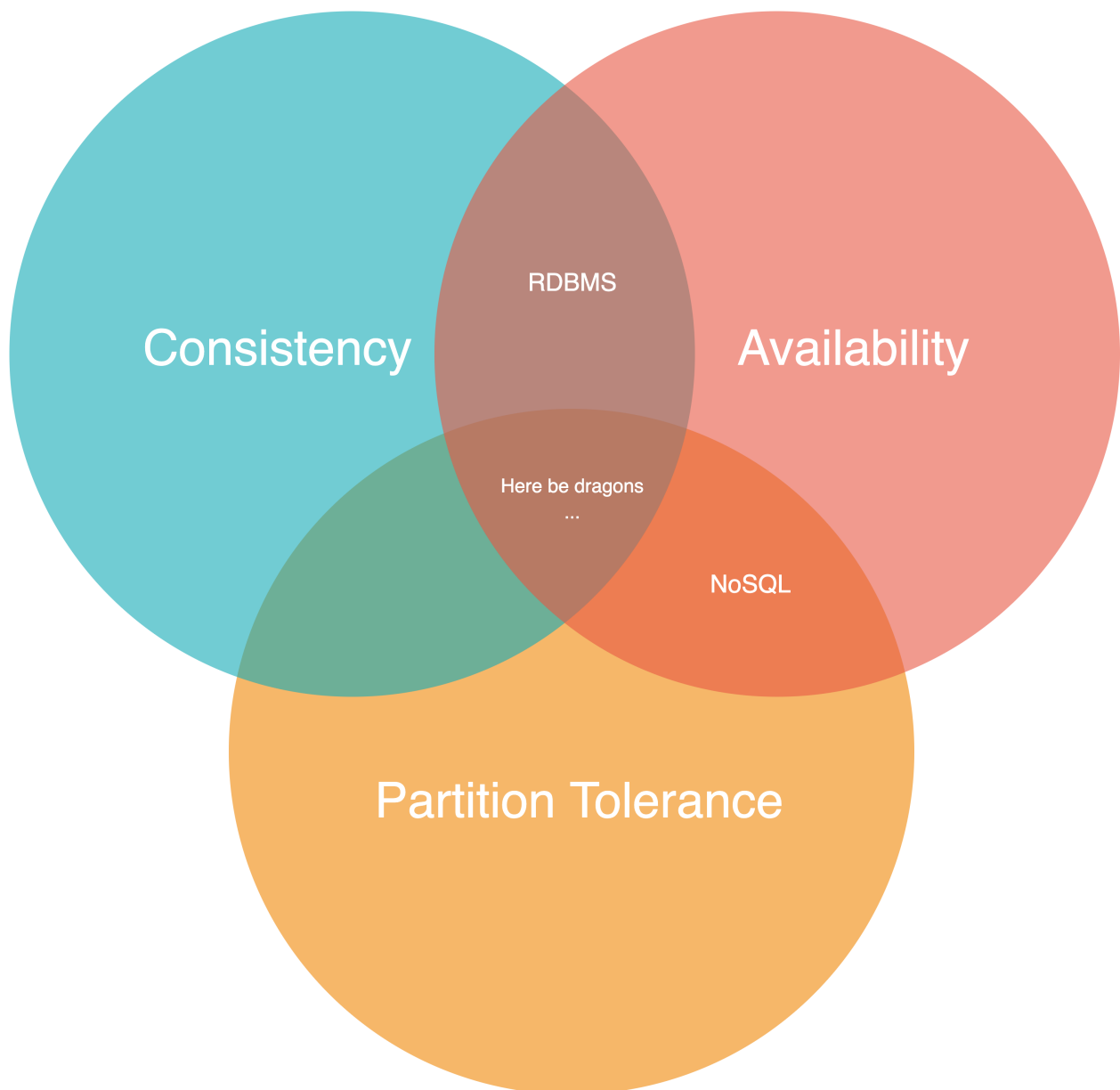
To reflect on the reason why we're going to use a NoSQL database, let's take a step back and consider what traits we usually expect our databases to have. There are two key concepts (and handy acronyms derived from those) in that respect: The **CAP theorem** and **ACID**.

## CAP Theorem

The **CAP theorem** (or “Brewer’s theorem” after computer scientist Eric Brewer) states that a distributed system can only make two of these three guarantees simultaneously:

- **Consistency:** Every node in a distributed system responds with the most recent data for a specific request. If an update is currently in progress the system will block the request until the update has finished.
- **Availability:** Every request receives a response, even if that request contains out-of-date data.
- **Partition tolerance:** The system continues to operate even if one or more nodes fail or messages have been dropped.

While RDBMS typically guarantee compliance with the consistency and availability stipulations, NoSQL database systems usually provide the latter two. There exists no distributed data store that can promise to maintain all three of those guarantees:



Consistency, Availability, and Partition Tolerance of RDBMS and NoSQL databases (CAP Theorem).

## ACID

So, while RDBMS focus on data being consistent and available, NoSQL database systems emphasize data availability and fault tolerance.

This has repercussions on another set of constraints database systems are often expected to satisfy. These constraints, commonly abbreviated as **ACID**, are:

- **Atomicity:** Each transaction is treated as a single unit. It either succeeds entirely or not at all.
- **Consistency:** Each transaction transitions a database from one valid state to another.
- **Isolation:** Concurrent transactions yield the same overall result as if they had been executed in sequence.
- **Durability:** Once a transaction has been committed the state resulting from it will be retained even after a subsequent system failure.

These properties focus on transactions as their main premise. They don't explicitly take distributed system behaviors into account.

Consequently, due to NoSQL database systems only guaranteeing data availability and fault tolerance they trade off ACID's strict consistency requirement for availability, performance, and scalability.

While with NoSQL database systems transactions usually are still atomic, durable, and – to some extent and depending on the exact implementation – isolated, these systems usually don't promise consistency but rather a property called *eventual consistency*.

This feature sometimes is summarised by the acronym *BASE*, which is derived from the real-world chemical opposite of an acid. This acronym in turn stands for these properties of a database system:

- **Basically available:** The system is always available for read and write operations. However, there's no guarantee whatsoever regarding the consistency of data these operations result in.



- **Soft state:** The state of a distributed database system might or might not have converged between nodes at any given point in time. There's merely a probability attached to knowing a state, rather than certainty for being able to do so.
- **Eventual consistency:** If a distributed database system runs long enough it will eventually achieve consistency, i.e. at some point in the future we will be able to know the state of the database for sure, and afterward that state will remain consistent and durable.

## Scenarios

Due to their focus on availability and fault tolerance and only a soft – or eventual – consistency requirement, NoSQL database systems shine when it comes to use cases where we need a system to be highly available and highly scalable.

Since NoSQL database systems promise partition tolerance they allow us to not only easily scale up (i.e. making it faster by adding more RAM or a faster CPU to a single instance) but also to scale out by adding additional instances (or: *replicas*) without compromising the overall availability of our distributed system.

This in turn lends itself to situations where we have a potentially large number of write operations but don't necessarily require data consistency at any given point in time.

## Benefits of DynamoDB

Now that we have addressed more general considerations for when to opt for a NoSQL database system instead of an RDBMS one, it's time to make the case for using DynamoDB specifically.

Amazon DynamoDB can be used as both a key-value store and a document

database. It allows us to use a flexible schema for our data with each item potentially having a different number of columns/attributes.

These data storage patterns are most suitable for situations where we either have very simple data structures such as lookup tables (key-value store) or highly unstructured, even somewhat unpredictable ones (document database).

Apart from these functional requirements covered by DynamoDB, this particular NoSQL database service also meets non-functional requirements such as:

- **Performance and scalability:** DynamoDB can handle more than 10 trillion requests per day with up to 20 million requests per second. We can have DynamoDB automatically replicate our data across different AWS regions, too.
- **Access control:** DynamoDB allows for [fine-grained control](#) over who can access which data entries from our DynamoDB tables. We can even define subsets of attributes to be visible only to specific IAM users, groups, or roles.
- **Event stream data:** DynamoDB keeps a record of item-level changes made in the last 24 hours. Using Amazon Kinesis Data Streams for DynamoDB we can capture these changes and persist them, e.g. for logging or data analysis purposes.
- **Encryption and automated backups:** By default, DynamoDB automatically creates backups and encrypts data at rest. We can also create and restore full backups of our DynamoDB tables on demand.
- **Time-to-live (TTL):** This DynamoDB feature allows us to define a table column containing [individual expiration times for each item](#). When the value defined in this column (as a UNIX Epoch timestamp) is less than the current timestamp the entry will be deleted automatically with no further business logic required.

These features are most suitable to scenarios where we're dealing with a larger number of events whose amount and/or frequency isn't known beforehand. Fine-grained access control and a time-to-live for each item allow us to automate processes and behaviors without having to implement those ourselves or drowning in event streaming data we couldn't possibly deal with manually.

Hence, we'll be using DynamoDB for event data, even though for our case of storing user flows in a sample application with just a few users we, admittedly, don't really need DynamoDB and the rich feature set and performance characteristics it provides.

## DynamoDB Terminology

Before we dive into designing a DynamoDB database for our use case, let's explore the key concepts of DynamoDB.

DynamoDB uses **tables** for storing **items**. Each of these items can have several scalar or nested **attributes**. Other than the key attributes, DynamoDB tables are **schemaless**, meaning that each item can have a different set of attributes.

A DynamoDB table has a unique name across an AWS account and region. Each table requires a **partition key** attribute, which serves as the primary key for identifying individual items in a table.

For scalability, DynamoDB stores the items of a table across many **partitions**. Too many read or write operations on a single partition in a short time frame can cause **throttling** – this effect is also called a **hot partition**. To avoid hot partitions, the partition keys of the items in a table should be as evenly distributed as possible.

Tables can also define an optional **sort key** attribute (also referred to **range key**). If such an attribute is provided, DynamoDB treats the partition key and the sort

key as a composite key. This allows us to have multiple items with the same partition key value, but a different value for the sort key, which in turn allows us to aggregate related data.

To query data using attributes other than the partition key, we can also define **secondary index** attributes. These allow us to access our data by alternative key attributes.

The throughput of DynamoDB tables is measured in **read capacity units** and **write capacity units**. A DynamoDB table can be priced **on-demand**, which means it automatically scales its partitions with the load of read and write capacity units, or it can be priced with **provisioned capacity**, which means that above a certain threshold of read and write capacity units the DynamoDB table will throttle access (and protect your wallet).

## Designing Data Schemas with DynamoDB

When using a NoSQL database system like DynamoDB, we have to approach the design of our data structures differently than if we were using an RDBMS to avoid maintainability and production performance problems.

One key consideration for example is which attributes the items in a particular table should have in common from the start. If we decide to add attributes to each already existing item later on, this might result in a huge amount of expensive write operations, depending on the number of items in our table at the time.

Working with NoSQL databases in general and DynamoDB in particular also requires a different way of thinking than working with RDBMS. With RDBMS, we usually work with normalized data and join data depending on our use case, while with NoSQL databases, we have to pre-join our data to achieve the best

possible performance.

Deciding which attributes to use as keys for searching and identifying items in our table is another vital aspect to consider when designing a DynamoDB table.

While RDBMS database design tends to prioritize flexibility, independence from specific business logic, and data normalization to avoid data duplication and the potential maintenance issues that arise from that, NoSQL table design favors query efficiency and data structures optimized for specific use cases.

Hence, when working with NoSQL databases, we want to design our data schema in a fashion that allows us to run the most common queries and operations in the most efficient manner possible at the expense of data normalization and flexibility.

This results in two main principles for designing NoSQL schemas:

1. Don't design your schema until you know the specific queries you want it to respond to.
2. Use as few tables as possible and try to keep related data together.

The designs and structures resulting from the second principle can also be described as *single-table design*.

In the next section, we'll design a DynamoDB table for our user tracing feature.



If you'd like to know more about best practices for designing DynamoDB schemas have a look at these resources: [Data modeling with Amazon DynamoDB AWS re:Invent talk](#), [The DynamoDB Book](#), [NoSQL Design for DynamoDB](#), [Best Practices for Designing and Architecting with DynamoDB](#),

## Implementing User Tracing in the Todo App

Now that we know what DynamoDB is about, let's start with actually using DynamoDB in our sample Todo application.

Following the packaging by feature approach, the new Java classes for this feature reside in `dev.stratospheric.todoapp.tracing`.

### Creating DynamoDB Tables with CDK

So, let's get started by creating the DynamoDB table for our tracing use case. As usual, we'll create a new app in our CDK project – this one called `DynamoDbApp`.

Here's the skeleton of that stack:

```
public class DynamoDbApp {

 public static void main(final String[] args) {
 App app = new App();

 // ...

 Stack dynamoDbStack = new Stack(app, "DynamoDbStack", StackProps.builder()
 .stackName(applicationEnvironment.prefix("DynamoDb"))
 .env(awsEnvironment)
 .build());

 new BreadcrumbsDynamoDbTable(
 dynamoDbStack,
 "BreadcrumbTable",
 applicationEnvironment,
 new BreadcrumbsDynamoDbTable.InputParameter("breadcrumb")
);

 app.synth();
 }
}
```

This CDK app creates a new stack called `DynamoDbStack`. As part of this stack, we commission one table to store our traces.

We’ve chosen the name “breadcrumb” here since, similar to the trail of breadcrumbs left in the forest by Hansel and Gretel, the feature we’re going to implement will allow us to trace back an individual user’s steps in our application.

This breadcrumb table is created by the `BreadcrumbsDynamoDbTable` construct:

```
public class BreadcrumbsDynamoDbTable extends Construct {

 public BreadcrumbsDynamoDbTable(
 final Construct scope,
 final String id,
 final ApplicationEnvironment applicationEnvironment,
 final InputParameter inputParameters
) {

 super(scope, id);

 new Table(
 this,
 "BreadcrumbsDynamoDbTable",
 TableProps.builder()
 .tableName(applicationEnvironment.prefix(inputParameters.tableName))
 .partitionKey(
 Attribute.builder().type(AttributeType.STRING).name("id").build()
)
 .encryption(TableEncryption.AWS_MANAGED)
 .billingMode(BillingMode.PROVISIONED)
 .readCapacity(10)
 .writeCapacity(10)
 .removalPolicy(RemovalPolicy.DESTROY)
 .build()
);

 record InputParameter(String tableName) {
 }
 }
}
```

The `Table` class is an AWS CDK level 2 construct that abstracts the underlying CloudFormation properties. Our custom construct takes the table name

as an input parameter and prefixes the environment and application name, e.g., `production-todo-app-breadcrumb`. This allows us to create the table for multiple environments and reuse the construct for different applications.

Next, we define the partition key. This `String`-type partition key acts as the primary key of our table and translates to the `id` attribute of our entity (aka “item”).

What’s left is to add a new `deploy` and `destroy` command to our `package.json`, similar to our existing CDK apps.

## Connecting to DynamoDB

First, we add the Spring Cloud AWS DynamoDB starter as a dependency to our application’s `build.gradle` file:

```
implementation 'io.awspring.cloud:spring-cloud-aws-starter-dynamodb'
```

This will provide us with autoconfigurations, APIs and convenient annotations for defining, creating, and accessing entities with DynamoDB.

There’s an additional community project called [Spring Data DynamoDB](#), which integrates with Spring Data and which provides additional features such as repositories, CRUD methods, projections, and REST support via [Spring Data REST](#).

While these are certainly useful features, particularly for more elaborate use cases, our use case is simple enough. We don’t need any of these features, which is why we’re doing without the complexity of adding another dependency here.

To establish DynamoDB connectivity, Spring Cloud AWS auto-configures a DynamoDB client for us. So, there’s nothing left for us to do in that respect.



For interacting with DynamoDB we'll use the auto-configured `DynamoDbTemplate` bean from Spring Cloud AWS that provides an API for basic CRUD operations.

Since a DynamoDB instance is automatically available for each AWS account in each region that's all we have to do.

## Mapping a DynamoDB Table to Java Objects

To store data in our new DynamoDB table from our Java application we need to map that table to a model class. Therefore, we create this new `Breadcrumb` class:

```
@DynamoDbBean
public class Breadcrumb {

 private String id;
 private String uri;
 private String username;
 private String timestamp;

 @DynamoDbPartitionKey
 public String getId() {
 return id;
 }

 // getters and setters
 // ...
}
```

The `@DynamoDbBean` annotation from the DynamoDB SDK marks a class as a representation of a DynamoDB table.

The `@DynamoDbPartitionKey` annotation denotes the `id` attribute to be used as the table's primary key.

Without further configuration, the Spring Cloud AWS DynamoDB integration

resolves the DynamoDB table name from the Java class name via the `DefaultDynamoDbTableNameResolver`.

To deploy our application to multiple stages and access a unique table per stage, we provide a custom implementation of the `DynamoDbTableNameResolver`. This allows us to prefix our table name with the environment and application name. This, in turn, allows us to deploy our application without any configuration change to multiple environments.

We'll add a new configuration class called `AmazonDynamoDBConfig` and provide a `DynamoDbTableNameResolver` bean to our Spring context:

```
@Configuration
public class AmazonDynamoDBConfig {

 @Bean
 public DynamoDbTableNameResolver
 dynamoDbTableNameResolver(Environment environment) {

 String environmentName = environment
 .getProperty("custom.environment");

 String applicationName = environment
 .getProperty("spring.application.name");

 return new DynamoDbTableNameResolver() {
 @Override
 public <T> String resolve(Class<T> clazz) {
 String className = clazz.getSimpleName().replaceAll("(.)\\p{Lu}", "$1_$2")\
 .toLowerCase(Locale.ROOT);
 return environmentName + "-" + applicationName + "-" + className;
 }
 };
 }
}
```

This custom implementation prefixes the sanitized and lowercased classname of the DynamoDB mapping class with the application's name and environment.

Given the following configuration inside our `application.yml`:

```
spring:
 application:
 name: todo-app

custom:
 environment: production
```

Our Java DynamoDB mapping entity class `Breadcrumb` will be resolved as `production-todo-app-breadcrumb`.

## Storing Items in DynamoDB via Spring Events

Now that we can connect to DynamoDB, have our DynamoDB table ready, and a model class mapped to that table we can tie it all together by creating a new `Breadcrumb` entity every time a user visits a specific page in our sample `Todo` application.

To keep the code for tracing users separate from our actual business logic and decouple the event of a user visiting a path from storing the data generated by that event we'll use Spring [application events](#). To this end, we'll introduce a new event class called `TracingEvent` and have it extend `ApplicationEvent`:

```
public class TracingEvent extends ApplicationEvent {

 private final String uri;
 private final String username;

 public TracingEvent(
 Object source,
 String uri,
 String username
) {
 super(source);

 this.uri = uri;
 this.username = username;
 }

 // getters
 // ...
}
```

Provided by the [Spring Integration messaging library](#), `ApplicationEvents` allow for the exchange of information between loosely coupled components using the familiar [publish-subscribe pattern](#).

Next, we create a data access object (DAO) as a Spring `@Component` named `TraceDao`, which depends on the `DynamoDbTemplate` bean to create a new `Breadcrumb` item whenever a `TracingEvent` is emitted. We use the `@EventListener` Spring annotation to listen to a specific type of `ApplicationEvent`, `TracingEvent` in our case:

```
@Component
public class TraceDao {

 private final DynamoDbTemplate dynamoDbTemplate;

 public TraceDao(DynamoDbTemplate dynamoDbTemplate) {
 this.dynamoDbTemplate = dynamoDbTemplate;
 }

 @Async
 @EventListener(TracingEvent.class)
 public Breadcrumb create(TracingEvent tracingEvent) {
 Breadcrumb breadcrumb = new Breadcrumb();
 // ...

 dynamoDbTemplate.save(breadcrumb);

 return breadcrumb;
 }
}
```

We use the `@Async` annotation to consume the `TracingEvent` asynchronously without blocking the thread and potentially delaying the incoming request.

We enable this support for asynchronous behavior in our Spring Boot application with `@EnabledAsync` and a custom configuration class called `AsyncConfig`:

```
@EnableAsync
@Configuration
public class AsyncConfig {

 @Bean
 @Primary
 public TaskExecutor taskExecutor() {
 ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
 executor.setMaxPoolSize(10);
 executor.initialize();
 return executor;
 }
}
```

Furthermore, we specify our primary `TaskExecutor` that's used to process async operations on a dedicated thread pool. The reason for this manual `TaskExecutor` definition is that we already have beans of type `TaskExecutor` in our `ApplicationContext` from Spring Cloud AWS messaging. Without marking one executor as `@Primary`, we wouldn't be able to use the same executor deterministically.

Finally, we can have Spring's `ApplicationEventPublisher` emit a new `TracingEvent` each time a user visits a page we'd like to trace. Here's an example of how to do this from our `IndexController`:

```
@Controller
public class IndexController {

 private final ApplicationEventPublisher eventPublisher;

 // ...

 @GetMapping
 @RequestMapping("/")
 public String getIndex(Principal principal) {
 this.eventPublisher.publishEvent(
 new TracingEvent(
 this,
 "index",
 principal != null
 ? principal.getName()
 : "anonymous"
)
);

 return "index";
 }
}
```

This aspect-oriented approach for the cross-cutting concern of user tracing allows us to easily modify or remove the code without affecting the business logic. Should we ever need to remove the user tracing feature or disable it in

specific regions (for example, to accommodate local privacy laws), we can easily achieve this by making our `TraceDao` component only available in specific AWS regions by using one of Spring's condition annotations.

## Setting up the Required IAM Permissions

Even though we don't need any additional infrastructure for using DynamoDB, there's one thing still missing: The IAM permissions for our application.

For the role assumed by our sample `Todo` application to be allowed to create new DynamoDB tables and items in those tables we add a new `IAM PolicyStatement` to the CDK `ServiceApp`:

```
public class ServiceApp {

 public static void main(final String[] args) {
 App app = new App();

 // ...

 new Service(
 serviceStack,
 "Service",
 awsEnvironment,
 applicationEnvironment,
 new Service.ServiceInputParameters(
 // ...
 .withTaskRolePolicyStatements(List.of(
 PolicyStatement.Builder.create()
 .sid("AllowDynamoTableAccess")
 .effect(Effect.ALLOW)
 .resources(
 List.of(String.format("arn:aws:dynamodb:%s:%s:table/%s",
 region, accountId, applicationEnvironment.prefix("breadcrumb")))
)
 .actions(List.of(
 "dynamodb:Scan",
 "dynamodb:Query",
```

```
 "dynamodb:PutItem",
 "dynamodb:GetItem",
 "dynamodb:BatchWriteItem",
 "dynamodb:BatchWriteGet"
))
 .build()
))
 // ...
);

// ...
}
}
```

With the next deployment, this policy will be added to the IAM policies for our sample Todo application, granting it the required permissions.

## Reading from a DynamoDB Table

So far, we're writing user breadcrumb events to DynamoDB, but we're not reading any data from that table. Use cases for reading the data could be:

- "I'd like to see all events for a user."
- "I'd like to see all events for a user in a specific time period."

To respond to these requests we make use of the timestamp sort key in our DynamoDB table. As explained above, a sort key allows us to aggregate data by a partition key. The items from that aggregated data set will then be distinguishable by the sort key. This in turn allows us to fetch items by a [partition key, sort key] tuple.

Our username partition key allows us to query for a list of all Breadcrumb items for a specific user as well as a list of events for a user within a specific time frame:



```
@Component
public class TraceDao {

 // ...

 public List<Breadcrumb> findAllEventsForUser(String username) {
 Breadcrumb breadcrumb = new Breadcrumb();
 breadcrumb.setUsername(username);

 return dynamoDbTemplate.query(
 QueryEnhancedRequest
 .builder()
 .queryConditional(
 QueryConditional.keyEqualTo(
 Key
 .builder()
 .partitionValue(breadcrumb.getId())
 .build()
)
)
 .build(),
 Breadcrumb.class
).items()
 .stream()
 .toList();
 }

 public List<Breadcrumb> findUserTraceForLastTwoWeeks(String username) {
 ZonedDateTime twoWeeksAgo = ZonedDateTime.now().minusWeeks(2);

 Breadcrumb breadcrumb = new Breadcrumb();
 breadcrumb.setUsername(username);

 return dynamoDbTemplate.query(
 QueryEnhancedRequest
 .builder()
 .queryConditional(
 QueryConditional.keyEqualTo(
 Key
 .builder()
 .partitionValue(breadcrumb.getId())
 .build()
)
)
)
```

```

)
 .filterExpression(
 Expression
 .builder()
 .expression("timestamp > :twoWeeksAgo")
 .putExpressionValue(":twoWeeksAgo",
 AttributeValue
 .builder()
 .s(twoWeeksAgo.toString())
 .build()
)
)
 .build(),
 Breadcrumb.class
)
 .items()
 .stream()
 .toList();
}
}

```

Both methods use an `QueryEnhancedRequest` for querying for a list of matching `Breadcrumb` items. In the `findUserTraceForLastTwoWeeks()` method we additionally provide a filter `Expression` to query for a two weeks time span.

## Enabling Local Development

We want our local development environment to be as close to production as possible. For example, we'd like to test our new feature before deploying it into our staging – let alone production – environment.

Therefore, we add `dynamodb` as an additional service to our `LocalStack` environment configuration in our `docker-compose.yml`:

```
services:
...
localstack:
...
environment:
- SERVICES=sqs,ses,dynamodb
```

As Spring Cloud AWS, specifically through its DynamoDB starter, configures the DynamoDB client, we only need to point the client to our LocalStack instance. By overriding the property `spring.cloud.aws.endpoint` we already override this endpoint configuration for all Java SDK clients.

What's left is to create the table as part of the LocalStack container bootstrapping phase. Therefore, we extend our LocalStack init script with the following command:

```
#!/bin/sh

... further initialization

awslocal dynamodb create-table \
 --table-name local-todo-app-breadcrumb \
 --attribute-definitions AttributeName=id,AttributeType=S \
 --key-schema AttributeName=id,KeyType=HASH \
 --provisioned-throughput ReadCapacityUnits=10,WriteCapacityUnits=10 \

echo "Initialized."
```

# Part III: Production Readiness

## with AWS

We have learned how to deploy a Spring Boot application to AWS and how to make it talk to different AWS services to implement certain use cases. That's pretty cool already!

However, that's not enough. How do we know that our application is healthy and doing what we expect it to do? In the cloud, no system administration team will notify us when something is amiss. We built it, we run it.

So, in the last part of this book, we'll implement some observability and alerting patterns to make sure that we can sleep at night, knowing that our application is running smoothly and bringing value to our users.

The main AWS service for observability is Amazon CloudWatch. We will send structured log data to CloudWatch and learn how we can query that data.

We will also learn how to send custom metrics to CloudWatch and how to query them together with default metrics from all the AWS services.

Then, we will set up alarms on those metrics, so that we get notified when certain thresholds are breached. This allows us to respond quickly to any incidents.

Finally, we'll make the application production-ready by securing it with HTTPS and hosting it on a custom domain.

# 15. Structured Logging with Amazon CloudWatch

What's the first thing we look at when something goes wrong? The logs. With a proper logging strategy, logs are a very valuable source of information about what users are doing in our application as well as for researching the root cause of errors.

I (Tom) remember projects where we had to send an email request to the system admin team, telling them that we need the logs from a certain time interval. A system admin would log into the production machines and copy the log file from each server into a secure place where we could download it via SFTP. Sometimes, this process would take hours because the system admin was on a break. Luckily, in the cloud, this cumbersome process is a thing from the past. Just imagine AWS system admins responding to log file requests from their millions of cloud customers!

A better solution is to send log information to a central server that gathers and stores log events from multiple instances. Developers get direct access to that logging server and can use it to query logs in real-time at any time. They don't even have to wake up a system admin when they have to investigate an error in the middle of the night.

Ideally, the log events not only contain plain text but structured information which we can use to filter log events. This structured information may contain the ID of the current user, for example, so we can collate log events from the same user.

The goal of this chapter is to implement a logging solution that allows us to search and filter structured log events from our Todo app with Amazon CloudWatch – Amazon’s main observability service, providing log query and dashboard capabilities for log data (among other observability capabilities that we’re going to look at in the next chapter).

## Logging with AWS

With the rise of DevOps, a logging server has become a de-facto standard in modern software operations. There are many logging solutions out there, and although we’re running our Todo app on AWS, this doesn’t mean we have to use an AWS service as our logging provider.

We could just as well configure our Spring Boot app to send its logs to another logging provider in the cloud like Splunk, Loggly, or Logz.io. Since these providers are specialized in logging, chances are high that they provide a more complete feature set around log ingestion, querying logs, and building dashboards than Amazon CloudWatch does.

However, an Amazon CloudWatch integration is “built into” many other AWS services. So even if its feature set does not match that of other logging providers, it provides the path of least resistance to implement a workable logging solution when our app is running on AWS.

Many AWS services can send their logs to CloudWatch, without us having to configure anything. We can have RDS send PostgreSQL logs, for example. If we chose to use a different logging provider, the configuration could become more complicated. In many cases, the logs have to go through CloudWatch anyway and are only sent to the external logging provider from there.

In this chapter, we focus on application logs, because they are the most impor-

tant in day-to-day operations.

## CloudWatch Logging Terminology

Before we dive into the details of logging with CloudWatch, let's discuss some CloudWatch vocabulary:

A **log stream** is, well, a stream of log events that is emitted by a certain component. In our case, each task running in ECS (i.e. each instance of our application) is emitting a stream of logs.

Log streams are aggregated into **log groups**. In our case, we have one log group for our Todo application that aggregates the log streams of all instances of the application. This log group allows us to query the logs of our Todo application across different instances of the app that might be running at the same time.

**CloudWatch Insights** is a service that provides a UI and a powerful query language to search through one or more log groups. We'll use it later to query our structured logs.

## Status Quo: Logging Unstructured Text

In the current state of our Todo application, we are already sending logs to CloudWatch. All we had to do for this to work is to configure ECS – where our Docker containers are running – to install the [CloudWatch agent](#) on our EC2 instances. The agent will take the standard output of our application and send it to CloudWatch.

Let's see how this is configured in our current setup and learn some CloudWatch concepts on the way.

## Configuring ECS to Send Logs to CloudWatch

In the chapter *Designing a Deployment Project with CDK*, we have built a CDK project that describes the infrastructure our application needs. One part of that infrastructure was the Service construct, which sets up an ECS task definition that takes our Docker image and deploys it to EC2 instances managed by ECS.

In the Service construct, we have already configured ECS to send logs to CloudWatch (see the full code [on GitHub](#)):

```
Role ecsTaskExecutionRole = Role.Builder.create(this, "ecsTaskExecutionRole")
...
.inlinePolicies(Map.of(
 applicationEnvironment.prefix("ecsTaskExecutionRolePolicy"),
 PolicyDocument.Builder.create()
 .statements(singleList(PolicyStatement.Builder.create()
 .effect(Effect.ALLOW)
 .resources(singleList("*"))
 .actions(Arrays.asList(
 ...
 "logs:CreateLogStream",
 "logs:PutLogEvents"))
 .build()))
 .build()))
 .build();

LogGroup logGroup = LogGroup.Builder.create(this, "ecsLogGroup")
 .logGroupName(applicationEnvironment.prefix("logs"))
 .retention(serviceInputParameters.logRetention)
 .removalPolicy(RemovalPolicy.DESTROY)
 .build();

CfnTaskDefinition.ContainerDefinitionProperty container =
 CfnTaskDefinition.ContainerDefinitionProperty.builder()
 ...
 .logConfiguration(CfnTaskDefinition.LogConfigurationProperty.builder()
 .logDriver("awslogs")
 .options(Map.of(
 "awslogs-group", logGroup.getLogGroupName(),
```



```
 "awslogs-region", awsEnvironment.getRegion(),
 "awslogs-stream-prefix", applicationEnvironment.prefix("stream"),
 "awslogs-datetime-format", serviceInputParameters.awslogsDateTimeFormat))
 .build()
 ...
 .build();
```

The role that is running the ECS tasks – the `ecsTaskExecutionRole` – will need to write logs to CloudWatch, so we give it the permissions `logs:CreateLogStream` and `logs:PutLogEvents`.

Next, we create a `LogGroup` with a specific `logGroupName`. The log group name will look something like this in our example: `prod-todo-app-logs`. While creating the log group, we can define how long we want to retain the logs and a removal policy to decide what happens if the log group is removed from the CDK project (with the `DESTROY` policy, the logs will be deleted if we re-deploy a version of the CDK stack that doesn't include the log group).

The most important part is the configuration of the ECS task definition. We configure the ECS tasks to use the [awslogs log driver](#). This driver is the piece of infrastructure that will take everything our Docker container logs to `STDOUT` and `STDERR` and send it to CloudWatch. In the options, we pass the name of our previously created log group, the AWS region, and a stream prefix to define where the logs should be sent.

An important piece of configuration is the `awslogs-datetime-format`. This option takes a datetime pattern that will tell the log driver how to parse the logs. The log driver expects each log entry to start with a date and time. Knowing the date time pattern, the log driver can look for it and decide where one (potentially multi-line) log event ends and a new log event starts. A wrong datetime pattern causes the logs in CloudWatch to become very (!) confusing, with log events being mixed up and timestamps all over the place, so it's important to get this right.

Once we deploy the `Service` CDK construct with this configuration, our application logs will be available in CloudWatch. However, we haven't configured our Spring Boot app to log in a specific format, yet. Furthermore, in our `ServiceApp` we're not passing any log-specific configuration into the `Service` construct and instead rely on the defaults defined by the `Service` construct. Let's see what the logs look like in CloudWatch with this "I feel lucky" configuration.

## **Querying Unstructured Logs with CloudWatch Logs Insights**

The CloudWatch Logs Insights screen in the AWS console looks something like this:

CloudWatch > CloudWatch Logs > Logs Insights

Select log group(s) ▼ 5m 30m **1h** 3h 12h Custom 📅

Clear staging-todo-app-logs ✕

```

1 fields @timestamp, @message
2 | sort @timestamp desc
3 | limit 20

```

Run query Save History

Logs Visualization Export results ▼ Add to dashboard ⚙️

Showing 4 of 4 records matched ⓘ  
4 records (3.0 kB) scanned in 3.1s @ 1 records/s (1,007.161 B/s) Hide histogram

#	@timestamp	@message
▶ 1	2021-05-02T06:18:22.009+10:00	2021-05-01 20:18:22.008 INFO 1 --- [MessageBroker-1] o.s.w.s.c.WebSocketMe
▶ 2	2021-05-02T06:18:20.381+10:00	2021-05-01 20:18:20.380 INFO 1 --- [MessageBroker-1] o.s.w.s.c.WebSocketMe
▶ 3	2021-05-02T05:48:22.008+10:00	2021-05-01 19:48:22.007 INFO 1 --- [MessageBroker-1] o.s.w.s.c.WebSocketMe
▶ 4	2021-05-02T05:48:20.380+10:00	2021-05-01 19:48:20.380 INFO 1 --- [MessageBroker-1] o.s.w.s.c.WebSocketMe

Running a query in CloudWatch Insights.

We can select one or more log groups to run a search query on, filter it by a time frame, and view the results in text or chart form.

To build queries, we can leverage a [query language](#) built around the pipe operator that has a very similar meaning in this context as it has in Unix (i.e. forwarding the output of one command as input into the next command). Note that log data might take a few seconds to be ingested before it's available to query.

Let's look at the default query that CloudWatch uses to pre-fill the query field:

```
fields @timestamp, @message
| sort @timestamp desc
| limit 20
```

The query combines multiple commands with the pipe operator:

- The `fields` command extracts a list of fields from the log messages. Fields that start with `@` are system fields that CloudWatch populates for us. In our case, we're extracting the fields `@timestamp` and `@message` from the raw log data.
- We pipe the result into the `sort` command to sort the data by the timestamp in descending order, so we see the latest log events first.
- Finally, we pipe the sorted log events into the `limit` command that limits the result to 20 log events.

The result is a table of log events with a `@timestamp` column containing the timestamp and a `@message` column containing the message content, sorted by timestamp in descending order, limited to 20 events.

However, if we look at the `@message` column, we see that it contains the timestamp, as well:

```
2021-05-01 21:18:22.010 INFO 1 --- [MessageBroker-1] ...
```

We have one timestamp in the `@timestamp` column and another timestamp in the `@message` column. This is the result of our configuration of the `awslogs` log driver. We configured the `awslogs-datetime-format` option with a pattern for a timestamp so that the log driver knows where one log event starts and another ends. And that configuration worked great because the log events are parsed correctly. However, the log driver does not cut the timestamp out of the log message itself, so we now have it twice. That's a bit awkward, but not a showstopper.

So far, so good, but logs are only valuable if we can search them for keywords and patterns. Say we want to find errors in the logs. We can use the `filter` command to filter the output to log events containing the string “error”:

```
fields @timestamp, @message
| filter @message like /error/
| sort @timestamp desc
| limit 20
```

Unsurprisingly, after running this query, the table of results will only show log events containing the string “error” in the `@message` field. However, since the `@message` field contains the log level as well as the actual log message, the results of this query will contain log events that have been logged to `ERROR` level as well as log events that contain the string “error” in the actual log message. This might suffice in applications with small amounts of log traffic, but will be very bothersome for applications with a lot of log events.

So let’s try to fix it by parsing the log message into distinct fields and make each of those fields filterable and searchable. Spring Boot’s default logging configuration structures log messages in a pattern that we can parse with CloudWatch (line break added for readability):

```
fields @timestamp, @message
| parse @message / (?<level>[A-Z]+) (?<pid>[0-9]+) --- \
 \[(?<thread>[^\]]+)\] (?<logger>[^\]+) (?<message>.*)/
| display @timestamp, level, thread, logger, message
| filter level = 'ERROR'
| sort @timestamp desc
| limit 20
```

We have added the `parse` command that parses the `@message` field with a regular expression. In the regex, we can define named matching groups within parentheses like this: `(?<name>...)`. The `?<name>` part gives the matching

group a name and the `...` part defines the regex to match this group. Each named group will be extracted to a new field with the name of the group.

With the regular expression in the query above, we're extracting the fields `level`, `thread`, `logger`, and `message` from the raw `@message` field.

With the `display` command, we select which fields we want to show in the table of search results. Using the `filter` command we filter these results to include only log events with the `ERROR` log level.

Parsing log events like this is quite a powerful feature and gives us a lot of options to query log events. However, there are still some drawbacks.

Building a query with a regex is quite cumbersome and error-prone. One wrong character in the regex often means that all fields will be empty.

Also, running a fairly complex regex across the log data of a heavily logging application doesn't feel like a good idea. The CloudWatch team has probably built in some nifty optimizations, but still ... parsing logs "on the fly" with a regex can't be good for performance.

In summary, the main drawback of the current solution is that we cannot log and query structured information. We could simulate this by logging all the fields we're interested in into a CSV-like log message to make it easier to use the `parse` command to search through those fields. However, we'd still have to parse the log messages on-the-fly with regular expressions, which is awkward at best. Let's explore how we can implement real structured logging, instead.

## Logging and Querying Structured Data

CloudWatch Logs Insights supports [reading JSON objects](#) from log events. It parses the first JSON object of each log event automatically and makes the fields

of the JSON object available as fields in the Insights query.

For this to work with our setup, we have to configure our Spring application to output JSON log events. To make the structured logging even more useful, we'll add a custom field to these log events that will later be queryable via CloudWatch Insights. Finally, we'll see how much easier it gets to query these structured log events.

## Adding Custom Fields to Log Events

Let's start with adding structured data to our log events.

Adding custom fields to log events is a great way of improving the observability of our app. Custom fields on log events make the difference between finding an issue instantly and having to spend days to add more logging information and waiting until the issue happens again.

Some examples for custom fields on log events are:

- **A user ID:** If we add the ID or username of the logged-in user to each log event, we can quickly filter log events by a specific user. This way, if a user has a support request, we can quickly filter the logs to the relevant log events.
- **An entry point:** Aside from a web UI, an application may have other entry points for incoming work. It may have scheduled jobs, for example, that run in regular intervals without a user request. It may also have an HTTP API that is not user-facing. If we add the entry point as a field to our log events, we'll know exactly which path a log event stems from, helping immensely in analyzing the logs of applications with multiple entry points.
- **The root cause of an error:** When an exception occurs, we usually see a stack trace in the logs. The first thing we often do is scroll to the bottom

of the stack trace to see the root cause of the exception. We can reduce the effort of analyzing stack traces by adding the name of the root cause exception as a field in the log event. This way, we can filter the logs by a specific root cause and see what impact that root cause had.

There are many more fields we can add to make log events more valuable with custom fields, many of them very specific to the application we're building.

As an example from our Todo app, we'll add the name of the logged-in user to each log event. We can log personally identifiable information like user names without thinking twice about it because we're just building a demo app. In a real application, you should consult your security expert to decide whether logging a user name is acceptable or not (and maybe log a UUID instead).

The logging abstraction that Spring Boot apps use by default is SLF4J. SLF4J supports the notion of a "Message Diagnostic Context" (or MDC in short) that allows us to add custom fields to log events.

To add the name of a user to each log event, we need to intercept all incoming web requests, get the logged-in user who initiated the request, and then add the user name to the MDC. We can do this by implementing a Spring Handler-Interceptor:



```

class LoggingContextInterceptor implements HandlerInterceptor {

 private final Logger logger =
 LoggerFactory.getLogger(LoggingContextInterceptor.class);

 @Override
 public boolean preHandle(
 final HttpServletRequest request,
 final HttpServletResponse response,
 final Object handler) {

 Authentication authentication =
 SecurityContextHolder.getContext().getAuthentication();
 String userId = getUserIdFromPrincipal(authentication.getPrincipal());
 MDC.put("userId", userId);
 return true;
 }

 private String getUserIdFromPrincipal(Object principal) {
 if (principal instanceof String) {
 // anonymous users will have a String principal with value "anonymousUser"
 return principal.toString();
 }

 if (principal instanceof OidcUser) {
 try {
 OidcUser user = (OidcUser) principal;
 if (user.getClaimAsString("name") != null) {
 return user.getClaimAsString("name");
 } else {
 logger.warn("could not extract userId from Principal");
 return "unknown";
 }
 } catch (Exception e) {
 logger.warn("could not extract userId from Principal", e);
 }
 }
 return "unknown";
 }

 @Override
 public void afterCompletion(
 final HttpServletRequest request,

```

```
 final HttpServletResponse response,
 final Object handler,
 final Exception ex) {
 MDC.clear();
 }
 }
}
```

This interceptor extracts the username from the current user's `Principal` object, which can be a `String` or an `OidcUser` in our app. It then adds the username to the `userId` field in the MDC. If there is an error while extracting the username, the interceptor sets the user ID to “unknown”.

Since the MDC is attached to the current thread, and since a thread might be re-used later on (because it's part of a thread pool), we have to remove the MDC once the request has been processed. We do this in the `afterCompletion()` method of our interceptor.

The only thing left to do is to configure Spring to use our `LoggingContextInterceptor`:

```
@Component
class LoggingContextConfiguration implements WebMvcConfigurer {

 @Override
 public void addInterceptors(InterceptorRegistry registry) {
 registry.addInterceptor(new LoggingContextInterceptor());
 }
}
```

Implementing a `WebMvcConfigurer` like above lets us add our `LoggingContextInterceptor` to Spring's `InterceptorRegistry`, thereby allowing it to intercept all incoming web requests.

With this code in place, all log events that are emitted on the code path starting with an incoming web request will now have a field `userId`. But Spring Boot's logging configuration doesn't log custom fields by default, so we haven't won

anything, yet. We need to configure our application to generate structured JSON logs to see this field.

## Configuring Spring Boot to Log JSON

Since CloudWatch supports parsing the first JSON object in a log event, we will just configure our logging so that each log event emits a single JSON object that contains the log message and all custom fields we might have added to the MDC.

Fortunately, we don't have to build that ourselves. There are many different libraries out there that we can use to do that. We have opted for the [awslogs-json-encoder library](#) since it's pre-configured to create logs that CloudWatch can ingest. Therefore, we don't have to configure anything.

To make this library available to our application, we include it in our `build.gradle` file:

```
implementation 'de.siegmar:logback-awslogs-json-encoder:1.1.0'
```

Next, we have to configure Spring Boot's logging to use this library. Spring Boot uses Logback as the default logging library. Logback is usually configured by putting a `logback.xml` file into the classpath. Spring Boot supports its own configuration file `logback-spring.xml`, which adds some dynamic features to the Logback configuration file. Our [logback-spring.xml](#) file looks like this:

```
<configuration>

 <appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
 <encoder class="de.siegmar.logbackawslogsjsonencoder.AwsJsonLogEncoder"/>
 </appender>

 <appender name="PLAIN" class="ch.qos.logback.core.ConsoleAppender">
 <layout class="ch.qos.logback.classic.PatternLayout">
 <Pattern>
 %cyan(%d{ISO8601}) %highlight(%-5level) [%blue(%-30t)] ...
 </Pattern>
 </layout>
 </appender>

 <springProfile name="dev">
 <root level="INFO">
 <appender-ref ref="PLAIN"/>
 </root>
 </springProfile>

 <springProfile name="aws">
 <root level="WARN">
 <appender-ref ref="JSON"/>
 </root>
 </springProfile>

</configuration>
```

Except for the `<springProfile>` tags, this file conforms to the standard format of a `logback.xml` file. We created an `<appender>` named `JSON` that uses the `AwsJsonLogEncoder` from the `awslogs-json-encoder` library we just imported. All log events sent to this appender will generate JSON-formatted logs. We also created another appender named `PLAIN` that generates plain text logs.

Then, we configure the root appender depending on the Spring profile the application is running in. If the application is running in the `dev` profile, we send all log events to the `PLAIN` appender, so that we still have the logs in a human-readable format (albeit without our custom fields). If the application

is running in the `aws` profile, we send all logs to the JSON encoder to produce JSON-formatted logs.

We could of course choose to also generate JSON logs during local development. However, reading textual logs is easier for us humans than reading JSON, and working locally we often don't have the support of a log server parsing the logs for us. Losing our custom fields in the textual logs isn't too bad for local development, usually, since we're not analyzing incidents in local logs. But if we wanted to, we could also add some custom fields to our textual logs by adding `%X{userId}` to the log pattern, for instance.

To test the JSON log format, we can replace the `PLAIN` log appender with the `JSON` log appender in the `dev` profile and start the application locally with `./gradlew bootrun`. We'll see that the generated logs now look something like this (line breaks added for readability):

```
{
 "timestamp":"2021-05-15T07:43:37.860000+1000",
 "level":"INFO",
 "logger":"dev.stratospheric.todoapp.todo.TODOController",
 "thread":"http-nio-8080-exec-5",
 "message":"successfully created todo",
 "mdc":{
 "userId":"duke"
 }
}
```

Now that the Spring Boot app generates JSON logs, we have to make sure that our ECS container sends them to CloudWatch correctly.

## Configuring ECS to Parse Logs

When we start the app in the `aws` profile, it will now send JSON-formatted logs to the standard output. As we've discussed above, ECS will pick up these logs

and send them to CloudWatch. Just a quick reminder: The configuration responsible for this is the `CfnTaskDefinition` construct in our [Service construct](#):

```
CfnTaskDefinition.ContainerDefinitionProperty container =
 CfnTaskDefinition.ContainerDefinitionProperty.builder()
 ...
 .logConfiguration(CfnTaskDefinition.LogConfigurationProperty.builder()
 .logDriver("awslogs")
 .options(Map.of(
 "awslogs-group", logGroup.getLogGroupName(),
 "awslogs-region", awsEnvironment.getRegion(),
 "awslogs-stream-prefix", applicationEnvironment.prefix("stream"),
 "awslogs-datetime-format", serviceInputParameters.awslogsDateTimeFormat))
 .build())
 ...
 .build();
```

As discussed above, the most important configuration here is the `awslogs-datetime-format`. The `awslogs` driver will use the pattern we define here to distinguish one log event from the next. If a line in the logs doesn't have a date in this format, it's considered to be part of the previous log event. This is to support multi-line log events.

To properly work with the `AwsJsonLogEncoder` we configured in the `logback-spring.xml` configuration file, we need to pass in the correct value for the `awslogs-datetime-format`. Since the `Service` construct takes this value as part of the `ServiceInputParameters` object, we can just pass the value in our [ServiceApp CDK app](#):

```
new Service(
 serviceStack,
 "Service",
 awsEnvironment,
 applicationEnvironment,
 new Service.ServiceInputParameters(...)
 .withAwsLogsDateTimeFormat("%Y-%m-%dT%H:%M:%S.%f%z")
 ...
 .build()
```

The date pattern `%Y-%m-%dT%H:%M:%S.%f%z` is the one that is used in the JSON log events (compare it with the date string `2021-05-15T07:43:37.860000+1000` from the log event in the JSON snippet above). With this configuration, we have made sure that the log driver can parse the logs correctly and the log events will be sent to CloudWatch without getting mangled.

Now, what can we do with these structured logs? Let's see how we can query them.

## Querying Structured Log Data with CloudWatch Insights

With the logs now formatted in JSON, CloudWatch will make every field from that JSON available in our log queries. This gives us a lot of power to write queries for answering our logging questions.

Let's say the user duke has reported an error while navigating the Todo app's web pages, and we'd like to investigate this further. We now can use the following query to show all `ERROR` log events that were logged during a web session of the user duke:

```
fields @timestamp, level, logger, message
| filter mdc.userId='duke'
| filter level='ERROR'
| sort @timestamp desc
| limit 20
```

Aside from the system field `@timestamp`, we can now also include the fields `level`, `logger`, `message` (without the `@`), and all other fields available in the JSON log events in our queries. The above query returns a structured table with the timestamp, log level, logger name, and message of each log event. The result is filtered by the `level` and `mdc.userId` fields to show us only the log events we are interested in at the moment.

The `mdc.userId` field also allows us to get a sense of how many active users the application has every day, which is often used as a success metric for an application:

```
stats count_distinct(mdc.userId) as distinct_users by bin(24h)
```

This query uses the `stats` operator to count the number of different users in 24-hour buckets. We'll get a table with a date and a `distinct_users` column telling us how many users were active on that date.

We can drill in a bit further to find out how many of those users were affected by errors:

```
filter level='ERROR'
| stats count_distinct(mdc.userId) as distinct_users by bin(24h)
```

Or, to get a general sense of the health of our logs, we can just count the number of log events for each log level:



```
stats count(*) by level
```

This query will show a table with the number of log events in each log level. Note that every time we run a query, we select a time interval to run the query on in the CloudWatch Insights UI, so we can decide for which time interval we want to see the log level count.

We can also filter by the `logger` field now, to investigate the errors logged by a certain component:

```
fields @timestamp, level, message
| filter level='ERROR'
| filter logger = 'dev.stratospheric.todoapp.todo.TODOController'
| sort @timestamp desc
| limit 20
```

Without the structured `logger` field, we'd have to filter the `@message` field for the logger name with the `like` operator. That would be unwieldy to write and less performant to execute.

These are just a few examples of what we can do with structured logs. If we add more fields to our JSON log events, like an entry point, a root cause in case of an exception, or any field that is meaningful in the context of the application, the logs become a highly valuable resource not only for investigating errors but also for getting usage and business metrics from our application.

We can even build dashboards from log query results, which we're going to look at that in the next chapter.

## 16. Metrics with Amazon CloudWatch

As of now, our Spring Boot application is running in production and it's sending logs to CloudWatch. The only way to inspect how our application performs is by frequently checking the logs. We're still more or less operating an opaque box. We can't accurately determine, for example, if our ECS Fargate resource settings (CPU and memory) fit our workloads.

Furthermore, by inspecting our Todo application logs, we understand what's going on only when actively looking at them. While it's important to look at our application's log regularly, we can't do this 24/7 – that would be boring and expensive. Similarly, we want to be actively informed when something goes south. Realizing that we had a system outage by reading user complaints isn't exactly desirable.

That's why we need a constant understanding of our operation through quantitative measures. We need metrics for both our application and also for the AWS services we're integrating. We'll want to admire visual metrics on an eye-pleasing dashboard, appreciate that machines are doing the work for us, and create alarms on key metrics to detect defects early and proactively.

With the help of metrics, we can also make business decisions and drive the evolution of our application. We can measure adoption rates and feature usages for this purpose. Another benefit is the additional support when investigating bugs or slow responses in the past. Having metrics for all key parts of our system in place, we can understand the utilization for any point in time in the past to identify the possible root cause.

With Amazon CloudWatch, AWS offers a full-blown monitoring and alerting

solution. As part of this chapter, we'll introduce the metrics-related part of Amazon CloudWatch. We're also going to send custom metrics from our Spring Boot backend and create a custom dashboard for our application. We'll discuss the alerting part in the chapter [Alerting with Amazon CloudWatch](#).

Let's get started with metrics.

## Introduction to Monitoring Metrics with Amazon CloudWatch

Amazon CloudWatch is the central monitoring and observability service of AWS. Apart from collecting and querying logs, CloudWatch is also a metrics registry. We can store, aggregate, and visualize metrics from AWS services like SQS or RDS as well as those from our own applications.

Similar to other AWS services, Amazon CloudWatch is a region-based service. We publish our metrics to a specific region, like `eu-central-1`. To avoid switching the AWS region when analyzing or debugging an outage for a service that spans over multiple AWS regions and availability zones, CloudWatch allows analyzing metrics [cross-region and even cross-account](#).

Amazon CloudWatch uses the concept of a *namespace* to group and isolate metrics from the same source or service. Each data point we send for our metrics needs to have a namespace. There is no default namespace available.

AWS uses the namespace `AWS/<SERVICE_NAME>` for its services (e.g., `AWS/Cognito`, `AWS/RDS`, or `AWS/SQS`). Most of the [AWS services already publish](#) metrics to CloudWatch out-of-the-box.

For a custom namespace, we have to choose a string with only valid XML characters and a maximum length of 256 characters. We'll consider this when

defining a namespace for our application later on.

A metric in the context of Amazon CloudWatch is a time-ordered set of data points. Each data point that we publish to Amazon CloudWatch requires a timestamp. If we don't associate time information with a metric, Amazon CloudWatch will use the time it received the data point. We can provide a timestamp up to two weeks in the past and two hours in the future.

To further categorize our metrics, we can add up to ten dimensions to each data point. These dimensions are key/value pairs and allow us to tag metrics. This way, we can create multiple variations of the same metric to count both the successful and failed user registrations, for example.

The `NumberOfMessagesSent` metric from the `AWS/SQS` namespace, for example, includes the dimension `QueueName` to identify to which Amazon SQS queue this metric belongs.

We'll add dimensions to our custom metrics to identify the application environment (staging or production) and (optionally) the outcome of an operation.

Amazon CloudWatch differentiates two metric resolutions:

- standard resolution for a one-minute granularity, and
- high resolution with granularity in seconds.

The standard resolution is the default one, and all AWS services produce metrics with a standard resolution by default. For our custom metrics, we define the resolution by the time frame we publish the metrics in from our application. We have to keep in mind that we're billed for each `PutMetricData` API call when using high-resolution metrics.

We can't actively delete metrics as they will automatically expire after 15 months if no new data points are published. Metrics where we continuously

publish data points, expire (i.e., are overridden and no longer available) after a pre-defined [retention period](#) depending on their resolution:

- Data points with a granularity of less than 60 seconds are available for 3 hours (high-resolution metrics)
- Data points with a granularity of 60 seconds (1 minute) are available for 15 days
- Data points with a granularity of 300 seconds (5 minutes) are available for 63 days
- Data points with a granularity of 3600 seconds (1 hour) are available for 455 days (15 months)

When it comes to [pricing](#), the number of metrics and the number of API requests (e.g., `PutMetricData` to send metrics) are the main cost drivers. On top of this, each custom dashboard that we create adds \$3 to our monthly bill.

Amazon CloudWatch treats each unique combination of dimensions as a separate metric. Furthermore, charges for custom metrics are prorated by the hour and metered only when we send metrics to CloudWatch. That's important to keep in mind when publishing custom metrics from our application.

For detailed pricing information with sample calculations, consider the [CloudWatch pricing calculator](#).

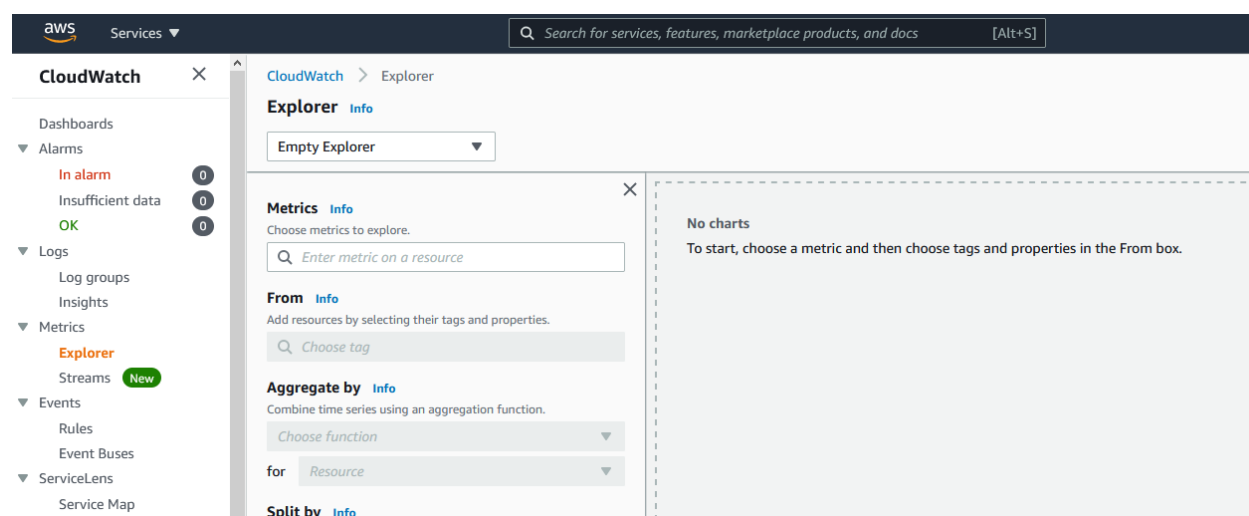
## **Sending Metrics from AWS Services**

[Almost all AWS services](#) already publish metrics to Amazon CloudWatch out-of-the-box. These default metrics help to monitor the underlying infrastructure of our application. This way, we for example get insights into the CPU utilization

of our ECS Fargate task or how many messages are inside our Amazon SQS dead-letter queue.

Depending on the AWS services, we can also turn on/off advanced monitoring. Amazon RDS, for example, provides an [Enhanced Monitoring](#) that provides some extra metrics.

With the help of the Amazon CloudWatch metrics explorer, we can investigate all available metrics for the different AWS services:



Exploring metrics with Amazon CloudWatch.

We'll take a brief look at important metrics for each AWS service we've already integrated with the following sections. Please note that the following list is not complete. However, there are [more metrics for each AWS service](#) available.

## Amazon ECS

The Amazon ECS Fargate cluster bears the brunt of our workload. Monitoring the resource usage of our containers is essential to detect over- and under-utilization of CPU and RAM.

If our application isn't using most of the resources for a continuous period,

we can tweak our Fargate settings to reduce our monthly AWS bill. If we see a constant 80–100% CPU utilization, we can resolve bottlenecks by adding more resources.

**Key metrics to monitor:**

- `CPUUtilization`: The percentage of CPU units used by the cluster or a service in the cluster.
- `MemoryUtilization`: The percentage of main memory (RAM) used by the cluster or a service in the cluster.

**AWS ELB**

AWS Elastic Load Balancing is the main entry point from the internet to our application. When debugging a scenario where requests don't arrive at our backend, it's important to look at the ELB metrics.

AWS differentiates between ELB and target response codes. The ELB response code is the response code that the ELB returned to the client and the target response code is the response code that the underlying target (in our case our Spring Boot application) returned to the ELB. In case of a misconfiguration of the load balancer, the incoming requests might not arrive at our application, and hence we won't see any failure inside our application logs.

Furthermore, with the ELB metrics, we can get a first overview of the number of requests and the average response times for a given period.

**Key metrics to monitor:**

- `HTTPCode_ELB_2XX_Count` (and 3XX, 4XX, 5XX): The number of HTTP response codes returned by the load balancer.

- `TargetResponseTime`: The time elapsed, in seconds, after the request leaves the load balancer until a response from the target is received.
- `RequestCount`: The number of requests processed over IPv4 and IPv6.

## Amazon Cognito

With Amazon Cognito being our identity provider, we can track and monitor the sign-ins and sign-ups per UserPool client.

### Key metrics to monitor:

- `SignUpSuccess`: The number of successful user registration requests for a particular user pool.
- `SignInSuccess`: The number of successful user authentication requests made to a particular user pool.

## Amazon SQS

For our todo sharing feature, we're using Amazon SQS to decouple the sending of emails from the rest of our application. Amazon SQS provides several metrics for inspecting the approximate queue size and the age of the oldest message. This helps with understanding if there's an issue with our processing.

Furthermore, it's important to keep track of the number of messages inside our dead-letter queue to identify failed message consumption.

### Key metrics to monitor:

- `NumberOfMessagesReceived`: The number of messages returned by calls to the `ReceiveMessage` action.



- `NumberOfMessagesDeleted`: The number of messages deleted from a queue.
- `ApproximateAgeOfOldestMessage`: The approximate age of the oldest non-deleted message in a queue.
- `ApproximateNumberOfMessagesVisible`: The number of messages available for retrieval from a queue.

## Amazon RDS

Whenever we detect performance issues for our application, the database is usually a good place to start the investigation. Inefficient SQL queries can have a considerable impact on the overall performance. Especially when working with ORM frameworks like Hibernate, the [N+1 select problem](#) can drain the performance significantly.

Monitoring the database helps to either disqualify or identify the database as the possible performance bottleneck.

### Key metrics to monitor:

- `CPUUtilization`: The percentage of CPU utilization.
- `ReadIOPS`: The average number of disk read I/O operations per second.
- `WriteIOPS`: The average number of disk write I/O operations per second.
- `DatabaseConnections`: The number of database connections currently in use.
- `FreeStorageSpace`: The amount of available storage space on disk.

## Amazon DynamoDB

With Amazon DynamoDB, we have to keep track of how we utilize the configured read and write capacity. We need to spring into action if we see constant

throttle events because we exceed our configured capacity.

**Key metrics to monitor:**

- `ReadThrottleEvents`: The number of requests that exceed the provisioned read capacity units for a table or a global secondary index.
- `WriteThrottleEvents`: The number of requests that exceed the provisioned write capacity units for a table or a global secondary index.

**Amazon SES**

Deliverability and bounce rates are key metrics to monitor when sending emails. Just because our application successfully sent an email to a user doesn't mean the message arrived in their inbox.

**Key metrics to monitor:**

- `Delivery`: The number of successfully delivered emails to the recipient's email server.
- `Reputation.BounceRate`: The bounce rate for our account. This includes both hard (the email address doesn't exist), and soft (the recipient's address is temporarily unable to receive messages) bounces.
- `Send`: The number of send email attempts from our account.

**Amazon MQ**

The Amazon MQ instance acts as the WebSocket broker for our real-time notification feature. It's vital to monitor the throughput and resource utilization of the broker instance.

**Key metrics to monitor:**

- `CpuUtilization`: The percentage of allocated Amazon EC2 compute units that the broker currently uses.
- `MessageCount`: The total number of ready and unacknowledged messages in the queues.
- `TotalMessageCount`: The number of messages stored on the broker.
- `TotalConsumerCount`: The number of message consumers subscribed to destinations on the current broker.
- `CurrentConnectionsCount`: The current number of active connections on the current broker.

## Amazon S3

Even though we don't use Amazon S3 (bucket store) for our Todo application, it's a commonly used AWS service. It's entirely conceivable to enhance our Todo application by managing file attachments for the todos and storing them within Amazon S3, too.

### Key metrics to monitor:

- `BucketSizeBytes`: The amount of data in bytes stored in a bucket.
- `NumberOfObjects`: The number of objects stored in a bucket for all storage classes

## AWS Lambda

For our deployment sequencing, we're using AWS Lambda. As this is a crucial part of our CI/CD pipeline, we should keep a close look at the outcome of the Lambda invocations. A Lambda might fail due to an error or a timeout, and as a result, we won't be able to deploy to production.

### Key metrics to monitor:

- **Invocations:** The number of total invocations for a function.
- **Duration:** The amount of time in milliseconds for an invocation.
- **Errors:** The number of invocations that result in an error.

## Sending Metrics from Our Spring Boot Application

Now it's time to configure our sample Todo application to send custom metrics to Amazon CloudWatch.

### Configuration

Spring Boot provides a single module to enable [production-ready features](#) for our backend: `spring-boot-actuator`. It only takes adding the Spring Boot Starter as an additional dependency, and our application is equipped with auditing, tracing, and monitoring capabilities.

A detailed introduction to all features of the Spring Boot Actuator is out of scope for this book. We're going to focus on the metrics-related parts.

With Spring Boot's auto-configuration mechanism and with having Actuator on the classpath, our application starts measuring key components like:

- core metrics (CPU, uptime, JVM information, etc.),
- Spring MVC metrics (incoming requests per controller),
- DataSource metrics (information about the JDBC connection pool, etc.),
- HTTP Client metrics (monitoring outgoing HTTP requests made with the `RestTemplate` or `WebClient`),
- and [much more](#).

Behind the scenes, the Actuator module uses [Micrometer](#) as an additional layer of abstraction.

Micrometer is described best by the following quote from its official documentation:

“Micrometer provides a simple facade over the instrumentation clients for the most popular monitoring systems, allowing you to instrument your JVM-based application code without vendor lock-in. Think SLF4J, but for application metrics! Application metrics recorded by Micrometer are intended to be used to observe, alert, and react to the current/recent operational state of your environment.”

With this application metric facade, we can define vendor-independent metrics for our application. This allows us to follow a well-defined standard for defining metrics instead of following a specific vendor's metric API and conventions. On the application side, the metrics are stored in a `MetricRegistry` and usually sent to a monitoring solution (like Amazon CloudWatch).

The conversion from the standardized metric format to a proprietary vendor format happens inside a concrete implementation of the `MetricRegistry`. There's a Micrometer `MetricRegistry` implementation for most of the well-known monitoring systems available. What's left for us to do is to configure it.

The Spring Cloud AWS project offers a module for a seamless Actuator and Amazon CloudWatch integration which is aligned with our application's AWS configuration.

All we need are the following two dependencies:

```
dependencies {
 implementation 'org.springframework.boot:spring-boot-starter-actuator'
 implementation 'io.micrometer:micrometer-registry-cloudwatch2'
}
```

Together with Spring Boot's auto-configuration, Spring Cloud AWS will configure the Amazon CloudWatch Java SDK Client for us. This configuration includes the global AWS credentials settings. Furthermore, it initializes a Spring Bean of type `CloudWatchMeterRegistry` that Micrometer uses internally to publish the metrics frequently.

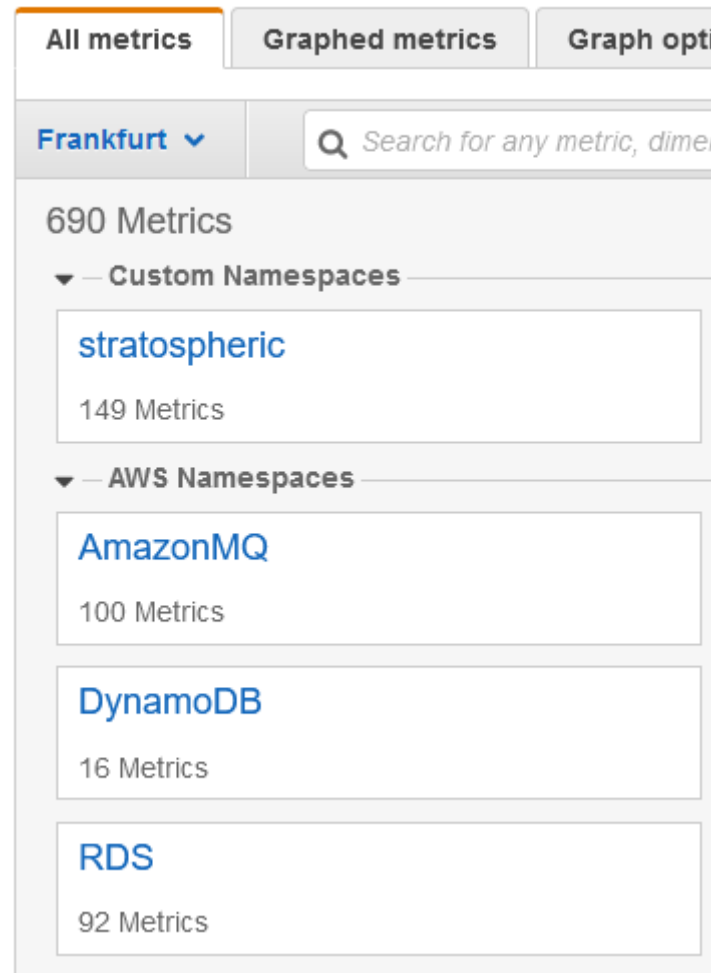
To avoid sending metrics to AWS CloudWatch for both our integration tests and when running locally, we disable the export in our `application-dev.yml` configuration file:

```
management:
 metrics:
 export:
 cloudwatch:
 enabled: false
```

For our default application profile (`application.yml`) we activate and configure the metrics export:

```
management:
 metrics:
 export:
 cloudwatch:
 enabled: true
 namespace: stratospheric
 step: 1m
 tags:
 environment: ${ENVIRONMENT_NAME}
```

The namespace configuration is required as with this identifier we'll group the metrics inside Amazon CloudWatch. As soon as we publish the first metric from our application, we'll see this namespace inside the CloudWatch console as part of the "Custom Namespaces" section:



Custom namespace inside the Amazon CloudWatch console.

Next, we configure the metrics publishing frequency using the `step` attribute. With `1m` we define a `Java Duration` that exports the metrics every minute. Please recall that this impacts how Amazon CloudWatch defines the resolution type of our metrics (standard vs. high-resolution).

The metrics are first buffered inside our application and then sent in batches to Amazon CloudWatch every minute. In case we need more real-time insights, we can reduce that duration. When choosing a sub-minute step, we're creating high-resolution metrics and should keep in mind the implications of this regarding additional costs and the reduced data retention of three hours.



With the `tags` attribute, we can define a set of metadata for all our custom metrics. These tags are then converted to Amazon CloudWatch dimensions. When adding tags to our data sets, we must keep in mind that Amazon CloudWatch comes with a limit of 10 dimensions per data set.

As an alternative to tagging the metrics with the environment name, we can also create a separate namespace for each environment.

There's one final configuration missing. To send metrics from our backend we have to add a new `PolicyStatement` to our ECS task role and allow the Amazon CloudWatch `PutMetricData` action.

For our setup, that means a minor change to the `Service` CDK construct in our `ServiceApp`:

```
.withTaskRolePolicyStatements(
 List.of(
 // ... existings statements
 PolicyStatement.Builder.create()
 .effect(Effect.ALLOW)
 .resources(singletonList("*"))
 .actions(singletonList("cloudwatch:PutMetricData"))
 .build()
)
)
```

CloudWatch [does not have any resource-level permissions](#) and hence we can safely use the wildcard `*` as the resources reference.

With this configuration in place, our application already publishes the default Actuator metrics to Amazon CloudWatch every minute:

All metrics	Graphed metrics	Graph options	Source
Frankfurt ▾	All > stratospheric-staging	Search for any metric, dimension or resource id	
172 Metrics			
exception, method, outcome, status, uri 72 Metrics	action, cause 12 Metrics	area, id 24 Metrics	
id 6 Metrics	level 5 Metrics	name 4 Metrics	
pool 19 Metrics	state 6 Metrics	Metrics with no dimensions 24 Metrics	

Actuator default metrics inside Amazon CloudWatch.

In the following section, we will create custom metrics to track specific parts of our Todo application.

## Emitting Custom Metrics from the Todo Application

Micrometer defines three metric types that we demonstrate as part of this chapter: Counter, Timer, and Gauge.

We interact with Micrometer to create custom metrics using the abstract `MeterRegistry` class. We can inject the `MeterRegistry` (with a runtime type of `CloudWatchMeterRegistry`) into any of our Spring components:

```
@Service
public class CognitoRegistrationService implements RegistrationService {

 // other fields omitted

 private final MeterRegistry meterRegistry;

 public CognitoRegistrationService(MeterRegistry meterRegistry) {
 this.meterRegistry = meterRegistry;
 }
}
```

With access to the `MeterRegistry`, we now want to count the number of user signups for our Todo application. We can achieve this by creating an instance of

Micrometer's Counter class and incrementing it each time a new user registers:

```
@Override
public void registerUser(Registration registration) {

 // prepare the payload to create a new user

 awsCognitoIdentityProvider.adminCreateUser(registrationRequest);

 Counter successCounter = Counter.builder("stratospheric.registration.users")
 .description("Number of user registrations")
 .tag("outcome", "success")
 .register(meterRegistry);

 successCounter.increment();
}
```

Using Micrometer tags, which are converted to Amazon CloudWatch dimensions, we can add information about the registration outcome. If the registration fails, we can add failure to the outcome tag to identify potential issues with our Cognito user signup.

We can also inline the metric creation, so we don't have to use the builder:

```
meterRegistry.counter(
 "stratospheric.registration.users",
 Tags.of("outcome", "success"))
.increment();
```

Apart from this imperative approach, we can also use Micrometer annotations to define custom metrics. This declarative approach makes it easy to, measure the invocation time of our methods using `@Timed`, for example:

```
@Timed(
 value = "stratospheric.collaboration.sharing",
 description = "Measure the time how long it takes to share a todo"
)
@PostMapping("/{todoId}/collaborations/{collaboratorId}")
public String shareTodoWithCollaborator() {
 // ...
}
```

When using `@Timed`, Micrometer automatically publishes four metrics instead of one. It already aggregates the values and also counts the invocations of this controller endpoint. That's why we'll see four different metrics inside Amazon CloudWatch:

```
stratospheric.collaboration.sharing.sum
stratospheric.collaboration.sharing.count
stratospheric.collaboration.sharing.avg
stratospheric.collaboration.sharing.max
```

In this particular example, we also get the timing and counter information for all our controller endpoints from the default `http.server.requests` metric.

For measuring invocations where timing information isn't needed, we can use the `@Counter` annotation instead.

Another metric type we can create with Micrometer is the so-called [gauge](#). With gauges, we monitor the current value of a component. In contrast to a simple Counter, which monotonically increases (like the number of invocations of a method), a gauge represents the current value of something that can increase and decrease over time.

Ideally, we should only use gauges for components with a natural upper bound like thread pools or the volume of a gas tank. Hence, tracking the number of incoming controller requests is not a good use case for a Gauge as there is no natural upper bound.

While we couldn't think of a meaningful metric to demonstrate a gauge for the Todo application, creating and tracking it is similar to what we've seen before with timers and counters:

```
Gauge gauge = Gauge.builder(
 "stratospheric.car.tank",
 () -> myCar.getTank().getFillingRatio()
).description("Measuring the current size of a car tank")
.register(meterRegistry);

// measuring happens when actively observing the current value
gauge.measure();
```

The value of a gauge is measured when actively sampling the current value. The Micrometer documentation provides further insights [on when and how to use gauges](#) for monitoring purposes.

With these three metric types – Counter, Timer, and Gauge – we can measure any custom metric for our Spring Boot application.

For us, there's nothing left to do, as Micrometer ensures to regularly send batches of the different data points to Amazon CloudWatch in the background. Depending on our Micrometer step configuration, the metrics will appear either in seconds or in a matter of minutes inside Amazon CloudWatch.

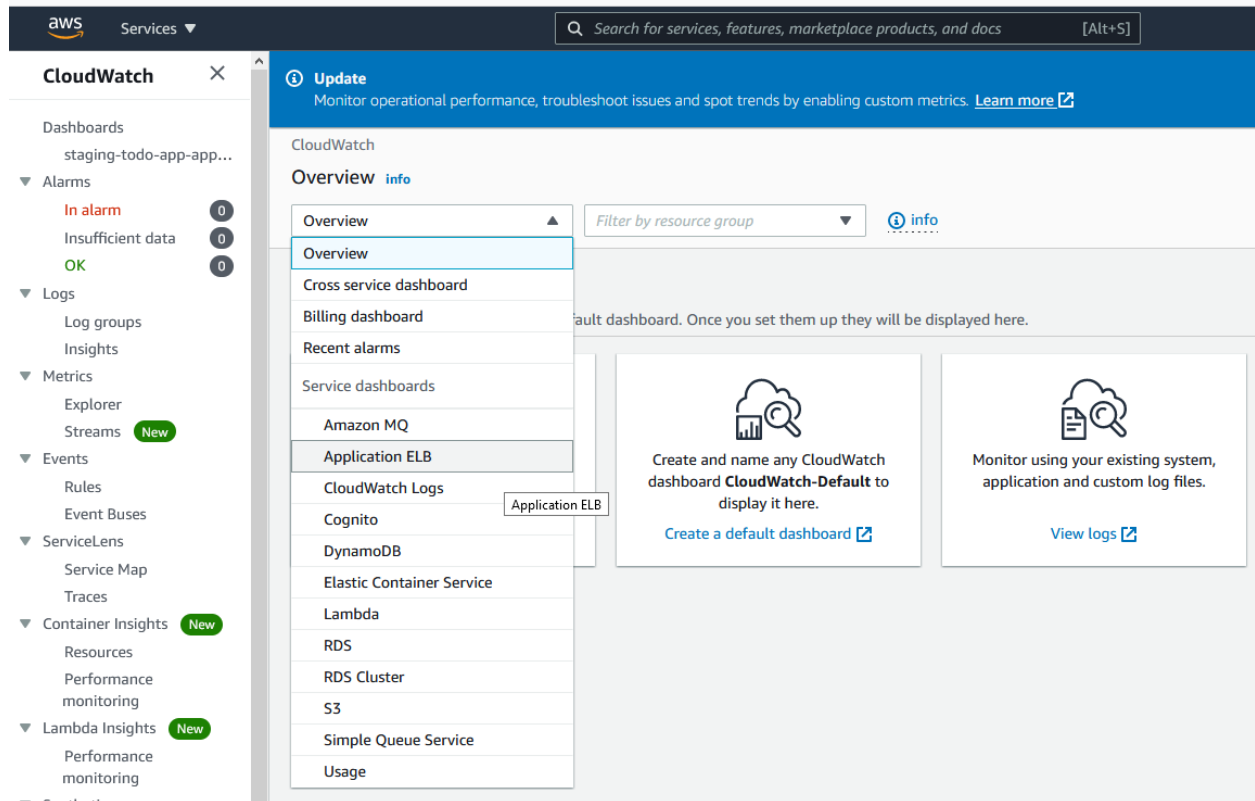
Having the metrics sent to Amazon CloudWatch, it's time to visualize them.

## Monitoring Metrics with Amazon CloudWatch

With our metrics now available inside AWS, we can now build eye-pleasing dashboards to visualize them and get insights into our operation.

## Creating Dashboards with Amazon CloudWatch

Each AWS service that publishes metrics to Amazon CloudWatch comes with a pre-defined dashboard that we can access as part of the Amazon CloudWatch console:



Access to the Service dashboards in Amazon CloudWatch.

These default service dashboards provide an overview to inspect the service's operations.

If we want to combine several AWS service metrics and include our custom metrics, we have to create our own dashboard(s).

For demonstration purposes, let's create a minimal dashboard that visualizes some metrics and grants us further insights into our application.

While Amazon CloudWatch provides a visual drag-and-drop dashboard builder,

we're favoring the infrastructure-as-code approach and will jump right into using the AWS CDK to create this dashboard.

The visual dashboard builder is an excellent place to start and explore the features of the dashboards we want to build, however. We can create the first draft of our dashboard by moving tiles inside the visual dashboard builder and then have the export functionality create a blueprint for our AWS CDK setup.

Let's get started by creating a new CDK app that encapsulates all monitoring-related infrastructure.

As part of the `MonitoringApp`, we're instantiating and synthesizing the Monitoring stack:

```
public class MonitoringApp {

 public static void main(final String[] args) {
 App app = new App();

 // context variables and environment object creation

 new MonitoringStack(app,
 "monitoring",
 awsEnvironment,
 applicationEnvironment);

 app.synth();
 }
}
```

Similar to all other CDK apps we've created so far, we're adding both `monitoring:deploy` and `monitoring:destroy` to our `package.json` for a convenient sync mechanism as part of our CI/CD pipeline.

The [Amazon CloudWatch CDK project](#) provides stable CDK level 2 constructs for creating dashboards. The CloudWatch CDK constructs are part of `aws-cdk-lib`

that we only have to add as a new dependency to the [pom.xml](#) of our cdk project if we're not already using `cdk-constructs`:

```
<!-- Both imports can be omitted when using cdk-constructs -->
<dependency>
 <groupId>software.amazon.awscdk</groupId>
 <artifactId>aws-cdk-lib</artifactId>
 <version>${aws-cdk-lib.version}</version>
</dependency>
<dependency>
 <groupId>software.constructs</groupId>
 <artifactId>constructs</artifactId>
 <version>${constructs.version}</version>
</dependency>
```

The CloudWatch CDK project also provides constructs for creating alarms, but for now, we'll be focusing on the Dashboard construct. We'll look at creating alarms in the chapter [Alerting with Amazon CloudWatch](#).

For the upcoming example, we'll be defining the sample dashboard in a custom CDK construct called [SampleCloudWatchDashboard](#). As we'll be focusing on the main building blocks for creating a CloudWatch dashboard with the AWS CDK, creating a full-blown operational dashboard is out of scope for this chapter. We've added a blueprint for a ready-to-use dashboard targeting Java / Spring Boot applications to the source code of the [OperationalCloudWatchDashboard](#) construct.

The Dashboard class is the main entry point for creating a dashboard with the CDK. As part of creating a new dashboard, we define both the layout and metadata using `DashboardProps`. The dashboard's metadata includes the name of the dashboard. We can optionally specify the default start and end time of the dashboard:



```
new Dashboard(this, "sampleApplicationDashboard",
 DashboardProps.builder()
 .dashboardName(applicationEnvironment.getApplicationName()
 + "-sample-application-dashboard")
 .start("-3H") // start three hours ago
 // ...
 .build());
```

We structure the dashboard's layout and content using the `widgets()` method of the `DashboardProps` builder. This method expects a nested list (`List<List<? extends IWidget>`) where the first dimension represents the dashboard row. The second dimension represents the column inside the row. Inside each column, we place a single or multiple widgets.



For more advanced and complex dashboard layouts with AWS CDK, please refer to the official [Amazon CloudWatch CDK documentation](#).

The Amazon CloudWatch CDK project defines the following widget types:

- `TextWidget`
- `SingleValueWidget`
- `GraphWidget`
- `LogQueryWidget`
- `AlarmWidget`
- `AlarmStatusWidget`

For demonstration purposes, let's fill the first row of our custom dashboard with the first four widget types. The `AlarmWidget` and the `AlarmStatusWidget` widgets are discussed in the chapter [Alerting with Amazon CloudWatch](#).

We're going to start with the `TextWidget`. With this widget, we can display arbitrary text-based information using Markdown syntax:

```
new Dashboard(this, "applicationDashboard", DashboardProps.builder()
 .dashboardName(applicationEnvironment + "-application-dashboard")
 .widgets(List.of(
 List.of(
 TextWidget.Builder
 .create()
 .markdown(
 "# Stratospheric Dashboard \n Created with AWS CDK." +
 " \n * IaC \n * Configurable \n * Nice-looking")
 .height(6)
 .width(6)
 .build()
)
)
 .build());
```

In our example, we display a trivial welcome message. We can use such text widgets to link to our runbooks, documentation, or any other static information that might help during an incident.

By default, each widget type comes with a pre-defined height and width. We can override both values to manually define the widget's size or to align the width of all widgets for instance. When manually specifying the width of our widgets, we must keep in mind that each row has a total width of 24.

Inside the next dashboard column, we'll display a `SingleValueWidget` to keep track of how many users registered for our application in any given period of time:

```
SingleValueWidget.Builder
 .create()
 .title("User Registrations")
 .setPeriodToTimeRange(true)
 .metrics(List.of(new Metric(MetricProps.builder()
 .namespace("stratospheric")
 .metricName("stratospheric.registration.signups.count")
 .region(awsEnvironment.getRegion())
 .statistic("sum")
 .dimensionsMap(Map.of(
 "outcome", "success",
 "environment", applicationEnvironment.getEnvironmentName()
)))
 .build()))
 .build()
```

By default, Amazon CloudWatch displays our metrics with a pre-defined period. This period defines the width of the time range of each data point on a graph. Let's say we visualize the user registration for the last 24 hours with a period of 1 hour. We would get a chart with 24 data points that show the aggregated sum of our metric every hour.

For our `SingleValueWidget`, we don't want this behavior. We want to display a single value representing the number of registrations of the last 24 hours. To avoid splitting the metric visualization into multiple periods, we set `periodToTimeRange` to `true`, and therefore the period (i.e., the width of our data points) equals the whole time range.

The metric definition happens inside the `metrics` method of the widget builder. This method expects a list of `Metric` instances. For this example, we're referring to one of our custom metrics and display the sum of user signups. It's important to add the correct environment dimension for the metric definition, as we're adding the environment tag to each custom metric.

When visualizing custom metrics from our application, we must keep in mind that we're running multiple application instances. All of our Spring Boot back-

end instances publish metrics to Amazon CloudWatch.

That doesn't have any repercussions when displaying a sum, but we might not detect outliers as soon as we're visualizing an average (avg). Imagine displaying the JVM memory consumption with a single avg value for all instances. This can lead to wrong interpretations as we won't detect if one instance is under heavy pressure while the other machines are idling.

In such cases, grouping the metric per instance can help.

The most complex widget type is the `GraphWidget`. As the name applies, we can visualize our metrics using bar, pie, or line charts. For demonstration purposes, we'll visualize two Amazon Cognito metrics:

```
GraphWidget.Builder.create()
 .title("User Sign In")
 .view(GraphWidgetView.BAR)
 .left(List.of(new Metric(MetricProps.builder()
 .namespace("AWS/Cognito")
 .metricName("SignInSuccesses")
 .region(awsEnvironment.getRegion())
 .period(Duration.minutes(15))
 .dimensionsMap(Map.of(
 "UserPoolClient", cognitoOutputParameters.getUserPoolClientId(),
 "UserPool", cognitoOutputParameters.getUserPoolId()))
 .statistic("sum")
 .build()))))
 .right(List.of(new Metric(MetricProps.builder()
 .namespace("AWS/Cognito")
 .metricName("TokenRefreshSuccesses")
 .region(awsEnvironment.getRegion())
 .period(Duration.minutes(15))
 .dimensionsMap(Map.of(
 "UserPoolClient", cognitoOutputParameters.getUserPoolClientId(),
 "UserPool", cognitoOutputParameters.getUserPoolId()))
 .statistic("sum")
 .build()))))
 .build()
```

To refer to the correct `UserPoolClient` and `UserPool`, we use the output pa-

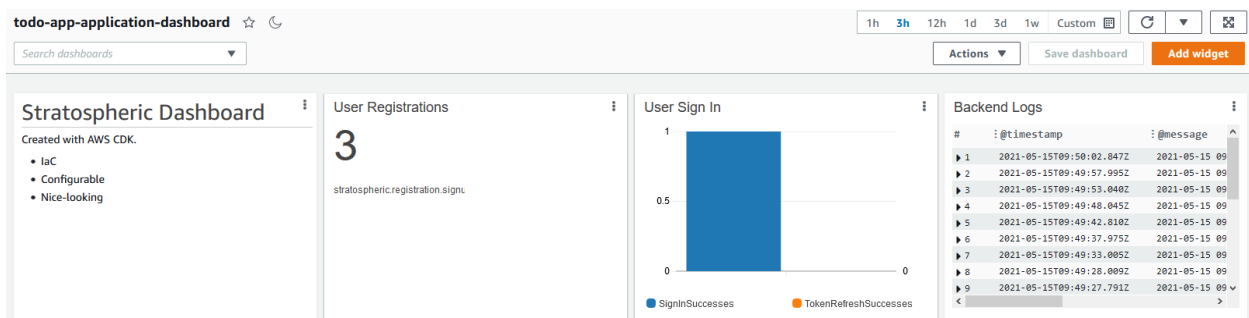
rameters of our AWS CDK CognitoStack. With `left` and `right` we define the metrics that are visualized at the left and right side of the Y-axis. The result will be a bar chart that displays the sum of user sign-ins and token refreshes that happened in a 15-minute time window.

Last but not least, we'll display the latest logs of our Spring Boot application using the `LogQueryWidget`:

```
LogQueryWidget.Builder
 .create()
 .view(LogQueryVisualizationType.TABLE)
 .title("Backend Logs")
 .logGroupNames(List.of(applicationEnvironment + "-logs"))
 .queryString(
 "fields @timestamp, @message" +
 "| sort @timestamp desc" +
 "| limit 20")
 .build()
```

When constructing this widget, we have to specify the query string and the list of log groups we want to query. The example above displays the last 20 loglines of our backend application.

Bringing it all together, our custom dashboard looks like the following:



A custom Amazon CloudWatch dashboard.

These four widgets represent a basic Amazon CloudWatch dashboard. Most of the widgets allow advanced configurations like coloring, axis labeling, custom

periods, etc. The [Amazon CloudWatch CDK documentation](#) provides further examples of how to use the different widgets.

Equipped with these four widget types, we can now build nifty dashboards for various purposes, for example to display:

- business metrics (conversion rates, number of orders, etc.),
- high-level system operations (SLO and SLIs), or
- to monitor external systems (answering questions such as: “How do our dependent systems perform?”).

For a ready-to-use operational dashboard blueprint for the Stratospheric sample application, head over to the GitHub repository and have a look at the [OperationalCloudWatchDashboard](#) construct.

The `OperationalCloudWatchDashboard` blueprint dashboard contains graphs for the key metrics of our Spring Boot application. This includes the latest error logs, the distribution of the different log levels, response times from the load balancer, and essential metrics for our RDS database and SQS queue.

## Alternatives to Amazon CloudWatch

When it comes to alternatives for Amazon CloudWatch, we have to differentiate between two use cases: storing metrics and creating dashboards.

There are many tools available that cover both use cases while others focus on only one part. [Prometheus](#), for example, is a time-series database that might be a perfect fit for a self-hosted on-premise solution. While it has a basic built-in metrics visualization, we're usually better off choosing a tool like [Grafana](#) for creating stylish dashboards.

The market for monitoring solutions, especially SaaS offerings, is huge. To name just some of the more well-known vendors, in no particular order or preference:

- Datadog
- New Relic
- Splunk
- Dynatrace
- Prometheus
- Grafana
- AppOptics
- etc.

Due to Amazon CloudWatch's excellent integration with the entire AWS infrastructure, one might start with Amazon's preferable monitoring solution. Nevertheless, having a closer look at the costs is important when using Amazon CloudWatch on a greater scale.

If we decide to change vendors for our monitoring, moving to a different monitoring stack is, most of the time, straightforward.

Thanks to the Micrometer abstraction, we can plug in a different `MeterRegistry` implementation at any time. This way, we start sending the same application metrics to Datadog or Prometheus with only a configuration change.

However, any custom Amazon CloudWatch dashboard has to be migrated to the dashboard syntax of the new provider. That's a one-time migration effort.

When it comes to exporting metrics of specific AWS services like the CPU utilization of our RDS database, we have to export them from AWS. Most of the well-known third-party monitoring solutions provide an adapter for regularly fetching metrics from AWS. These adapters need to be configured only once. It's

worth checking a monitoring provider's documentation in case there's a plan to move away from Amazon CloudWatch.

Grafana, for example, already comes with native [Amazon CloudWatch support](#). In case our provider doesn't integrate with CloudWatch, we can also fallback to [Amazon CloudWatch metric streams](#) to make the data points available in third-party systems.



## 17. Alerting with Amazon CloudWatch

Metrics, dashboards, and insights to the logs of our application make operating our application transparent. At any time, we know what's going on and we can identify bottlenecks of our infrastructure, architecture, or implementation.

Frequently checking our dashboards and filtering for error logs is a routine worth doing. When working in the same office, it's beneficial to display our dashboards on a dedicated monitor. This way, everyone can take a brief look at the health of our system on their way to grab a coffee.

However, we shouldn't rely on identifying outages and incidents while actively inspecting our dashboards and logs. Humans tend to forget things or spend their days in meetings. Furthermore, with the current trend toward remote-first work, we're no longer working together from the same physical location. Everyone is sitting at home and might expect another team member to observe the application's health frequently.

Even worse, what about the operation of our application at night? We don't want to have to look through the logs all night!

We need an automated solution that identifies failures in our systems as soon as possible. This allows us to start fixing an incident before the first user even complains.

Fortunately, an automated solution for this use case exists: alarms.

With alarms, we can define thresholds based on our application or infrastructure metrics to identify a possible failure of our system. Once a metric breaches

the threshold (for example more than 100 5xx HTTP responses in 5 minutes), we'll automatically get notified.

That's the theory. In practice, there are many nuances to this, and getting the correct alarms with meaningful thresholds in place is not trivial.

With this chapter, we'll introduce Amazon CloudWatch's alerting capabilities, see how we can create alarms with the AWS CDK, and discuss best practices for handling failures.

Let's get started.

## Introduction to Alerting with Amazon CloudWatch

With Amazon CloudWatch, we can create two types of alarms:

- **Metric alarms:** We continuously evaluate a single CloudWatch metric or the result of a mathematical expression based on up to ten CloudWatch metrics.
- **Composite alarms:** We consider the state of several alarms and define a logical expression on them (for example `ALARM("alarm-a") AND (ALARM("alarm-b") OR ALARM("alarm-c"))`).

We'll demonstrate both alarm types and create at least one for each type using the AWS CDK in the following sections.

Every alarm is in one of the following three states at any given time:

- **OK:** Yay – Everything's okay. We're below our defined threshold.
- **ALARM:** Ouch – The threshold for our metric or mathematical expression has been breached.

- **INSUFFICIENT\_DATA:** No or not enough data points for the metric are available. This is usually the case when we've just created the alarm, or our metric is not being emitted anymore.

Once an alarm transitions into a new state, we can trigger automatic actions with Amazon CloudWatch. We'll explore the available actions in the section [Adding Alarm Actions](#) below.

At its core, every alarm continuously evaluates an expression (based on metrics, or the state of other alarms) to identify a failure in our system. Four components go into this failure detection:

- **Period:** How often do we evaluate the metric or mathematical expression? When the period is set to five minutes, the alarm evaluates the metric every five minutes.
- **Evaluation Periods:** How many recent periods are we evaluating to determine the state of the alarm? If set to three, for example, the data of three consecutive five-minute periods will be evaluated.
- **Threshold:** Which amount identifies a potential incident? For example, we can set this to 10 if we want to alert on 10 or more 5XX HTTP responses.
- **Data points to alarm:** How many data points within our evaluation periods need to breach the threshold? For example, we could decide to trigger an alarm only if the threshold has been breached five times within the three evaluation periods.

Tweaking these four values is an ongoing process. As a first step, it's important to understand how these parameters influence the evaluation of an alarm. The [Amazon CloudWatch documentation](#) explains how they affect each other with concrete examples and timeline illustrations.

Another essential configuration to make for each alarm is how it handles missing metric data. Should missing data breach our threshold, can we ignore it, or should the alarm transition to the `INSUFFICIENT_DATA` state?

This highly depends on the metric we're evaluating. For example, we might treat missing data as breaching for metrics where we expect continuous data like the CPU utilization metric of our ECS task. On the other hand, we can simply ignore missing data for metrics like the 5xx API count as this is the expected (non-error) state.

The missing data configuration has an impact on the evaluation of our alarm's state. Head over to the official Amazon CloudWatch documentation to learn about the different scenarios for [how missing data is evaluated](#).

Amazon CloudWatch also offers an anomaly detection feature. This allows us to apply statistical and machine learning algorithms on past data points to automatically define an expected range of our metrics. We'll not cover anomaly detection with this chapter, as we're focusing on creating the alarms ourselves. For more information about this promising failure detection feature, consider the [CloudWatch anomaly detection documentation](#).

One last word about the costs of our alarms. We're billed depending on the resolution (standard vs. high resolution) of the alarm metric we're evaluating. Furthermore, each composite alarm and anomaly detection comes with a [fixed price per month](#).

So, let's create our first alarm with the AWS CDK.

## Creating Alarms with the AWS CDK

As our alarms fall into our application's operation and monitoring domain, we'll enrich the existing CDK MonitoringStack and add further constructs to it. You can have a look at that CDK stack [on GitHub](#).

### Creating Metric Alarms

We'll start with a basic alarm that monitors our Todo application for slow responses. Slow responses can have multiple root causes. For example, slow responding remote systems we depend on, general hiccups, or high pressure on the database. For mission-critical APIs that we might even expose to external clients, it's vital to keep track of how fast our backend responds.

The Amazon CloudWatch CDK project offers stable level 2 constructs for conveniently creating alarms. These constructs are part of the `aws-cdk-lib` Java dependency, which we've already included in our CDK project as part of the chapter *Metrics with Amazon CloudWatch*:

```
<!-- Both imports can be omitted when using cdk-constructs -->
<dependency>
 <groupId>software.amazon.awscdk</groupId>
 <artifactId>aws-cdk-lib</artifactId>
 <version>${aws-cdk-lib.version}</version>
</dependency>
<dependency>
 <groupId>software.constructs</groupId>
 <artifactId>constructs</artifactId>
 <version>${constructs.version}</version>
</dependency>
```

For this alarm example, we're using a metric the AWS ELB emits by default. While our Spring Boot backend also emits timing information about our con-

trollers, the response time of the ELB is more relevant as this includes the whole round-trip time for our clients and not just how long controller invocation took.

The `TargetResponseTime` metric that this alarm continuously checks comes from the `AWS/ApplicationELB` namespace. As part of our alarm definition, we have to refer to the correct load balancer precisely. This includes both the AWS region and the load balancer's name. Therefore, we pass both as metric dimensions.

The information about the AWS region is available to all our stacks as part of the `Environment` object we pass to our `MonitoringStack` constructor:

```
public class MonitoringStack extends Stack {

 public MonitoringStack(
 // ...
 final Environment awsEnvironment) {
 // ...
 }
}
```

What's left is to determine the name of the load balancer. Our `Network` stack returns the ARN of the load balancer as one of its output parameters. A load balancer ARN looks something like this:

```
arn:aws:elasticloadbalancing:{REGION}:{ACCOUNT_ID}:
loadbalancer/app/staging-loadbalancer/2280d6069948c49e
```

We have to modify the ARN slightly because the metric expects the name of the load balancer in a specific format:

```
app/staging-loadbalancer/2280d6069948c49e
```

To extract the last part of the load balancer ARN, we're splitting the output parameter of the `Network` construct:

```

Network.NetworkOutputParameters networkOutputParameters =
 Network.getOutputParametersFromParameterStore(this,
 applicationEnvironment.getEnvironmentName());

String loadBalancerName = Fn
 .split(":loadbalancer/", networkOutputParameters.getLoadBalancerArn(), 2)
 .get(1);

```

As we're storing our stack output parameters inside the AWS Parameter Store, those parameters get resolved to a Token at the construction phase of our stack:

```

System.out.println(networkOutputParameters.getLoadBalancerArn());

${Token[TOKEN.60]}

```

This token points to the actual parameter stored in the SSM Parameter Store but gets resolved during the prepare phase. That's why we can't use Java's string manipulation: The value is not yet resolved at the construction phase. Hence, we're using the [built-in CloudFormation function Fn::Split](#) to manipulate the parameter right after it gets resolved.

With the metric dimensions in place, we can create an instance of the Alarm object and configure our alerting rules:

```

Alarm elbSlowResponseTimeAlarm = new Alarm(this, "elbSlowResponseTimeAlarm",
 AlarmProps.builder()
 .alarmName("slow-api-response-alarm")
 .alarmDescription("Indicating potential problems with the Spring Boot Backend")
 .metric(new Metric(MetricProps.builder()
 .namespace("AWS/ApplicationELB")
 .metricName("TargetResponseTime")
 .dimensionsMap(Map.of(
 "LoadBalancer", loadBalancerName
))
 .region(awsEnvironment.getRegion())
 .period(Duration.minutes(5))
 .statistic("avg")
 .build()))

```

```
.treatMissingData(TreatMissingData.NOT_BREACHING)
.comparisonOperator(ComparisonOperator.GREATER_THAN_OR_EQUAL_TO_THRESHOLD)
.evaluationPeriods(3)
.threshold(2)
.actionsEnabled(true)
.build();
```

Besides setting the name of the alarm and adding a descriptive text, we're defining the metric this alarm continuously monitors. Our alarm evaluates the average target response time of our AWS ELB for five-minute time windows.

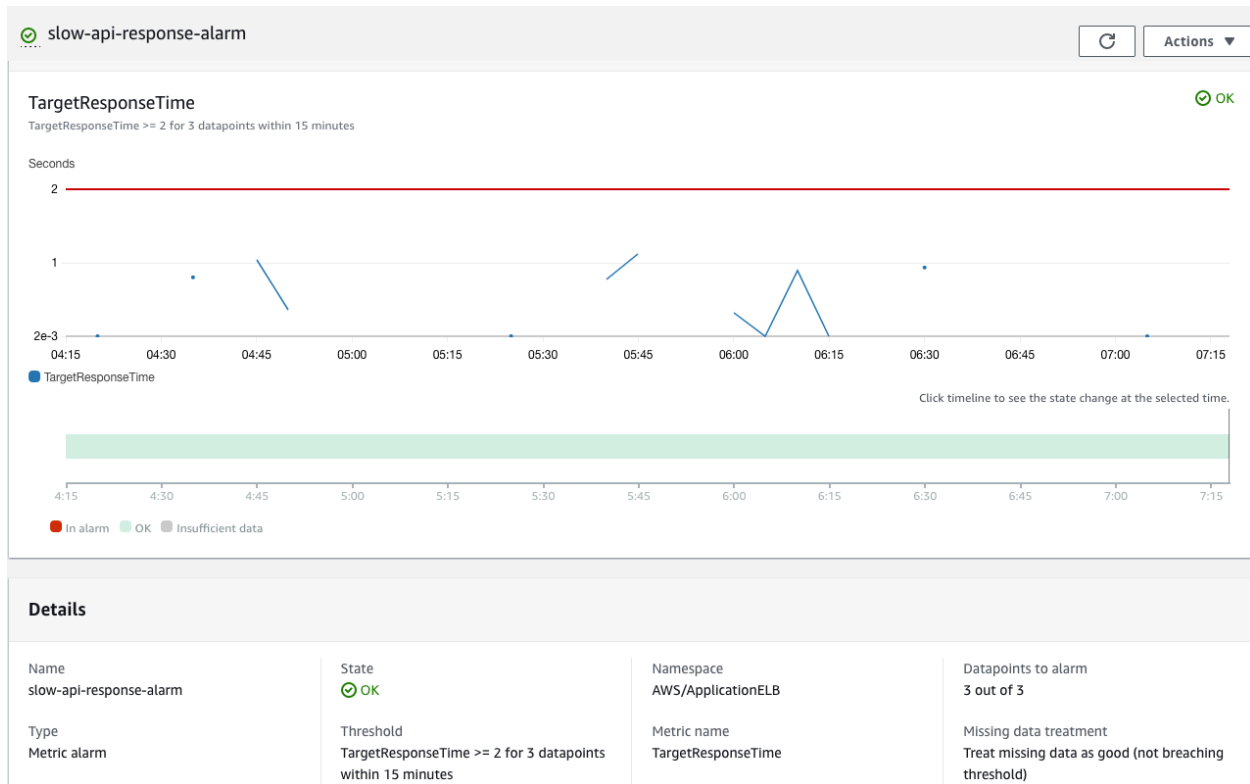
What's next is to define how our alarm treats missing data. For our example, there are no data points for this metric whenever there's no traffic to our application (at night, for example). Hence, missing data is expected and won't breach the threshold.

We define the alerting threshold by a combination of `comparisonOperator`, `evaluationPeriods`, and `threshold`. In our example, the alarm transitions into the ALARM state whenever the target response time is greater than or equal to 2 seconds within fifteen minutes (three 5-minute evaluation periods).

With `actionsEnabled` we enable or disable any attached action that this alarm triggers whenever the alarm's state changes.

Once we now deploy our CDK `MonitoringApp` with `npm monitoring:deploy`, we'll see the following result in the Amazon CloudWatch console inside the Alarm section:





The “Slow API Response” alarm in the Amazon CloudWatch console.

In the context of our Todo application, other alarms worth creating are the following:

- Monitor the number of messages inside our DLQ (dead-letter queue) to identify problems with the processing of collaboration requests.
- Monitor the email delivery rate of SES.
- Monitor the essential resource utilization (CPU, RAM) of our ECS tasks.

As a next step, let’s add an action to our alarm to get actively notified whenever our backend responds slowly.

## Adding Alarm Actions

We can trigger automatic actions whenever our alarm transitions to a new state (OK, ALARM, INSUFFICIENT\_DATA). Amazon CloudWatch currently supports four actions:

- Trigger EC2 actions, for example to reboot or terminate an instance.
- Perform an autoscaling operation, for example to scale up when the resource utilization is high.
- Send a notification to an SNS topic.
- Create an incident or OpsItem inside the [AWS Systems Manager](#).

For our slow response time alarm, we'll send a notification to an SNS topic as soon as the metric breaches our threshold. This way we can further route the alarm to different systems by adding several subscribers to the SNS topic.

SNS offers several subscriber types to notify, route, trigger actions whenever there's a new notification:

- Trigger an AWS Lambda function.
- Send a message to an Amazon SQS queue.
- Send an email.
- Invoke an HTTP/HTTPS endpoint with the notification payload.
- Send an SMS.

We're going to inform the developers of our Todo application by emailing them.

As a first step, we create an Amazon SNS topic that acts as a hub to route the alarm notifications further:

```
Topic snsAlarmingTopic = new Topic(this, "snsAlarmingTopic", TopicProps.builder()
 .topicName(applicationEnvironment + "-alarming-topic")
 .displayName("SNS Topic to further route Amazon CloudWatch Alarms")
 .fifo(false)
 .build());
```

Setting up the SNS topic using the CDK level 2 construct is straightforward. We don't need strict ordering of the alarm notifications as they'll usually arrive infrequently and not concurrently (at least, that's what we hope). Hence, we disable the FIFO option.

Next, we add our subscriptions. For our example, we're using the `EmailSubscription`:

```
snsAlarmingTopic.addSubscription(EmailSubscription.Builder
 .create(confirmationEmail)
 .build()
);
```



We have to [confirm every Amazon SNS subscription](#) before we can receive notifications via this channel. For our example, we confirm the subscription by following a link that AWS sent to `info@stratospheric.dev`.

We can add further subscriptions to our SNS topic and integrate third-party incident management tools by adding an `UrlSubscription`. SNS will [perform an HTTP POST with the message payload](#) and inform other systems about a given alarm:

```
snsAlarmingTopic.addSubscription(UrlSubscription.Builder
 .create("https://my-alarming-tool.com/alarms/incoming")
 .build());
```

With the `UrlSubscription` construct, we can also integrate Slack/Microsoft Teams/Discord to ring a bell whenever there's an alarm state change.

Right after creating the SNS topic, we can integrate it with our Amazon CloudWatch alarm to send notifications whenever we breach our threshold:

```
elbSlowResponseTimeAlarm.addAlarmAction(new SnsAction(snsAlarmingTopic));
```

We can also define separate actions for each alarm state transition and handle them differently:

```
elbSlowResponseTimeAlarm.addInsufficientDataAction(new SnsAction(anotherTopic));
elbSlowResponseTimeAlarm.addOkAction(new AutoScalingAction(...));
```

## Creating Alarms Based on Logs

As a next step, we'll create an alarm based on our Spring Boot application logs. Thanks to the preparation work of the chapter [Structured Logging with Amazon CloudWatch](#), we can filter and count the log output by log level with ease.

We'll put an alarm in place for our upcoming example that continuously monitors the amount of ERROR logs our application produces.

There are numerous reasons why our backend might output ERROR statements. Not all of them are worth waking someone up at night. Imagine a malicious attacker trying to penetrate our APIs. The `StrictHttpFirewall` (part of the Spring Security FilterChain) will produce quite some noise on stdout for each malicious request. As this is expected and as nothing is currently *going wrong* (except being attacked – which might also be worth noting), the developer on-call wouldn't necessarily have to leave their bed.

However, most of the time, we can correlate an increasing number of error logs to a failure in our system. In short, be aware of the potential false positives when relying (solely) on this kind of alarm.

Before we can put a threshold for our error logs in place, we have some ground-work left to do.

As our alarms are based on metrics, we somehow need a metric based on our backend logs. Amazon CloudWatch provides a [metric filter](#) functionality for such use cases. With this filter, we define our log query once (i.e., filter all ERROR logs or count the occurrences of a specific term inside the message). Then, we can create a custom metric (and alarm) on top of it.

The log level is directly accessible from the custom field `level` of our JSON-based logs. Hence, the following pattern filters out all error logs and achieves what we're looking for:

```
{ $.level = "ERROR" }
```

Based on this filter pattern, we can now create the metric filter for our log group. The CloudWatch CDK module provides a level 2 construct for this purpose:

```
MetricFilter errorLogsMetricFilter = new MetricFilter(this, "errorLogsMetricFilter",
 MetricFilterProps.builder()
 .metricName("backend-error-logs")
 .metricNamespace("stratospheric")
 .metricValue("1")
 .defaultValue(0)
 .logGroup(LogGroup
 .fromLogGroupName(this, "applicationLogGroup",
 applicationEnvironment + "-logs"))
 .filterPattern(FilterPattern.stringValue("$.level", "=", "ERROR"))
 .build());
```

While creating the `MetricFilter`, we define the metric's name, the custom namespace where it belongs, and information on how to measure. For our example, the metric value is numeric as we count each error log line.

With `logGroup`, we refer to the Amazon CloudWatch log group we want to apply the filter pattern. The metric filter pattern itself is then defined right before we

build the `MetricFilter` instance.

We can now create the actual metric right out of the `MetricFilter` instance and pass further configuration options (e.g., duration, statistical operator, etc.):

```
MetricFilter errorLogsMetricFilter = // ...

Metric errorLogsMetric = errorLogsMetricFilter.metric(MetricOptions.builder()
 .period(Duration.minutes(5))
 .statistic("sum")
 .region(awsEnvironment.getRegion())
 .build());
```

As a final step, we create an alarm based on this metric. We construct the alarm right from the metric instance (vs. `new Alarm()` used for the previous alarm). This way, we save some keystrokes for the metric definition and only have to configure the threshold and metadata for the alarm:

```
MetricFilter errorLogsMetricFilter = // ...

Metric errorLogsMetric = errorLogsMetricFilter
 .metric();

Alarm errorLogsAlarm = errorLogsMetric
 .createAlarm(this, "errorLogsAlarm", CreateAlarmOptions.builder()
 .alarmName("backend-error-logs-alarm")
 .alarmDescription("Alert on multiple ERROR backend logs")
 .treatMissingData(TreatMissingData.NOT_BREACHING)
 .comparisonOperator(ComparisonOperator.GREATER_THAN_OR_EQUAL_TO_THRESHOLD)
 .evaluationPeriods(3)
 .threshold(5)
 .actionsEnabled(false)
 .build());
```

That's the beauty of CDK in action!

For this example, the alarm will transition to the `ALARM` state once we have more than (or equal to) five error logs for three data points within a fifteen-minute time window.

As already outlined at the beginning of this section, the alarm fatigue for monitoring error logs can be quite high. That's why we disable any actions that this alarm would trigger with `actionsEnabled`.

You might wonder: "What's the benefit of an alarm that doesn't even ring a bell?". We'll use the error log alarm and combine it with other alarm states into a *composite alarm*.

## Creating Composite Alarms

With [composite alarms](#), we can reduce the alarm fatigue as we combine multiple alarm states to identify an incident. We define a logical expression to have a more fine-grained rule when to get thrown out of bed at night:

- We have multiple error logs AND our 5xx responses are high.
- We have multiple error logs AND all remote systems are healthy. Therefore, the error doesn't originate from an external dependency and must lie in our application.
- Our 5xx responses are high AND either our database or message broker reports a high CPU utilization.
- etc.

We don't necessarily have to always refer to the `ALARM` state as part of our logical expression. We can also refer to any other alarm state (`OK` or `INSUFFICIENT_DATA`) of one of the alarms that we're combining. If we want, we can even include another composite alarms.

For demonstration purposes, we'll use our already available Amazon CloudWatch alarms for high error log occurrences that don't trigger any action right now. As an additional outage indicator, we create an alarm for 5xx HTTP

responses from our ELB. The alarm definition for this alarm is similar to our `elbSlowResponseTimeAlarm`. The only differences is that we refer to the `HTTPCode_ELB_5XX_Count` metric and disable any automatic action:

```
Alarm elb5xxAlarm = new Alarm(this, "elb5xxAlarm", AlarmProps.builder()
 .alarmName("5xx-backend-alarm")
 .alarmDescription("Alert on multiple HTTP 5xx ELB responses")
 .metric(new Metric(MetricProps.builder()
 .namespace("AWS/ApplicationELB")
 .metricName("HTTPCode_ELB_5XX_Count")
 .dimensionsMap(Map.of(
 "LoadBalancer", loadBalancerName
))
 .region(awsEnvironment.getRegion())
 .period(Duration.minutes(5))
 .statistic("sum")
 .build()))
 .treatMissingData(TreatMissingData.NOT_BREACHING)
 .comparisonOperator(ComparisonOperator.GREATER_THAN_OR_EQUAL_TO_THRESHOLD)
 .evaluationPeriods(3)
 .threshold(5)
 .actionsEnabled(false)
 .build());
```

Next, we want to combine both alarms into a composite alarm and only get notified when both alarms are in the ALARM state:



```
Alarm elb5xxAlarm = // ...;
Alarm errorLogsAlarm= // ...;

CompositeAlarm compositeAlarm = new CompositeAlarm(this, "basicCompositeAlarm",
 CompositeAlarmProps.builder()
 .actionsEnabled(true)
 .alarmDescription("Showcasing a Composite Alarm")
 .compositeAlarmName("Backend API Failure")
 .alarmRule(AlarmRule.allOf(
 AlarmRule.fromAlarm(elb5xxAlarm, AlarmState.ALARM),
 AlarmRule.fromAlarm(errorLogsAlarm, AlarmState.ALARM)
)
)
 .build());
```

Our AlarmRule creates a logical AND expression for both alarms:

```
(ALARM("5xx-backend-alarm") AND ALARM("backend-error-logs-alarm"))
```

As this combined alarm is now a better indicator for a possible outage, we enable the actions that this alarm triggers with `actionsEnabled(true)`. Furthermore, we add an `SnsAction` to publish a notification to our existing SNS topic whenever this composite alarm is in the ALARM state:

```
compositeAlarm.addAlarmAction(new SnsAction(snsAlarmingTopic));
```

Whenever both “child alarms” are in ALARM state simultaneously, Amazon CloudWatch will notify us via email.

## Working and Living with Alarms and Incidents

In the previous section, we saw how we can define Amazon CloudWatch alarms for our system to get proactively notified about a potential incident. Sending out emails or notifications on time is the first step to effectively dealing with alarms and incidents.

The incident management process doesn't stop here.

The automated actions (sending a notification and performing autoscaling) are somewhat limited and by themselves might not help to resolve all our alarms on autopilot. While the well-known “Have you tried turning it on and off again?” (i.e. rebooting an instance) might help for some alarms, in general, human interaction is necessary.

Betting on the full brainpower of a developer waking up at 3 AM is something we should avoid. The person on-call might not even be familiar with our codebase. As a best practice, we can (and should) place [runbooks](#) inside our repository or document them elsewhere.

With runbooks, we ...

“Enable consistent and prompt responses to well understood events by documenting procedures [...]. Runbooks are the predefined procedures to achieve a specific outcome. Runbooks should contain the minimum information necessary to successfully perform the procedure. Start with a valid effective manual process, implement it in code and trigger automated execution where appropriate. This ensures consistency, speeds responses, and reduces errors caused by manual processes.”

(Excerpt from the [AWS Well-Architected Framework](#))

Basic Markdown files are sufficient for this purpose. If possible, each alarm should have its own runbook.

Next, we can link the runbook of a particular alarm as part of the alarm's description. This way, our firefighter doesn't have to search for the documents in the first place. We must also ensure the person on-call has access to them!

These runbooks should contain clear instructions on what to immediately check for and potential hints to mitigate the symptom or fix the underlying issue.

Something similar to the checklists that pilots have in their cockpit to handle engine failures.

We can't expect the person to immediately work with 100% brain capacity right after a harsh wake-up call at night. The responsible developer on duty shouldn't blindly scrape log files or watch numerous graphs on several dashboards. They need clear instructions with either deep links to prepared log queries or copy-pasteable instructions.

As a first step when analyzing an alarm's cause, it's important to identify to which extent the current incident propagates to other systems. For example, is it just a hiccup inside our service that doesn't affect the whole company, or is a heavily used API unavailable and grinding the entire company's operations to a halt?

Once we identify the impact on a high level, the person on-call should decide what to do next. For example, is it worth further investigating the issue, waking up other people to fix it, or going back to sleep?

Imagine a semi-important reporting functionality or batch job failing at night. In this case, it's sufficient to tackle it right at the beginning of the following day instead of fixing it with a sleepy mood at midnight.

For a real-world operation of our application, we can also define escalation policies to propagate the alarm to multiple teams or individual people if we fail to acknowledge the alarm on time, for example when the on-call person forgot to unmute their phone.

Furthermore, with fine-grained alarm routing rules, we can trigger different notifications based on the alarm's priority and the time of the day. Low priority alarms, for example, should ping us immediately via Slack during working hours but only send an email at night as they're not mission-critical. High priority alarms, however, should call the chosen developer at any time of the

day.

Coming up with a list of useful alarms is an ongoing process and depends on our system and architecture. Technically we can create an alarm for almost everything, but that would amount to little more than cargo cult. From a developer's point of view, we might have a gut feeling for those parts of our system that – should they fail – have a considerable impact on our overall operation. That's a good starting point for our alarms.

Next, the business side may (and should) also have their definition of measuring the success or failure of our product. However, as a rule of thumb, we should at least cover the SLOs (service level objectives) of our application with alarms. The freely available [SRE \(Site Reliability Engineering\) book](#) from Google is a great source of inspiration for this topic and highly recommend.

Configuring the correct thresholds, setting the proper alarm priority, and adding valuable instructions is something we learn over time. If our alarms trigger too often and there's no real damage, we must adjust the configuration. If there's too much alarm noise, our developers might soon ignore them as they lack significance.

On the other hand, if we miss identifying a critical production outage because the thresholds are too high or low, we also need to adjust them. Revisiting our alarms is an [essential part of a good postmortem](#).

The whole incident management process is a vast topic for itself. Each company usually (hopefully) defines its own standardized procedures and best practices.

When it comes to tooling support, a lot of SaaS applications have emerged for smooth incident handling:

- Opsgenie
- Splunk On-Call

- AlertOps
- AWS Incident Manager (part of AWS Systems Manager)
- etc.

Most of these tools provide seamless integration with our Amazon CloudWatch Alarms either directly or by configuring an HTTPS subscription for the SNS notification topic.

With these basic alarms for our application in place, let's take a look at a technique for preventing (or reducing the likelihood of) incidents in the first place in the next chapter.

# 18. Synthetic Monitoring with Amazon CloudWatch

Our application is now humming along in a staging and production environment. We have logging and metrics to provide us with the information we need to analyze issues and make decisions. We also have alarms on some logs and metrics in place to warn us when something goes wrong before the first user can even submit a support request.

But we still can't be sure at this point if our application's most important use cases are working as expected. There may be something wrong in the UI, in our application, or in a service the application depends on that does not create error logs or metrics that we alert on.

In this chapter, we're going to close the observability loop by adding synthetic monitoring to our Todo application. With synthetic monitoring, we regularly verify that the main features of our application are working as expected by running an automated script. This way we're alerted as early as possible when we have broken something with our recent code changes or something else has broken the app.

Since these synthetic checks are rather flaky at times, and thus take some effort to maintain, we don't cover all our application's use cases with these checks, but rather only the most important ones.

Synthetic monitoring is also known as “end-to-end testing” or “canary testing”. In many contexts, these terms mean the same thing. It's called “synthetic” because it's generating artificial traffic on our application (as opposed to

the “organic” traffic that our users are generating). The term “canary testing” comes from the rather morbid practice of taking canary birds down into mines to act as an early warning of poisonous and odorless gases. Due to their faster oxygen consumption, the birds would drop dead before any human would notice a change.

We’ll start by looking into the tools that AWS provides for synthetic monitoring and then we’ll build a canary script for our Todo application.



In this chapter, the term “canary” refers to a script that we run against our application which is deployed to a staging or production environment to check if the application behaves as expected. It’s not to be confused with a “canary deployment” or “canary release”, which means that we’re deploying a second version of the application and distribute part of the traffic to this canary instance. This is also possible with AWS, but not covered in this book.

## Introduction to CloudWatch Synthetics

The solution AWS offers for synthetic monitoring is part of the CloudWatch umbrella and is called “[CloudWatch Synthetics](#)”. Any monitoring scripts we create and run with this service are called “canaries”, so we’ll stick with this term.

CloudWatch Synthetics offers a UI in which we can create simple canaries based on a blueprint:

- a **heartbeat canary** calls a single URL and checks its HTTP response code,
- an **API canary** calls multiple endpoints of an application’s HTTP API and checks if they respond as expected,

- a **broken link checker** crawls a web page for outgoing links and checks if they are valid.

We can also use the **GUI workflow builder** to build a canary that goes through an application's web UI step by step. CloudWatch even offers a browser plugin to record these steps, which we will later use to create an initial version of our canary script. We can configure the scripts to take a screenshot with each step so that we get visual feedback.

In all cases, the result is a Node or Python script running against the Synthetics API. In the case of a Node-based canary, we can use the Puppeteer library to remote control a headless browser. In Python-based libraries, we can use Selenium instead. We can modify this script as we see fit via the CloudWatch UI, or we can copy it into a CloudFormation or CDK project to automate the process of creating the canaries.

Now that we have a shared understanding of what canaries are, let's go through the process of creating one for our Todo application.

## Recording a Canary Script for the Todo App

For our Todo application, we want to build a GUI workflow canary that goes through the main use case of the application: creating a todo item.

Here's what the workflow should look like:

1. Open the start page of the app.
2. Click the "Login" button.
3. Log in with a previously created test user.
4. Click the "Create a new Todo" button.



5. Fill in the todo form and click the “Create” button.
6. Verify that we’ve been redirected to the dashboard.
7. Verify that the new todo shows up on the dashboard.
8. Click the “Delete” button for the todo we just created.
9. Verify that the new todo no longer shows up on the dashboard.

This canary script checks three main use cases of the app: the login, creating a todo, and deleting a todo. The canary won’t detect bugs in other use cases, but we can be fairly certain that if the canary runs successfully, most users will be able to work with the app as well.

We could, of course, extend the canary script to go through all the other use cases as well. We could check if all buttons in the UI work as expected, if the pagination on the dashboard view works properly, if we can edit a todo item and add a note to it, and if the “share” functionality works.

However, canary scripts that rely on the web UI of an application are very vulnerable to change. If the `id` of a button or the layout of a page changes, for instance, a canary script may fail even though a real user is not impacted at all. The more use cases our canary scripts cover, the flakier they become and the more expensive they are to maintain.

So, the compromise is to select the few use cases that make the app usable for a user and only cover those with one or more canaries. This is why we have chosen the script above for the Todo app. For other apps, the main use cases will look completely different. If an app has a very high impact and changes very rarely, it might even make sense to cover all use cases.

In the end, the decision of which use cases to cover with a canary comes down to the very “soft” matter of feeling confident. If we as the development team feel confident that a canary will catch the worst issues when running against the app every couple of minutes, this is a signal that the coverage is good enough. A

big part of application monitoring is about improving the quality of life for the people operating the app, after all.

Note that we have created a test user specifically for this canary. In a canary test, we don't want to log in as a real user for two reasons: we'd have to share the password of the real user with the canary script, and if something changes in the real user's configuration, it might break the canary.

To build the initial canary script, we're going to use the "Canary Recorder". In the "Create Canary" dialog of the [CloudWatch Synthetics Console](#), we choose "Use a blueprint" and then "Canary Recorder". The UI will show instructions on how to install the required browser plugin which is based on the open-source [Headless Recorder](#). Once started, we hit "record" in the plugin and then navigate to the URL against which to run the canary – in our case this is `https://app.stratospheric.dev`.

Then we go through the steps we want to record in the browser as if we're a normal user of the app. Once the recording is done, we can copy the generated script from the browser plugin into the "Create Canary" UI in the CloudWatch Synthetics Console and fill out the rest of the form.

The script will look something like this:

```
var synthetics = require('Synthetics');
const log = require('SyntheticsLogger');

const recordedScript = async function () {
 let page = await synthetics.getPage();

 const navigationPromise = page.waitForNavigation()

 await synthetics.executeStep('Go to home page', async function() {
 await page.goto(
 process.env.TARGET_URL,
 {waitUntil: 'domcontentloaded', timeout: 30000})
 })

 await page.setViewport({ width: 1853, height: 949 })

 await navigationPromise

 await synthetics.executeStep('Click "Login" button', async function() {
 await page.waitForSelector('.container > .section > .container > div > .btn')
 await page.click('.container > .section > .container > div > .btn')
 })

 await navigationPromise

 // ... more steps

};
exports.handler = async () => {
 return await recordedScript();
};
```

The script imports the AWS packages Synthetics and SyntheticsLogger and then executes all the steps that we performed during the recording. The steps are initially named something like “Click\_1” (when the step is about clicking a button) or “Type\_2” (when the step is about typing something into a field), so we have to rename them manually after recording. You can find the complete script [on GitHub](#).

By default, the recorder uses CSS selectors like `.container > .section >`

`.container > div > .btn` to identify an element in the HTML of the web page. Note that this is not optimal, as it is very vulnerable to a change in the HTML layout. If we want to make it more robust, we should give each item that we want to interact with an `id` attribute and then use the `#id` selector in the script, because it is much less likely to change.

The Synthetics package is based on Puppeteer, so we have access to the [Puppeteer API](#) to interact with the website.

Since the canary should be able to run against different application environments, we make the url against which the script should run configurable by using the `TARGET_URL` environment variable.

Similarly, we want to configure the username and password the script uses for the login step to make the script independent of a given user:

```
await synthetics.executeStep('Type username', async function() {
 await page.type(
 'div:nth-child(2) > div > div > .cognito-asf #signInFormUsername',
 process.env.USER_NAME)
})

await synthetics.executeStep('Type password', async function() {
 await page.type(
 'div:nth-child(2) > div > div > .cognito-asf #signInFormPassword',
 process.env.PASSWORD)
})
```

We can enter all environment variables in the CloudWatch UI. Sadly, CloudWatch Synthetics doesn't support secret environment variables, so the password is visible for anyone who has the permission to edit the canary script.

After each change to the script, we can save the canary and let it run via the CloudWatch UI. The result of each step is displayed in the UI so that we can debug the script until it's running as expected.

## Keeping it Simple

As you might have guessed from the previous section, creating a canary script with CloudWatch Synthetics involves editing a script in a text area in the CloudWatch web UI. While that text area has syntax highlighting, it does not replace an IDE and is not suited to edit complex scripts.

At the time of writing, there is no support for developing and testing canary scripts in an IDE. The node packages `Synthetics` and `SyntheticsLogger` are not available publicly on NPM. We could probably replace the `Synthetics` package with an equivalent `Puppeteer` package for local development, but that would be on us to explore and set up since there is no official support for this.

So, our recommendation is to only use CloudWatch Synthetics for rather simple scripts. The login script is simple enough to build and debug via a web UI. If it gets more involved, you may want to choose a different tool to run your synthetic checks. You can either build one yourself (for example, by building a custom, cron-triggered Lambda that runs a headless browser script) or choose a tool from a vendor like Datadog or Splunk.

There is an argument to keep canary scripts simple enough so you *can* use CloudWatch Synthetics, however. Often, a simple HTTP status check for a REST API endpoint or a simple headless browser script like our login script is sufficient to act as a canary. After all, if the login fails, we know something is wrong and we will have to investigate. This is exactly what a canary script is for.

If a canary script grows too complex, something may be wrong with the script itself! The script's maintenance will require a lot of effort with each change of the application and the developer team's resistance to change will grow.

## Automating Canary Deployment with CDK

Once we have created and tested our canary, we want to ensure that it is deployed every time our application is deployed. For this, we will extend our existing CDK project to include the canary script.

To get access to the CloudWatch Synthetics CDK constructs and build our own construct on top of them, we have to add the following dependencies to the `pom.xml` of our CDK project:

```
<!-- Both imports can be omitted when using cdk-constructs -->
<dependency>
 <groupId>software.amazon.awscdk</groupId>
 <artifactId>aws-cdk-lib</artifactId>
 <version>${aws-cdk-lib.version}</version>
</dependency>
<dependency>
 <groupId>software.constructs</groupId>
 <artifactId>constructs</artifactId>
 <version>${constructs.version}</version>
</dependency>
```

We will build a `CanaryStack` and `CanaryApp` in the structure that we introduced in the chapter *[Designing a Deployment Project with CDK](#)*. You can find the full code of both classes [on GitHub](#).

The `CanaryApp` just serves as an entry point to create a `CanaryStack`, so we're not going to look at its code here.

The `CanaryStack` creates an S3 bucket, an IAM role, and the canary itself.

The S3 Bucket is used by CloudWatch to store the screenshots it makes during a canary run. To create it, we use the `Bucket level 2 Construct`, which is very convenient to use:

```

Bucket bucket = Bucket.Builder.create(this, "canaryBucket")
 .bucketName(applicationEnvironment.prefix("canary-bucket"))
 .removalPolicy(RemovalPolicy.DESTROY)
 .build();

```

We prefix the name of the bucket with the application name and environment to make it unique and we set the removal policy to DESTROY so that the bucket will be deleted when we delete the CanaryStack. If we don't do this, the bucket will stay alive even when we destroy the stack, and the next deployment will fail due to a "Bucket already exists" error.

Next, we create the IAM role that the canary script will assume while it's running:

```

Role executionRole = Role.Builder.create(this, "canaryExecutionRole")
 .roleName(applicationEnvironment.prefix("canary-execution-role"))
 .assumedBy(new AnyPrincipal())
 .inlinePolicies(Map.of(
 applicationEnvironment.prefix("canaryExecutionRolePolicy"),
 PolicyDocument.Builder.create()
 .statements(singletonList(PolicyStatement.Builder.create()
 .effect(Effect.ALLOW)
 .resources(singletonList("*"))
 .actions(Arrays.asList(
 "s3:PutObject",
 "s3:GetBucketLocation",
 "s3:ListAllMyBuckets",
 "cloudwatch:PutMetricData",
 "logs:CreateLogGroup",
 "logs:CreateLogStream",
 "logs:PutLogEvents"))
 .build()))
 .build()))
 .build();

```

This is a simple role with an inline policy that contains exactly the permissions that a canary needs to do its job. It needs to upload screenshots to S3, and send metrics and logs to CloudWatch.

Finally, we create the canary script itself:

```
String canaryName = applicationEnvironment.prefix("canary", 21);

CfnCanary.Builder.create(this, "canary")
 .name(canaryName)
 .runtimeVersion("syn-nodejs-puppeteer-3.9")
 .artifactS3Location(bucket.s3UrlForObject("create-todo-canary"))
 .startCanaryAfterCreation(Boolean.TRUE)
 .executionRoleArn(executionRole.getRoleArn())
 .schedule(ScheduleProperty.builder()
 .expression("rate(15 minutes)")
 .build())
 .runConfig(RunConfigProperty.builder()
 .environmentVariables(Map.of(
 "TARGET_URL", targetUrl,
 "USER_NAME", username,
 "PASSWORD", password
))
 .timeoutInSeconds(30)
 .build())
 .code(CodeProperty.builder()
 .handler("recordedScript.handler")
 .script(getScriptFromResource("canaries/create-todo-canary.js"))
 .build())
 .build();
}
```

Note that we're using the CfnCanary level 1 construct here, not the more convenient Canary level 2 construct. The reason for this is that the Canary construct is in experimental mode at the time of writing, so we to fall back to the CfnCanary construct which is more verbose, but gives us full control.

We pass in the location of the S3 bucket and the role we just created and set a schedule for the canary to run every 15 minutes. Then we pass in the three environment variables the script needs and set the timeout, so the script doesn't wait more than 30 seconds for a response before failing. As the final step, we pass in the script we created earlier from a local file in the codebase.



The method `getScriptFromResource()` just reads the script from the file as a string.

In the CDK project's `package.json` file, we then add the scripts `canary:deploy` and `canary:destroy` so we can use those scripts from the command line and our continuous deployment pipeline.

## Alerting on Canary Failure

According to our configuration, the canary will now be running every 15 minutes. In the CloudWatch UI, we can see successful and failed canary runs, but as discussed in the chapter [Alerting with Amazon CloudWatch](#), we don't want to actively monitor for failures, but instead create an alarm that will notify us of failures.

So, we'll add an Alarm construct to our `CanaryStack`:

```
Alarm canaryAlarm = new Alarm(this, "canaryAlarm", AlarmProps.builder()
 .alarmName("canary-failed-alarm")
 .alarmDescription("Alert on multiple Canary failures")
 .metric(new Metric(MetricProps.builder()
 .namespace("CloudWatchSynthetics")
 .metricName("Failed")
 .dimensionsMap(Map.of(
 "CanaryName", canaryName
))
 .region(awsEnvironment.getRegion())
 .period(Duration.minutes(50))
 .statistic("sum")
 .build()))
 .treatMissingData(TreatMissingData.NOT_BREACHING)
 .comparisonOperator(ComparisonOperator.GREATER_THAN_OR_EQUAL_TO_THRESHOLD)
 .evaluationPeriods(1)
 .threshold(3)
 .actionsEnabled(false)
 .build());
```

The alarm listens to the `Failed` metric in the `CloudWatchSynthetics` namespace. It will transition into the failed state if the canary has failed three consecutive times within 50 minutes. We set the period to 50 minutes because we expect the canary to run three times within 45 minutes. If it fails three times in a row, these three failures will have been within the last 45 minutes, which we rounded up to 50 minutes to allow for some margin of error.

This example shows that with all the knobs and dials we have at our disposal, it's not trivial to set up alerting properly. If we want to be warned earlier, we have to reduce the interval in which the canary runs *and* the evaluation period of the alarm. This is why we put the alarm into the `CanaryStack` and not together with the other alarms into the `MonitoringStack` we used in earlier chapters. Putting things together that change together is a good idea in software code and infrastructure code alike.

Finally, we might want to add an action to the alarm to notify people as was described in the previous chapter.

The alarms we created in the previous chapter and the synthetic monitoring we set up in this chapter will allow us to sleep at night knowing that the application is running smoothly and that we will be alerted if something goes south.

# Closing Thoughts

# Conquer the Cloud

Cloud development is all about a self-service approach. As developers, we can leverage this approach to use a suite of services that fit our needs without having to overcome organizational barriers.

The self-service aspect of CDK and CloudFormation is what got us hooked and Spring Boot's easy integration with all kinds of AWS services is what made us write this book.

If you haven't built software "in the cloud", yet, we hope this book sparked some interest in cloud development in general and building software with Spring Boot and AWS in particular. Maybe, even enough to suggest starting on the path toward cloud development at your place of work?

Even if your workplace isn't ready for cloud development, you might build your next SaaS side project with Spring Boot and deploy it to AWS. From our own experience, we know that it's a lot of fun to build a side project like this. Let us know when you made your first million from your SaaS business!

We love tweaking software to continually improve it, and a continuous deployment pipeline like the one discussed in this book is perfect for that.

If you're already a cloud developer, you know that software development in the DevOps world is not only about writing code, but also about getting the software into production. We hope this book helped you with getting a bit more of an in-depth perspective on the subjects of deployment and observability.

Finally, if you're making decisions in your workplace, we hope that the book made it clear that cloud development isn't a complex mystery only reserved for

high skilled developers working for tech giants. It's rather simple at its core: using and combining cloud services to create value. The self-service paradigm empowers software teams to be self-sufficient and self-organizing, unleashing their creativity and productivity. Introducing cloud development requires a culture shift, though. Hopefully, we've motivated you to get started on that path.

If you enjoyed reading this book and like our hands-on approach, check out our [Stratospheric online course](#). While this ebook constitutes a baseline for the online course, during the course we cover various topics in greater detail and provide a more dynamic, "Choose Your Own Adventure"-type of learning experience.

In any case, let us know how your cloud journey went! Drop us an email at [info@stratospheric.dev](mailto:info@stratospheric.dev). We'd love to hear your story!

# Additional Resources

Recommended resources and reading suggestions for eager and motivated developers (including special discounts & deals for existing Stratospheric reader):

- [Get Your Hands Dirty on Clean Architecture](#) (by Tom)
  - A hands-on guide to creating clean web applications with code examples in Java.
  - This book discusses how the Hexagonal Architecture style tries to fulfill this goal and translates the concepts into actual code to provide inspiration for software developers.
  - If you prefer a print version of the book, you can get it on [Amazon](#).
- [Java Testing Toolbox – 30 Testing Tools and Libraries Every Java Developer Must Know](#) (from Philip)
  - Explore the Java testing ecosystem with bite-sized introductions to the tools and libraries.
  - Think of the Java Testing Toolbox as a cookbook: Practical, concise & hands-on.
  - Covering test frameworks, assertion libraries, mocking frameworks, as well as performance testing, infrastructure, and utility libraries.
  - Get the ebook for \$9 with the discount code **u13VONHHTIy6**

- [Testing Spring Boot Applications Masterclass](#) (by Philip)
  - Deploy Spring Boot applications with more confidence and fewer regressions.
  - A comprehensive testing course to boost your confidence and productivity.
  - All Spring Boot testing best practices in a single place.
  - Get a 30% discount as a Stratospheric reader with this coupon code: **STRATOSPHERIC30**
  
- [Spring Boot course](#)
  - hands-on Spring Boot knowledge
    - \* fundamentals: dependency injection, convention over configuration, Spring and the Spring context
    - \* REST APIs with Spring Boot
    - \* testing
    - \* working with databases
    - \* security
  - for Spring Boot rookies as well as more experienced Spring developers
  - English and German language
  - Attend online via Zoom.
  - Get a 10% discount as a Stratospheric reader with this coupon code: **STRATOSPHERIC10**
  
- [AWS Fundamentals book](#): Covering all aspects of AWS, from getting started to core building blocks. Programming-language and framework-agnostic:
  - Covering all major categories of services (Compute, Storage, Networking, etc.).
  - From traditional computing to serverless computing.

- The important config options you need to know and nothing more.
- The basics of modern infrastructure-as-code (CloudFormation, CDK, Serverless).
- Nice-looking and printable infographics for every service covered in the book.
- Get a [30% discount](#) as a Stratospheric reader



# Appendix

## Technical GitHub Actions IAM User

To mirror our GitHub Actions setup, create an IAM user and attach the following permission policies:

- AmazonRDSFullAccess
- AmazonEC2ContainerRegistryFullAccess
- SystemAdministrator
- AmazonEC2FullAccess
- AmazonECS\_FullAccess
- AWSCertificateManagerFullAccess
- AWSLambda\_FullAccess
- AWSCloudFormationFullAccess
- AmazonCognitoPowerUser
- AmazonS3FullAccess
- AmazonSSMFullAccess
- IAMReadOnlyAccess
- AWSKeyManagementServicePowerUser
- AWSCloudFormationFullAccess
- AWSLambda\_FullAccess
- AmazonMQApiFullAccess

- CloudWatchSyntheticsFullAccess

Since AWS imposes a limit of 10 policies per user (“Cannot exceed quota for PoliciesPerUser: 10”), you need to create user groups, attach the policies to the groups, and add the user to those groups.

For example, you can create a group called `EC2_users`, attach the following policies, and subsequently add your IAM user to that group (repeat for any other required permission policies):

- AmazonEC2ContainerRegistryFullAccess
- AmazonEC2FullAccess
- AmazonECS\_FullAccess

If you plan to automate the deployment for AWS services we didn’t cover in this book, make sure to add the required permissions to this technical user.

To simplify this permission setup we’ve added a [GitHubActionsApp CDK App](#) to the [Stratospheric repository](#) and a corresponding NPM script, which will automatically create the requisite permissions in your AWS account (using your own AWS profile instead of `stratospheric`, of course):

```
npm run github-actions:deploy -- --profile stratospheric
```

## Deployment Guide

Follow this step-by-step guide if you want to deploy the entire Stratospheric infrastructure including the sample application to your AWS account.

Prerequisites:

- You have a custom domain (e.g., `mycompany.io`) hosted within Amazon Route53. You can also host your domain at a different provider (e.g., GoDaddy, Namecheap, Hetzner, etc.). However, this involves additional manual effort to correctly set up SSL.
- You've created an SSL certificate within the AWS Certificate Manager for that domain and have the ARN for the SSL certificate.
- You've [configured a named profile](#) for the AWS CLI (e.g., `stratospheric`) with sufficient privileges to create / delete resources.
- Your Docker Engine is up and running.
- You have Node  $\geq 16$  installed: `node -v`
- You have Java 17 installed: `java -version`
- You're using an x64 processor, or you're able to create a Docker image for this architecture (see this [article](#) if you're using an Apple M1).

Bootstrapping the entire infrastructure from scratch takes 20 – 30 minutes. You can speed up the process by increasing the instances sizes for the database, ActiveMQ and the ECS tasks.

**IMPORTANT NOTE:** Deploying this infrastructure will result in recurring costs if you don't clean up the resources afterward. Closely follow the progress of the stack deletion and check the CloudFormation overview in the AWS console afterward. There shouldn't be any stack definitions left.

## Step 1: Deploy the Surrounding Infrastructure

We're assuming you're using a named profile for the AWS CLI called `stratospheric`. If you're using the default profile, you can remove `-- --profile stratospheric` from all the following commands:

1. Clone the [Stratospheric repository](#) from GitHub

2. Navigate to the `cdk` folder

3. Adjust the configuration in `cdk.json`:

- `applicationName`: The name of your application. Make sure the application and environment name combination is unique as we're creating resources that require a unique name, e.g., `todo-app`.
- `region`: The region you want to deploy the infrastructure to, e.g., `eu-central-1`.
- `accountId`: The account ID of your AWS account, e.g., `221875718260`.
- `dockerRepositoryName`: The name of your Docker repository. Unless you want to deploy a Docker image from another registry, this should be equal to the `applicationName`, e.g., `todo-app`.
- `dockerImageTag`: The Docker image you want to deploy. Update the tag whenever you want to deploy a new version of the application, e.g., `1`.
- `applicationUrl`: The full application URL of your application, e.g., `https://app.stratospheric.dev`.
- `loginPageDomainPrefix`: This becomes the subdomain for the Cognito login form, e.g., `stratospheric-staging`.
- `environmentName`: The name of the application environment, e.g., `staging` or `prod`.
- `springProfile`: The Spring profile that should be activated for the running ECS container, e.g., `aws`.
- `activeMqUsername`: The name of the ActiveMQ root user, e.g., `activemqUser`.
- `canaryUsername`: (Optional) If you plan to deploy the Canary stack, add the name of the user to log in. You first have to create this user manually via the sample application, e.g., `canary`.
- `canaryUserPassword`: (Optional) If you plan to deploy the `CanaryApp`, add the password of the user to log in, e.g., `s3cr3t`.

- `confirmationEmail`: (Optional) The email for receiving CloudWatch alerts, e.g., `info@stratospheric.dev`.
- `applicationDomain`: The domain of your application, without protocol, e.g., `app.stratospheric.dev`.
- `sslCertificateArn`: The arn for the SSL certificate for your custom domain. Step #5 will explain how to get this parameter, e.g., `arn:aws:acm:REGION:ID:certificate/UUID`.
- `hostedZoneDomain`: The domain name for the hosted zone within Route53, e.g., `stratospheric.dev`.
- `githubToken`: (Optional) If you plan to deploy the deployment sequencer, add the access token for GitHub to trigger GitHub Action workflow, e.g. `s3cr3t`.

#### 4. Bootstrap CDK for your AWS account:

```
npm run bootstrap -- --profile stratospheric
```

#### 5. Create an SSL certificate for your domain:

If you're using Route 53 for your domain, make sure to configure the `applicationDomain` and `hostedZoneDomain` parameters. Next, deploy the `CertificateApp` by running this command:

```
npm run certificate:deploy -- --profile stratospheric
```

Afterwards, copy the `sslCertificateArn` to the `cdk.json` file.

If you're not using Route 53, follow the instructions in the [Creating an SSL Certificate with CDK](#) chapter on how to generate and validate an SSL certificate manually.

#### 6. Deploy the `NetworkStack`-dependent infrastructure:

```
npm run network:deploy -- --profile stratospheric
npm run database:deploy -- --profile stratospheric
npm run activeMq:deploy -- --profile stratospheric
```

## 7. (or in parallel to #6) Deploy NetworkStack-independent infrastructure:

```
npm run repository:deploy -- --profile stratospheric
npm run messaging:deploy -- --profile stratospheric
npm run cognito:deploy -- --profile stratospheric
```

## 8. Route traffic from your custom domain to the ELB:

If you're using Route 53 for your domain, you can automate this process with this command:

```
npm run domain:deploy -- --profile stratospheric
```

Otherwise, you have to create the DNS A record with your domain registrar. This A record has to point to the default DNS name of the ELB. You can get this information from the AWS console: Look for 'EC2' -> 'Load Balancers' -> 'Select the load balancer'

## Step 2: Build and Push the First Docker Image

Build the first Docker image:

```
cd application
./gradlew build
```

```
docker build -t <accountId>.dkr.ecr.<region>.amazonaws.com/<applicationName>:1 .
```

```
aws ecr get-login-password --region <region> --profile stratospheric | docker login \
--username AWS --password-stdin <accountId>.dkr.ecr.<region>.amazonaws.com
```

```
docker push <accountId>.dkr.ecr.<region>.amazonaws.com/<applicationName>:1
```

On Apple M1:

```
docker buildx build --platform linux/amd64,linux/arm64 --push -t <accountId>.dkr.ecr\
.<region>.amazonaws.com/todo-app:1 .
```

### Step 3: Deploy the Docker Image to the ECS Cluster

1. Customize the `dockerImageTag` property inside the `cdk/cdk.json` file to match the Docker image tag you've just pushed:

```
npm run service:deploy -- --profile stratospheric
```

Afterwards, you'll be able to access the application from your custom domain.

Please consider the following:

The sharing functionality only works if either you:

1. Request production access for SES and verify the domain from which your application sends emails.
2. Manually verify all addresses you're about to send emails to and also verify the domain from which your application sends emails.

### Step 4: (Optional) Deploy the Monitoring Infrastructure

1. Deploying the Amazon CloudWatch dashboard and alarms:

```
cd cdk
npm run monitoring:deploy -- --profile stratospheric
```

2. You'll receive an email to verify the SNS subscription based on what you've configured for `confirmationEmail`.

## Step 5: (Optional) Deploy the Canary Stack

1. Create an application user within the sample application.
2. Update the `canaryUsername` and `canaryUserPassword` inside the `cdk/cdk.json` file.
3. Deploy the canary stack.

```
cd cdk
npm run canary:deploy -- --profile stratospheric
```

## Step 6: Destroy Everything

Run all `npm run *:destroy -- --profile stratospheric` scripts in the reverse order the resource were created in.

Visit the CloudFormation web console to ensure all stacks have been removed.



# Changelog

Notes about the changes in each revision of this book.

- **Revision 0.1 (2020-11-08)** – First published version, containing the chapters *Introduction*, *The Sample Todo Application*, *Getting Warm with AWS*, and *Overview of AWS Services*.
- **Revision 0.2 (2020-12-20)** – Added the chapters *Managing Permissions with IAM*, *The Evolution of Automated Deployments*, *First Steps with CDK*, and *Local Development*.
- **Revision 0.3 (2021-02-14)** – Added the chapters *Designing a Deployment Project with CDK*, *Building User Registration and Login with Cognito*, and *Connecting to a Database With RDS*. Added preview for chapters in Part 3 (Operations).
- **Revision 0.4 (2021-04-10)** – Added the chapters *Building a Continuous Deployment Pipeline*, *Sharing Todos with SQS and SES*, and *Push Notifications with Amazon MQ*.
- **Revision 1.0 (2021-08-01)** – Added the chapters *Tracing User Actions with DynamoDB*, *Structured Logging with Amazon CloudWatch*, *Metrics with Amazon CloudWatch*, *Alerting with Amazon CloudWatch*, *Synthetic Monitoring with Amazon CloudWatch*, and *Configuring HTTPS and a Custom Domain with Route 53 and ELB*. Polished and proofed all the things.
- **Revision 1.1 (2021-09-28)** – Clarifications for the section about scaling out the *User Registration and Login with Cognito* as Spring Session is also a valid alternative to sticky sessions. Clarifications around using an SSL certificate in the *Designing a Deployment Project with CDK* chapter. Unifying the look and feel of some diagrams using *Excalidraw*.

- **Revision 1.2 (2021-10-24)** – New section on securing parameters with the AWS Secrets Managers as part of the [User Registration and Login with Cognito](#) chapter. Migration to Spring Cloud AWS 2.3.2 and its new home at `io.awspring`. This dependency update brings various configuration improvements, especially for the AWS SDK clients.
- **Revision 1.3 (2021-11-23)** – Improved [SSL and custom domain setup](#); using a `cdk.json` file to pass context and variables to CDK scripts; added a note about the GitHub Actions concurrency feature to the [Building a Continuous Deployment Pipeline](#).
- **Revision 1.4 (2022-01-18)** – Moved the “resources” section from the back to the front because people kept asking about how to join the Slack community. Migration to the AWS CDK v2. Adapting the [First Steps with CDK](#), [SSL and custom domain setup](#), [User Registration and Login with Cognito](#), [Metrics with Amazon CloudWatch](#), [Alerting with Amazon CloudWatch](#), [Synthetic Monitoring with Amazon CloudWatch](#) chapters for AWS CDK v2.
- **Revision 1.5 (2022-02-13)** – Upgrade the sample Todo application and the CDK infrastructure project to Java 17.
- **Revision 1.6 (2022-03-31)** – Added missing information about bootstrapping an AWS environment. Two new sections were added to the beginning of the book: [Bootstrapping the AWS Environment for a CDK Deployment](#) and [Bootstrapping a New Environment](#). Upgrade PostgreSQL to 12.9.
- **Revision 1.7 (2022-07-13)** – Added local development support for the Apple M1 (ARM64) processor by upgrading the Keycloak and LocalStack Docker image. Refactored the `TodoController` and `TodoService` from the [Connecting to a Database With RDS](#) chapter. Favor loading different Spring beans via custom properties instead of profiles (see rationale [here](#)).
- **Revision 1.8 (2022-08-01)** – Provision the DynamoDB table with the AWS CDK and publish Spring events asynchronously ([Tracing User Actions with DynamoDB](#)). Apply the principle of least privilege to the IAM role of the ECS

Task by removing any wildcard: \* (see `ServiceApp`). Refactor the Spring Security configuration (deprecation of `WebSecurityConfigurerAdapter`) to use a component-based security configuration ([Building User Registration and Login with Cognito](#)).

- **Revision 1.9 (2022-10-04)** - Add a blueprint for a ready-to-use Amazon CloudWatch dashboard targeting Java / Spring Boot applications to the sources ([Creating Dashboards with Amazon CloudWatch](#)). Mention the complementary Stratospheric online course in the [Outro](#) and [Introduction](#). Fixing various typos and stylistic improvements.
- **Revision 1.10 (2023-02-21)** - Migration of the Stratospheric sample Todo application to Spring Boot 3.0 and Spring Cloud AWS 3.0. This includes the required namespace changes from `javax` to `jakarta` as well as a transition from the AWS Java SDK v1 to v2. Furthermore, we're now using the Spring Cloud AWS DynamoDB support to reduce the manual setup efforts in the [Tracing User Actions with DynamoDB](#) chapter.
- **Revision 1.11 (2023-03-14)** - Updating the required policies for the GitHub Actions user in the [Appendix](#). Fixing syntax errors in the manuscript that caused issues with the ePUB version of the ebook.
- **Revision 1.12 (2023-04-21)** - Adding a section for [Additional Resources](#) to the appendix of the ebook.
- **Revision 1.13 (2024-07-23)** - Links that previously led to chapter-specific example applications now lead to the main example application to avoid the chapter-specific apps to become outdated. Updated the chapter [Designing a CDK Project](#) to make it easier to find the IP address of the deployed service. Added a [GitHubActionsApp CDK App](#) to the repository and to the appendix that automatically creates the prerequisite IAM permissions for deploying from a GitHub Actions workflow.